A SELF-STUDY COURSE

Fine-tuning and Reinforcement Learning

Theory, mathematics, and code — from tensors to RLHF, DPO and GRPO

41 chapters · 31 companion scripts · built 2026-10-27

Contents

Part 0 — Getting started

Fine-tuning and reinforcement learning — theory, mathematics, and code

A practical learning map

Theory and code: complete reading index

Requested RL syllabus: prerequisites, mathematics, and code

The mathematical language used in this course

Practical tricks, with the reason behind each one

Vocabulary beside the code

Part 1 — Foundations

1.1 Python, vectors, softmax, and gradient descent + `01_python_and_math.py`

1.2 Tensors, automatic differentiation, and checking gradients + `02_tensors_autograd.py`

1.3 Neural networks and the complete training loop + `03_neural_network_training.py`

1.4 Tokens, embeddings, and causal attention + `04_tokens_attention.py`

1.5 A tiny transformer language model from start to sampling + `05_tiny_transformer.py`

Part 2 — Fine-tuning

2.1 Transfer learning: reuse features, then adapt them + `01_transfer_learning.py`

2.2 Instruction tuning, response-only loss, and token masks + `02_instruction_data_and_masks.py`

2.3 LoRA: adapting a model through low-rank weight updates + `03_lora_from_scratch.py`

2.4 Fine-tuning a pretrained language model: objective, data, and updates + `04_huggingface_sft.py`

2.5 Reloading models and generating tokens + `05_reload_and_generate.py`

Part 3 — Reinforcement learning

3.1 Bandits: learn which button gives the most rewards + `01_bandits.py`

3.2 MDPs, value iteration, and policy iteration: planning a route to reward + `02_mdp_value_iteration.py`

3.3 Q-learning and SARSA: learn a route by trying actions + `03_q_learning_sarsa.py`

3.4 DQN and Double DQN: learning action values with a neural network + `04_dqn.py`

3.5 Policy gradients, REINFORCE, and a learned baseline + `05_reinforce_actor_critic.py`

3.6 PPO and GAE: from rewards to a stable policy update + `06_ppo_and_gae.py`

3.7 Monte Carlo: model-based planning and model-free learning + `07_monte_carlo.py`

3.8 A2C and A3C: advantage actor-critic + `08_a2c_a3c.py`

3.9 TRPO: constrain each policy update + `09_trpo.py`

Part 4 — Alignment

4.1 Reward models: learning scores from pairwise preferences + `01_reward_model.py`

4.2 RLHF and KL regularization: improve reward without unrestricted drift + `02_rlhf_kl_penalty.py`

4.3 DPO: derive preference optimization from a regularized reward objective + `03_dpo.py`

4.4 GRPO: compare groups of answers instead of learning a value critic + `04_grpo.py`

4.5 SDPO, RLSD, and OPSD: learning from a privileged self-teacher + `05_sdpo_rlsd_opsd.py`

Part 5 — Advanced topics

5.1 Efficient training: accumulation, precision, clipping, schedules, and checkpoints + 01_training_efficiency.py
5.2 Soft Actor-Critic: continuous actions, entropy, and twin value estimates + 02_sac_update.py
5.3 Offline RL and the conservative Q-learning penalty + 03_offline_rl_cql.py
5.4 Distillation and quantization: two different ways to make models smaller + 04_distillation_and_quantization.py
5.5 Evaluation and uncertainty: how much should you trust an improvement? + 05_evaluation_and_uncertainty.py
5.6 Activation checkpointing: recompute values to save training memory + 06_activation_checkpointing.py
5.7 Distributed data parallel training: split data and average gradients + 07_distributed_training.py

Part 6 — Projects and appendices

Capstones and thesis experiments
Primary sources for deeper study
Verification record

PART 0

Getting started

README.md

Fine-tuning and reinforcement learning — theory, mathematics, and code

For the requested RL syllabus, start with the RL coverage and prerequisites guide.

Start with the theory and code index, then follow the learning map. Every one of the 31 lesson scripts has a matching Markdown chapter beside it: for example, `06_ppo_and_gae.md` explains the mathematics behind `06_ppo_and_gae.py`.

Read the chapter first for intuition, definitions, derivations, worked numerical examples, and a map from equations to code. Then run and modify the Python file. Start with the math primer if derivatives, expectations, entropy, or KL divergence are unfamiliar.

This is a broad beginner-to-advanced foundation, not an exhaustive treatment of every research method. Complete trainers and isolated equation labs are clearly identified. Small toy successes are not evidence of real-world model quality.

Setup

Use Python 3.11 or newer. Run these commands in this folder:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python 01_foundations/01_python_and_math.py
python 01_foundations/02_tensors_autograd.py
```

On Windows, activate with `.venv\Scripts\activate` instead. For a CPU-only PyTorch installation on Linux/Windows, you can use `python -m pip install torch --index-url https://download.pytorch.org/whl/cpu`. For CUDA, follow the PyTorch installation selector. No GPU or account is needed for the core lessons. The first Python/math lesson, bandits, value iteration, Q-learning/SARSA, and bootstrap evaluation need only Python.

Reading order

Folder	What you learn
00_start	Study order, terminology, experiments, troubleshooting
01_foundations	Python/math, tensors, autograd, neural networks, attention, tiny transformer
02_finetuning	Transfer learning, SFT labels, LoRA, real Hugging Face training, reload/inference
03_reinforcement_learning	Bandits, MDPs, value/policy iteration, Monte Carlo, SARSA/Q-learning, DQN, policy gradients, A2C/A3C, TRPO, PPO/GAE
04_alignment	Preference reward models, RLHF objective, DPO, GRPO, SDPO/RLSD/OPSD
05_advanced	Efficient training, SAC, offline CQL, distillation, quantization, uncertainty, checkpointing, DDP
06_projects	Capstone experiments and thesis directions

Read matching `.md` and `.py` files together in numerical order within each folder. The real-model scripts are optional: you can continue into RL without downloading any pretrained model.

Run and check the offline lessons

```
python run_lessons.py --list
python run_lessons.py --stdlib-only
python run_lessons.py
python -m unittest discover -s tests -v
```

The runner excludes external model downloads and the multi-process DDP lesson. It runs each lesson separately with a timeout and reports failures. To run DDP:

```
torchrun --standalone --nproc_per_node=2 05_advanced/07_distributed_training.py
```

Fine-tune an actual pretrained model

```
python -m pip install -r requirements-huggingface.txt
python 02_finetuning/04_huggingface_sft.py --model sshleifer/tiny-gpt2 --steps 5
python 02_finetuning/05_reload_and_generate.py
# A larger pretrained base, still small by LLM standards:
python 02_finetuning/04_huggingface_sft.py --model distilgpt2 --method lora --steps 20
# Full parameter fine-tuning:
python 02_finetuning/04_huggingface_sft.py --method full --output outputs/full
```

The tiny GPT-2 option checks the pipeline; expect poor generated text. Even DistilGPT-2 with eight teaching examples will not become a useful assistant. First runs require internet and disk space for model downloads. CPU training of the larger model is slower and requires more RAM. Outputs are saved locally, never uploaded. Select a new `--output` directory to preserve previous runs.

For the optional NVIDIA GPU QLoRA branch, install `bitsandbytes`, then add `--qlora --method lora`. That branch is hardware-dependent. Larger model choices need a compatible tokenizer/template and enough GPU memory. QLoRA support is based on the Transformers quantization documentation.

How to study a file

1. Read the matching Markdown chapter and derive its worked example by hand.
2. Open the Python file and connect each main operation to its equation.
3. Predict the tensor shapes and output before running it.
4. Run once, then change one hyperparameter and record the result.
5. Complete the exercise in the comments. Explain why the result changed.
6. Reimplement the central equation without looking, then compare.

Keep an experiment journal: hypothesis, seed, configuration, data split, metrics, failure cases, and conclusion. Consult tricks and debugging, the glossary, and primary sources. See verification details for what was actually executed.

A practical learning map

Every lesson now has a matching `.md` explanation and `.py` implementation. Use the paired index to read theory before code. For notation and mathematical prerequisites, see the math primer.

Prerequisites: basic computer use and willingness to read Python. The first lesson introduces lists, functions, loops, dot products, softmax and derivatives; if those are completely new, pause and practice each with your own numbers. You do not need to know deep learning before starting.

Stage 1: Learn what training actually does

Read `01_foundations/01` through `03`. Understand a prediction, loss, derivative, parameter update, minibatch, and held-out evaluation. Derive the two gradients in linear regression by hand. Explain why removing all nonlinearities makes a multilayer network equivalent to one linear map.

Checkpoint: fit a different synthetic function and explain training versus validation performance without using the word “magic.”

Stage 2: Understand language models

Read foundations `04` and `05`. Trace text -> token IDs -> embeddings -> causal attention -> logits -> next-token loss. Track [batch, sequence, vocabulary] shapes. Distinguish pretraining from fine-tuning and inference.

Checkpoint: explain why a causal model cannot see tomorrow's target token, why generation repeatedly calls the model, and what temperature changes.

Stage 3: Fine-tune responsibly and efficiently

Read all files in `02_finetuning`. Start with a source/target task, then learn response-only supervision and LoRA algebra. Use the external model workflow only after understanding the offline versions. Read the saved adapter config and identify which original base weights are needed for reload.

Checkpoint: explain the difference between the attention mask and label mask; show that frozen parameters do not change; compare full tuning and LoRA at the same evaluation budget. QLoRA means a quantized frozen base plus trainable adapters.

Stage 4: Learn RL from small environments

Read `03_reinforcement_learning` in order. Distinguish reward (one step) from return (discounted future sum). Compute a Bellman backup by hand. Move from a table to a neural Q function, then to an explicitly stochastic policy.

Checkpoint: explain on-policy versus off-policy, why replay helps DQN but stale replay is problematic for ordinary PPO, and why termination differs from a collector's time limit. Explain why GAE uses two different masks.

Stage 5: Connect LLM fine-tuning to preference optimization

Read `04_alignment`. SFT imitates demonstrations. A reward model learns pairwise preferences. RLHF optimizes generated behavior against rewards and a reference constraint. DPO learns directly from fixed preference pairs. GRPO compares multiple sampled answers to the same prompt without a learned value critic.

Checkpoint: trace which tensors receive gradients. Explain the separate roles of the reference policy and the old rollout policy. They are NOT interchangeable. The alignment examples use tiny feature/categorical problems to expose the objectives; they do not implement production LLM alignment pipelines.

Stage 6: Study advanced objectives and systems

Read `05_advanced`. Reproduce each equation before extending it. SAC and CQL are explicit equation labs; extend them into full agents only after understanding replay collection, target updates, evaluation, and environment action bounds. Read the roadmap below for subjects that require separate larger projects.

Checkpoint: distinguish memory for parameters, gradients, optimizer states, and activations. Explain how mixed precision, LoRA, checkpointing, DDP and FSDP address different parts of the problem.

Further advanced topics (reading/research extensions, not implemented here)

Topic	Central idea	Useful next experiment
TRPO	Constrain policy updates using a KL-based trust region	Compare measured KL with PPO clipping
TD3	Twin critics, delayed actor updates, target action noise	Compare with SAC on bounded continuous actions
Distributional RL	Learn a return distribution rather than just its mean	Implement categorical return projection
Prioritized replay	Sample important transitions, correct sampling bias	Compare TD-error sampling with uniform replay

Topic	Central idea	Useful next experiment
n-step returns	Trade bootstrap bias against return variance	Sweep horizon at fixed environment steps
Model-based RL	Learn dynamics and plan imagined futures	Measure compounding model error
IQL / offline RL	Learn without querying unsupported actions	Compare offline policy with behavior cloning
Contextual bandits	Choose actions conditional on features	Add observations to the bandit lesson
Multi-agent RL	Other learning agents make the environment nonstationary	Start with cooperative matrix games
Hierarchical RL	Learn temporally extended skills/options	Separate navigation subgoals from control
RNN policies / POMDPs	Infer hidden state from history	Hide velocity in a control environment
Curriculum / sparse rewards	Improve learning signal and task sequence	Compare shaping with unmodified task return
Prefix/prompt tuning, adapters, DoRA	Alternative parameter-efficient updates	Compare parameter and activation budgets
FSDP / ZeRO	Shard weights, gradients, and/or optimizer states	Profile peak memory across ranks
Sequence packing / attention kernels	Reduce padding and attention overhead	Ensure packed documents have correct boundaries
Long-context tuning / RoPE	Adapt positional behavior and context usage	Evaluate retrieval across position and length
Continual learning	Adapt while limiting catastrophic forgetting	Measure both source and target tasks
RLAIF, process rewards, verifiable rewards	Change feedback source or granularity	Audit verifier errors and length shortcuts
Preference variants (IPO, KTO, ORPO)	Change preference assumptions/objectives	Derive one objective and test label noise
Tool-using agents	Actions may call tools over multiple turns	Define tool costs, observations, and termination

Choose one capstone in `06_projects/PROJECTS.md`. A narrow, controlled study is more useful than running many methods without understanding their comparisons.

Requested RL sequence

Use the RL syllabus guide for MDP/RL setup, policy iteration, Monte Carlo, A2C/A3C, TRPO, PPO/GRPO, and SDPO/RLSD/OPSD with prerequisite links and code.

Theory and code: complete reading index

Read the explanation first, work through its numerical example, then open the matching code. Each row pairs files with exactly the same filename stem.

If the notation is unfamiliar, start with the math primer. For the overall progression, see the learning map.

01 foundations

Topic	Mathematical explanation	Runnable code
Python, vectors, softmax, and gradient descent	Read theory	Open Python
Tensors, automatic differentiation, and checking gradients	Read theory	Open Python
Neural networks and the complete training loop	Read theory	Open Python
Tokens, embeddings, and causal attention	Read theory	Open Python
A tiny transformer language model from start to sampling	Read theory	Open Python

02 finetuning

Topic	Mathematical explanation	Runnable code
Transfer learning: reuse features, then adapt them	Read theory	Open Python
Instruction tuning, response-only loss, and token masks	Read theory	Open Python
LoRA: adapting a model through low-rank weight updates	Read theory	Open Python
Fine-tuning a pretrained language model: objective, data, and updates	Read theory	Open Python
Reloading models and generating tokens	Read theory	Open Python

03 reinforcement learning

Topic	Mathematical explanation	Runnable code
Bandits: learning which action is rewarding	Read theory	Open Python
MDPs, Bellman equations, value iteration, and policy iteration	Read theory	Open Python
Q-learning and SARSA: Bellman learning from experience	Read theory	Open Python
DQN and Double DQN: learning action values with a neural network	Read theory	Open Python
Policy gradients, REINFORCE, and a learned baseline	Read theory	Open Python
PPO and GAE: from rewards to a stable policy update	Read theory	Open Python
Model-based rollout planning and model-free Monte Carlo control	Read theory	Open Python
A2C and concurrent A3C	Read theory	Open Python
TRPO constrained update	Read theory	Open Python

04 alignment

Topic	Mathematical explanation	Runnable code
Reward models: learning scores from pairwise preferences	Read theory	Open Python
RLHF and KL regularization: improve reward without unrestricted drift	Read theory	Open Python
DPO: derive preference optimization from a regularized reward objective	Read theory	Open Python
GRPO: compare groups of answers instead of learning a value critic	Read theory	Open Python
SDPO, RLSD, and OPSD objectives	Read theory	Open Python

05 advanced

Topic	Mathematical explanation	Runnable code
Efficient training: accumulation, precision, clipping, schedules, and checkpoints	Read theory	Open Python
Soft Actor-Critic: continuous actions, entropy, and twin value estimates	Read theory	Open Python
Offline RL and the conservative Q-learning penalty	Read theory	Open Python
Distillation and quantization: two different ways to make models smaller	Read theory	Open Python
Evaluation and uncertainty: how much should you trust an improvement?	Read theory	Open Python
Activation checkpointing: recompute values to save training memory	Read theory	Open Python
Distributed data parallel training: split data and average gradients	Read theory	Open Python

Every explanation includes the problem, key notation, mathematical reasoning, a worked calculation, the connection to its implementation, and exercises or interpretation checks. Some advanced Python files are equation labs; their companion chapters identify the additional work needed for a full trainer.

See the requested RL syllabus for prerequisites and topic-by-topic coverage.

Requested RL syllabus: prerequisites, mathematics, and code

Start here to study the requested topics. Paths are relative to this file; run commands from the `finetuning_and_rl` folder. Core code is offline and CPU-sized.

Prerequisite sequence

1. Python loops, functions, lists, classes, random sampling, and numerical debugging: Python and math.
2. Sums, vectors/matrices, conditional probability, expectations, variance, derivatives/chain rule, softmax, logarithms, entropy and KL: math primer. Practice each short calculation there.
3. Tensors, gradients, gradient descent and loss functions: autograd and neural network training.
4. For deep RL: function approximation and optimizer behavior. For TRPO add Hessians and solving linear systems; its chapter introduces local curvature.
5. For GRPO and self-distillation: token/attention basics, causal language modeling, and response masking.

RL setup and the difference between an MDP and RL

An MDP is a mathematical problem specification $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0)$: states, actions, transition law, expected reward, discount, and initial-state distribution. The Markov property is $P(S_{t+1}, r_t \mid S_0, A_0, \ldots, S_t, A_t) = P(S_{t+1}, r_t \mid S_t, A_t)$. RL is a learning framework: an agent uses experience to improve decisions, frequently without knowing P or R. An MDP may be solved by planning with a known model; RL may learn a model or learn values/policies directly. A partially observed agent generally needs history or belief state rather than the raw observation.

$$S_0 \sim \rho_0, \quad A_t \sim \pi_\theta(\cdot \mid S_t), \quad (S_{t+1}, r_t) \sim p(\cdot, \cdot \mid S_t, A_t),$$

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t r_t\right].$$

Here r_t denotes the reward received after action A_t; some books call it R_{t+1}. Define $V^\pi(s) = E[G_t \mid S_t = s]$, $Q^\pi(s,a) = E[G_t \mid S_t = s, A_t = a]$, and $A^\pi = Q^\pi - V^\pi$. The agent/environment loop is:

```
state = env.reset()
while True:
    action = agent.act(state)       # exploration is part of behavior policy
    nxt, reward, terminated, truncated = env.step(action)
    agent.observe(state, action, reward, nxt, terminated, truncated)
    if terminated or truncated:
        state = env.reset()
```

```
    else:
        state = nxt
```

This is interface pseudocode; the runnable lessons define their own tiny environments. A real Gymnasium `reset()` returns `(observation, info)` and `step()` includes an `info` dictionary. A true terminal suppresses bootstrap; an external continuing-task time limit bootstraps from the final observation while stopping the trace at reset. Lesson 06 implements both masks explicitly.

Model-based means using a transition model for prediction or planning; model-free means learning without that model. On-policy means targeting the behavior policy; off-policy allows a different target policy. Bootstrapping means using a learned estimate inside a target. These are separate distinctions.

Coverage for item 2

Requested topic	Theory and prerequisite	Runnable code
MDP, RL setup, MDP versus RL	This guide, then Bellman equations	corridor model
Value iteration and policy iteration	evaluation, improvement, convergence; probability and discounted return	both algorithms
Model-based Monte Carlo	simulation and rollout planning; sample means	rollout planner
Model-free Monte Carlo	first-visit MC control; epsilon-greedy exploration	MC control
SARSA and Q-learning	on/off-policy TD updates; Bellman targets	both tabular learners
Deep Q-learning (DQN)	replay, target networks, DQN versus Double DQN; neural nets	DQN / Double DQN trainer

SARSA uses $Q \leftarrow Q + \alpha[r + \gamma Q(s', a') - Q]$ with sampled next action. Q-learning uses $Q \leftarrow Q + \alpha[r + \gamma \max_b Q(s', b) - Q]$. Terminal targets are r. DQN regresses $Q_\theta(s, a)$ toward $y = r + \gamma(1 - d) \max_b Q_{target}(s', b)$ with replay and frozen targets. Double DQN selects $b^* = \arg\max_b Q_\theta(s', b)$ then evaluates $Q_{target}(s', b^*)$. The default uses Double DQN; pass `--vanilla-dqn` for original DQN.

Coverage for item 3

Requested topic	Theory	Runnable code and scope
Policy gradient / REINFORCE	likelihood-ratio derivation and baseline	complete MC-return trainer
A2C and A3C	n-step advantage and synchronization	synchronous and concurrent trainers
TRPO	KL constraint and curvature	exact-bandit update lab
PPO	clipped surrogate and GAE	complete toy trainer
GRPO	group rewards and critic-free advantages	one-token response trainer

For a discounted episodic objective, the likelihood-ratio estimator is $\nabla J = E[\sum_t \gamma^t \nabla \log \pi_\theta(A_t \mid S_t)(G_t - b(S_t))]$. An action-independent baseline cancels because $\sum_a \pi(a \mid s) \nabla \log \pi(a \mid s) = \nabla \sum_a \pi(a \mid s) = 0$. Many practical implementations, including the small teaching trainer, average unweighted per-step terms; distinguish that convention from the exact discounted start-state objective when deriving an estimator. PPO maximizes $E[\min(\rho \hat{A}, \text{clip}(\rho, 1-\epsilon, 1+\epsilon)\hat{A})]$. GRPO constructs $\hat{A}_i = (r_i - \bar{r})/(\sigma_r + \varepsilon)$ within each prompt's group, then uses a related surrogate. Zero reward variance gives zero reward advantage.

Coverage for item 4

SDPO, RLSD, and OPSD chapter provides paper-specific definitions, equations, a worked calculation, gradient handling, and links to primary sources. Companion code implements masked reverse KL, forward KL/JSD, and RLSD token credit plus a policy surrogate. These are objective labs, not complete LLM training pipelines. Read GRPO and the KL sections before this chapter.

Suggested execution order

```
python3 03_reinforcement_learning/02_mdp_value_iteration.py
python3 03_reinforcement_learning/07_monte_carlo.py
python3 03_reinforcement_learning/03_q_learning_sarsa.py
python3 03_reinforcement_learning/04_dqn.py
python3 03_reinforcement_learning/05_reinforce_actor_critic.py
python3 03_reinforcement_learning/08_a2c_a3c.py
python3 03_reinforcement_learning/09_trpo.py
python3 03_reinforcement_learning/06_ppo_and_gae.py
python3 04_alignment/04_grpo.py
python3 04_alignment/05_sdpo_rlsd_opsd.py
```

Mastery check: derive each target, identify which distribution generates data, mark every detached quantity, explain terminal handling, and predict one update by hand. Compare multiple seeds and held-out metrics before claiming improvement.

The mathematical language used in this course

Read this when an equation's notation gets in the way of the idea. You do not need to memorize every identity before starting. Return to the relevant section when it first appears in a lesson.

1. Indices, sums, and shapes

x_i means entry i of a vector; x_{ij} means row i, column j of a matrix. $\sum_{i=1}^{N} x_i$ means add entries 1 through N. Thus $\frac{1}{N}\sum_i x_i$ is their arithmetic mean. A superscript such as x^2 usually means a power, but V^{π} means "value under policy pi," not exponentiation.

A matrix $X \in \mathbb{R}^{B\times d}$ has B rows and d columns of real numbers. If $W \in \mathbb{R}^{d\times k}$, then XW has shape $[B, k]$:

$$(XW)_{ij} = \sum_{l=1}^{d} X_{il} W_{lj}.$$

The transpose X^T swaps rows and columns. PyTorch's `nn.Linear(d,k)` stores weights as $[k, d]$ and therefore calculates `x @ weight.T + bias` for row batches. Always write shapes beside an unfamiliar equation before implementing it.

2. Functions and derivatives

A function maps inputs to outputs: $f(w) = w^2$ maps 3 to 9. Its derivative $f'(w) = 2w$ describes local sensitivity. At $w = 3$, a small increment h changes the function by approximately $6h$:

$$f(w + h) \approx f(w) + hf'(w).$$

For several variables, a partial derivative changes one variable while holding the others fixed. The gradient stacks all partial derivatives into a vector:

$$\nabla_\theta L = [\partial L/\partial\theta_1, \ldots, \partial L/\partial\theta_d]^T.$$

The chain rule differentiates a composition. If $u = 3w$ and $L = u^2$, then $dL/dw = (dL/du)(du/dw) = 2u \cdot 3 = 18w$. Backpropagation repeatedly applies this rule to a computation graph, adding contributions when a parameter affects the output through several paths.

Gradient descent is $\theta \leftarrow \theta - \eta\nabla L$, with learning rate $\eta > 0$. Locally it moves against increasing loss. In a nonconvex network, this does not guarantee finding the global best parameters.

3. Probability and expectation

A discrete probability distribution has $p_i \geq 0$ and $\sum_i p_i = 1$. $X \sim p$ means sample a random variable from distribution p. The expectation of a function is a probability-weighted average:

$$\mathbb{E}_{X\sim p}[f(X)] = \sum_x p(x) f(x).$$

For rewards 0 and 1 with probabilities 0.2 and 0.8, expected reward is 0.8. A single sample is still either 0 or 1. An expectation describes an average, not what must happen on each attempt. A sample mean estimates an expectation; its reliability depends on the number and dependence of observations.

Conditional probability $p(y \mid x)$ means probability of y given x. An RL policy $\pi(a \mid s)$ is the action distribution conditional on state. The law of total expectation lets us average first over a next state and then over actions; this is the basis of Bellman expectation equations.

4. Logarithms, exponentials, and likelihood

Natural log and exponential are inverse functions: $\log(e^u) = u$. Products become sums: $\log(ab) = \log a + \log b$. A sequence of conditional token probabilities multiplies to a sequence probability, but their logs add. Working in log space avoids underflow from multiplying many tiny numbers.

A likelihood treats observed data as fixed and model parameters as variable. Maximizing its log is equivalent to minimizing negative log-likelihood. For a correct class with model probability p, its loss is $-\log p$. Probability 1 gives loss 0; probability 0.1 gives about 2.303.

Softmax converts logits z_i into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Subtracting the same constant from all logits preserves these probabilities. Subtract the maximum to improve numerical stability. See the worked softmax lesson.

5. Variance, bias, and standardization

Variance measures expected squared deviation from the mean:

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Standard deviation is its square root. Standardization subtracts a mean and divides by a standard deviation, usually adding a tiny denominator constant for numerical stability. Standardized data is not automatically Gaussian.

For an estimator $\hat{q}$ of q, bias is $\mathbb{E}[\hat{q}] - q$. A method can be consistently wrong (bias) or fluctuate greatly (variance). Under the usual squared-error decomposition:

$$\mathbb{E}[(\hat{q} - q)^2] = \mathrm{Var}(\hat{q}) + (\mathbb{E}[\hat{q}] - q)^2.$$

GAE trades reliance on learned value estimates against noisy observed returns. These are tendencies, not universal numeric guarantees for every parameter choice.

6. Entropy, cross-entropy, and KL divergence

For distributions p and q over the same discrete outcomes:

$$H(p) = -\sum_i p_i \log p_i,$$

$$H(p, q) = -\sum_i p_i \log q_i,$$

$$D_{KL}(p \| q) = \sum_i p_i \log \frac{p_i}{q_i} = H(p, q) - H(p).$$

Entropy measures uncertainty within p. Cross-entropy measures the average negative log-probability that q assigns to outcomes drawn from p. KL compares distributions and is nonnegative when well-defined, but is not a symmetric distance. If $q_i = 0$ where $p_i > 0$, KL is infinite.

For a one-hot true label, $H(p) = 0$, so cross-entropy becomes the negative log of the predicted probability of that label. For distillation, the teacher provides a soft target distribution instead.

A sampled log-ratio $\log(p_i/q_i)$ can be negative. Only its expectation under p is constrained to be nonnegative. Continuous-action differential entropy has different properties from discrete entropy and may itself be negative.

7. Optimization versus evaluation

$\arg\max_a f(a)$ returns an input action achieving a maximum. $\max_a f(a)$ returns the maximum value itself. Double DQN deliberately uses one network for the argmax selection and another to evaluate the selected action.

A training loss is the quantity optimized. A task metric is the quantity you care about, such as held-out correctness or episode return. They may disagree: a better reward-model score can reflect a shortcut, and a lower training loss can reflect memorization.

A `stopgrad` or `detach` operation preserves a value in the forward calculation while treating it as constant during differentiation. This distinction is central to target networks, policy-gradient advantages, and frozen teachers.

Practice before the advanced lessons

- Compute softmax for `[0,0]`: answer `[0.5,0.5]`.
- Differentiate $(2w - 1)^2$: answer $4(2w - 1)$.
- Compute expected reward for values `[0,2]` with probabilities `[0.75,0.25]`: answer 0.5.
- Compute entropy of two equally likely outcomes: answer $\log 2 \approx 0.693$ nats.
- For rewards `[0,1]` and discount 0.9, the initial return is 0.9, not 1.

Continue with the theory and code index.

8. Linear systems and curvature for policy iteration and TRPO

A linear system $Ax = b$ asks for a vector whose matrix product equals b. For $A = \text{diag}(2, 4)$ and $b = (6, 8)^T$, the solution is $(3, 2)^T$. Numerically solve the system rather than explicitly forming A^{-1}. Policy evaluation solves $(I - \gamma P_\pi)V = r_\pi$; each row couples a state's value to expected successor values.

A Hessian is the matrix of second partial derivatives, $H_{ij} = \partial^2 f / (\partial\theta_i \partial\theta_j)$. For $f(\theta) = \theta_1^2 + 2\theta_2^2$, it is $\text{diag}(2, 4)$. Taylor's local quadratic approximation is

$$f(\theta + d) \approx f(\theta) + \nabla f(\theta)^T d + \tfrac{1}{2} d^T H d.$$

A symmetric matrix is positive definite if $d^T H d > 0$ for every nonzero d; positive semidefinite permits zero. KL curvature at equal distributions is semidefinite. Adding ηI, with $\eta > 0$, makes the system easier to solve and handles redundant softmax logit shifts. TRPO solves a curvature system to scale steps according to policy change rather than raw parameter distance. Conjugate gradient approximates such a solve using matrix-vector products, without storing the full matrix. The small TRPO lab forms the matrix explicitly.

Practical tricks, with the reason behind each one

Before training

- Overfit 8–32 examples first. Failure usually means the data, loss, mask, or optimizer is wrong; a larger GPU rarely fixes it.
- Print decoded inputs and supervised labels. Check EOS, padding, truncation, template boundaries, and whether any response tokens survive.
- Split by source/document/user before chunking. Overlapping windows from the same document in both splits can create misleading validation scores.
- Record model revision, tokenizer, template, dataset hash, seed, library versions and configuration. A seed alone does not guarantee reproducibility.
- Start with a simple baseline and a held-out evaluation set. For RL include a random policy; for offline RL include behavior cloning.

When loss behaves badly

Symptom	Likely checks
Loss is NaN	Empty supervised masks, invalid log values, excessive LR, FP16 overflow
Loss never changes	Frozen parameters, missing backward/step, detached computation
Loss falls but generation is bad	Leakage, wrong template, tiny/poor data, wrong task metric
CUDA out of memory	Sequence length, activation size, batch size, optimizer state
Training resumes differently	Missing RNG/sampler/optimizer/scheduler state
LoRA has no effect	Target module names, adapter loading, actual trainable parameters
RL reward suddenly collapses	Update size, stale rollouts, incorrect advantage/bootstrap masks
High reward but poor behavior	Reward hacking, verifier bugs, evaluation mismatch

Fine-tuning techniques

Use smaller learning rates when modifying pretrained weights. Warmup avoids large early updates; clipping limits large gradients. Neither replaces correct loss scaling. Select learning rate on validation, not a fixed universal rule.

Try data quality and coverage before more epochs. Monitor forgetting on the original capabilities as well as target performance. Increase LoRA rank only when the adaptation capacity appears limiting; compare under a fixed budget.

Use gradient accumulation for larger effective batches, mixed precision for appropriate hardware, and activation checkpointing for long sequences. A 4-bit base still has floating-point activations and adapter optimizer state. Sequence length can dominate memory even when weights are heavily compressed.

RL techniques

Separate training exploration from evaluation. Evaluate without updates and with a defined action-selection rule. Report multiple training seeds and several evaluation episodes per trained agent.

Keep rewards and observations at manageable scales. Fit normalization using training experience and freeze it during evaluation. Reward normalization and reward clipping change optimization; reward clipping can change the task.

For Gymnasium, step returns `(observation, reward, terminated, truncated, info)`. Reset after either end flag. Bootstrap across a time-limit truncation when the underlying task continues, using the final observation before reset; stop the GAE trace at both kinds of episode boundary. See the official API guide.

For PPO, log entropy, value loss, approximate KL, clip fraction, and return. For DQN/SAC, inspect target values, replay diversity, and update-to-data ratio. Small surrogate loss alone does not prove the policy is improving.

Honest experiments

Change one factor at a time. Compare at equal data/compute budgets and keep a final test set untouched. Report failures and confidence intervals along with averages. For repeated tasks from the same source, resample at the source level.

Vocabulary beside the code

Term	Meaning
Parameter / weight	A learned number inside a model
Hyperparameter	A choice such as learning rate, rank or batch size
Tensor	A multidimensional array with shape and numeric type
Logit	Unnormalized score before softmax
Softmax	Converts scores into probabilities summing to one
Log-probability	Log of a probability; products become sums
Cross-entropy	Negative log-probability assigned to the target
Gradient	Derivatives of an objective with respect to parameters
Autograd	Automatic differentiation using the chain rule
Backpropagation	Reverse traversal that computes those derivatives
Optimizer	A rule for updating parameters from gradients
AdamW	Adaptive gradient optimizer with decoupled weight decay
Epoch / step	Pass through data / a training update (check local meaning)
Generalization	Performance on unseen data from the intended task
Leakage	Evaluation information accidentally entering training/selection
Embedding	Learned vector representing a token or other discrete item
Attention	Content-dependent weighted aggregation of representations
Causal mask	Prevents attending to future positions
Pretraining	Initial broad learning, often next-token prediction
SFT	Supervised fine-tuning on desired examples
PEFT / LoRA / QLoRA	Parameter-efficient tuning / low-rank updates / quantized-base LoRA
Policy pi(a given s)	Distribution of actions given state/observation
State / observation	Environment's underlying condition / what the agent sees
Reward r	Immediate feedback after an action
Return G	Sum of future rewards, usually discounted
Discount gamma	Weight on future rewards; often between zero and one
V(s)	Expected return from a state under a policy
Q(s,a)	Expected return after a state/action pair under a policy
Advantage A(s,a)	Q(s,a)-V(s): better/worse than the state baseline
TD error	Sampled reward plus bootstrap value minus current estimate
Bootstrap	Use a learned future estimate in a training target
On-policy	Learn from the policy currently being improved
Off-policy	Learn a target policy using behavior from another policy

Term	Meaning
Replay buffer	Stored transitions reused for learning
Actor / critic	Policy model / value estimator
Entropy	Uncertainty of an action distribution
KL divergence	Directional measure of difference between distributions
GAE	Smoothed sum of temporal-difference errors estimating advantage
RLHF / RLAIF	Reinforcement learning using human / AI-derived feedback
DPO	Direct preference optimization from chosen/rejected examples
GRPO	Group-relative policy optimization using within-prompt rewards
Quantization	Represent numbers using fewer bits and scaling/codebooks
Distillation	Train a student to approximate a teacher's behavior
Ablation	Remove/change one component to test its contribution

Notation: B=batch size, T=sequence length, V=vocabulary size, d=hidden width, r=LoRA rank, alpha=adapter scale (or an unrelated SAC temperature), beta=reference regularization scale. Symbols can mean different things in different algorithms; read each local definition.

PART 1

Foundations

1.1

Python, vectors, softmax, and gradient descent

Companion code: 01_python_and_math.py. Read this explanation before running the code.

What you are learning and why

A model is a parameterized calculation. Training chooses its parameters so predictions match examples. Before a neural network, understand a model with only two learned numbers: $\hat{y} = wx + b$. Here x is an input, w is a slope, and b is an offset. The code learns the hidden rule $y = 2x + 3$ from five pairs.

Python lists store the examples, a `for` loop repeats updates, and a function packages a reusable calculation. `zip(a, b)` pairs corresponding list entries; `sum(...)` adds them. A list comprehension `[f(x) for x in xs]` applies a calculation to every item and returns the results as a list.

Vectors and dot products

A scalar is one number; a vector is an ordered list. For vectors of length d:

$$a \cdot b = \sum_{j=1}^{d} a_j b_j.$$

For `[1, 2]` and `[3, 4]`, this is $1(3) + 2(4) = 11$. In a model, the dot product combines input features with their learned weights. A matrix stores several vectors as rows or columns, allowing many dot products in one operation.

Softmax: formula, code, and numerical stability

Logits are arbitrary real scores. To obtain a probability distribution:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

Exponential makes each term positive, and dividing by their sum makes the probabilities add to one. Higher logits produce higher probabilities. These probabilities are model outputs, not automatically calibrated confidence.

A direct implementation is:

```
exponentials = [math.exp(score) for score in logits]
denominator = sum(exponentials)
probabilities = [value / denominator for value in exponentials]
```

The actual `softmax` function subtracts $m = \max_j z_j$ first:

$$\frac{e^{z_i-m}}{\sum_j e^{z_j-m}} = \frac{e^{-m}e^{z_i}}{e^{-m}\sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

The common factor cancels, so this is the same formula. But the largest exponential is now $e^0 = 1$, avoiding overflow from numbers such as e^{1000}. For `[1,2,3]`, exponentiate `[-2,-1,0]` to get approximately `[0.1353,0.3679,1]`. Divide by 1.5032 to get `[0.0900,0.2447,0.6652]`. In the original short function, `shifted` contains these **exponentials**, not just the shifted scores. The final list comprehension normalizes them.

For a correct class k, negative log-likelihood is $-\log p_k$. If the correct class has probability 0.6652, the loss is about 0.4076. Giving it probability 0.01 produces loss about 4.6052. Minimizing this loss encourages more probability on the correct class.

Derivatives: which direction reduces error?

Define error $e_i = wx_i + b - y_i$ and mean squared error:

$$L(w,b) = \frac{1}{N}\sum_i e_i^2.$$

A derivative measures local change. The chain rule gives:

$$\frac{\partial L}{\partial w} = \frac{2}{N}\sum_i e_i x_i, \qquad \frac{\partial L}{\partial b} = \frac{2}{N}\sum_i e_i.$$

Why? The derivative of e_i^2 is $2e_i$ times the derivative of e_i; $\partial e_i/\partial w = x_i$ and $\partial e_i/\partial b = 1$.

For the code's five inputs `[-2,-1,0,1,2]`, targets are `[-1,1,3,5,7]`. At $w = b = 0$, errors are `[1,-1,-3,-5,-7]`. Therefore $\partial L/\partial w = -8$ and $\partial L/\partial b = -6$.

Gradient descent subtracts the gradient:

$$w \leftarrow w - \eta\frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta\frac{\partial L}{\partial b}.$$

With learning rate $\eta = 0.1$, the first updated weights are $w = 0.8$, $b = 0.6$. The negative gradient means increasing these parameters initially reduces loss. This follows from the local approximation $L(\theta + \Delta) \approx L(\theta) + \nabla L^T\Delta$: choosing $\Delta = -\eta\nabla L$ predicts a decrease of $\eta\|\nabla L\|^2$. For too large a step, the approximation can fail and loss may grow.

Connect to the script

Code	Meaning
`errors = [...]`	Calculate $\hat{y}_i - y_i$
`loss = ... / len(xs)`	Average squared error over examples
`dw`, `db`	The two derivatives derived above
`weight -= learning_rate * dw`	Gradient descent for the slope
`bias -= learning_rate * db`	Gradient descent for the offset

`random.seed(7)` has no effect on this deterministic dataset; it is a reproducibility habit for later randomized examples. The code reports loss before its update and weights after its update, so they refer to adjacent points in the training loop.

Run, experiment, and check

Run `python 01_foundations/01_python_and_math.py`. Expect the dot product 11, softmax probabilities summing to one, and weights approaching 2 and 3. Try $y = -3x + 1$; the learned weights should approach -3 and 1. Explain why adding 100 to every logit leaves softmax unchanged. Check the cancellation above. A frequent error is adding the gradient instead of subtracting it: that performs ascent on the loss.

Next: tensors and automatic differentiation.

Companion code: `01_python_and_math.py`

01_foundations/01_python_and_math.py

```python
"""Python and the mathematics hiding inside training.
Run: python3 01_foundations/01_python_and_math.py
Requires: Python only. Read each comment, then change one number and rerun.

A scalar is one number; a vector is a list of numbers; a matrix is a table.
A parameter is a number learned from data. A hyperparameter (e.g. learning
rate) is chosen by us. Loss measures how wrong predictions are.
Exercise: change the target rule to y = -3*x + 1 and recover its parameters.
"""
import math
import random

# A function packages a reusable calculation. zip pairs corresponding elements.
def dot(a, b):
    assert len(a) == len(b), 'Dot products require matching dimensions.'
    return sum(x * y for x, y in zip(a, b))


def softmax(logits):
    # Subtracting the maximum prevents exp(large_number) from overflowing.
    shifted = [math.exp(x - max(logits)) for x in logits]
    return [x / sum(shifted) for x in shifted]


def main():
    random.seed(7)  # Reproducibility: the same pseudorandom sequence each run.
    print('dot product:', dot([1, 2], [3, 4]))
    probabilities = softmax([1, 2, 3])
    print('probabilities:', probabilities, 'sum:', sum(probabilities))
    # Cross-entropy for a single correct class = -log(probability of that class).
    print('loss when class 2 is correct:', -math.log(probabilities[2]))
    xs = [-2, -1, 0, 1, 2]
    ys = [2*x + 3 for x in xs]
    weight, bias, learning_rate = 0.0, 0.0, 0.1
    for step in range(100):
        errors = [weight*x + bias - y for x, y in zip(xs, ys)]
        loss = sum(e*e for e in errors) / len(xs)
        # Derivatives tell us how loss changes when each parameter changes.
        dw = 2 * sum(e*x for e, x in zip(errors, xs)) / len(xs)
        db = 2 * sum(errors) / len(xs)
        weight -= learning_rate * dw  # Descend: move opposite the gradient.
        bias -= learning_rate * db
        if step % 25 == 0:
            print(f'step={step:3} loss={loss:.6f} w={weight:.3f} b={bias:.3f}')
    assert abs(weight - 2) < 0.01 and abs(bias - 3) < 0.01

if __name__ == '__main__':
    main()
```

1.2

Tensors, automatic differentiation, and checking gradients

Companion code: 02_tensors_autograd.py. Read this explanation before running the code.

The question this example answers

How does PyTorch know which weight changes reduce loss, and how can we check its answer? We compute the same derivative in three ways: by hand, with `backward()`, and with small numerical perturbations. Then we examine a shape mistake that changes the problem without producing an error.

Read gradient descent first. A tensor is an array with shape, numeric type, and device. `requires_grad=True` enables derivative tracking for a floating-point tensor. Tracking is different from updating.

Inputs and matrix multiplication

The arrays are:

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad w = \begin{bmatrix} 0.2 \\ 0.3 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Rows of X are examples. Columns are features. There are two weights and no bias in this model. Matrix multiplication gives:

$$\hat{y} = Xw = \begin{bmatrix} 1(0.2) + 2(0.3) \\ 3(0.2) + 4(0.3) \end{bmatrix} = \begin{bmatrix} 0.8 \\ 1.8 \end{bmatrix}.$$

The shape rule is `[2,2] @ [2,1] -> [2,1]`. In Python, `@` is matrix multiplication; `*` would be elementwise multiplication with broadcasting.

Loss and an explicit derivative

$$L = \frac{1}{2}[(0.8 - 1)^2 + (1.8 - 2)^2] = 0.04.$$

For N examples, the gradient of $L = \|Xw - y\|^2/N$ is:

$$\nabla_w L = \frac{2}{N} X^T (Xw - y).$$

For this batch, $Xw - y = [-0.2, -0.2]^T$, so:

$$\frac{\partial L}{\partial w_1} = (-0.2)(1) + (-0.2)(3) = -0.8,$$

$$\frac{\partial L}{\partial w_2} = (-0.2)(2) + (-0.2)(4) = -1.2.$$

The graph is `w -> predictions -> errors -> squared errors -> mean`. Autograd traverses it backwards and multiplies local derivatives using the chain rule. `loss.backward()` accumulates the resulting derivatives in `w.grad`, approximately `[[-0.8],[-1.2]]`. No optimizer is present, so the weights remain unchanged after this call.

`analytical = w.grad.clone()` saves an independent copy. The name distinguishes this chain-rule derivative from a finite-difference estimate; PyTorch is doing automatic differentiation, not symbolic algebra or finite differences.

Finite differences: measuring a slope without autograd

For a smooth one-variable function:

$$f'(u) \approx \frac{f(u+\epsilon) - f(u-\epsilon)}{2\epsilon}.$$

Taylor expansion gives $f(u \pm \epsilon) = f(u) \pm \epsilon f'(u) + \epsilon^2 f''(u)/2 \pm \epsilon^3 f^{(3)}(u)/6 + \cdots$. Subtracting and dividing cancels the even terms, leaving $f'(u) + O(\epsilon^2)$. Very small ϵ can amplify floating-point roundoff; smaller is not always more accurate.

The script holds $w_2 = 0.3$ fixed and perturbs only w_1 with $\epsilon = 0.001$:

w_1	Predictions	Mean squared error
0.201	[0.801, 1.803]	0.039205
0.199	[0.799, 1.797]	0.040805

The estimated derivative is $(0.039205 - 0.040805)/0.002 = -0.8$. Restoring the original weight is essential: this is a diagnostic, not a training update.

Code	Purpose
`with torch.no_grad()`	Avoid recording the diagnostic edits/calculations
`w[0,0].item()`	Read one weight as a Python number
`plus`, `minus`	Evaluate the loss on either side of the original weight
`(plus-minus)/(2*eps)`	Central finite-difference estimate
`allclose(..., atol=1e-3)`	Compare with absolute tolerance plus default relative tolerance

Why broadcasting can silently break a loss

Correct: predictions and targets both have shape `[2,1]`. Each prediction is compared with its matching target, giving errors `[[-0.2],[-0.2]]`.

`target.squeeze(1)` removes its length-one second dimension, producing shape `[2]`. PyTorch aligns dimensions from the right. `[2,1]` and `[2]` broadcast to `[2,2]`, producing all pairwise differences:

$$\begin{bmatrix} 0.8-1 & 0.8-2 \\ 1.8-1 & 1.8-2 \end{bmatrix} = \begin{bmatrix} -0.2 & -1.2 \\ 0.8 & -0.2 \end{bmatrix}.$$

The wrong mean squared error would be $(0.04 + 1.44 + 0.64 + 0.04)/4 = 0.54$, not 0.04. Always inspect shapes before computing a regression loss.

Run and check understanding

Run `python 01_foundations/02_tensors_autograd.py`. Expect numerical derivative near -0.8, automatic gradient near `[[-0.8],[-1.2]]`, and shape outputs `[2,2]` and `[2,1]`. Independently perturb the second weight: its estimate should be near -1.2. Try `eps=0.1` and `eps=1e-8`; this quadratic has no central-difference truncation error in exact arithmetic, but float32 roundoff can spoil the tiny step.

`manual_seed` has no effect here because the tensors are explicit constants. `set_num_threads` affects execution scheduling, not the mathematics. Finally, `if __name__ == '__main__': main()` runs the demonstration only when the file is executed directly.

Reference: PyTorch automatic differentiation.

Companion code: `02_tensors_autograd.py`

01_foundations/02_tensors_autograd.py

```python
"""Tensor shapes, broadcasting, derivatives, and gradient checks.
Requires: torch. A batch has shape [B, features]; a weight matrix [features, out].
Autograd records operations and applies the chain rule in reverse.
Exercise: deliberately omit zero_grad in 03_neural_network_training.py and observe
accumulation.
"""
import torch


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    x = torch.tensor([[1., 2.], [3., 4.]])  # Shape [2 examples, 2 features].
    w = torch.tensor([[0.2], [0.3]], requires_grad=True)
    target = torch.tensor([[1.], [2.]])
    loss = ((x @ w - target)**2).mean()
    loss.backward()  # Populates w.grad; it does NOT update w.
    analytical = w.grad.clone()
    # Finite differences independently check a derivative on this tiny problem.
    eps = 1e-3
    with torch.no_grad():
        original = w[0, 0].item()
        w[0, 0] = original + eps
        plus = ((x @ w - target)**2).mean()
        w[0, 0] = original - eps
        minus = ((x @ w - target)**2).mean()
        w[0, 0] = original
    numerical = (plus - minus) / (2*eps)
    print('gradient:', analytical.tolist(), 'finite difference:', numerical.item())
    assert torch.allclose(analytical[0, 0], numerical, atol=1e-3)
    # Broadcasting [2,1] against [2] produces [2,2], often a SILENT loss bug.
    print('wrong subtraction shape:', (x @ w - target.squeeze(1)).shape)
    print('correct subtraction shape:', (x @ w - target).shape)

if __name__ == '__main__':
    main()
```

1.3

Neural networks and the complete training loop

Companion code: 03_neural_network_training.py. Read this explanation before running the code.

Why use a neural network?

A linear classifier makes a straight decision boundary. The script labels an example positive when $x_1 x_2 > 0$: both coordinates have the same sign. Positive examples occupy two opposite quadrants. One straight line cannot separate these from the other two quadrants, so we need a nonlinear model.

Prerequisite: tensors and gradients. Our network is:

$$h = \mathrm{ReLU}(W_1 x + b_1), \qquad z = W_2 h + b_2.$$

h has 32 hidden features; z has two class scores. ReLU is $\max(0, u)$ elementwise. It creates different linear behavior in different regions. Without ReLU, composing the layers gives $z = W_2 W_1 x + W_2 b_1 + b_2$, still just one affine map.

From scores to a trainable objective

For class k, softmax gives $p_k = e^{z_k} / \sum_j e^{z_j}$. If the correct class is y, the per-example loss is:

$$\ell = -\log p_y = -z_y + \log \sum_j e^{z_j}.$$

Differentiating gives a useful identity:

$$\frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y].$$

For probabilities `[0.7,0.3]` and correct class 1, gradients on the logits are `[0.7,-0.7]`. Gradient descent pushes the wrong score down and correct score up. The chain rule then propagates through the output layer, ReLU, and first layer. This is why decreasing cross-entropy encourages correct classification.

`cross_entropy(model(inputs), labels)` accepts raw logits and integer class indices. It combines log-softmax and negative log-likelihood internally. Passing softmax probabilities instead applies the wrong transformation.

Minibatches, epochs, and AdamW

The empirical objective averages losses across training examples:

$$L(\theta) = \frac{1}{N} \sum_i \ell_i(\theta).$$

A shuffled minibatch estimates its gradient using a smaller average. An epoch visits all training examples once. In this file, 400 examples with batch size 32 produce 13 batches, the last one containing 16 examples.

Adam keeps moving averages of gradients and their squares:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

Bias-corrected versions divide by $1-\beta_1^t$ and $1-\beta_2^t$. The adaptive update scales each coordinate roughly by $\hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$. AdamW additionally shrinks parameters by a factor approximately $1 - \eta\lambda$ through decoupled weight decay. This discourages unnecessarily large weights; it does not guarantee generalization.

Global gradient clipping replaces g by $g \min(1, c/\|g\|_2)$, with threshold $c = 1$ here. It limits an unusually large gradient before the optimizer uses it.

Map the loop to its purpose

Code	Meaning
`optimizer.zero_grad(...)`	Remove gradients left by the previous minibatch
`loss.backward()`	Compute this minibatch's parameter derivatives
`clip_grad_norm_`	Limit the global gradient norm
`optimizer.step()`	Apply AdamW's update
`model.train()`	Enable training behavior in layers such as dropout
`model.eval()` and `no_grad()`	Select evaluation behavior and disable graph recording
`copy.deepcopy(model.state_dict())`	Preserve the actual best weights, not references to changing tensors

Why separate training, validation, and test data?

The first 400 examples update weights. The next 100 select the stopping point. The final 100 are evaluated once after restoring the best validation checkpoint. This separation reduces optimistic estimates caused by choosing settings on the same examples used to report final quality.

If validation loss fails to improve for eight epochs, training stops. This is patience-based early stopping. Because validation influenced model selection, its score is not an untouched final test result.

Accuracy is the fraction whose largest logit selects the correct class. A model can improve confidence and lower cross-entropy without changing any argmax predictions, so accuracy and loss measure different things.

Run and investigate

Run `python 01_foundations/03_neural_network_training.py`. Expect falling validation loss and high test accuracy on this easy synthetic task; exact numbers are not universal guarantees. Remove ReLU and explain the remaining classification errors. Reduce training examples and increase width: compare training and validation behavior. Do not repeatedly use the final test split to decide which modification to keep.

Next: tokens and attention.

Companion code: `03_neural_network_training.py`

01_foundations/03_neural_network_training.py

```python
"""A complete supervised training loop with unseen validation data.
An epoch visits the training set once. A minibatch is part of that set.
ReLU supplies nonlinearity; without it, stacked linear layers remain linear.
Train data updates weights; validation selects settings; test data is held
until the final comparison. Never tune repeatedly on the test set.
Exercise: increase width to 256 and shrink training data to see overfitting.
"""
import copy
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    x = torch.randn(600, 2)
    y = ((x[:, 0] * x[:, 1]) > 0).long()  # XOR-like classification.
    train = DataLoader(TensorDataset(x[:400], y[:400]), batch_size=32, shuffle=True)
    model = nn.Sequential(nn.Linear(2, 32), nn.ReLU(), nn.Linear(32, 2))
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.01, weight_decay=0.01)
    best_loss, best_state, stale = float('inf'), None, 0
    for epoch in range(60):
        model.train()  # Activates dropout / training behavior if present.
        for inputs, labels in train:
            optimizer.zero_grad(set_to_none=True)
            # CrossEntropyLoss expects raw logits, NOT already-softmaxed values.
            loss = nn.functional.cross_entropy(model(inputs), labels)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
        model.eval()  # eval() does not disable gradients; no_grad does.
        with torch.no_grad():
            val = nn.functional.cross_entropy(model(x[400:500]), y[400:500]).item()
        if val < best_loss:
            best_loss, best_state, stale = val, copy.deepcopy(model.state_dict()), 0
        else:
            stale += 1
        if epoch % 10 == 0:
            print(f'epoch={epoch} validation_loss={val:.4f}')
        if stale >= 8:  # Early stopping avoids training past the best validation point.
            break
    model.load_state_dict(best_state)
    with torch.no_grad():
        accuracy = (model(x[500:]).argmax(-1) == y[500:]).float().mean().item()
    print(f'held-out test accuracy: {accuracy:.1%}')

if __name__ == '__main__':
    main()
```

1.4

Tokens, embeddings, and causal attention

Companion code: 04_tokens_attention.py. Read this explanation before running the code.

Why attention is useful

To predict the next token, a model needs context. In “the cat sat,” a prediction may depend on several earlier tokens. Attention lets each position choose a weighted mixture of available contextual representations.

Read softmax first. The script is a mechanics experiment, not a trained language model.

Text becomes discrete IDs, then continuous vectors

For `cat sat`, the sorted character vocabulary is `[' ', 'a', 'c', 's', 't']`. The encoded IDs are `[2,1,4,0,3,1,4]`. IDs are lookup addresses, not numerical measurements: ID 4 is not “twice as meaningful” as ID 2.

An embedding table $E \in \mathbb{R}^{V \times d}$ contains one learned vector per token. Looking up token i returns row E_i. Here $V = 5$ and $d = 8$. During training, gradients update the rows used by examples; the coordinates are learned representations, not predefined linguistic labels.

For next-token prediction, inputs are IDs except the last, while labels are IDs except the first. Position t uses context through t to predict $t + 1$.

For example, use the character tokens of `cat sat`, including its space:

```
ids = [2, 1, 4, 0, 3, 1, 4]  # c a t SPACE s a t
inputs = ids[:-1]             # [2, 1, 4, 0, 3, 1] = "cat sa"
labels = ids[1:]              # [1, 4, 0, 3, 1, 4] = "at sat"
```

Positions start at zero. Each output predicts the token immediately after its input position, using all allowed context:

Position t	Input token	Allowed context	Correct next token
0	`c`	`c`	`a`
1	`a`	`ca`	`t`
2	`t`	`cat`	space
3	space	`cat`	`s`
4	`s`	`cat s`	`a`
5	`a`	`cat sa`	`t`

At position 4, the model predicts `a` from `cat s`. The causal mask prevents it from reading the `a` at position 5, even though that token is in the input. The model can compute all six predictions in one forward pass with this mask.

For original IDs $z_0, \ldots, z_{N-1}$, the aligned input and target are $x_t = z_t$ and $y_t = z_{t+1}$ for $t = 0, \ldots, N-2$. The training objective is:

$$\mathcal{L} = -\frac{1}{N-1} \sum_{t=0}^{N-2} \log p_\theta(z_{t+1} \mid z_0, \ldots, z_t).$$

The last token is excluded from inputs because this sequence provides no following token as its target. The first token is excluded from labels because there is no preceding input position to predict it. Seven tokens therefore provide six input-target pairs. The companion script demonstrates the attention mechanics; it does not train this predictor.

Derive scaled dot-product attention

Let $X \in \mathbb{R}^{T \times d}$ contain one representation per position. A full attention head learns projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

A query asks what a position is looking for, a key describes what a position can match, and a value is the information it contributes. Similarity scores are:

$$S_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}.$$

When query and key components are roughly independent, have mean zero, and have unit variance, a dot product of d_k terms has variance roughly d_k. Dividing by $\sqrt{d_k}$ keeps score scale more stable across dimensions and helps avoid overly saturated softmax.

To see why, let q and k be one query and one key, each with d_k coordinates. Their dot product is a sum of coordinate products:

$$D = q \cdot k = \sum_{r=1}^{d_k} q_r k_r.$$

For the following idealized calculation, assume all coordinates are mutually independent, with $\mathbb{E}[q_r] = \mathbb{E}[k_r] = 0$ and $\mathrm{Var}(q_r) = \mathrm{Var}(k_r) = 1$. Unit variance means expected squared deviation from the mean is 1; it does not mean each value is 1. Since the means are zero, $\mathbb{E}[q_r^2] = \mathbb{E}[k_r^2] = 1$. Each product then has mean zero and variance one:

$$\mathbb{E}[q_r k_r] = \mathbb{E}[q_r]\mathbb{E}[k_r] = 0,$$

$$\mathrm{Var}(q_r k_r) = \mathbb{E}[q_r^2 k_r^2] - (\mathbb{E}[q_r k_r])^2 = \mathbb{E}[q_r^2]\mathbb{E}[k_r^2] = 1.$$

Independent terms have zero covariance, so their variances add:

$$\mathrm{Var}(D) = \sum_{r=1}^{d_k} \mathrm{Var}(q_r k_r) = d_k.$$

For a concrete random example, imagine every coordinate is independently chosen to be -1 or $+1$, each with probability $1/2$. Each coordinate and each product has mean zero and variance one. With $d_k = 4$, one draw could be:

$$q = [1, -1, 1, 1], \qquad k = [1, 1, -1, 1],$$

$$D = 1 - 1 - 1 + 1 = 0.$$

Other draws give dot products of $-4, -2, 0, 2$, or 4, with probabilities $1/16, 4/16, 6/16, 4/16, 1/16$, respectively. Across these draws the mean is zero and the variance is:

$$\mathbb{E}[D^2] = \frac{16 + 4(4) + 6(0) + 4(4) + 16}{16} = 4 = d_k.$$

Variance describes the spread across possible draws, not the value of one particular dot product. Its square root, the standard deviation, is $\sqrt{d_k}$. Scaling a random quantity by c scales its variance by c^2:

$$\operatorname{Var}\left(\frac{D}{\sqrt{d_k}}\right) = \frac{1}{d_k}\operatorname{Var}(D) = 1.$$

Key dimension d_k	Raw dot-product standard deviation	Divisor	Scaled standard deviation
4	2	2	1
64	8	8	1
256	16	16	1

Larger score differences make softmax more extreme. For example, scores `[0, 8]` give probabilities approximately `[0.0003, 0.9997]`. Dividing these scores by 8 gives `[0, 1]`, whose probabilities are approximately `[0.269, 0.731]`. Scaling helps prevent dimension alone from driving softmax toward near-zero and near-one probabilities, where its gradients can be small.

This is a motivation, not a guarantee about learned queries and keys. Their coordinates may be correlated or have different means and variances. In particular, the companion script uses $Q = K = X$, so a self-dot-product does not satisfy the independent-query-and-key assumption above.

Normalize each row over source positions, then form a weighted sum:

$$P_{ij} = \frac{e^{S_{ij}}}{\sum_l e^{S_{il}}}, \qquad O_i = \sum_j P_{ij} V_j.$$

A two-position worked example

For easy arithmetic, take one-dimensional queries, keys, and values $Q = K = V = [1, 2]^T$. Then $S = [[1, 2], [2, 4]]$. At position 0, causal attention allows only position 0, so weights are `[1,0]` and output is 1. At position 1, softmax of `[2,4]` is approximately `[0.1192,0.8808]`, giving output $0.1192(1) + 0.8808(2) = 1.8808$.

Each row's weights sum to one. Different query positions can produce different mixtures of the same value vectors.

Why the future must be masked

For causal language modeling, position i may see only $j \leq i$:

$$S_{ij} \leftarrow -\infty \quad \text{when } j > i.$$

Because $e^{-\infty} = 0$, future tokens receive zero probability. Without this mask, the model could read the token it is supposed to predict during training. It could appear successful while failing at generation, where that token does not yet exist.

Code	Formula/meaning
`x @ x.T / sqrt(dimension)`	Scores with the simplifying choice $Q = K = V = X$
`triu(diagonal=1)`	True above the main diagonal: future positions
`masked_fill(..., -inf)`	Eliminate forbidden positions before normalization
`.softmax(-1)`	Normalize over keys for each query row
`probabilities @ x`	Weighted value mixtures

The script's embeddings are randomly initialized and never trained. It also omits learned Q/K/V projections, multiple heads, and position embeddings. Therefore its printed weights demonstrate valid shapes and masking, not meaningful learned language relationships.

Run and check

Run `python 01_foundations/04_tokens_attention.py`. Expect a `[6,8]` output and a lower-triangular `[6,6]` attention matrix with row sums one. Change only the final input vector: earlier outputs must not change. Remove the mask and repeat: earlier outputs can now depend on the changed future token.

Next: a complete tiny transformer. Reference: Attention Is All You Need.

Companion code: `04_tokens_attention.py`

01_foundations/04_tokens_attention.py

```python
"""Token IDs, embeddings, causal attention, and next-token labels.
Real tokenizers usually use subword units; characters make the mechanics clear.
Attention(Q,K,V) = softmax(Q K^T / sqrt(d)) V. A causal mask blocks the future.
Exercise: remove the mask and explain why the resulting training task leaks.
"""
import math
import torch
from torch import nn


def causal_attention(x):
    # Here Q=K=V=x for clarity; a transformer learns separate projections.
    length, dimension = x.shape
    scores = x @ x.T / math.sqrt(dimension)
    future = torch.ones(length, length, dtype=torch.bool).triu(diagonal=1)
    probabilities = scores.masked_fill(future, float('-inf')).softmax(-1)
    return probabilities @ x, probabilities


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    text = 'cat sat'
    vocabulary = sorted(set(text))
    token_to_id = {character: index for index, character in enumerate(vocabulary)}
    ids = torch.tensor([token_to_id[c] for c in text])
    embedding = nn.Embedding(len(vocabulary), 8)
    inputs, labels = ids[:-1], ids[1:]  # Each position predicts the next character.
    output, attention = causal_attention(embedding(inputs))
    print('vocabulary:', token_to_id)
    print('input IDs:', inputs.tolist(), 'next-token IDs:', labels.tolist())
    print('attention weights:', attention.detach().round(decimals=2))
    assert torch.all(attention.triu(1) == 0)
    assert torch.allclose(attention.sum(-1), torch.ones(len(inputs)))
    print('output shape:', output.shape)
    # Full transformers also use positional information, multiple heads,
    # residual additions, normalization and feed-forward blocks: see
05_tiny_transformer.py.

if __name__ == '__main__':
    main()
```

1.5

A tiny transformer language model from start to sampling

Companion code: 05_tiny_transformer.py. Read this explanation before running the code.

The learning objective

This file trains a small character language model from random initialization. It learns one repeated sentence, so it demonstrates pretraining mechanics and memorization, not broad language understanding. Read attention and training loops first.

A causal model factorizes a sequence probability:

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

Taking negative logs turns products into sums. The training objective is the average next-token negative log-probability. We can predict every next token in a window in parallel because the causal mask prevents future leakage.

What each part of the architecture contributes

Input token x_t becomes $h_t^{(0)} = E[x_t] + P[t]$. E stores token embeddings; P stores learned position embeddings. Adding positions lets the model represent order: the same token at two locations need not have the same input.

The block uses four attention heads. With width 32, each head uses width 8. Independent projected queries/keys/values allow different heads to aggregate different relationships. Head outputs are concatenated and projected back to width 32.

The default encoder layer here uses residual additions and post-layer normalization, with dropout set to zero:

$$u = \mathrm{LayerNorm}(h + \mathrm{MHA}(h)), \qquad h' = \mathrm{LayerNorm}(u + \mathrm{FFN}(u)).$$

The feed-forward network is $W_2\,\mathrm{ReLU}(W_1 u + b_1) + b_2$, with intermediate width 64. Residual paths make it easier to propagate information and gradients without forcing every block to relearn an identity mapping. Layer normalization standardizes features within a token representation and then applies learned scale and offset; it does not normalize a whole batch.

Although the PyTorch class is named `TransformerEncoder`, supplying a causal mask makes this use causal self-attention. There is no cross-attention or separate encoder-decoder architecture in this example.

Shapes and loss

Stage	Shape
Input IDs	`[B,T]`
Embeddings and block output	`[B,T,32]`
Vocabulary logits	`[B,T,V]`
Flattened logits for cross-entropy	`[B*T,V]`
Flattened next-token labels	`[B*T]`

A sampled batch contains 25 tokens per example. The first 24 are inputs; the last 24, offset by one, are targets. The output projection produces a score for every vocabulary token at every input position.

$$L = -\frac{1}{BT}\sum_{b,t}\log p_\theta(x_{b,t+1} \mid x_{b,\le t}).$$

Each of 120 updates uses 16 randomly selected windows. This code has no held-out split, so reported loss measures fitting the repeated training sentence only. If evaluating a larger corpus, split source documents before making overlapping windows; otherwise train and validation can contain nearly identical contexts.

Why generation is sequential

Training knows all target tokens, but generation must create them. Start with `a`, obtain the last position's logits, sample one token, append it, and repeat. Temperature $\tau > 0$ changes the distribution:

$$p_i(\tau) = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}.$$

For two logits differing by 2, their probability odds are $e^{2/\tau}$. At $\tau = 1$ the odds are about 7.39; at $\tau = 0.5$ they are about 54.6. Lower temperature sharpens preferences, but cannot make an inaccurate model correct. `multinomial` samples rather than always taking the largest score.

Only the last 24 tokens are passed back to the model. Their learned positions restart at zero for each window; this is a simple sliding-context demonstration, not a KV-cached long-context implementation. `eval()` and `no_grad()` prepare inference behavior and prevent unnecessary backward graphs.

Run and experiment

Run `python 01_foundations/05_tiny_transformer.py`. Loss should fall and sampled text should resemble the repeated sentence. Try a new starting character and a different temperature. Add a held-out corpus and compare loss: memorizing one sentence does not establish generalization. Check causality by changing a future input ID and confirming earlier logits stay unchanged in evaluation mode.

Reference: original transformer architecture. Next: transfer learning.

Companion code: `05_tiny_transformer.py`

01_foundations/05_tiny_transformer.py

```python
"""Train and sample a tiny causal transformer entirely offline.
This is pretraining from random initialization, NOT fine-tuning a pretrained LLM.
A single repeated sentence demonstrates mechanics, not language competence.
Exercise: use a richer corpus, split it BEFORE making overlapping windows,
and report validation cross-entropy instead of memorization loss.
"""
import torch
from torch import nn


class TinyLM(nn.Module):
    def __init__(self, vocab_size, width=32, context=24):
        super().__init__()
        self.context = context
        self.tokens = nn.Embedding(vocab_size, width)
        self.positions = nn.Embedding(context, width)
        block = nn.TransformerEncoderLayer(width, 4, 64, dropout=0.0, batch_first=True)
        self.blocks = nn.TransformerEncoder(block, num_layers=1,
enable_nested_tensor=False)
        self.head = nn.Linear(width, vocab_size)

    def forward(self, ids):
        length = ids.shape[1]
        x = self.tokens(ids) + self.positions(torch.arange(length, device=ids.device))
        mask = torch.ones(length, length, device=ids.device, dtype=torch.bool).triu(1)
        return self.head(self.blocks(x, mask=mask))


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    corpus = 'a cat sits on a mat. ' * 40
    chars = sorted(set(corpus))
    encoded = torch.tensor([chars.index(c) for c in corpus])
    model = TinyLM(len(chars))
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.005)
    for step in range(120):
        starts = torch.randint(len(encoded)-model.context-1, (16,))
        batch = torch.stack([encoded[s:s+model.context+1] for s in starts])
        logits = model(batch[:, :-1])
        loss = nn.functional.cross_entropy(logits.reshape(-1, len(chars)), batch[:,
1:].reshape(-1))
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        if step % 40 == 0:
            print('step:', step, 'training loss:', round(loss.item(), 3))
    model.eval()
    ids = torch.tensor([[chars.index('a')]])
    with torch.no_grad():
        for _ in range(60):
            logits = model(ids[:, -model.context:])[:, -1]
            # Temperature <1 sharpens probabilities; >1 increases randomness.
            next_id = torch.multinomial((logits / 0.8).softmax(-1), 1)
            ids = torch.cat([ids, next_id], dim=1)
    print('sample:', ''.join(chars[i] for i in ids[0].tolist()))
```

```
56
57 if __name__ == '__main__':
58     main()
```

PART 2

Fine-tuning

2.1

Transfer learning: reuse features, then adapt them

Companion code: 01_transfer_learning.py. Read this explanation before running the code.

What changes when we fine-tune?

Training from scratch initializes parameters without task-specific knowledge. Transfer learning begins with parameters learned on a source task, then adapts them to a target task. The hope is that useful source features reduce target data needs. Transfer can also hurt if the source representation is unsuitable.

The code provides a small, fully local experiment. A backbone maps two input features to 16 hidden features; a head maps those features to two logits.

$$h = f_\phi(x), \qquad z = g_\psi(h), \qquad L(\phi, \psi) = \frac{1}{N} \sum_i -\log p_{\phi,\psi}(y_i \mid x_i).$$

ϕ names backbone parameters, and ψ names head parameters. Read neural-network training for cross-entropy and backpropagation.

Source and target tasks in the script

Source labels are $y_s = \mathbf{1}[x_1 > 0]$. Target labels are $y_t = \mathbf{1}[x_1 + 0.5x_2 > 0]$. Both depend on similar inputs, but the target boundary is tilted. For input $[-0.2, 1]$, the source label is 0 while the target label is 1 because $-0.2 + 0.5(1) = 0.3 > 0$.

The script uses the first 300 inputs for training in both phases and the final 100 only for final target evaluation. Source and target training tasks share inputs intentionally; they do not share the held-out evaluation inputs.

The three training phases

1. **Source training:** optimize both ϕ and ψ_s for 100 steps.
2. **Head adaptation:** replace the head with randomly initialized ψ_t, freeze ϕ, and train only ψ_t for 60 steps.
3. **Full adaptation:** unfreeze ϕ and train both for 60 steps at a smaller learning rate.

While frozen, the backbone is still evaluated in the forward pass. “Frozen” means no learned parameter update, not that its output is constant for all inputs. The head learns a new decision rule over the existing representation:

$$\psi_t \leftarrow \psi_t - \eta \nabla_{\psi_t} L, \qquad \phi \text{ unchanged.}$$

After unfreezing, both derivatives matter. A lower learning rate reduces the size of initial changes to reused features. It is a heuristic to preserve useful structure, not a guarantee against forgetting.

How the code implements the idea

Code	Purpose
`model[1] = nn.Linear(16, 2)`	Discard the source decision head and create a target head
`p.requires_grad_(False)`	Exclude backbone parameters from gradient computation
Filter `model.parameters()` by `requires_grad`	Give the optimizer only the intended parameters
Call `fit` again after unfreezing	Create a new optimizer that includes the newly trainable backbone
`with torch.no_grad()` on final inputs	Evaluate without modifying parameters

Each `fit` call creates a new AdamW optimizer, so optimizer moments restart at each stage. That is a teaching choice. Some practical workflows preserve or carefully rebuild optimizer state when changing trainable parameter groups.

Model weights and optimizer memory are different

The model stores learned weights and biases. AdamW separately stores a step counter and two moving averages for each parameter it updates. For a scalar parameter with gradient g_t, these averages are:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

The first moment m_t tracks the recent gradient direction; the second moment v_t tracks squared gradient magnitude (it is not the centered variance). For tensors, these calculations happen element by element. Both start at zero. AdamW uses bias-corrected versions, $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$, in its update:

$$\theta_t = (1-\eta\lambda)\theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Here η is the learning rate, λ is weight decay, and ϵ is a small positive number for numerical stability. The optimizer therefore uses recent gradient history as well as the current gradient.

For example, take $\beta_1 = 0.9$ and $\beta_2 = 0.999$. If the first gradient is $g_1 = 2$, then $m_1 = 0.2$ and $v_1 = 0.004$. If the next gradient is -1, continuing with that same optimizer gives:

$$m_2 = 0.9(0.2) + 0.1(-1) = 0.08, \qquad v_2 = 0.999(0.004) + 0.001(1) = 0.004996.$$

Recreating the optimizer before that second gradient instead gives fresh first-step values $m_1 = -0.1$ and $v_1 = 0.001$. Even with the same model weight and current gradient, the gradient-based update can differ because the history and step counter differ. These numbers are the raw moments, before bias correction.

In this script, the state transitions are:

Stage	Model parameters	AdamW state
Source training	Train backbone and source head	Start fresh
Head adaptation	Keep learned backbone; replace source head with a random target head	Start fresh for target head only
Full adaptation	Keep the backbone and the now-trained target head; unfreeze backbone	Start fresh for both

Creating a new optimizer does **not** reinitialize the model. In particular, the third `fit` call keeps the target head's learned weights but discards its 60 steps of optimizer history. Only the explicit `nn.Linear(16, 2)` replacement creates a new head with new initial weights.

An alternative at unfreezing is to keep the head optimizer and add the newly trainable backbone with `optimizer.add_param_group(...)`. That retains the head's history while giving the newly added parameters fresh state; learning rates must also be set as intended for each group. Unfreezing alone does not add parameters to an optimizer that excluded them. Replacing a head also requires updating optimizer parameter references: an old optimizer still refers to the old head. Preserving state is a deliberate choice, since source gradient history may not suit a changed target objective.

Freezing parameters is different from freezing BatchNorm statistics

For a vision backbone containing BatchNorm, freezing weights alone does not freeze running statistics; train/eval mode needs separate attention. This small backbone has only a linear layer and ReLU, so that issue is absent here.

BatchNorm normally has two kinds of stored state:

State	How it changes
Learned scale γ and shift β	Gradient-based optimizer updates
Running mean and running variance	Updates during forward passes in training mode

Setting `requires_grad_(False)` stops new gradients for the learned parameters. It does not change the layer's training mode, so BatchNorm can still update its running statistics when the frozen backbone processes a batch. Even `torch.no_grad()` does not stop those training-mode updates.

For example, suppose a BatchNorm feature has running mean 10. A target batch has mean 20, and BatchNorm momentum is 0.1. A training-mode forward pass updates:

$$\text{running mean}_{\text{new}} = 0.9(10) + 0.1(20) = 11.$$

That change happens even if every backbone parameter is frozen and no optimizer step runs. The running variance updates separately as well. During training, BatchNorm normalizes using the current batch statistics; during evaluation, standard BatchNorm uses the stored running statistics. Thus, the representation at evaluation can change even though the weights did not.

If the intention is to keep both the frozen backbone's parameters and its BatchNorm statistics fixed while training the head, a typical setup is:

```
for p in backbone.parameters():
    p.requires_grad_(False)

model.train()      # Put the overall model in training mode first.
backbone.eval()    # Then keep the backbone in evaluation mode.
# Train using an optimizer containing only the head parameters.
```

Calling `model.train()` again also puts its backbone back into training mode, so reapply `backbone.eval()` afterward when needed. Conversely, `eval()` by itself does not disable gradients or freeze parameters. These are separate controls. The example assumes BatchNorm tracks running statistics, its default behavior; evaluation mode also disables dropout in a backbone containing it.

Sometimes adapting BatchNorm statistics to target data is intentional. The choice is whether to retain source statistics or update them using target batches. None of this changes the supplied script: its `Linear` and `ReLU` backbone has no running statistics or dropout.

What the result means

Run `python 02_finetuning/01_transfer_learning.py`. Expect high target accuracy. The file illustrates the mechanics but does not prove transfer helped: there is no matched target-only baseline in its printed result. To test that claim, train a fresh model on the same target data and compare multiple seeds under a fair budget. Also evaluate the source task after adaptation to measure forgetting.

Exercise: train with only 20 target examples. Compare head-only, full adaptation, and scratch training. Predict which has lower variance, then check rather than assuming pretrained features always win.

Companion code: `01_transfer_learning.py`

02_finetuning/01_transfer_learning.py

```python
"""Pretrain, replace the head, freeze, then gradually unfreeze.
Fine-tuning starts from weights learned on a previous task. Feature extraction
trains only the new head; full fine-tuning updates the backbone too.
Synthetic source/target tasks keep this runnable without downloading anything.
Exercise: compare head-only and full tuning on a small target training set.
"""
import torch
from torch import nn


def fit(model, x, y, steps, lr):
    optimizer = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad], lr=lr)
    for _ in range(steps):
        loss = nn.functional.cross_entropy(model(x), y)
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    x = torch.randn(400, 2)
    source_labels = (x[:, 0] > 0).long()
    target_labels = (x[:, 0] + 0.5*x[:, 1] > 0).long()
    backbone = nn.Sequential(nn.Linear(2, 16), nn.ReLU())
    model = nn.Sequential(backbone, nn.Linear(16, 2))
    fit(model, x[:300], source_labels[:300], 100, 0.03)
    model[1] = nn.Linear(16, 2)  # New task, new head.
    for p in backbone.parameters():
        p.requires_grad_(False)
    fit(model, x[:300], target_labels[:300], 60, 0.02)
    for p in backbone.parameters():
        p.requires_grad_(True)
    # Recreate the optimizer so newly unfrozen parameters are included.
    fit(model, x[:300], target_labels[:300], 60, 0.002)
    with torch.no_grad():
        accuracy = (model(x[300:]).argmax(-1) == target_labels[300:]).float().mean()
    print('target test accuracy:', accuracy.item())
    # Real transfer: use a lower backbone LR, monitor source-task forgetting,
    # and handle BatchNorm running statistics when freezing vision backbones.

if __name__ == '__main__':
    main()
```

2.2

Instruction tuning, response-only loss, and token masks

Companion code: 02_instruction_data_and_masks.py. Read this explanation before running the code.

What supervised fine-tuning teaches

SFT trains a model to reproduce desired responses to prompts. Given prompt x and response tokens $y_1, \ldots, y_M$, the conditional likelihood is:

$$p_\theta(y \mid x) = \prod_{t=1}^{M} p_\theta(y_t \mid x, y_{<t}).$$

Minimizing its negative logarithm teaches the response while using the prompt as context. This is supervised learning: desired responses are supplied, unlike RL where the agent learns from consequences or rewards.

Read next-token prediction first. This file constructs a batch and checks its loss; it does not train an LLM.

Tokens, formatting, and boundaries

The teaching tokenizer represents UTF-8 bytes as IDs 0–255, with EOS=256 and PAD=257. Non-ASCII characters may use several bytes. A real model must use its own tokenizer and expected prompt/chat format.

For each example, the script concatenates a prefix such as `User: 2+2?\nAssistant:` with the answer `4` and an EOS token. The target labels for the prefix are -100, while labels for the answer and EOS are their actual token IDs. -100 is a loss sentinel, not a token passed through the embedding table.

Why supervise EOS? The model needs to learn when to stop. Why not supervise the prompt? In response-only SFT, the goal is conditional response behavior; prompt reconstruction would spend part of the objective on a different task.

Two masks do different jobs

An **attention mask** says which input positions are real rather than padding. A **label mask** says which next-token predictions contribute to the objective. A prompt can be visible to attention while excluded from direct loss.

The loss with label mask m_{bt} is:

$$L = -\frac{\sum_{b,t} m_{bt} \log p_\theta(u_{bt} \mid u_{b,<t})}{\sum_{b,t} m_{bt}}.$$

The denominator counts supervised tokens, not all positions or examples. Consequently, examples with longer responses contribute more token terms. That is a weighting choice, not an unavoidable property of SFT.

Ignoring prompt labels does **not** stop all gradients through prompt representations: response predictions depend on them through attention. It removes direct prompt-token prediction loss.

Why we shift labels exactly once

Suppose the full IDs are `[10,20,30,256]`, where `[10,20]` is the prompt and `[30,256]` is the response plus EOS. Labels are `[-100,-100,30,256]`.

Logit position	Predicts full-sequence position	Supervised target
0	1	ignored
1	2	30, the first response token
2	3	256, EOS

The code pairs `logits[:, :-1]` with `labels[:, 1:]`. Notice that the first answer token is predicted from the final prompt position. Excluding all logits at prompt positions would incorrectly remove this important prediction.

Hugging Face causal models usually perform this shift internally when passed `labels`. The local helper performs it manually because it receives raw logits. Doing both would make every target off by one.

Map to code and inspect the outcome

`make_batch` creates equal-length ID and label rows. It pads inputs with PAD and labels with -100. `ids.ne(pad)` creates the attention mask. `response_loss` flattens batch and sequence dimensions, leaving vocabulary as the class axis, and calls cross-entropy with ignored targets excluded.

The random logits in `main` are a diagnostic tensor with gradients enabled. There is no attention model in this file, so the returned attention mask is inspected, not passed into a neural network. The gradient check confirms the loss is differentiable; it does not show a model has learned anything.

Run `python 02_finetuning/02_instruction_data_and_masks.py`. The two examples have 2 and 7 supervised tokens: answer bytes plus EOS. Verify this manually. Try changing logits whose shifted labels are -100: the loss should not change. If truncation leaves zero response tokens, drop or fix that example; an empty supervision denominator can produce an undefined loss.

Finally, split by source before chunking and deduplicate across splits. Perfect label masking cannot repair train/test leakage in the data itself.

Companion code: `02_instruction_data_and_masks.py`

02_finetuning/02_instruction_data_and_masks.py

```python
"""Instruction examples, response-only loss, padding, and leakage.
SFT = supervised fine-tuning: maximize likelihood of desired responses.
Prompt tokens provide context; only response tokens contribute to this loss.
-100 is PyTorch cross-entropy's default ignored target. It is NOT a token ID.
Exercise: add a second example of different length and inspect every label.
"""
import torch
from torch import nn


def make_batch(examples):
    # A byte tokenizer is only a teaching aid. IDs 256 and 257 are EOS and PAD.
    # Real chat models need their OWN tokenizer and chat template.
    eos, pad = 256, 257
    sequences, labels = [], []
    for prompt, answer in examples:
        prefix = list(('User: ' + prompt + '\nAssistant: ').encode('utf-8'))
        completion = list(answer.encode('utf-8')) + [eos]
        sequences.append(prefix + completion)
        labels.append([-100]*len(prefix) + completion)
    length = max(map(len, sequences))
    ids = torch.tensor([s + [pad]*(length-len(s)) for s in sequences])
    targets = torch.tensor([s + [-100]*(length-len(s)) for s in labels])
    return ids, targets, ids.ne(pad)


def response_loss(logits, labels):
    # logits at position t predict token t+1. Shift EXACTLY ONCE.
    # Hugging Face causal LM forward(labels=...) generally shifts internally.
    return nn.functional.cross_entropy(logits[:, :-1].reshape(-1, logits.size(-1)),
                                       labels[:, 1:].reshape(-1), ignore_index=-100)


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    ids, labels, attention_mask = make_batch([('2+2?', '4'), ('Say hello', 'Hello!')])
    print('batch shape:', ids.shape)
    print('supervised tokens per example:', labels.ne(-100).sum(-1).tolist())
    print('padding tokens:', (~attention_mask).sum().item())
    logits = torch.randn(*ids.shape, 258, requires_grad=True)
    loss = response_loss(logits, labels)
    loss.backward()
    assert torch.all(labels[~attention_mask] == -100)
    assert logits.grad.isfinite().all()
    print('response-only loss:', loss.item())
    # Split by document/user/source BEFORE chunking. Deduplicate across splits.
    # An attention mask controls visibility; a label mask controls supervision.
    # If truncation removes every response token, drop that training example.

if __name__ == '__main__':
    main()
```

2.3

LoRA: adapting a model through low-rank weight updates

Companion code: 03_lora_from_scratch.py. Read this explanation before running the code.

Why train fewer parameters?

A pretrained model may contain millions or billions of weights. Full tuning stores gradients and optimizer states for all of them. LoRA freezes the base weights and trains a smaller structured update. The assumption is that a useful task adaptation may be expressible in a lower-dimensional family than an arbitrary full weight change. This need not hold for every task.

Prerequisites: matrix multiplication and transfer learning.

Factor the update, not the original weight

For a linear layer, PyTorch stores $W \in \mathbb{R}^{d_{out} \times d_{in}}$ and computes $XW^T + b$. LoRA defines:

$$W_{eff} = W_0 + sBA, \quad s = \alpha / r,$$

where $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$ are trainable. Since $\text{rank}(BA) \leq r$, the added update is low-rank. The original W_0 itself can remain full-rank.

For row-oriented batches:

$$XW_{eff}^T = XW_0^T + sXA^TB^T.$$

This explains the exact transpose order in `x @ self.a.T @ self.b.T`. For example, input `[128,8]` multiplied by $A^T[8,2]$ becomes `[128,2]`, then by $B^T[2,4]$ becomes `[128,4]`, matching the base output.

Parameter and rank arithmetic

A full update requires $d_{out}d_{in}$ parameters. LoRA uses $r(d_{in} + d_{out})$. In this lesson, full weights contain $4 \times 8 = 32$ entries and LoRA uses $2(8 + 4) = 24$. The toy saving is modest. A 4096-by-4096 layer at rank 8 would use 65,536 adapter parameters instead of 16,777,216 full weight parameters, excluding any biases and other trainable modules.

Low-rank tuning reduces trainable state. It does not automatically eliminate base weights, forward computation, or activation memory.

Why initialize one factor to zero?

The script initializes A randomly and $B = 0$. Then $BA = 0$, so attaching the adapter initially preserves the base model's predictions.

Let $G = \partial L / \partial W_{eff}$. Matrix derivatives give:

$$\frac{\partial L}{\partial B} = sGA^T, \qquad \frac{\partial L}{\partial A} = sB^TG.$$

Initially, B can receive a nonzero gradient because A is nonzero. A's first gradient is zero because $B = 0$; after B changes, A can learn too. If both factors started at zero, both gradients would be zero and this adapter would be stuck. This is a concrete reason for the asymmetric initialization.

What the experiment actually learns

The code constructs targets using a frozen base weight plus a synthetic rank-2 update. Thus a rank-2 adapter has enough representational capacity to reproduce the target. It fits predictions using mean squared error on 128 random inputs. This is an intentionally favorable reconstruction problem, not proof that rank 2 is enough for language-model adaptation.

Code	Meaning
`self.base.requires_grad_(False)`	Freeze the original layer
`self.scale = alpha / rank`	Scale the learned update
Optimizer filters `requires_grad`	Optimize only adapter factors
`merged_weight()`	Construct $W_0 + sBA$
Frozen-weight assertion	Check the original model was preserved during adaptation
Merged-output assertion	Check the factorized and merged computations agree

Merging preserves the output of this simple adapter in exact arithmetic. The base bias must still be included if present. Real adapters may have dropout, quantized bases, or other variants that require additional handling.

Run and reason

Run `python 02_finetuning/03_lora_from_scratch.py`. Expect 24 trainable parameters and very small final reconstruction error. Try rank 1: it cannot represent a general rank-2 target update exactly. Increase rank and compare error, parameter count, and convergence rather than assuming bigger always wins.

An adapter checkpoint needs its compatible original base weights to work. Saving A and B alone does not save a standalone pretrained model. Reference: LoRA paper.

Companion code: `03_lora_from_scratch.py`

02_finetuning/03_lora_from_scratch.py

```python
"""Low-rank adaptation (LoRA), trainable counts, and merging.
W_eff = W_frozen + (alpha/r) B A. For W[out,in], A[r,in], B[out,r].
LoRA reduces trainable parameters and optimizer state, not all activation memory.
Exercise: vary rank 1, 2, 4, 8 and compare reconstruction error and parameter count.
"""
import torch
from torch import nn


class LoRALinear(nn.Module):
    def __init__(self, base, rank=2, alpha=4):
        super().__init__()
        self.base = base
        self.base.requires_grad_(False)
        self.a = nn.Parameter(torch.randn(rank, base.in_features) * 0.02)
        self.b = nn.Parameter(torch.zeros(base.out_features, rank))
        self.scale = alpha / rank

    def forward(self, x):
        # B starts at zero: attaching the adapter initially preserves the base.
        return self.base(x) + (x @ self.a.T @ self.b.T) * self.scale

    def merged_weight(self):
        return self.base.weight + self.scale * (self.b @ self.a)


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    base = nn.Linear(8, 4, bias=False)
    adapter = LoRALinear(base)
    x = torch.randn(128, 8)
    original = base.weight.detach().clone()
    with torch.no_grad():
        target_weight = original + torch.randn(4, 2) @ torch.randn(2, 8) * 0.1
        y = x @ target_weight.T
    optimizer = torch.optim.Adam([p for p in adapter.parameters() if p.requires_grad],
lr=0.03)
    for _ in range(200):
        loss = ((adapter(x)-y)**2).mean()
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
    assert torch.equal(base.weight, original), 'Frozen weights must not change.'
    assert torch.allclose(adapter(x), x @ adapter.merged_weight().T, atol=1e-5)
    print('trainable parameters:', sum(p.numel() for p in adapter.parameters() if
p.requires_grad))
    print('final mean squared error:', loss.item())
    # Save the adapter AND base-model identity. Adapter weights alone are not
    # a standalone model. Quantized base merging needs special handling.

if __name__ == '__main__':
    main()
```

2.4

Fine-tuning a pretrained language model: objective, data, and updates

Companion code: 04_huggingface_sft.py. Read this explanation before running the code.

What this workflow does

This is the real-model version of response-only SFT. It downloads a pretrained causal model, formats local prompt/response pairs, trains either all weights or LoRA adapters, measures validation loss, and saves the result. Unlike the tiny transformer lesson, it starts from existing weights. The bundled eight examples teach the workflow, not a useful domain assistant.

The default is a base completion model, DistilGPT-2. It is not a chat model. Changing the model ID may require changing tokenization and prompt formatting; a model's conversation template is part of the data interface it learned.

The objective and its exact weighting

For example i with prompt x_i and response y_i, including EOS:

$$\ell_i = -\sum_{t=1}^{M_i} \log p_\theta(y_{it} \mid x_i, y_{i,<t}).$$

The batch loss is $\sum_i \ell_i / \sum_i M_i$, the average over supervised tokens. Prompt labels and padding labels are -100, so they do not enter this sum. The model still conditions on the prompt. Gradients can flow through prompt representations because they affect response predictions.

`model(**batch).loss` performs the causal shift internally. `collate` therefore passes unshifted full-sequence labels. The first answer label aligns with the logit computed at the last prompt position after the model's internal shift.

The script encodes the prompt and the response separately, then concatenates the resulting IDs. This defines a clear token boundary for the label mask; it need not equal tokenizing the concatenated string with every tokenizer. Use and verify the model's own template/masking procedure for a real chat corpus.

Padding and EOS can share an ID without sharing a role

For GPT-2 tokenizers, padding is often assigned the EOS ID. Consider a sequence whose final real token is EOS and which needs one additional pad position. The two IDs are identical, but the real EOS must be predicted and the pad ignored.

That is why the collator uses lengths to construct masks:

```
attention_mask = [1] * real_length + [0] * padding_length
labels = real_labels + [-100] * padding_length
```

Masking every token equal to the padding ID would incorrectly remove genuine EOS supervision. The batch's attention mask and labels have separate purposes.

Full tuning, LoRA, and QLoRA

Full tuning directly changes the original parameters θ. LoRA substitutes updates $W = W_0 + (\alpha/r)BA$ and changes only the adapter parameters. Read the LoRA derivation first.

The optional QLoRA path stores a quantized frozen base, while training floating-point adapters. Conceptually:

$$W_{eff} \approx \text{dequantize}(Q(W_0)) + sBA.$$

A quantizer stores a small code plus scaling information; its reconstruction is an approximation. NF4 uses a nonuniform 4-bit codebook suited to roughly normal weight distributions. Double quantization also compresses quantization constants. The code delegates those details to bitsandbytes; it does not implement NF4 arithmetic itself. Training still needs activations, temporary computation, and adapter optimizer state. Four-bit storage is not four-bit end-to-end backpropagation. See the official quantization guide.

Follow one update through the script

1. Load and validate nonempty JSONL data; reject exact overlapping prompts.
2. Encode examples and reject overlong ones rather than silently cutting answers.
3. Collate two examples into a padded tensor batch.
4. Clear gradients, run the causal model, and compute masked token loss.
5. Backpropagate, clip gradient norm to 1, and apply AdamW.
6. Repeat until the requested number of updates has run, cycling the data loader.

The learning rate is 2×10^{-4} for LoRA and 2×10^{-5} for full tuning. Those are example settings, not guaranteed suitable rates. `steps` counts optimizer updates, not epochs. With eight examples and batch size two, four updates traverse the dataset once.

Validation is aggregated by supervised-token count. If batch A has 10 tokens and mean loss 2, and batch B has 30 tokens and mean loss 4, the correct combined mean is $(10 \cdot 2 + 30 \cdot 4)/40 = 3.5$, not 3. The script does exactly this. Checking exact prompt overlap catches one leak type, not paraphrase or source leakage; real data needs stronger provenance-aware splits.

Run and interpret

Install the optional requirements, then try:

```
python 02_finetuning/04_huggingface_sft.py --model sshleifer/tiny-gpt2 --steps 5
```

This tiny model tests plumbing. Its generated text may be repetitive and its validation loss may not improve. Save paths contain either full model weights or adapter weights, a tokenizer, and a run configuration. The saved configuration does not contain optimizer state, so it is for inference and documentation, not exact continuation of an interrupted training run.

Exercise: print the decoded supervised tokens for one batch; verify that they contain the response and EOS only. Compare full and LoRA training on a larger held-out task before making quality claims. Continue to reload and generation.

Companion code: `04_huggingface_sft.py`

02_finetuning/04_huggingface_sft.py

```python
"""Actual pretrained causal-LM fine-tuning with full tuning or LoRA.
Optional dependencies: requirements-huggingface.txt. First run downloads a model.
Run: python3 02_finetuning/04_huggingface_sft.py --method lora --steps 20
For a tiny plumbing check: add --model sshleifer/tiny-gpt2.
For NVIDIA QLoRA: add --qlora (install bitsandbytes separately).

This deliberately uses a plain PyTorch loop to expose labels and updates.
The tiny bundled dataset teaches the workflow; it cannot create a capable bot.
Default distilgpt2 is a base completion model, not an instruction/chat model.
For chat models adapt formatting to their chat template and verify the exact
assistant-token mask; generic string concatenation is not a universal template.
"""
import argparse
import json
from pathlib import Path
import torch
from torch.utils.data import DataLoader


def encode_row(row, tokenizer, max_length):
    prefix = tokenizer.encode('Question: '+row['prompt']+'\nAnswer:', add_special_tokens=False)
    answer = tokenizer.encode(' '+row['response'], add_special_tokens=False) + [tokenizer.eos_token_id]
    # Preserve full examples. Silently truncating a response can train bad EOS behavior.
    if len(prefix)+len(answer) > max_length:
        raise ValueError('Example exceeds --max-length; shorten it or increase the limit.')
    return {'input_ids': prefix+answer, 'labels': [-100]*len(prefix)+answer}


def collate(rows, pad_id):
    size = max(len(row['input_ids']) for row in rows)
    batch = {'input_ids': [], 'labels': [], 'attention_mask': []}
    for row in rows:
        length = len(row['input_ids'])
        batch['input_ids'].append(row['input_ids']+[pad_id]*(size-length))
        batch['labels'].append(row['labels']+[-100]*(size-length))
        # Mask by position, not token equality: PAD may share the EOS ID.
        batch['attention_mask'].append([1]*length+[0]*(size-length))
    return {key: torch.tensor(value) for key, value in batch.items()}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--model', default='distilgpt2')
    parser.add_argument('--method', choices=['full', 'lora'], default='lora')
    parser.add_argument('--steps', type=int, default=20)
    parser.add_argument('--max-length', type=int, default=128)
    parser.add_argument('--data', type=Path, default=Path(__file__).parent/'data/train.jsonl')
    parser.add_argument('--validation', type=Path, default=Path(__file__).parent/'data/validation.jsonl')
    parser.add_argument('--output', type=Path, default=Path(__file__).resolve().parents[1]/'outputs/sft')
    parser.add_argument('--qlora', action='store_true')
    args = parser.parse_args()
    if args.steps < 1:
```

```
        parser.error('--steps must be positive')
    if args.qlora and (args.method != 'lora' or not torch.cuda.is_available()):
        parser.error('This QLoRA example requires --method lora and an NVIDIA CUDA GPU.')
    from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
    torch.manual_seed(7)
    torch.set_num_threads(2)
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    tokenizer = AutoTokenizer.from_pretrained(args.model)
    if tokenizer.eos_token_id is None:
        raise ValueError('This example requires an EOS token.')
    if tokenizer.pad_token_id is None:
        tokenizer.pad_token = tokenizer.eos_token
    kwargs = {}
    if args.qlora:
        compute_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
        kwargs.update(quantization_config=BitsAndBytesConfig(
            load_in_4bit=True, bnb_4bit_quant_type='nf4',
            bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=compute_dtype),
            device_map={'': 0})
        # NF4 compresses frozen base weights; adapters remain trainable floats.
    model = AutoModelForCausalLM.from_pretrained(args.model, **kwargs)
    if args.qlora:
        from peft import prepare_model_for_kbit_training
        model = prepare_model_for_kbit_training(model)
    else:
        model.to(device)
    if args.method == 'lora':
        from peft import LoraConfig, get_peft_model
        config = LoraConfig(task_type='CAUSAL_LM', r=8, lora_alpha=16,
                            lora_dropout=0.05, target_modules='all-linear')
        model = get_peft_model(model, config)
        model.print_trainable_parameters()
    model.config.use_cache = False  # KV caching is for generation, not this training loop.
    def load(path):
        rows = [json.loads(line) for line in path.read_text().splitlines() if line.strip()]
        if not rows:
            raise ValueError(f'Empty dataset: {path}')
        return rows
    train_rows, val_rows = load(args.data), load(args.validation)
    if {r['prompt'] for r in train_rows} & {r['prompt'] for r in val_rows}:
        raise ValueError('Overlapping prompts between train and validation.')
    batcher = lambda rows: collate(rows, tokenizer.pad_token_id)
    train = DataLoader([encode_row(r, tokenizer, args.max_length) for r in train_rows],
                       batch_size=2, shuffle=True, collate_fn=batcher)
    val = DataLoader([encode_row(r, tokenizer, args.max_length) for r in val_rows],
                     batch_size=2, collate_fn=batcher)
    optimizer = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad],
                                  lr=2e-4 if args.method == 'lora' else 2e-5)
    def evaluate():
        model.eval()
        total, count = 0.0, 0
        with torch.no_grad():
            for batch in val:
                batch = {k: v.to(device) for k, v in batch.items()}
                n = batch['labels'][:, 1:].ne(-100).sum().item()
                total += model(**batch).loss.item()*n
                count += n
        return total/count  # Token-weighted mean, not a mean of unequal batches.
    print('validation before:', evaluate())
    iterator = iter(train)
    for step in range(args.steps):
        try:
```

```
            batch = next(iterator)
        except StopIteration:
            iterator = iter(train)
            batch = next(iterator)
        model.train()
        batch = {k: v.to(device) for k, v in batch.items()}
        optimizer.zero_grad(set_to_none=True)
        loss = model(**batch).loss  # Model shifts labels internally.
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        if step % 5 == 0:
            print('step:', step, 'train loss:', loss.item())
    print('validation after:', evaluate())
    args.output.mkdir(parents=True, exist_ok=True)
    model.save_pretrained(args.output)
    tokenizer.save_pretrained(args.output)
    (args.output/'run_config.json').write_text(json.dumps(vars(args), default=str,
indent=2))
    print('saved to:', args.output)
    # LoRA output contains adapters; reload with the ORIGINAL base model plus
    # PeftModel.from_pretrained(base, output). Full tuning saves full weights.

if __name__ == '__main__':
    main()
```

2.5

Reloading models and generating tokens

Companion code: 05_reload_and_generate.py. Read this explanation before running the code.

What has to be saved to reproduce behavior?

A language model's behavior depends on its architecture, learned weights, tokenizer, input format, and generation rule. Weights without their compatible tokenizer can turn text into the wrong IDs. A LoRA adapter additionally needs the original base model from which it was trained.

Read the SFT workflow. This script performs inference, not optimization: there is no loss, backward pass, or optimizer update.

Full checkpoint versus adapter checkpoint

A full checkpoint represents the tuned weights W_{full}. An adapter checkpoint represents the update factors, conceptually A, B in:

$$W_{effective} = W_{base} + (\alpha/r)BA.$$

The presence of `adapter_config.json` selects the adapter-loading branch. `PeftConfig` identifies the compatible base model; the base is loaded, then `PeftModel.from_pretrained` attaches the saved adapters. Otherwise, `AutoModelForCausalLM.from_pretrained` loads the full checkpoint directly.

The tokenizer is loaded from the output directory in both cases. For strict reproducibility, also pin base model revisions, package versions, and template behavior rather than relying only on a mutable repository name.

Autoregressive generation, mathematically

For prompt tokens x and generated tokens y:

$$p(y \mid x) = \prod_{t=1}^{M} p(y_t \mid x, y_{<t}).$$

At every step the model produces vocabulary logits. Greedy decoding chooses:

$$y_t = \arg\max_k z_{t,k}.$$

The script uses `do_sample=False`, so generation is greedy. Greedy token choices do not necessarily maximize the probability of the whole completed sequence. For example, a first token with probability 0.6 followed by a best continuation of probability 0.1 gives joint

probability 0.06, whereas another first token with probability 0.4 and continuation probability 0.9 gives 0.36. Greedy decoding picks the first option without exploring that future.

Sampling instead draws from $\mathrm{softmax}(z/\tau)$, with temperature τ. Top-p sampling keeps a prefix of probability-ranked tokens whose mass reaches a chosen threshold, renormalizes, and samples. These alter decoding; they do not update or repair the trained model.

Connect the code to behavior

Code	Purpose
`model.eval()`	Disable training-specific behavior such as dropout
`with torch.no_grad()`	Avoid recording a backward graph
`'Question: '+prompt+'\nAnswer:'`	Match the format used by the SFT lesson
`max_new_tokens=40`	Limit additional tokens, excluding the original prompt
`output[0, input_length:]`	Decode only the new suffix, not prompt plus answer
`skip_special_tokens=True`	Hide tokens such as EOS in the displayed text

Generation can stop before 40 tokens if EOS is produced. The CPU reload path loads an unquantized base even for a QLoRA-trained adapter; this can require substantially more RAM than the quantized training base. Other hardware/loading configurations need explicit setup.

Run and check

After running the SFT lesson, execute `python 02_finetuning/05_reload_and_generate.py`. Use `--checkpoint outputs/full` for the full-model example. Expect a string of new tokens, not necessarily a correct answer from a tiny teaching model.

Exercise: compare greedy and sampled output using the same saved model. Explain which source of variation changed: decoding randomness, not model parameters. For a stricter save/reload check, compare evaluation-mode logits on fixed input IDs before saving and after loading, allowing for floating-point tolerances.

Companion code: `05_reload_and_generate.py`

02_finetuning/05_reload_and_generate.py

```python
"""Reload a full checkpoint OR a LoRA adapter and generate a response.
Run after 04_huggingface_sft.py. No training occurs here. Only new tokens are decoded.
Exercise: compare greedy generation with temperature/top-p sampling.
"""
import argparse
from pathlib import Path
import torch


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    parser = argparse.ArgumentParser()
    parser.add_argument('--checkpoint', type=Path,
default=Path(__file__).resolve().parents[1]/'outputs/sft')
    parser.add_argument('--prompt', default='What is a gradient?')
    args = parser.parse_args()
    from transformers import AutoModelForCausalLM, AutoTokenizer
    tokenizer = AutoTokenizer.from_pretrained(args.checkpoint)
    if (args.checkpoint/'adapter_config.json').exists():
        from peft import PeftConfig, PeftModel
        config = PeftConfig.from_pretrained(args.checkpoint)
        base = AutoModelForCausalLM.from_pretrained(config.base_model_name_or_path)
        model = PeftModel.from_pretrained(base, args.checkpoint)
    else:
        model = AutoModelForCausalLM.from_pretrained(args.checkpoint)
    # This reloads even a QLoRA-trained adapter on an unquantized CPU base.
    # Large bases may require quantized/GPU loading instead; see 04_huggingface_sft.py.
    model.eval()
    inputs = tokenizer('Question: '+args.prompt+'\nAnswer:', return_tensors='pt')
    with torch.no_grad():
        output = model.generate(**inputs, max_new_tokens=40, do_sample=False,
                                pad_token_id=tokenizer.pad_token_id)
    print(tokenizer.decode(output[0, inputs['input_ids'].shape[1]:],
skip_special_tokens=True))

if __name__ == '__main__':
    main()
```

PART 3

Reinforcement learning

3.1

Bandits: learn which button gives the most rewards

Companion code: 01_bandits.py. This chapter assumes no previous reinforcement learning knowledge. Read sections 1–6 before the algorithm.

1. Imagine three mystery buttons

You can press one button each round. After pressing it, you receive either one coin or no coin. Your goal is to collect as many coins as possible.

Action you choose	Possible result	What you initially know
Button A	1 coin or 0 coins	Its chance of giving a coin is unknown
Button B	1 coin or 0 coins	Its chance of giving a coin is unknown
Button C	1 coin or 0 coins	Its chance of giving a coin is unknown

An **agent** is the decision maker: you, or the program choosing buttons. An **action** is a choice, such as pressing C. A **reward** is the numerical feedback after an action: here, 1 for a coin and 0 for no coin. The **environment** is the mechanism producing that reward.

A bandit is this simple repeated decision problem. An “arm” means an available action, borrowing the name from a slot machine's lever. A three-armed bandit has three choices. Here the reward probabilities stay fixed, and each pull is sampled independently. There is no changing location or future state to plan around. Bandits are a basic RL setting; MDPs later add state transitions and future consequences.

2. What does success probability mean?

Here, **success means receiving reward 1 on a particular press**. It does not mean the agent has learned successfully or chosen the best button.

Imagine the environment secretly uses these probabilities:

Button	Probability of reward 1	Probability of reward 0
A	0.2 = 20%	0.8 = 80%
B	0.5 = 50%	0.5 = 50%
C	0.8 = 80%	0.2 = 20%

The notation $P(R = 1 \mid A = C) = 0.8$ reads: “Given that we choose button C, the probability of reward 1 is 0.8.” Here P means probability, R means reward, and the vertical bar means “given.” In this equation A names the action variable; its value C identifies the button.

An 80% success probability does **not** mean you receive 0.8 of a coin on a press. You receive either 1 or 0. It also does not guarantee exactly eight successes in every ten presses. A possible ten-press record for C is:

```
Press:   1  2  3  4  5  6  7  8  9 10
Reward:  1  1  0  1  1  1  0  1  1  1
```

This particular example has eight coins. Another ten presses might give six, nine, or even zero. Across many presses, the success fraction tends toward 0.8. The agent sees the reward record, not the hidden probability table.

3. Expected reward: why is it 0.8?

An **expected value** is a probability-weighted average. For one press of C:

$$E[R \mid C] = (1 \text{ coin})(0.8) + (0 \text{ coins})(0.2) = 0.8 \text{ coins}.$$

We multiply each possible reward by its probability, then add. Although 0.8 is not a possible single outcome, it is the long-run average reward per press.

Similarly:

$$E[R \mid A] = 1(0.2) + 0(0.8) = 0.2,$$

$$E[R \mid B] = 1(0.5) + 0(0.5) = 0.5.$$

C has the highest expected reward. It can still fail on a particular press; A can still succeed. “Best” refers to average performance, not guaranteed success. For these 0/1 rewards, expected reward equals success probability. That equality is specific to this reward scheme: if success paid 5 coins, C's expected reward would be $5(0.8) + 0(0.2) = 4$ coins, while its success probability would remain 0.8.

4. Why is expected total reward 0.8T?

T is the total number of rounds. $0.8T$ means $0.8 \times T$. If we choose C every round, each round contributes expected reward 0.8.

Number of presses of C	Expected total reward
1	0.8
2	$0.8 + 0.8 = 1.6$
10	$10 \times 0.8 = 8$
100	$100 \times 0.8 = 80$
T	$T \times 0.8$

For example, the actual total over two presses can only be 0, 1, or 2 coins. The expectation 1.6 is an average over hypothetical repetitions of that experiment.

Using notation, let R_t be the reward at round t. The symbol $\sum$ means “add all these terms”:

$$\sum_{t=1}^{T} R_t = R_1 + R_2 + \cdots + R_T.$$

Expected values add, giving:

$$E\left[\sum_{t=1}^{T} R_t\right] = \sum_{t=1}^{T} E[R_t] = \underbrace{0.8 + 0.8 + \cdots + 0.8}_{T \text{ terms}} = 0.8T.$$

This addition rule is called **linearity of expectation**. It does not require independence. We do assume here that choosing C continues to give mean 0.8. The result is a benchmark for always choosing the best button. An agent still learning which button is best will generally earn a smaller expected total.

5. What is an action value?

An **action value** answers: "How much reward should I expect, on average, when I choose this action?" For this bandit, define the true value as

$$q(a) = E[R \mid \text{choose action } a].$$

The lowercase a is a placeholder for any button. Thus $q(A) = 0.2$, $q(B) = 0.5$, and $q(C) = 0.8$. The environment knows these values, but the agent does not.

We use uppercase $Q(a)$ for the agent's **estimate** of that true value. The distinction is crucial:

Quantity	Meaning	Available to the learner?
$q(C) = 0.8$	Hidden true mean reward of C	No
$Q(C) = 0.6$, for example	Mean of rewards observed so far from C	Yes
Latest reward, such as 0	Result of one press	Yes

One reward is an observation. A value estimate summarizes many observations.

6. Estimate a value by averaging observed rewards

Suppose the first five times we choose C produce rewards `[1, 0, 1, 1, 0]`. These are five pulls **of C**; presses of other buttons do not enter C's average.

Number of C pulls n	New reward	Total coins from C	Estimate $Q_n(C)$
1	1	1	$1/1 = 1.00$
2	0	1	$1/2 = 0.50$
3	1	2	$2/3 \approx 0.667$
4	1	3	$3/4 = 0.75$
5	0	3	$3/5 = 0.60$

The estimate 0.60 does not change the true probability 0.8. It reflects limited, noisy evidence. More samples generally improve reliability, but an estimate does not necessarily move closer to the truth after every individual sample.

The sample-average formula is:

$$Q_n(a) = \frac{R_1^{(a)} + R_2^{(a)} + \cdots + R_n^{(a)}}{n} = \frac{1}{n}\sum_{i=1}^{n} R_i^{(a)}.$$

Here $R_i^{(a)}$ means the reward from the i-th time action a was selected; it is an index label, not a power. n counts observations of that particular action. An untried action has no sample average yet. Initializing its estimate to zero is a programming convention, not evidence that it never pays.

7. Update the average without storing every reward

After each new reward, the code uses:

$$\boxed{Q_{new} = Q_{old} + \frac{1}{n}(R_{new} - Q_{old})}$$

Read this as **old estimate + step size × prediction error**:

Part	Meaning
Q_{old}	What we expected before this observation
R_{new}	What we actually received
$R_{new} - Q_{old}$	How much the observation differs from our estimate
$1/n$	How strongly this observation changes the estimate

For C's third observation in the table, $Q_{old} = 0.5$, $R_{new} = 1$, and $n = 3$:

$$Q_{new} = 0.5 + \frac{1}{3}(1 - 0.5) = 0.5 + 0.1667 \approx 0.6667.$$

For its fifth observation, $Q_{old} = 0.75$, $R_{new} = 0$, and $n = 5$:

$$Q_{new} = 0.75 + \frac{1}{5}(0 - 0.75) = 0.75 - 0.15 = 0.60.$$

A reward above the estimate moves it upward; one below moves it downward. Later observations have smaller individual influence because more evidence has already accumulated. This is ordinary averaging, not a separate approximation. To derive it, the previous $n - 1$ observations have total $(n-1)Q_{old}$:

$$Q_{new} = \frac{(n-1)Q_{old} + R_{new}}{n} = Q_{old} + \frac{R_{new} - Q_{old}}{n}.$$

You can run this tiny example independently:

```
estimate = 0.0
for count, reward in enumerate([1, 0, 1, 1, 0], start=1):
```

```
    estimate += (reward - estimate) / count
    print(count, round(estimate, 3))
# 1 1.0
# 2 0.5
# 3 0.667
# 4 0.75
# 5 0.6
```

8. Why must the agent explore?

Imagine the agent tries each button once and observes:

Button	Observed reward	Current estimate
A	1	1.0
B	0	0.0
C	0	0.0

A happened to get lucky; C happened to get unlucky. Choosing only the highest current estimate can neglect C, even though C is truly best.

Exploitation means choosing the action with the highest current estimate. **Exploration** means trying actions to collect more information. An **epsilon-greedy** rule balances them. Epsilon, written ϵ, is a chosen exploration probability; it is not a button's success probability.

With $\epsilon = 0.1$:

```
Each round:
    10% chance -> explore -> choose A, B, or C uniformly
    90% chance -> exploit -> choose the highest current estimate
Then observe one reward and update only the chosen button's estimate.
```

Suppose C now has the highest estimate. It is chosen through two routes: exploitation, or exploration that happens to select C. Therefore:

$$P(\text{choose C}) = 0.9 + 0.1 \times \frac{1}{3} \approx 0.9333.$$

A and B are each chosen with probability $0.1/3 \approx 0.0333$. Do not confuse $P(\text{choose C}) \approx 0.9333$ with $P(\text{reward } 1 \mid C) = 0.8$: one describes the agent's choice, the other describes the environment's response.

When C is the selected greedy action, expected reward per round is:

$$0.0333(0.2) + 0.0333(0.5) + 0.9333(0.8) \approx 0.77.$$

Equivalently: 90% of rounds use C, and exploratory rounds choose uniformly:

$$0.9(0.8) + 0.1\left(\frac{0.2 + 0.5 + 0.8}{3}\right) = 0.72 + 0.05 = 0.77.$$

Persistent exploration costs some immediate reward. Before C is identified as best, the expected reward may be lower still. The actual observed average fluctuates around its expectation.

9. Connect each code line to the story

The companion script uses indices `0, 1, 2` for A, B, C:

```
true_means = [0.2, 0.5, 0.8]  # environment's hidden probabilities
counts = [0, 0, 0]            # number of presses of each button
estimates = [0.0, 0.0, 0.0]  # learner's current Q estimates
```

`rng.random()` produces a random number in `[0, 1)`. The expression `rng.random() < 0.1` is true with probability 0.1, triggering exploration. `rng.randrange(3)` chooses a random button index. Otherwise, `max` selects the index with the largest estimate. Initial equal estimates are broken in favor of the first index in this implementation.

The environment generates a reward using:

```
reward = float(rng.random() < true_means[action])
```

For C, a random number below 0.8 gives reward 1. For example, 0.37 gives 1, while 0.91 gives 0. The agent's action-selection rule never reads `true_means`. The simulator needs them to produce feedback; the learner sees only that feedback.

Then the learner updates:

```
counts[action] += 1  # n must include this new observation
estimates[action] += (reward - estimates[action]) / counts[action]
```

Only the chosen button's count and estimate change. `total_reward / 2000` is the observed overall reward per round, mixing all three buttons. It is different from an individual button's estimate.

Run from the course folder:

```
python3 03_reinforcement_learning/01_bandits.py
```

Expect C to receive most selections. Its estimate should be near 0.8 after many pulls, and the overall average near 0.77 once the agent mostly identifies C. Neither number is guaranteed in a short run.

10. Check your understanding

1. A button pays 1 with probability 0.3 and otherwise 0. What is its expected reward? **0.3**, though one press gives only 0 or 1.
2. Press it 20 times. What total reward do you expect? **6**, not a guaranteed 6.
3. Observe rewards `[0, 1, 1, 0]`. What is your estimate? **2/4 = 0.5**.

4. The next reward is 1. Update without storing the history: $0.5 + (1 - 0.5)/5 = 0.6$.
5. Does that make the true success probability 0.6? **No. It changes our estimate.**
6. Why explore? **Early estimates can rank the buttons incorrectly.**

After these examples make sense, compare epsilon values 0, 0.1, and 0.5 across several seeds. More exploration gathers information but also spends more rounds on weaker actions. Next: MDPs and future consequences.

Companion code: `01_bandits.py`

03_reinforcement_learning/01_bandits.py

```python
"""Rewards, exploration, and incremental value estimates.
A bandit has actions but no changing state. RL adds state transitions and
long-term consequences. Epsilon-greedy explores randomly with probability eps.
Exercise: compare eps=0, 0.1, 0.5 across ten seeds, not just one lucky run.
"""
import random


def main():
    rng = random.Random(7)
    true_means = [0.2, 0.5, 0.8]
    counts, estimates = [0]*3, [0.0]*3
    total_reward = 0
    for step in range(2000):
        action = rng.randrange(3) if rng.random() < 0.1 else max(range(3),
key=estimates.__getitem__)
        reward = float(rng.random() < true_means[action])
        counts[action] += 1
        estimates[action] += (reward-estimates[action]) / counts[action]
        total_reward += reward
    print('action counts:', counts, 'estimated rewards:', estimates)
    print('average reward:', total_reward/2000)
    # The environment knows true_means; the learning agent only sees samples.

if __name__ == '__main__':
    main()
```

3.2

MDPs, value iteration, and policy iteration: planning a route to reward

Companion code: 02_mdp_value_iteration.py. Prerequisites: the bandits chapter, probability-weighted averages, and basic Python loops. This chapter explains each new symbol before using it.

1. What problem are we solving?

Imagine a robot standing in a corridor:

```
State:       0          1          2          3          4
          [END] <--> [room] <--> [START] <--> [room] <--> [GOAL]
Arrival
reward:      0          0          0          0         +1
```

The robot starts at state 2. It can move left or right. Entering either edge, 0 or 4, ends the episode. Only entering state 4 gives reward 1. An **episode** is one run from a starting state until the task ends.

We want to answer two questions:

1. How valuable is each starting room, considering rewards we can obtain later?
2. Which action should we choose in each room?

In a bandit, pressing a button affects the current reward. Here an action also changes our location and therefore our future opportunities. Moving right from 2 gives no immediate reward, but puts us one step away from the goal.

The solution is a policy: choose right in states 1, 2, and 3. Value iteration works out why these actions are best by calculating their future consequences.

2. The vocabulary of an MDP

An MDP, or **Markov decision process**, specifies a sequential decision problem.

Term	Symbol	Meaning in our corridor
State	s	Current position: 0, 1, 2, 3, or 4
Action	a	Left (`0`) or right (`1`)
Next state	s'	Position after the move; the prime means “next”
Reward	r	1 when entering 4; otherwise 0
Transition model	$p(s', r \mid s, a)$	Probabilities of next positions and rewards
Discount factor	γ	How strongly future rewards count; code uses 0.95

Term	Symbol	Meaning in our corridor
Policy	$\pi(a \mid s)$	Rule giving action probabilities in each state
Terminal state	—	State 0 or 4; no further decisions or rewards

“Markov” means the present state contains the information needed to predict the next transition, given our action. For this corridor, knowing the current position is enough; we need not remember how we got there.

The model is deterministic: from state 2, action right always produces `(next_state=3, reward=0, terminated=False)`. From state 3, right produces `(4, 1, True)`. Deterministic transitions are a special case of probabilities: the stated outcome has probability 1 and all other outcomes have probability 0.

An MDP describes the problem. **Planning** uses a known model to solve it. **Reinforcement learning** learns from experience, often without knowing that model. This script performs planning: it already has `transition(state, action)`. It does not need to explore the corridor and collect training episodes.

3. Reward and return are different

A reward is feedback for one move. A **return** adds rewards over future moves. Let r_t mean the reward immediately after the action at time t:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

G_t is the return from time t. The discount γ multiplies each reward once for each extra step we wait. With $\gamma = 0.95$:

Future timing	Contribution of reward 1 to current return
On the current move	1
One move later	0.95
Two moves later	$0.95^2 = 0.9025$
Three moves later	$0.95^3 = 0.857375$

This is not a success probability. The robot's moves are certain in this example; 0.95 is a chosen weight making delayed rewards count less.

From state 2, move right twice:

```
2 --right / reward 0--> 3 --right / reward 1--> 4 (end)
```

Its return is $0 + 0.95(1) = 0.95$. From state 1, moving right three times gives $0 + 0.95(0) + 0.95^2(1) = 0.9025$. The first reward is not discounted, so three moves to the goal mean two discount factors, not three.

Taking a detour from state 2, `2 -> 1 -> 2 -> 3 -> 4`, gives rewards `[0, 0, 0, 1]` and return $0.95^3 = 0.857375$. Both routes eventually earn one coin, but the shorter route has higher discounted return.

4. What is a state value?

A state value answers: **"Starting here, how much future return should I expect?"**

$$V^\pi(s) = E[G_t \mid S_t = s, \text{ follow policy } \pi].$$

The superscript π means "under this policy"; it is not a power. A policy that always goes left has value zero everywhere in this corridor. A policy that always goes right has values:

State	Rewards along the rightward route	Value
1	`[0, 0, 1]`	0.9025
2	`[0, 1]`	0.95
3	`[1]`	1
0 or 4	Already finished; no future rewards	0

Why is $V(4) = 0$ rather than 1? The reward 1 was already received when entering 4. Starting at 4 means the episode has ended. Counting another 1 would count the same goal reward twice.

$V^*(s)$ means the best achievable expected return from s, over all policies. The star means "optimal." The values above are also optimal for this corridor.

5. The Bellman idea: split the future into one move plus the rest

The return can be rearranged as:

$$G_t = r_t + \gamma G_{t+1}.$$

Consequently, a move's value is:

$$\text{immediate reward} + \gamma \times \text{value of the next state}.$$

Suppose our current estimates are $V(1) = 0.9025$ and $V(3) = 1$. At state 2, compare:

Candidate action	Next state	Immediate reward	Candidate value
Left	1	0	$0 + 0.95(0.9025) = 0.857375$
Right	3	0	$0 + 0.95(1) = 0.95$

Choose right because 0.95 is larger. Notice both immediate rewards are zero; it is the **future value** that distinguishes the actions.

For a known deterministic transition, value iteration uses:

$$V_{new}(s) = \max_a \left[r(s,a) + \gamma(1-d)V_{old}(s') \right].$$

Here $d = 1$ for a terminal transition, and $d = 0$ otherwise. Thus $(1 - d)$ removes future continuation after termination. `max` chooses the largest candidate value. It returns a number; `argmax` would return the action achieving that number.

6. Follow value iteration sweep by sweep

We do not begin with the answer. Initialize all estimates to zero:

```
values = [0.0] * 5
```

A **sweep** updates all nonterminal states. Every update within one sweep uses the previous sweep's values. This is why the code creates `updated = values.copy()`.

Sweep	$V(0)$	$V(1)$	$V(2)$	$V(3)$	$V(4)$
Initial	0	0	0	0	0
1	0	0	0	1	0
2	0	0	0.95	1	0
3	0	0.9025	0.95	1	0
4	0	0.9025	0.95	1	0

Sweep 1: State 3 discovers the immediate reward: right has candidate 1. State 2 still sees the *old* $V(3) = 0$, so its new value remains zero.

Sweep 2: State 2 now sees the previous sweep's $V(3) = 1$, so right gives $0 + 0.95(1) = 0.95$. State 1 still sees the previous $V(2) = 0$.

Sweep 3: State 1 now sees $V(2) = 0.95$, giving $0 + 0.95(0.95) = 0.9025$.

Sweep 4: No value changes. The equations and the estimates agree.

```
Knowledge of reward moves backward:
    sweep 1:                       state 3 knows
    sweep 2:            state 2 knows
    sweep 3: state 1 knows
```

The robot is not moving backward here. These are calculations over a model.

7. What exactly converges, and what is solved?

The entries of `values` approach V^*: optimal expected discounted returns. A **fixed point** is a vector unchanged by another Bellman optimality update. The loop stops when the largest absolute change is below `1e-10`:

```
change = max(abs(a-b) for a, b in zip(values, updated))
```

This checks numerical stability of the values; it is not a count of successful robot episodes. In this example the values stabilize exactly after a few sweeps. More general problems may need many sweeps and only reach an approximation.

Once values are known, compare action candidates in each state to extract a policy. In states 1, 2, and 3 the best action is right. The problem is solved when we have a decision rule maximizing expected discounted return, not merely a list of numbers. The values explain and support that decision rule.

For a finite discounted MDP, the Bellman optimality operator T satisfies

$$\|TV - TU\|_\infty \leq \gamma \|V - U\|_\infty.$$

$\|V - U\|_\infty$ means the largest absolute difference between two value vectors. The inequality says one backup shrinks that difference by at least the factor $\gamma < 1$. This contraction gives a unique fixed point and convergence. It does not guarantee convergence of arbitrary neural-network RL algorithms.

8. What changes when transitions are random?

Suppose an example action gives reward 0 and goes to a state valued at 1 with probability 0.8, or a terminal state valued at 0 with probability 0.2. Its expected candidate value is

$$0.8[0 + 0.95(1)] + 0.2[0 + 0.95(0)] = 0.76.$$

We average over possible outcomes **for each action**, then pick the action with the highest expected candidate. We do not pick whichever random outcome we would prefer. The general optimality equation is:

$$V^*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V^*(s')].$$

The summation means “add each outcome's probability times its return.” Terminal state values are zero here. For evaluating a fixed stochastic policy, also average over its action choices:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma V^\pi(s')].$$

The first equation chooses the best action; the second evaluates the given policy.

9. Policy iteration: a different way to obtain the same answer

Value iteration repeatedly updates values using a maximum over actions. **Policy iteration** alternates two separate jobs:

1. **Policy evaluation:** Hold the policy fixed. Calculate its expected returns.
2. **Policy improvement:** At each state, choose the action giving the largest immediate reward plus discounted evaluated continuation.

Repeat until the policy no longer changes.

The script initializes the policy to left everywhere. With the code's consistent left-first tie rule, its nonterminal policies evolve as follows:

Evaluated policy at states `[1,2,3]`	Its values at `[1,2,3]`	Next improved policy
`[left,left,left]`	`[0,0,0]`	`[left,left,right]`
`[left,left,right]`	`[0,0,1]`	`[left,right,right]`
`[left,right,right]`	`[0,0.95,1]`	`[right,right,right]`
`[right,right,right]`	`[0.9025,0.95,1]`	Unchanged

For example, when state 3 is known to be valuable, improvement changes state 2 to right. Evaluating that new policy makes state 2 valuable; a later improvement changes state 1. Policy evaluation can itself require many iterative sweeps.

In matrix notation, evaluation solves

$$V^\pi = r_\pi + \gamma P_\pi V^\pi, \qquad (I - \gamma P_\pi)V^\pi = r_\pi.$$

P_π contains transition probabilities under the policy, r_π contains expected immediate rewards, and I is the identity matrix. The script uses iterative evaluation rather than a matrix solver. See the math primer for linear systems and curvature.

With exact evaluation, finite states/actions, $\gamma < 1$, and consistent tie handling, policy iteration reaches an optimal policy. The code approximates policy evaluation using a small numerical tolerance.

10. Read the Python and predict its output

Code	Meaning
`transition(state, action)`	Ask the known model about a hypothetical move
`range(1, 4)`	Update states 1, 2, 3; terminals stay zero
`reward + gamma * (not terminated) * values[nxt]`	One move plus discounted continuation
`max(candidates)`	Keep the best action's candidate value
`updated = values.copy()`	Keep old and new sweeps separate
`policy_iteration()`	Alternate fixed-policy evaluation and improvement

Run from the course folder:

```
python3 03_reinforcement_learning/02_mdp_value_iteration.py
```

Expected optimal values: `[0, 0.9025, 0.95, 1, 0]`. Policy iteration also prints `[1,1,1]`, meaning right in all three nonterminal states.

11. Practice before moving on

1. At state 3, why is right worth 1 rather than 0.95? **Its reward is immediate.**
2. Why is the goal state's future value zero? **The episode has already ended.**
3. If $\gamma = 0.5$, what are optimal values at states 1–3? **[0.25, 0.5, 1].**
4. If all values start at zero, why does state 2 not improve in sweep 1? **It uses the old value of state 3, which is still zero.**
5. Did value iteration need sampled episodes? **No; it used the known model.**
6. What if the model is unknown? **Learn from observed transitions, as in Q-learning and SARSA.**

Companion code: `02_mdp_value_iteration.py`

03_reinforcement_learning/02_mdp_value_iteration.py

```python
"""Markov decision processes and the Bellman equation.
An MDP specifies states, actions, transition probabilities, rewards, discount.
V*(s) = max_a E[r + gamma V*(s')]. Dynamic programming assumes a known model.
This five-state chain terminates at either edge; reaching the right earns +1.
Exercise: add a step penalty or change gamma and explain the resulting values.
"""


def transition(state, action):
    next_state = state + (1 if action == 1 else -1)
    terminated = next_state in (0, 4)
    reward = float(next_state == 4)
    return next_state, reward, terminated


def value_iteration(gamma=0.95):
    values = [0.0]*5
    for _ in range(100):
        updated = values.copy()
        for state in range(1, 4):
            candidates = []
            for action in (0, 1):
                nxt, reward, terminated = transition(state, action)
                candidates.append(reward + gamma * (not terminated) * values[nxt])
            updated[state] = max(candidates)
        change = max(abs(a-b) for a, b in zip(values, updated))
        values = updated
        if change < 1e-10:
            break
    return values


def policy_iteration(gamma=0.95):
    """Evaluate a fixed policy to convergence, then greedily improve it."""
    policy = [0]*5
    values = [0.0]*5
    for _ in range(100):
        for _ in range(10000):
            updated = values.copy()
            for state in range(1, 4):
                nxt, reward, done = transition(state, policy[state])
                updated[state] = reward + gamma*(not done)*values[nxt]
            change = max(abs(a-b) for a, b in zip(values, updated))
            values = updated
            if change < 1e-12:
                break
        stable = True
        for state in range(1, 4):
            candidates = []
            for action in (0, 1):
                nxt, reward, done = transition(state, action)
                candidates.append(reward + gamma*(not done)*values[nxt])
            best = max(range(2), key=lambda a: candidates[a])
            stable &= best == policy[state]
            policy[state] = best
        if stable:
            return values, policy
```

```
    raise RuntimeError('Policy iteration did not stabilize')


if __name__ == '__main__':
    v = value_iteration()
    print('optimal state values:', v)
    assert abs(v[1] - 0.95**2) < 1e-9
    # Optimal policy: move right from every nonterminal state.

    pv, policy = policy_iteration()
    print('policy iteration:', pv, policy[1:4])
    assert max(abs(a-b) for a,b in zip(v,pv)) < 1e-9
```

3.3

Q-learning and SARSA: learn a route by trying actions

Companion code: 03_q_learning_sarsa.py. Prerequisites: bandit averages and MDPs, return, and value iteration. Read this chapter with those two examples in mind; no neural network is needed.

1. What problem are we solving now?

Use the same corridor:

```
      0          1          2          3          4
[END] <--> [room] <--> [START] <--> [room] <--> [GOAL]
 reward 0                                      reward +1
```

Left is action `0`; right is action `1`. Each move gives reward zero except entering state 4, which gives 1. Entering 0 or 4 ends the episode.

The goal is to learn which move to choose in each room to obtain a high **discounted total reward**, not just an immediate reward. Both methods should learn a greedy decision rule that moves right in all three nonterminal rooms.

Value iteration knew the model and calculated hypothetical moves. These learners instead choose an action, observe what happens, and update from that experience. The script still needs environment code to simulate movement, but the learner's update only uses the observed transition. It does not enumerate a known transition model to plan. This is the meaning of **model-free** here.

2. What information does one experience contain?

An observed transition might be:

```
Current state: 2
Chosen action: right
Observed reward: 0
Next state: 3
Terminated: False
```

We abbreviate this as (s, a, r, s', d):

Symbol	Meaning	Example
s	Current state	2
a	Selected action	Right
r	Reward just received	0

Symbol	Meaning	Example
s'	Next state; prime means “next”	3
d	1 if task ended, otherwise 0	0

A terminal example is $(3, \text{right}, 1, 4, 1)$. The transition is a sample of experience, not a table of every possible outcome.

3. What exactly is a Q-value?

A state value $V(s)$ asks, “How valuable is being here?” An action value $Q(s, a)$ asks, **“How valuable is taking this particular action here, including what happens afterward?”** The continuation policy must also be specified.

For a fixed policy π, the true action value is

$$q^{\pi}(s, a) = E[G_t \mid S_t = s, A_t = a, \text{ then follow } \pi].$$

$G_t = r_t + \gamma r_{t+1} + \cdots$ is discounted return. Uppercase Q below is the learner's table of estimates. Unlike a bandit's $Q(a)$, it needs a state index, because “right” can have different consequences in different locations.

The code creates a table with five rows and two columns:

```
q = [[0.0, 0.0] for _ in range(5)]
# q[state][0] estimates left; q[state][1] estimates right
```

State	Estimate for left	Estimate for right
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

Zero initialization means no useful evidence has been incorporated yet. It does not mean the agent knows that all actions truly have zero future value. Only the selected state/action entry changes on each update.

4. The basic update: move an estimate toward a target

Both algorithms use

$$\boxed{Q_{new}(s, a) = Q_{old}(s, a) + \alpha[y - Q_{old}(s, a)]}.$$

Part	Interpretation
$Q_{old}(s, a)$	Current prediction of future return

Part	Interpretation
y	New target, constructed from the observed reward and estimated future
$y - Q_{old}(s, a)$	Prediction error, called the temporal-difference or TD error
α	Learning rate: how far to move toward the target

The code uses $\alpha = 0.2$. With old estimate 0.4 and target 0.9:

$$Q_{new} = 0.4 + 0.2(0.9 - 0.4) = 0.4 + 0.1 = 0.5.$$

Equivalently, $Q_{new} = (1 - \alpha)Q_{old} + \alpha y$: retain 80% of the old estimate and incorporate 20% of the new target. The target is not necessarily the true answer; it often contains another imperfect estimate.

This resembles the bandit update $Q \leftarrow Q + (R - Q)/n$. There are two changes: we use a constant step size 0.2 rather than $1/n$, and the target includes **future return**, not just the newest immediate reward.

5. Q-learning: build the target using the best estimated next action

For a nonterminal transition, Q-learning uses

$$\boxed{y_Q = r + \gamma \max_{a'} Q(s', a')}.$$

The symbol a' means a candidate next action. `max` takes the largest value in the next state's row. The code uses $\gamma = 0.95$.

Suppose we move right from 2 to 3, receiving reward 0, and the next-state row is

```
Q(3, left)  = 0.2
Q(3, right) = 0.8
```

Then

$$y_Q = 0 + 0.95 \max(0.2, 0.8) = 0.95(0.8) = 0.76.$$

If $Q(2, \text{right}) = 0.4$, its updated value is

$$0.4 + 0.2(0.76 - 0.4) = 0.472.$$

The prediction increases because the observed move leads to a state with a promising estimated continuation. We need not receive the final reward on this particular step to improve the estimate.

6. Watch reward information travel backward through actual experience

Start with all entries zero. Consider these selected transitions in chronological order; the table shows only the entry updated on that transition:

Observation	Next-state best estimate	Target	Old entry	New entry
$2 \to 3$, right, reward 0	0	0	$Q(2, R) = 0$	0
$3 \to 4$, right, reward 1, terminal	No continuation	1	$Q(3, R) = 0$	0.2
Later: $2 \to 3$, right, reward 0	0.2	$0.95(0.2) = 0.19$	$Q(2, R) = 0$	0.038
Later: $3 \to 4$, right, reward 1, terminal	No continuation	1	$Q(3, R) = 0.2$	0.36
Later: $1 \to 2$, right, reward 0	0.038	$0.95(0.038) = 0.0361$	$Q(1, R) = 0$	0.00722

The first episode's $2 \to 3$ update remains zero because the learner has not yet learned that state 3 is valuable. A later visit can use the newly improved row. The learner does not automatically revise every earlier transition after finding the goal. This script has no replay or backward episode-return pass.

For repeated terminal transitions $3 \to 4$, the estimate follows `0 -> 0.2 -> 0.36 -> 0.488 -> ...`, approaching 1. Each update closes 20% of the remaining gap. In this special deterministic terminal case, $Q_k(3, R) = 1 - 0.8^k$ if it began at zero and received k such updates.

Using an estimate inside another estimate's target is called **bootstrapping**. Using a reward plus a next-state estimate rather than waiting for the complete observed return is the central temporal-difference idea.

7. SARSA: use the next action actually selected

SARSA uses the same update structure, but its nonterminal target is

$$\boxed{y_{SARSA} = r + \gamma Q(s', a')}.$$

Here a' is the next action **sampled from the agent's behavior rule**, not an argmax placeholder. SARSA stands for **State, Action, Reward, next State, next Action**.

Use exactly the earlier numbers: $Q(3, L) = 0.2$, $Q(3, R) = 0.8$, reward 0, $\gamma = 0.95$, current $Q(2, R) = 0.4$, and $\alpha = 0.2$.

Next action selected	Q-learning target	SARSA target	SARSA updated $Q(2, R)$
Right	0.76	$0.95(0.8) = 0.76$	0.472
Left, due to exploration	0.76	$0.95(0.2) = 0.19$	$0.4 + 0.2(0.19 - 0.4) = 0.358$

In the second row, Q-learning still assumes the best estimated continuation; SARSA accounts for the exploratory left action the agent will actually take. Neither calculation is mistaken. They are estimating different continuations. When the sampled next action is greedy, their targets agree for that transition.

The script saves `next_action` and actually executes it next. Choosing a fresh, different action before the next move would break this sampled SARSA sequence.

8. On-policy and off-policy in ordinary language

A **policy** is a rule for choosing actions. The **behavior policy** generates the agent's experience. The **target policy** is the policy whose return it learns.

Method	How it acts during training	What its update assumes afterward
Q-learning	Epsilon-greedy, sometimes random	Greedy continuation: pick the best estimated action
SARSA	Epsilon-greedy, sometimes random	Continue with that same exploratory policy

Q-learning is **off-policy** because behavior and target can differ. SARSA is **on-policy** because its target uses the same policy that generates experience. “Off-policy” does not mean the program stops following a policy.

The script uses $\epsilon = 0.2$ and two actions. When right uniquely has the higher estimate, right is chosen with probability $0.8 + 0.2/2 = 0.9$ and left with probability 0.1. Equal-value ties are broken randomly.

With the next-state row `[0.2,0.8]`, SARSA's conditional expected next-value sample is $0.1(0.2) + 0.9(0.8) = 0.74$, giving an expected target of $0.95(0.74) = 0.703$. Ordinary SARSA samples either value on each update; it does **not** explicitly calculate this average. Using that average directly would be Expected SARSA, a related algorithm that is not implemented in this file.

Continued exploration can delay reward or terminate at the wrong end. SARSA's values can therefore be lower even when its greedy action choices match Q-learning.

9. Why can risky routes expose the difference?

Imagine a separate grid task with a short route beside a cliff and a longer route away from it. A greedy agent can follow the short route safely, but a random exploratory action near the cliff can incur a large penalty.

Q-learning's target values greedy future decisions, so it can favor the short route while the actual exploratory behavior still falls occasionally. SARSA includes those exploratory consequences in its targets and can prefer the safer route. This illustrates a tendency in such tasks, not a guarantee that SARSA is always safer or better. The provided corridor has no cliff and does not run that experiment.

10. Terminal states and collection limits

The complete target uses a terminal indicator d:

$$y_Q = r + \gamma(1 - d)\max_{a'} Q(s', a'),$$

$$y_{SARSA} = r + \gamma(1 - d)Q(s', a').$$

On entering state 4, $r = 1$ and $d = 1$, so both targets are exactly 1. There is no future decision after termination. On entering state 0, both targets are 0.

For example, with terminal reward 1, old Q 0.5, and learning rate 0.2:

$$Q_{new} = 0.5 + 0.2(1 - 0.5) = 0.6.$$

The code also stops collecting after 30 steps. This is a practical collection limit, not an extra terminal state in the underlying corridor. A nonterminal 30th transition still includes its next-state bootstrap. The next collection starts at 2. If a deadline were an actual task rule affecting optimal behavior, we would need to model the remaining time as part of the state.

The code calls `choose(nxt)` even on a terminal transition. That selected action is never executed and its value is multiplied by zero, so it does not affect the target. It does consume a random-number draw, affecting the later random sequence.

11. What converges, and what has been solved?

For Q-learning, the goal is a table approximating **optimal action values** Q^*. At the optimal fixed point in this corridor, the nonterminal rows are:

State	$Q^*(s, \text{left})$	$Q^*(s, \text{right})$	Greedy action
1	0	$0.95^2 = 0.9025$	Right
2	$0.95^3 = 0.857375$	0.95	Right
3	$0.95^2 = 0.9025$	1	Right

Why is moving left from 3 worth 0.9025 rather than zero? It leads to state 2; optimal continuation can turn around: `3 -> 2 -> 3 -> 4`, rewards `[0,0,1]`. A suboptimal first action need not make eventual success impossible.

For SARSA with fixed exploration, the goal is consistent action values for its continuing exploratory policy, not necessarily Q^*. Since the policy changes as the table changes, values and behavior improve together during training.

The code runs 500 episodes with constant learning rate 0.2. It does not use a convergence stopping test. A finite run can produce useful decisions without establishing mathematical convergence. Classical tabular convergence results need additional conditions, including sufficient state/action visits and suitable step sizes; optimal-control SARSA also needs exploration to become greedy under appropriate conditions. Here persistent exploration remains part of behavior.

What is solved in the demonstration? From observed rewards, both learners find table entries that favor right in states 1–3. The extracted greedy policy reaches the goal by the shortest route. Training behavior still sometimes explores.

12. Map the explanation to the script

Code	Meaning
`choose(state)`	Sample an epsilon-greedy behavior action
`nxt = state + ...`	Simulate the chosen move in the environment

Code	Meaning
`reward = float(nxt == 4)`	Environment returns 1 only on goal arrival
`next_action = choose(nxt)`	Select the next behavior action
`max(q[nxt])`	Q-learning's greedy next-value estimate
`q[nxt][next_action]`	SARSA's sampled next-value estimate
`target = reward + 0.95 * ...`	Construct the TD target
`q[state][action] += 0.2 * (target-q[state][action])`	Update one visited entry
`state, action = nxt, next_action`	Carry the sampled next action into the next move

Run from the course folder:

```
python3 03_reinforcement_learning/03_q_learning_sarsa.py
```

Each printed row contains `[left value, right value]`; the output shows only states 1–3. The assertion checks that right has the larger value in each row. It does not check that every number has converged or that SARSA equals Q^*.

13. Practice calculations

1. A nonterminal transition gives reward 0, next values `[0.3,0.9]`, and $\gamma = 0.95$. What is Q-learning's target? $0.95(0.9) = 0.855$**.**
2. If SARSA samples the first action, what is its target? $0.95(0.3) = 0.285$**.**
3. With old value 0.5 and $\alpha = 0.2$, what are the two updated values? **Q-learning: 0.571. SARSA: 0.457.**
4. If the same transition were terminal with reward 0, what would both targets be? **Zero, regardless of the next-state table.**
5. What would $\alpha = 1$ do? **Replace the old estimate with the latest target.**
6. Is $\gamma = 0.95$ the exploration probability? **No. Gamma discounts future rewards; epsilon controls random action selection; alpha controls update size.**

Try changing one of these three parameters at a time and compare several seeds. Explain both the learned numbers and the greedy actions. Next: DQN: replacing a Q-table with a neural network.

Companion code: `03_q_learning_sarsa.py`

03_reinforcement_learning/03_q_learning_sarsa.py

```python
"""Model-free tabular Q-learning and SARSA.
Q(s,a) estimates discounted future return after action a in state s.
Q-learning bootstraps with max_a Q(s',a): off-policy (greedy target).
SARSA bootstraps with the action actually sampled: on-policy.
A terminal transition has no future value. A time limit is different: it
usually still bootstraps because the underlying task did not truly terminate.
Exercise: add a cliff penalty to see how on/off-policy targets differ.
"""
import random


def train(method='q_learning', seed=7):
    rng = random.Random(seed)
    q = [[0.0, 0.0] for _ in range(5)]
    def choose(state):
        if rng.random() < 0.2:
            return rng.randrange(2)
        # Randomize ties to avoid systematically favoring action zero.
        best = max(q[state])
        return rng.choice([a for a in (0, 1) if q[state][a] == best])
    for _ in range(500):
        state, action = 2, choose(2)
        for step in range(30):
            nxt = state + (1 if action else -1)
            terminated = nxt in (0, 4)
            reward = float(nxt == 4)
            next_action = choose(nxt)
            bootstrap = max(q[nxt]) if method == 'q_learning' else q[nxt][next_action]
            target = reward + 0.95 * (not terminated) * bootstrap
            q[state][action] += 0.2 * (target-q[state][action])
            if terminated:
                break
            state, action = nxt, next_action
            # Ending at 30 steps truncates collection, not the Bellman target.
    return q


if __name__ == '__main__':
    for method in ('q_learning', 'sarsa'):
        q = train(method)
        print(method, 'Q values:', q[1:4])
        assert all(row[1] > row[0] for row in q[1:4])
```

3.4

DQN and Double DQN: learning action values with a neural network

Companion code: 04_dqn.py. Requires PyTorch. Prerequisites: MDPs, return, and value iteration, Q-learning and SARSA, and neural network training. Read this explanation before running the code.

1. What problem are we solving now?

The script keeps exactly the same corridor as the Q-learning lesson, so that the only new ingredient is the neural network:

```
      0          1          2          3          4
[END] <--> [room] <--> [START] <--> [room] <--> [GOAL]
 reward 0                                      reward +1
```

Action `0` is left and action `1` is right. Entering state 4 gives reward 1; every other move gives 0. Entering 0 or 4 ends the episode. The discount is $\gamma = 0.95$, and the goal is again a greedy rule that moves right in states 1–3.

In lesson 03 we stored one number per state/action pair in a table `q[state][action]`. That works for 5 states. It does not work for an Atari screen, a robot's joint angles, or any large or continuous observation: there are too many possible inputs to give each one its own row, and most would never be visited even once.

DQN (deep Q-network) replaces the table with a function $Q_\theta(s, a)$ computed by a neural network with parameters θ. The network reads an encoding of the state and outputs one estimate per action. Because all states share the same weights, a training step on one state can also change the predictions for other states. That **generalization** is the point, but it also combines three things that are individually fine and jointly risky: bootstrapped targets, function approximation, and off-policy data. Stability is no longer as simple as in the tabular case, which is why DQN adds a replay buffer and a target network.

The corridor is far too easy to *need* a network. It is used so you can check every number by hand and isolate what neural approximation changes.

2. Notation

Symbol	Meaning	Example / Code
s, a, r, s', d	One transition: state, action, reward, next state, terminated flag	`(state, action, float(nxt == 4), nxt, terminated)`
$x(s)$	Network input for state s: a one-hot vector of length 5	`features[state]`, e.g. `features[2] = [0,0,1,0,0]`

Symbol	Meaning	Example / Code
$Q_\theta(s, \cdot)$	**Online** network: outputs `[Q(s,left), Q(s,right)]`	`net(features[s])`
θ	Online network parameters, trained by Adam	`net.parameters()`
$Q_{\bar{\theta}}(s, \cdot)$	**Target** network: a lagged copy used to build targets	`target(features[ns])`
$\bar{\theta}$	Target parameters, copied from θ every 50 updates	`target.load_state_dict(net.state_dict())`
y	Regression target built from r and the next state	`targets`
γ	Discount factor, 0.95	`0.95*(1-done)*next_q`
ϵ	Exploration probability	`epsilon = max(0.05, 1-episode/150)`
$\mathcal{D}$	Replay buffer of past transitions, at most 2000	`replay = deque(maxlen=2000)`
B	Minibatch size, 32	`rng.sample(list(replay), 32)`

The bar over $\bar{\theta}$ means "the frozen copy", not an average.

3. What does the network compute?

```
net = nn.Sequential(nn.Linear(5, 32), nn.ReLU(), nn.Linear(32, 2))
```

Input: a one-hot vector of length 5. Hidden layer: 32 ReLU units. Output: two numbers, the estimated values of left and right. One forward pass gives the values of **all** actions in a state, which makes $\max_a$ and $\arg\max_a$ cheap.

With one-hot inputs, the first layer effectively looks up one column of its weight matrix per state, so this network is close to a table. It is still not exactly a table: the hidden bias, the ReLU, and the whole second layer are shared by every state. An update for state 2 therefore also nudges the outputs for states 1, 3, and even the terminal states 0 and 4.

Shapes in a minibatch of $B = 32$:

Expression	Shape	Meaning
`features[s]`	`[32, 5]`	One-hot rows for the sampled states
`net(features[s])`	`[32, 2]`	Both action values for each sampled state
`a[:, None]`	`[32, 1]`	The actions that were actually taken
`.gather(1, a[:, None])`	`[32, 1]`	Pick, in each row, the column of the taken action
`.squeeze(1)`	`[32]`	Drop the extra dimension: `predicted`
`r`, `done`, `targets`	`[32]`	Must also be one-dimensional

Worked `gather` example with a batch of 3:

```
net rows:  [[0.2, 0.5],    a = [1, 1, 0]   -> gather -> [0.5, 0.7, 0.3]
            [0.1, 0.7],
            [0.3, 0.0]]
```

Only the value of the action actually taken can be compared with the observed outcome; the other action's output receives no gradient from that row. If `predicted` had shape `[32,1]` and `targets` had shape `[32]`, subtraction would silently broadcast to `[32,32]`. Matching one-dimensional shapes avoid that bug.

4. Regression toward a detached Bellman target

Q-learning's update moved a table entry toward $y = r + \gamma \max_{a'} Q(s', a')$. DQN turns the same idea into a **regression problem**: make the network's prediction $Q_\theta(s, a)$ close to a label y. Classic DQN builds the label with the separate target network $Q_{\bar{\theta}}$:

$$\boxed{y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a')}$$

Part	Interpretation
r	Reward observed on this transition
$(1 - d)$	0 for a terminal transition, so no future value is added
$\max_{a'} Q_{\bar{\theta}}(s', a')$	Best estimated continuation according to the frozen network

The targets are computed inside `torch.no_grad()`. That is essential: y is treated as a fixed label. The update should move the prediction toward the target, not also move the target toward the prediction through the same backward pass.

Worked target example. Suppose three sampled transitions are

Transition	r	d	Target-net row $Q_{\bar{\theta}}(s', \cdot)$	y
$2 \to 3$, right	0	0	`[0.6, 0.9]`	$0 + 0.95(0.9) = 0.855$
$3 \to 4$, right	1	1	`[5.0, 7.0]`	$1 + 0.95(0)(7.0) = 1$
$1 \to 0$, left	0	1	`[2.0, 3.0]`	0

The rows for terminal states 0 and 4 can contain arbitrary numbers. Those states never appear as the *current* state `s` in replay, so the network is never trained on them. The mask $(1 - d)$ makes their values irrelevant. (In the default run, an instrumented copy of the script printed values near 0.5 for those rows, which is meaningless but harmless.)

5. The loss: Smooth L1 and gradient clipping

For each sampled transition let $u = Q_\theta(s, a) - y$ be the prediction error. The script uses `nn.functional.smooth_l1_loss`, whose default threshold is 1:

$$\boxed{\ell(u) = \begin{cases} u^2/2 & |u| < 1 \\ |u| - 1/2 & |u| \geq 1 \end{cases}}$$

and averages ℓ over the 32 transitions. Its derivative is u near zero and ± 1 for large errors, so one extreme TD target cannot produce an arbitrarily large gradient. This is the same idea as the Huber loss.

Continue the example with predictions `[0.5, 0.7, 0.3]` from section 3 and targets `[0.855, 1, 0]`:

Row	$u =$ prediction $-$ target	$\ell(u)$	$\partial(\text{mean loss})/\partial\,\text{prediction} = u/3$
1	$0.5 - 0.855 = -0.355$	$0.355^2/2 = 0.0630125$	-0.1183
2	$0.7 - 1 = -0.3$	0.045	-0.1
3	$0.3 - 0 = 0.3$	0.045	$+0.1$

Mean loss $= (0.0630125 + 0.045 + 0.045)/3 \approx 0.0510$. Gradient descent raises predictions 1 and 2 and lowers prediction 3, each toward its target.

Large-error comparison: prediction 0.5, target 3.5, so $u = -3$. Smooth L1 gives loss $3 - 0.5 = 2.5$ and derivative -1. Plain $u^2/2$ would give loss 4.5 and derivative -3. The robust loss limits the push from a single surprising label.

After `loss.backward()`, `clip_grad_norm_(net.parameters(), 10.0)` rescales the whole parameter gradient if its norm exceeds 10. This is an additional bound on the size of one optimizer step, applied to parameters rather than to errors.

6. Why a target network? Watch the label move

Without a target network, the label is built from the same θ that is being updated. Raising $Q_\theta(2, R)$ also changes shared weights, which can raise $Q_\theta(3, \cdot)$, which raises the next label for $Q_\theta(2, R)$. The regression keeps chasing a target that runs away from it.

The script keeps a frozen copy:

```
target = copy.deepcopy(net).requires_grad_(False)
...
if updates % 50 == 0:
    target.load_state_dict(net.state_dict())
```

Between copies, labels for a given transition do not change, so for 50 updates the online network solves an ordinary, fixed regression problem. Then the target jumps to the improved network, and the next phase starts. Copying every update weakens this effect; copying very rarely makes targets stale and slows the backward spread of reward information you saw in lesson 03.

In the default run, an instrumented copy of the script counted 559 optimizer updates, so the target was refreshed $\lfloor 559/50 \rfloor = 11$ times.

7. Why replay? Reuse experience and break correlations

Consecutive transitions are strongly correlated: `2 -> 3 -> 2 -> 3 -> ...` Training only on the newest transition would fit a few states at a time and then overwrite them. A **replay buffer** stores past transitions and samples a random minibatch for each update.

In this script:

1. Every environment step appends one tuple to `replay` (capacity 2000; the oldest are dropped first when full).
2. Once `len(replay) >= 32`, **every** environment step also performs one gradient update on 32 transitions sampled without replacement by `rng.sample(list(replay), 32)`.
3. A transition can therefore be reused in many updates.

This reduces within-batch temporal correlation. It is not an unbiased sample of the current policy's state distribution: old transitions came from older, more random behavior. Q-learning targets are off-policy (they use $\max_{a'}$, not the behavior's next action), which is what makes learning from old data legitimate.

Script-specific caveat: in the default run the instrumented copy counted 590 stored transitions in total, so the 2000-transition capacity is never reached and nothing is ever evicted. On larger tasks the capacity matters.

8. Exploration schedule

```
epsilon = max(0.05, 1-episode/150)
```

Episode	ϵ	Behavior
0	1.0	Fully random
50	$1 - 50/150 \approx 0.667$	Mostly random
75	0.5	Half random
100	≈ 0.333	Mostly greedy
142	≈ 0.0533	Almost at the floor
143–199	0.05	Floor: 5% random actions

The greedy branch uses the **online** network, `net(features[state]).argmax()`. Each episode starts at state 2 and stops at a terminal state or after 30 steps. `terminated` masks bootstrap; the 30-step collection limit does not mark a transition as terminal (as in lesson 03, it is a collection limit, not a task rule).

9. Why Double DQN separates selection and evaluation

Maximizing noisy estimates favors actions whose errors happen to be positive. Tiny example: both actions in s' truly have value 0.5, but each estimate is independently 0.4 or 0.6 with equal probability. The max is 0.4 only if both are low (probability 1/4):

$$E[\max] = 0.25(0.4) + 0.75(0.6) = 0.55 > 0.5.$$

The target is biased upward even though each individual estimate is unbiased. Through bootstrapping, that bias can be copied into earlier states.

Double DQN **selects** the next action with the online network and **evaluates** it with the target network:

$$\boxed{a^* = \arg\max_{a'} Q_\theta(s', a'), \qquad y = r + \gamma(1-d)\, Q_{\bar{\theta}}(s', a^*)}$$

If the evaluator's errors were independent of the selector's, a lucky selection would no longer also be scored with its own lucky error. The online and target networks are not truly independent (the target is a lagged copy), so this **reduces** overestimation; it does not eliminate all estimation error.

Example from the original lesson: online next-state values are `[3,2]` and target values are `[1,4]`. Ordinary target-network DQN bootstraps from $\max(1, 4) = 4$. Double DQN selects action 0 from the online values, then evaluates it as 1 with the target network.

Redo the first row of section 4 with Double DQN. Suppose the online row for state 3 is `[0.8, 0.7]` while the target row is still `[0.6, 0.9]`:

Rule	Next action	Next value	Target y	Error u for prediction 0.5
DQN	$\arg\max$ of target row = right	0.9	0.855	-0.355
Double DQN	$\arg\max$ of online row = left	0.6	$0.95(0.6) = 0.57$	-0.07

Terminal rows are unchanged (both still give 1 and 0), so the three-row Smooth L1 loss becomes $(0.00245 + 0.045 + 0.045)/3 \approx 0.0308$ instead of 0.0510. Neither number is "correct"; the two rules define different regression labels.

10. Run either target rule

The default uses Double DQN. Run

```
python3 03_reinforcement_learning/04_dqn.py --vanilla-dqn
```

to use the original DQN target $r + \gamma(1-d)\max_a Q_{target}(s', a)$. Both branches share the same environment, replay buffer, optimizer, and target network update schedule, making the target-selection difference directly inspectable. Because the targets differ, the networks and therefore the sampled trajectories diverge after the first updates, so the two runs do not see identical data.

11. What should the learned values be?

The optimal action values from lesson 03 are the reference:

State	$Q^*(s, \text{left})$	$Q^*(s, \text{right})$	Greedy action
1	0	$0.95^2 = 0.9025$	Right
2	$0.95^3 = 0.857375$	0.95	Right
3	$0.95^2 = 0.9025$	1	Right

The script prints only greedy actions, not values. For checking, a scratch copy of the script that additionally printed `net(features)` gave, after training, approximately `[0.0, 0.903]`, `[0.858, 0.95]`, `[0.903, 1.0]` for states 1–3 in the default (Double DQN) run and `[0.0, 0.902]`, `[0.857, 0.95]`, `[0.903, 1.0]` with `--vanilla-dqn`. Both are close to Q^* here. This tiny deterministic task gives little room for overestimation, so it does not demonstrate Double DQN's advantage; that shows up on noisier tasks with many actions.

12. Map the explanation to the script

Code	Meaning
`net = nn.Sequential(nn.Linear(5, 32), nn.ReLU(), nn.Linear(32, 2))`	Online network Q_θ: 5 one-hot inputs, 2 action values
`target = copy.deepcopy(net).requires_grad_(False)`	Frozen target network $Q_{\bar{\theta}}$
`torch.optim.Adam(net.parameters(), lr=0.003)`	Only the online network is optimized
`replay = deque(maxlen=2000)`	Replay buffer $\mathcal{D}$
`features = torch.eye(5)`	One-hot state encodings
`epsilon = max(0.05, 1-episode/150)`	Linear exploration decay with floor 0.05
`rng.randrange(2) if rng.random() < epsilon else net(features[state]).argmax().item()`	Epsilon-greedy behavior from the online net
`replay.append((state, action, float(nxt == 4), nxt, terminated))`	Store (s, a, r, s', d)
`if len(replay) >= 32:`	Start updating once a minibatch is available
`zip(*rng.sample(list(replay), 32))`	Random minibatch without replacement
`net(features[ns]).argmax(-1, keepdim=True)`	Double DQN: online network **selects** a^*
`target(features[ns]).gather(1, next_actions).squeeze(1)`	Double DQN: target network **evaluates** a^*
`target(features[ns]).max(-1).values`	Vanilla DQN: target network selects and evaluates
`targets = r + 0.95*(1-done)*next_q`	Bellman target y, computed under `no_grad`
`net(features[s]).gather(1, a[:, None]).squeeze(1)`	Prediction $Q_\theta(s, a)$ for taken actions, shape `[32]`
`nn.functional.smooth_l1_loss(predicted, targets)`	Robust regression loss
`nn.utils.clip_grad_norm_(net.parameters(), 10.0)`	Bound the gradient norm
`if updates % 50 == 0: target.load_state_dict(net.state_dict())`	Refresh target every 50 updates
`net(features[1:4]).argmax(-1).tolist()`	Final greedy actions for states 1–3

The closing comment mentions **prioritized replay**, an extension not implemented here: sample large-TD-error transitions more often and use importance weights to correct the changed sampling distribution.

13. Run and investigate

Run from the course folder:

```
python3 03_reinforcement_learning/04_dqn.py
python3 03_reinforcement_learning/04_dqn.py --vanilla-dqn
```

Both commands print

```
greedy actions for states 1,2,3: [1, 1, 1]
```

meaning the greedy policy from the online network moves right in all three nonterminal states. The script contains **no assertion**: it prints the actions and you inspect them. It also does not print Q-values, losses, or returns, and it evaluates only the greedy actions, not the exploratory actions used during training.

Experiments: remove the target delay (copy every update) or replace the replay sample with only the newest transition, one change at a time, and compare several seeds. This easy corridor may still work after removing a stabilizer; that does not disprove its value on harder tasks.

References: DQN, Double DQN.

14. Practice calculations

1. A non-terminal transition has $r = 0$, target-network next row `[0.4, 0.7]`, $\gamma = 0.95$. What is the DQN target? $0.95(0.7) = 0.665$**.**
2. The online next row is `[0.9, 0.2]`. What is the Double DQN target? **Online selects left; target evaluates it:** $0.95(0.4) = 0.38$**.**
3. The transition $3 \to 4$ has $r = 1$, $d = 1$, and a target-network row `[8, 9]`. What is y? **1. The terminal mask removes the next-state value.**
4. Prediction 0.2, target 0.665. What is the Smooth L1 loss? $u = -0.465$**,** $\ell = 0.465^2/2 \approx 0.1081$**.**
5. Prediction 0, target 2.5. Smooth L1 loss and its derivative? $u = -2.5$**;** $\ell = 2.5 - 0.5 = 2$**; derivative** -1**.**
6. What is ϵ at episode 120? $1 - 120/150 = 0.2$**.**
7. If `net(features[s])` has shape `[32,2]`, what shape does `gather(1, a[:, None])` return? `[32,1]`**;** `squeeze(1)` **makes it** `[32]`**.**
8. Why does one update on state 2 change the output for state 1? **The hidden bias and second layer are shared by all states.**

Next: policy gradients, REINFORCE, and a learned baseline.

Companion code: `04_dqn.py`

03_reinforcement_learning/04_dqn.py

```python
"""Deep Q-networks: replay, target networks, and Double DQN.
A neural net replaces the Q table. Replay breaks temporal correlations.
A delayed target network reduces moving-target instability. Double DQN uses
the online net to SELECT an action and the target net to EVALUATE it.
Exercise: remove replay/target networks one at a time and compare seed variance.
"""
import argparse
import copy
import random
from collections import deque
import torch
from torch import nn


def main(double=True):
    rng = random.Random(7)
    torch.manual_seed(7)
    torch.set_num_threads(1)
    net = nn.Sequential(nn.Linear(5, 32), nn.ReLU(), nn.Linear(32, 2))
    target = copy.deepcopy(net).requires_grad_(False)
    optimizer = torch.optim.Adam(net.parameters(), lr=0.003)
    replay = deque(maxlen=2000)
    features = torch.eye(5)
    updates = 0
    for episode in range(200):
        state = 2
        epsilon = max(0.05, 1-episode/150)
        for step in range(30):
            with torch.no_grad():
                action = rng.randrange(2) if rng.random() < epsilon else
net(features[state]).argmax().item()
            nxt = state + (1 if action else -1)
            terminated = nxt in (0, 4)
            replay.append((state, action, float(nxt == 4), nxt, terminated))
            if len(replay) >= 32:
                s, a, r, ns, done = zip(*rng.sample(list(replay), 32))
                s, a, ns = map(lambda z: torch.tensor(z, dtype=torch.long), (s, a, ns))
                r, done = torch.tensor(r), torch.tensor(done, dtype=torch.float32)
                with torch.no_grad():
                    if double:
                        next_actions = net(features[ns]).argmax(-1, keepdim=True)
                        next_q = target(features[ns]).gather(1, next_actions).squeeze(1)
                    else:
                        next_q = target(features[ns]).max(-1).values
                    targets = r + 0.95*(1-done)*next_q
                predicted = net(features[s]).gather(1, a[:, None]).squeeze(1)
                loss = nn.functional.smooth_l1_loss(predicted, targets)
                optimizer.zero_grad(set_to_none=True)
                loss.backward()
                nn.utils.clip_grad_norm_(net.parameters(), 10.0)
                optimizer.step()
                updates += 1
                if updates % 50 == 0:
                    target.load_state_dict(net.state_dict())
            state = nxt
            if terminated:
                break
```

```
    with torch.no_grad():
        print('greedy actions for states 1,2,3:', net(features[1:4]).argmax(-1).tolist())
    # Prioritized replay is an extension: sample large TD errors more often,
    # but use importance weights to correct the changed sampling distribution.

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--vanilla-dqn', action='store_true', help='Use original DQN target instead of Double DQN')
    args = parser.parse_args()
    main(double=not args.vanilla_dqn)
```

3.5

Policy gradients, REINFORCE, and a learned baseline

Companion code: 05_reinforce_actor_critic.py. Requires PyTorch. Prerequisites: MDPs, returns, and values, Q-learning and SARSA, DQN, and tensors and autograd. Read this explanation before running the code.

1. What problem are we solving now?

Same corridor, same rewards, $\gamma = 0.95$:

```
      0          1          2          3          4
[END] <--> [room] <--> [START] <--> [room] <--> [GOAL]
 reward 0                                     reward +1
```

Q-learning and DQN learn **values** and then act by choosing the action with the largest value. A **policy-gradient** method instead learns the **action probabilities** directly. It parameterizes a policy $\pi_\theta(a \mid s)$ and adjusts θ so that actions which led to high return become more probable.

Why bother, when the value methods already work here?

- The policy is naturally stochastic, so exploration is built in.
- It extends directly to continuous actions (output a Gaussian mean, say), where $\max_a Q(s, a)$ would itself be an optimization problem.
- It is the family that PPO, and LLM fine-tuning with RL, belong to.

The script trains two small networks:

- an **actor** that outputs two logits per state, turned into a categorical distribution over left/right by softmax;
- a **critic** that outputs one number $V_\phi(s)$, a prediction of return used only as a **baseline**. The critic never chooses actions.

The goal of the run is a high probability of moving right in states 1–3.

2. Notation

Symbol	Meaning	Example / Code
$\pi_\theta(a \mid s)$	Probability of action a in state s	`Categorical(logits=actor(features[state]))`
θ	Actor parameters	`actor = nn.Linear(5, 2)`
z_{left}, z_{right}	The two logits; $\pi(\text{right}) = e^{z_R}/(e^{z_L} + e^{z_R})$	`actor(features[state])`
$V_\phi(s)$	Critic's estimate of return from s	`critic(features[state]).squeeze(-1)`

Symbol	Meaning	Example / Code
ϕ	Critic parameters	`critic = nn.Linear(5, 1)`
r_t	Reward after the action at step t	`rewards[t]` , 1 only on entering 4
G_t	Discounted return from step t to the end of the episode	`returns[t]`
$\hat{A}_t$	Advantage estimate $G_t - V_\phi(s_t)$	`advantage[t]`
$H(\pi_t)$	Entropy of the action distribution at step t	`distribution.entropy()`
$J(\theta)$	Expected return of the policy: what we want to increase	(not computed directly)

3. What does the actor look like at the start?

`actor = nn.Linear(5, 2)` on a one-hot input means the logits for state s are column s of the weight matrix plus a bias vector that **every state shares**. With `torch.manual_seed(7)` , the initial probabilities of moving right are

State	1	2	3
Initial $\pi(\text{right} \mid s)$	0.434	0.426	0.370

So the agent begins slightly biased toward left. The initial critic values are all negative (about -0.64, -0.43, -0.51 for states 1–3), although every true return is between 0 and 1. Both networks have to learn.

4. The log-probability trick on a one-step problem

Start with a single decision, like a bandit. Let $R(a)$ be the reward of action a. The expected reward is

$$J(\theta) = \sum_a \pi_\theta(a) R(a).$$

Its gradient is $\sum_a \nabla \pi_\theta(a) R(a)$. Using $\nabla \pi = \pi \nabla \log \pi$ (just the chain rule for log):

$$\boxed{\nabla J(\theta) = \sum_a \pi_\theta(a)\, \nabla \log \pi_\theta(a)\, R(a) = \mathbb{E}_{a\sim\pi}\big[\nabla \log \pi_\theta(a)\, R(a)\big]}$$

It is an **expectation under the policy**, so one sampled action gives an unbiased estimate $\nabla \log \pi_\theta(a) R(a)$ without differentiating through the environment. A high-reward sampled action becomes more probable; a poor one less probable.

For a softmax policy the score has a simple form: $\partial \log \pi(a) / \partial z_k = 1[k = a] - \pi(k)$.

Worked example. $\pi(\text{right}) = 0.4$, $R(\text{right}) = 1$, $R(\text{left}) = 0$. Look at the derivative with respect to z_R:

Sampled action	Probability	Score $\partial \log \pi(a) / \partial z_R$	Score $\times R(a)$
Right	0.4	$1 - 0.4 = 0.6$	0.6

Sampled action	Probability	Score $\partial \log \pi(a)/\partial z_R$	Score $\times R(a)$
Left	0.6	$0 - 0.4 = -0.4$	0

Average: $0.4(0.6) + 0.6(0) = 0.24$. Directly: $J = \pi_R$, and $\partial \pi_R / \partial z_R = \pi_R(1 - \pi_R) = 0.4(0.6) = 0.24$. They agree.

5. From one step to whole episodes

An episode's probability is a product of action probabilities (times environment probabilities that do not depend on θ), so its log is a sum of $\log \pi_\theta(a_t \mid s_t)$ terms. An action at time t cannot cause rewards that arrived before it, so its score term can be weighted by the **future** return only:

$$\boxed{G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = r_t + \gamma G_{t+1}}$$

`discounted_returns` computes this backwards. For rewards `[0, 0, 1]`:

Step (backwards)	Calculation	G_t
$t = 2$	$1 + 0.95(0)$	1
$t = 1$	$0 + 0.95(1)$	0.95
$t = 0$	$0 + 0.95(0.95)$	0.9025

Returns: `[0.9025, 0.95, 1]`. An episode ending at state 0 has all rewards zero and all returns zero. An episode is collected **completely** before any update, which is what makes this a **Monte Carlo** method.

Estimator convention: for the exact discounted start-state objective, the expected sum includes an outer γ^t as well as discounting inside G_t. Like many practical small implementations, this script instead averages per-step score terms with discounted returns and no explicit outer γ^t. That is its estimator convention, not a claim of exact equivalence to every choice of discounted objective.

6. Why subtract a baseline?

A return of 1 is impressive in a state normally worth 0.1, but disappointing in a state normally worth 5. Without a reference point, every positive return pushes its action up, and only the *size* of the push tells good from great. Subtract a state-dependent baseline:

$$\boxed{\hat{A}_t = G_t - V_\phi(s_t)}$$

For a baseline that does not depend on the sampled action:

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a \mid s)\, V(s)] = V(s) \sum_a \nabla \pi(a \mid s) = V(s) \nabla \sum_a \pi(a \mid s) = V(s) \nabla 1 = 0.$$

So subtracting it does not change the expected gradient at a fixed state, while it can greatly reduce variance.

Worked example. Repeat section 4 with baseline $b = 0.4$ (the true expected reward):

Sampled action	Probability	Score w.r.t. z_R	$R - b$	Product
Right	0.4	0.6	0.6	0.36
Left	0.6	-0.4	-0.4	0.16

	Mean	$E[x^2]$	Variance
No baseline (values 0.6, 0)	0.24	$0.4(0.36) = 0.144$	$0.144 - 0.0576 = 0.0864$
Baseline 0.4 (values 0.36, 0.16)	$0.144 + 0.096 = 0.24$	$0.4(0.1296) + 0.6(0.0256) = 0.0672$	$0.0672 - 0.0576 = 0.0096$

Same mean, nine times smaller variance. Notice the left action now also pushes z_R **up**: left was worse than average, so the update makes it less likely.

The critic is trained to approximate returns so that it can play the role of b. It does not need to be accurate for the expected gradient to stay unbiased; an accurate critic just reduces variance more.

7. Losses and gradient flow

The script minimizes, over the T steps of one episode:

$$\boxed{L_{actor} = -\frac{1}{T}\sum_t \log \pi_\theta(a_t \mid s_t)\ \text{stopgrad}(\hat{A}_t)}$$

$$L_{critic} = \frac{1}{T}\sum_t (G_t - V_\phi(s_t))^2, \qquad \boxed{L = L_{actor} + 0.5 L_{critic} - 0.01\frac{1}{T}\sum_t H(\pi_t)}$$

Term	Purpose
Minus sign in L_{actor}	Optimizers minimize; minimizing $-\log \pi \cdot \hat{A}$ raises $\log \pi$ when $\hat{A} > 0$
`advantage.detach()`	Stop-gradient: the actor loss must not change the critic
$0.5 L_{critic}$	Squared-error regression of V_ϕ toward observed returns
$-0.01 H$	Entropy bonus: rewards keeping some randomness, encouraging exploration

Without `detach()`, the actor loss would also send gradients into the critic through $\hat{A}$, encouraging it to manipulate the baseline rather than just predict returns. Too large an entropy weight keeps the policy too random.

One-term example: advantage 0.6 and selected-action probability 0.25 give $-\log(0.25)(0.6) \approx 0.8318$. Raising that probability to 0.5 reduces the term to $-\log(0.5)(0.6) \approx 0.4159$. For negative advantage the incentive reverses: the loss falls when the probability falls.

8. Trace one update by hand

To see every number, take the initial networks from section 3 and **suppose** the first episode were $2 \to 3 \to 4$ (right, right). This is a chosen example, not the episode the seeded script actually samples first.

Collected data.

t	s_t	$\pi(\text{right} \mid s_t)$	$\log \pi(a_t \mid s_t)$	$H(\pi_t)$	$V_\phi(s_t)$	r_t
0	2	0.4260	-0.8534	0.6821	-0.4310	0
1	3	0.3699	-0.9946	0.6589	-0.5077	1

Returns and advantages. $G = [0.95, 1]$, so $\hat{A}_0 = 0.95 - (-0.4310) = 1.3810$ and $\hat{A}_1 = 1 - (-0.5077) = 1.5077$. Both are positive: reaching the goal was much better than the (pessimistic) critic predicted.

Losses.

$$L_{actor} = -\tfrac{1}{2}[(-0.8534)(1.3810) + (-0.9946)(1.5077)] = \tfrac{1}{2}(1.1785 + 1.4995) \approx 1.3390$$

$$L_{critic} = \tfrac{1}{2}(1.3810^2 + 1.5077^2) = \tfrac{1}{2}(1.9071 + 2.2733) \approx 2.0902, \qquad \bar{H} = \tfrac{1}{2}(0.6821 + 0.6589) \approx 0.6705$$

$$L = 1.3390 + 0.5(2.0902) - 0.01(0.6705) \approx 2.3774.$$

Actor gradient. Using $\partial \log \pi / \partial z_R = 1 - \pi_R$ for a right action, the gradient of L_{actor} with respect to state 2's right logit is $-\tfrac{1}{2}(1.3810)(1 - 0.4260) \approx -0.3964$, and for state 3 it is $-\tfrac{1}{2}(1.5077)(1 - 0.3699) \approx -0.4750$. Left logits get the opposite signs. Gradient descent therefore **raises** the right logits. The shared right-logit bias receives the sum, -0.8714. Weight columns for states 0, 1, 4 get zero gradient because their one-hot inputs were zero.

Critic gradient. $\partial(0.5 L_{critic})/\partial V_t = -\tfrac{1}{2}\hat{A}_t$: -0.6905 and -0.7539, so descent raises both values toward the returns.

One Adam step (lr 0.03). On its first step Adam moves each parameter with a nonzero gradient by almost exactly 0.03 against the gradient's sign. The right-minus-left logit gap for states 2 and 3 grows by $4 \times 0.03 = 0.12$ (weight and bias, on both logits); for state 1 only the shared bias moves, so the gap grows by 0.06. Running this in PyTorch gives:

State	$\pi(\text{right})$ before	after one step
1 (not visited)	0.434	0.449
2	0.426	0.456
3	0.370	0.398

State 1 changed even though it was never visited, because of the shared bias. That is generalization through shared parameters, the same effect as in DQN.

9. What kind of algorithm is this?

This is **Monte Carlo REINFORCE with a learned baseline**, not TD-based A2C: the advantage uses complete observed returns G_t, and the critic is never bootstrapped from $V(s_{t+1})$. Bootstrapped advantages appear in A2C/A3C and PPO and GAE.

The episode loop runs at most 30 steps. Here that horizon is treated as the endpoint of the finite collection task: there is no bootstrap at that endpoint, and the return of a 30-step episode that never reaches a terminal state is just its (zero) observed rewards. Since the actor and critic observe only position and not remaining time, this finite-horizon setup does not represent a fully time-aware optimal value function. It is a simplifying choice in this toy demonstration. For a continuing-task time limit, use bootstrap as taught in PPO and GAE.

Each episode is used for exactly **one** gradient step and then discarded. Unlike DQN there is no replay: the score-function estimator requires actions sampled from the current policy, so old episodes would need importance correction (one motivation for PPO's probability ratio).

10. Map the explanation to the script

Code	Meaning
`discounted_returns(rewards, gamma=0.95)`	Backward recursion $G_t = r_t + \gamma G_{t+1}$
`actor, critic = nn.Linear(5, 2), nn.Linear(5, 1)`	Logits for 2 actions; one value per state
`torch.optim.Adam(list(actor.parameters())+list(critic.parameters()), lr=0.03)`	One optimizer for both networks
`features = torch.eye(5)`	One-hot state encodings
`for episode in range(250):`	250 episodes, one update each
`for _ in range(30):`	At most 30 steps per episode
`distribution = Categorical(logits=actor(features[state]))`	Softmax policy $\pi_\theta(\cdot \mid s)$
`action = distribution.sample()`	Sample, do not take the argmax
`logs.append(distribution.log_prob(action))`	Store $\log \pi_\theta(a_t \mid s_t)$ with its graph
`entropies.append(distribution.entropy())`	Store $H(\pi_t)$
`values.append(critic(features[state]).squeeze(-1))`	Store $V_\phi(s_t)$ as a scalar
`state += 1 if action.item() else -1`	Move right for action 1, left for 0
`rewards.append(float(state == 4))`	Reward 1 only on entering the goal
`if state in (0, 4): break`	Terminal states end the episode
`advantage = returns-values`	$\hat{A}_t = G_t - V_\phi(s_t)$
`actor_loss = -(torch.stack(logs)*advantage.detach()).mean()`	L_{actor} with stop-gradient
`critic_loss = advantage.square().mean()`	L_{critic}

Code	Meaning
`loss = actor_loss + 0.5*critic_loss - 0.01*entropy`	Total loss L
`actor(features[1:4]).softmax(-1)[:, 1].tolist()`	Final $\pi(\text{right})$ for states 1–3

11. Run and experiment

Run from the course folder:

```
python3 03_reinforcement_learning/05_reinforce_actor_critic.py
```

With the supplied seed the output is

```
probability of right: [0.9134044647216797, 0.9554551839828491, 0.9237444400787354]
```

These are $\pi(\text{right} \mid s)$ for states 1, 2, 3, up from 0.434, 0.426, 0.370 at initialization. The policy remains stochastic (about 4–9% left), partly because the entropy bonus resists full determinism and partly because training stops after 250 episodes. The script contains **no assertion**; it prints the probabilities for inspection. Separately, `tests/test_learning_invariants.py` checks `discounted_returns([0., 0., 1.], 0.5)` equals `[0.25, 0.5, 1.]`.

Experiments:

- Replace `advantage.detach()` in the actor loss with `returns` to obtain a no-baseline REINFORCE comparison.
- Compare variability across several seeds, not one final probability.
- Change the entropy weight 0.01 to 0 and to 0.1 and compare final probabilities.
- Explain why subtracting a baseline that depends arbitrarily on the action would invalidate the zero-expectation argument in section 6.

12. Practice calculations

1. Rewards `[0, 0, 0, 1]`, $\gamma = 0.95$. Returns? **`[0.857375, 0.9025, 0.95, 1]`.**
2. $G_t = 0.9025$ and $V(s_t) = 0.5$. Advantage? 0.4025**.**
3. Selected-action probability 0.8, advantage -0.5. Actor loss term $-\log \pi \cdot \hat{A}$? $-(-0.2231)(-0.5) \approx -0.1116$**; minimizing it pushes the probability down.**
4. Softmax with $\pi(\text{right}) = 0.3$, right sampled, advantage 2. Gradient of $-\log \pi \cdot \hat{A}$ with respect to z_R? $-(1 - 0.3)(2) = -1.4$**; descent raises** z_R**.**
5. In section 4's example, use the action-dependent "baseline" $b(\text{right}) = 1$, $b(\text{left}) = 0$. Expected gradient? $0.4(0.6)(0) + 0.6(-0.4)(0) = 0 \neq 0.24$**: biased.**
6. Why is there no replay buffer here? **REINFORCE needs samples from the current policy; old episodes would need importance weights.**

Next: PPO and GAE: from rewards to a stable policy update.

Companion code: `05_reinforce_actor_critic.py`

`03_reinforcement_learning/05_reinforce_actor_critic.py`

```python
"""Policy gradients, REINFORCE, and a learned value baseline.
Policy pi(a|s) directly produces action probabilities. REINFORCE weights
log pi(a|s) by observed return. Subtracting V(s) reduces variance without
changing the expected policy gradient (when treated as a baseline).
This uses Monte Carlo returns with a learned critic, not TD-based A2C.
Exercise: replace advantage with raw returns and compare learning variance.
"""
import torch
from torch import nn
from torch.distributions import Categorical


def discounted_returns(rewards, gamma=0.95):
    result, total = [], 0.0
    for reward in reversed(rewards):
        total = reward + gamma*total
        result.insert(0, total)
    return torch.tensor(result)


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    actor, critic = nn.Linear(5, 2), nn.Linear(5, 1)
    optimizer = torch.optim.Adam(list(actor.parameters())+list(critic.parameters()),
lr=0.03)
    features = torch.eye(5)
    for episode in range(250):
        state, logs, values, rewards, entropies = 2, [], [], [], []
        # Here the 30-step horizon is PART OF THE DEFINED finite task: no
        # bootstrap at its end. A continuing-task time limit needs bootstrap.
        for _ in range(30):
            distribution = Categorical(logits=actor(features[state]))
            action = distribution.sample()
            logs.append(distribution.log_prob(action))
            entropies.append(distribution.entropy())
            values.append(critic(features[state]).squeeze(-1))
            state += 1 if action.item() else -1
            rewards.append(float(state == 4))
            if state in (0, 4):
                break
        returns = discounted_returns(rewards)
        values = torch.stack(values)
        advantage = returns-values
        # detach prevents the actor objective from modifying the baseline.
        actor_loss = -(torch.stack(logs)*advantage.detach()).mean()
        critic_loss = advantage.square().mean()
        entropy = torch.stack(entropies).mean()
        loss = actor_loss + 0.5*critic_loss - 0.01*entropy
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        print('probability of right:', actor(features[1:4]).softmax(-1)[:, 1].tolist())

if __name__ == '__main__':
    main()
```

3.6

PPO and GAE: from rewards to a stable policy update

Companion code: 06_ppo_and_gae.py. Read this explanation before running the code.

1. The problem we are solving

Imagine an agent in a five-position corridor, numbered 0 through 4. It starts at position 2 and chooses left or right. Position 0 ends the episode with zero reward; entering position 4 ends it with reward 1. Other transitions reward 0. We want the agent to learn to move right without supplying the correct action as a supervised label.

The policy produces probabilities, not a command that is always correct. Early on it may choose right only half the time. Learning should increase the probability of actions that led to better-than-expected outcomes. However, large changes based on a few lucky episodes can destroy useful behavior.

Two components address different parts of this problem:

- **GAE, generalized advantage estimation:** estimate how much better an action was than the policy's usual expected outcome at that state.
- **PPO, proximal policy optimization:** improve action probabilities using those estimates while reducing the incentive for excessively large updates.

GAE is an estimator, not an optimizer. PPO can use other advantage estimators; GAE can be used with other policy-gradient algorithms.

Prerequisites: returns and policy gradients, MDPs and values, and softmax and derivatives.

2. Notation: read the symbols before the equations

Symbol	Meaning	Code
s_t	State before taking action at step t	`state`
a_t	Sampled action, 0=left or 1=right	`action`
r_t	Reward from that action	`rewards[t]`
$\pi_\theta(a \mid s)$	Action probability under actor parameters θ	`Categorical(logits=actor(...))`
$V_\phi(s)$	Critic's estimate of expected future return	`critic(...)`
γ	Discount factor, here 0.95	`gamma`
λ	GAE trace-decay setting, here 0.95	`lam`
$\hat{A}_t$	Estimated advantage	`advantages[t]`
ρ_t	New action probability divided by old probability	`ratio[t]`

Symbol	Meaning	Code
ϵ	PPO clip width, here 0.2	bounds `0.8, 1.2`
d_t	1 if transition truly terminates the task	`terminated[t]`
e_t	1 if the collected episode ends, including truncation	`episode_end[t]`

$\mathbb{E}$ means an average weighted by probabilities. A hat, as in $\hat{A}$, means an estimate. ∇_θ means derivatives with respect to all entries of the actor's parameter vector.

3. Rewards, returns, values, and advantages

A reward describes one transition. A return includes future rewards:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

If rewards are `[0, 0, 1]` and $\gamma = 0.9$, the three returns are `[0.81, 0.9, 1.0]`. Earlier actions receive credit because they made the later reward possible. Discounting also prefers a sooner reward to an equally large later reward.

The state value is an average over possible continuations:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s].$$

The action value fixes the first action before averaging the continuation:

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a].$$

Their difference is the advantage:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s).$$

If the state normally yields return 0.4, but taking a particular action yields expected return 0.7, its advantage is 0.3. Positive advantage means better than the state baseline; negative advantage means worse. It does not mean the immediate reward itself must be positive or negative.

4. Why a policy gradient can improve behavior

The basic objective is expected return, $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G_0]$, where τ is a trajectory. For a probability p_θ:

$$\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta.$$

Substituting this identity into the derivative of an expectation gives the score-function estimator. The probability of a trajectory contains a product of action probabilities; taking its logarithm turns this product into a sum. For a discounted episodic objective, one exact reward-to-go form is:

$$\nabla_\theta J = \mathbb{E}\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t\right].$$

In practical PPO, rollout transitions are typically averaged uniformly, using discounted advantage estimates, rather than inserting explicit γ^t factors. This is the convention used in our code and its surrogate objective.

For a fixed state, subtracting an action-independent baseline does not change the expected score-function term:

$$\sum_a \pi_\theta(a \mid s)\nabla_\theta \log \pi_\theta(a \mid s)b(s) = b(s)\nabla_\theta \sum_a \pi_\theta(a \mid s) = 0.$$

That is the mathematical reason a value baseline can reduce noise without introducing baseline bias. Approximate bootstrapped advantage estimates can still introduce bias; that is a separate issue.

A sample with positive advantage rewards increasing its log-probability. A negative-advantage sample rewards decreasing it. The advantages are held fixed while differentiating the actor loss, so the actor cannot improve that loss by merely changing the critic's estimate.

5. First build a one-step advantage estimate

The one-step temporal-difference residual is:

$$\delta_t = r_t + \gamma(1 - d_t)V_\phi(s_{t+1}) - V_\phi(s_t).$$

It compares an updated estimate (reward plus discounted next value) with the old estimate at the current state. If the value function is accurate, averaging this residual conditional on (s_t, a_t) gives the action's advantage.

Example: $r_t = 0$, $V(s_t) = 0.4$, $V(s_{t+1}) = 0.7$, $\gamma = 0.9$, and no termination:

$$\delta_t = 0 + 0.9(0.7) - 0.4 = 0.23.$$

Only one observed transition is used, so this can be relatively low-variance, but it depends strongly on the accuracy of the critic's next-state estimate.

Code:

```
deltas = rewards + gamma * (1 - terminated) * next_values - values
```

6. Deriving GAE by combining several horizons

An n-step estimate uses more observed rewards before bootstrapping:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}) - V(s_t).$$

Away from terminal boundaries, expand the TD residuals and cancel intermediate value terms to obtain:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}.$$

GAE mixes these estimates with geometrically decreasing weights. In the infinite-horizon expression for $0 \le \lambda < 1$:

$$\hat{A}_t^{\mathrm{GAE}} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.$$

Why does the second equality hold? The residual δ_{t+l} occurs in every n-step estimate with $n \ge l+1$. Its total coefficient is:

$$\gamma^l (1-\lambda) \sum_{n=l+1}^{\infty} \lambda^{n-1} = \gamma^l \lambda^l.$$

For a finite rollout, compute the available residual sum backwards:

$$\hat{A}_t = \delta_t + \gamma\lambda(1-e_t)\hat{A}_{t+1}.$$

This recurrence is the `carry` variable in the code. Initialize the carry to zero at the end of the collected buffer. If that boundary is not terminal, the final residual still contains the next-state bootstrap value.

- $\lambda = 0$: use only the one-step TD residual.
- λ closer to 1: use longer observed reward sequences, usually reducing reliance on intermediate value predictions while increasing sampling noise.
- $\lambda = 1$ on a complete terminal episode: telescoping gives $G_t - V(s_t)$. On a truncated rollout, it gives a bootstrapped return minus the value instead.

These are bias/variance tendencies, not guarantees for every environment.

7. A full numerical GAE example

Use a three-step terminal episode. To keep the arithmetic easy, take $\gamma = 0.9$, $\lambda = 0.8$ (the script uses 0.95 for both).

t	Reward	$V(s_t)$	$V(s_{t+1})$	Terminal?	δ_t
0	0	0.4	0.5	No	$0 + 0.9(0.5) - 0.4 = 0.05$
1	0	0.5	0.6	No	$0 + 0.9(0.6) - 0.5 = 0.04$
2	1	0.6	ignored	Yes	$1 - 0.6 = 0.4$

Since $\gamma\lambda = 0.72$, work backwards:

$$\hat{A}_2 = 0.4, \qquad \hat{A}_1 = 0.04 + 0.72(0.4) = 0.328,$$

$$\hat{A}_0 = 0.05 + 0.72(0.328) = 0.28616.$$

The critic regression targets are:

$$\hat{R}_t = \hat{A}_t + V_{\mathrm{old}}(s_t) = [0.68616,\ 0.828,\ 1.0].$$

These are GAE-derived lambda-return targets, not necessarily the raw Monte Carlo returns `[0.81, 0.9, 1.0]`. `gae()` returns both arrays before the script normalizes the advantages for the actor.

8. Why the code needs two episode masks

Termination: the task actually ends. Future task value is zero.

Truncation: collection stops at a time limit even though the underlying continuing task could proceed. We usually bootstrap from the final observation. The next stored transition may belong to a newly reset episode, so we must not propagate its GAE residual backwards into the old one.

Thus:

Transition	Bootstrap next value?	Continue GAE into next stored transition?
Ordinary step	Yes	Yes, if available in buffer
True terminal	No	No
Time-limit reset	Yes	No

Example: a truncated step has reward 1, current value 0, next value 2, and $\gamma = 0.5$. Its residual is $1 + 0.5(2) = 2$. Do not add a following reset episode's reward of 100 to that estimate. Use the observation from **before resetting** for `next_values`.

If a finite deadline is genuinely part of the task definition, its endpoint is terminal instead. Remaining time may also need to be part of the state. See Gymnasium's explanation of time limits.

9. Why PPO stores old log-probabilities

Collect experience using a fixed rollout policy π_{old}. After one optimization step, the actor has changed, but we would like to reuse this batch for a few more steps.

For a fixed state, importance weighting gives:

$$\sum_a \pi_\theta(a \mid s) f(a) = \sum_a \pi_{\text{old}}(a \mid s) \frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} f(a),$$

assuming the old distribution has support wherever the new one needs it. Define the sampled-action ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} = \exp(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{old}}(a_t \mid s_t)).$$

That last expression is exactly:

```
ratio = (new_logs - old_logs).exp()
```

The simple surrogate is $\mathbb{E}_{\text{old}}[\rho_t \hat{A}_t]$. It reweights sampled actions, but does not fully correct the changing state visitation distribution. Consequently, it is a local approximation to policy improvement, not an exact new-policy return evaluator.

At the beginning of an update the policies agree, so ratios are 1. They change as optimization proceeds. `old_logs` must remain fixed throughout those updates.

10. Deriving the clipped objective's behavior

The PPO-Clip objective to **maximize** is:

$$L^{\text{clip}}(\theta) = \frac{1}{N} \sum_t \min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t \right).$$

The minimum chooses the more pessimistic of the ordinary and clipped surrogate. The sign of advantage determines which direction should stop receiving extra incentive.

Take $\epsilon = 0.2$. The clip interval is `[0.8, 1.2]`:

Advantage	Ratio	Ordinary term	Clipped term	Minimum
+2	1.1	2.2	2.2	2.2
+2	1.5	3.0	2.4	2.4
+2	0.5	1.0	1.6	1.0
-2	0.5	-1.0	-1.6	-1.6
-2	1.5	-3.0	-2.4	-3.0

For a good action ($\hat{A} > 0$), increasing its probability helps until the ratio exceeds 1.2; beyond that, this sample's clipped surrogate stops giving additional benefit. Decreasing a good action's probability remains penalized.

For a bad action ($\hat{A} < 0$), decreasing its probability helps until the ratio falls below 0.8. Increasing a bad action's probability remains penalized.

For example, if the old action probability was 0.4 and the new probability is 0.6, the ratio is 1.5. It is the **ratio**, not the probability, that is clipped. A ratio bound of 1.2 would correspond here to probability 0.48.

PyTorch minimizes losses, so the script negates this maximization objective:

```
policy_loss = -torch.minimum(unclipped, clipped).mean()
```

Clipping does not force all ratios to remain in the interval. Shared parameters, other samples, and optimizer momentum can still move them outside. PPO does not guarantee monotonic return improvement or a strict KL bound. The clipped surrogate is the central mechanism described in the PPO paper.

11. Train the critic and maintain exploration

The critic learns the fixed targets computed from the rollout:

$$L_V(\phi) = \frac{1}{N}\sum_t (V_\phi(s_t) - \hat{R}_t)^2.$$

The entropy of a discrete policy is:

$$H(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s)\log \pi(a \mid s).$$

For two equally likely actions, entropy is $\log 2 \approx 0.693$ nats. For a nearly deterministic policy it is near zero. An entropy bonus discourages premature collapse to one action.

The script minimizes:

$$L_{\text{total}} = -L^{\text{clip}} + 0.5L_V - 0.01\overline{H}.$$

The signs matter: decrease policy loss and value error, but increase entropy. The coefficients are teaching choices, not universal optimal settings.

The actor advantages are standardized:

$$\tilde{A}_t = \frac{\hat{A}_t - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + 10^{-8}}.$$

This controls their scale within an update. It is a practical heuristic, not required by the definition of GAE; finite-batch centering may change individual advantage signs. The **critic targets must use the unnormalized advantages**, which the code computes before this step.

12. Map the entire script to the math

Code section	Mathematical role
`features = torch.eye(5)`	One-hot state encoding; each state has its own input coordinate
`actor = nn.Linear(5, 2)`	Two logits per state; the categorical distribution applies softmax
`critic = nn.Linear(5, 1)`	One scalar value prediction per state
`with torch.no_grad()` collection	Store fixed old-policy statistics without a backward graph
64 collection steps	One rollout batch, potentially containing several episodes
`next_values` before reset	Bootstrap from the actual next state
`gae(...)`	TD residuals, backward advantage recursion, critic targets
Four update epochs	Reuse this batch briefly, always comparing with original old logs
`clip_grad_norm_(..., 0.5)`	Separate safeguard on gradient magnitude; not PPO ratio clipping
Next outer iteration	Collect fresh transitions using the newly updated actor

The example uses one full rollout batch per update epoch, separate linear actor/critic networks, and one shared optimizer. Larger implementations often use shuffled minibatches, separate learning rates, and additional diagnostics. This code has no KL early stopping or clipped value loss.

13. What to expect and what can go wrong

Run from the course directory:

```
python 03_reinforcement_learning/06_ppo_and_gae.py
```

The final probability of moving right should become high for the nonterminal states under the supplied seed. Intermediate "successful terminal transitions" are counts within 64 collection steps, not an evaluation success percentage. Shorter successful episodes can increase that count.

Common mistakes include recomputing `old_logs` during updates, letting gradients flow into advantage targets, failing to reset after truncation, using the reset observation for bootstrap, normalizing critic targets accidentally, or treating PPO clipping as ordinary gradient clipping.

Useful extra diagnostics: average evaluation return, entropy, fraction of ratios outside the clip interval, critic explained variance, and approximate KL. One sample-based KL diagnostic is the average of $\rho - 1 - \log \rho$ over old-policy samples; its population expectation equals $D_{KL}(\pi_{old} \| \pi_{new})$ at fixed states under matching support. It is still only a finite-sample estimate, not a hard constraint.

14. Exercises with checks

1. Set $\lambda = 0$ in the numerical example. Expected advantages: `[0.05, 0.04, 0.4]`.
2. Set $\lambda = 1$. Expected advantages: `[0.41, 0.4, 0.4]`; adding the old values gives the Monte Carlo returns `[0.81, 0.9, 1.0]`.
3. Use advantage -3 and ratio 0.6 with clip width 0.2. The selected surrogate is `min(-1.8, -2.4) = -2.4`; the minimized policy loss contribution is +2.4.
4. Explain why a positive reward can still produce negative advantage. Check: the outcome can be worse than an even higher expected baseline.
5. Compare one, four, and twenty update epochs across several seeds. Measure policy change and held-out returns rather than assuming more updates help.

For the estimator's original motivation see the GAE paper. For an implementation-oriented reference see Spinning Up PPO. The corridor arithmetic and code walkthrough above are worked specifically for this learning folder.

Companion code: `06_ppo_and_gae.py`

03_reinforcement_learning/06_ppo_and_gae.py

```python
"""Proximal Policy Optimization and generalized advantage estimation.
PPO reuses a fresh on-policy rollout for a FEW epochs, then collects new data.
ratio = exp(log_pi_new - log_pi_old). Clipping limits the surrogate incentive
for large policy changes; it is not a hard KL constraint or safety guarantee.
This is a complete tiny chain trainer; no Gymnasium dependency is required.
Exercise: vary clip=0.1/0.2/0.4 and log approximate KL and success rate.
"""
import torch
from torch import nn
from torch.distributions import Categorical


def gae(rewards, values, next_values, terminated, episode_end, gamma=0.95, lam=0.95):
    # terminated masks value bootstrap; episode_end (terminated OR truncated)
    # prevents the trace from crossing into a reset episode.
    deltas = rewards + gamma*(1-terminated)*next_values-values
    advantages = torch.zeros_like(rewards)
    carry = 0.0
    for t in reversed(range(len(rewards))):
        carry = deltas[t] + gamma*lam*(1-episode_end[t])*carry
        advantages[t] = carry
    return advantages, advantages+values


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    actor, critic = nn.Linear(5, 2), nn.Linear(5, 1)
    parameters = list(actor.parameters())+list(critic.parameters())
    optimizer = torch.optim.Adam(parameters, lr=0.02)
    features = torch.eye(5)
    state, age = 2, 0
    for iteration in range(35):
        states, actions, old_logs, rewards, values, next_values, terminals, ends = [], [],
[], [], [], [], [], []
        with torch.no_grad():
            for _ in range(64):
                distribution = Categorical(logits=actor(features[state]))
                action = distribution.sample()
                nxt = state + (1 if action.item() else -1)
                age += 1
                terminal, truncated = nxt in (0, 4), age >= 20
                states.append(state)
                actions.append(action.item())
                old_logs.append(distribution.log_prob(action))
                values.append(critic(features[state]).squeeze())
                # Use the final observation, BEFORE resetting, for bootstrap.
                next_values.append(critic(features[nxt]).squeeze())
                rewards.append(float(nxt == 4))
                terminals.append(float(terminal))
                ends.append(float(terminal or truncated))
                state = nxt
                if terminal or truncated:
                    state, age = 2, 0
        states, actions = torch.tensor(states), torch.tensor(actions)
        old_logs, values, next_values = map(torch.stack, (old_logs, values, next_values))
        advantages, returns = gae(torch.tensor(rewards), values, next_values,
```

```
                                    torch.tensor(terminals), torch.tensor(ends))
        advantages = (advantages-advantages.mean()) / (advantages.std(unbiased=False)+1e-8)
        # Targets, advantages and old log-probabilities stay fixed across epochs.
        for _ in range(4):
            distribution = Categorical(logits=actor(features[states]))
            new_logs = distribution.log_prob(actions)
            ratio = (new_logs-old_logs).exp()
            unclipped = ratio*advantages
            clipped = ratio.clamp(0.8, 1.2)*advantages
            policy_loss = -torch.minimum(unclipped, clipped).mean()
            value_loss = (critic(features[states]).squeeze(-1)-returns).square().mean()
            loss = policy_loss + 0.5*value_loss - 0.01*distribution.entropy().mean()
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            nn.utils.clip_grad_norm_(parameters, 0.5)
            optimizer.step()
        if iteration % 10 == 0:
            print('iteration:', iteration, 'successful terminal transitions:',
int(sum(rewards)))
    with torch.no_grad():
        print('right probabilities:', actor(features[1:4]).softmax(-1)[:, 1].tolist())

if __name__ == '__main__':
    main()
```

3.7

Monte Carlo: model-based planning and model-free learning

Companion code: 07_monte_carlo.py. Requires only Python. Prerequisites: MDPs, return, and value iteration and Q-learning and SARSA. You should be comfortable with sample averages (bandits), discounted returns, and epsilon-greedy action selection. No neural network is needed.

1. What problem are we solving now?

Use the same corridor as the earlier lessons:

```
   0          1          2          3          4
[END] <--> [room] <--> [START] <--> [room] <--> [GOAL]
 reward 0                                     reward +1
```

Left is action `0`; right is action `1`. Every move gives reward 0 except entering state 4, which gives 1. Entering 0 or 4 ends the episode. The discount is $\gamma = 0.95$.

Value iteration (lesson 02) used the full transition model. Q-learning and SARSA (lesson 03) learned from one transition at a time and **bootstrapped**: their target $r + \gamma Q(s', a')$ contained another estimate.

Monte Carlo (MC) methods take a different route: **play complete episodes, add up what actually happened, and average.** No estimate appears inside a target. The script shows two uses of that idea:

1. **Model-based rollout planning:** a planner that can *simulate* the corridor tries each first action many times in imagination and picks the better one.
2. **Model-free first-visit MC control:** a learner that can only *act* in the corridor plays real episodes and averages the returns it observed.

Both should prefer moving right, but they estimate different quantities, so their numbers differ. Understanding *why* they differ is the main goal.

2. Notation

Symbol	Meaning	Example / Code
s, a	State and action	`state`, `action`
r_t	Reward received for the move made at step t	`reward`
T	Number of moves in the episode	`len(trajectory)`
γ	Discount factor	`gamma=0.95`
G_t	Discounted return from step t to the end	`returns[t]`

Symbol	Meaning	Example / Code
π	Policy used to choose actions	random, or epsilon-greedy
$Q^\pi(s, a)$	True expected return of a in s, then following π	—
$\hat{Q}(s, a)$	Our estimate (a sample average)	`q[state][action]`
$N(s, a)$	How many returns have been averaged for (s, a)	`counts[state][action]`
ϵ	Probability of a uniformly random action	`epsilon=0.2`

3. What does Monte Carlo estimate?

Monte Carlo replaces an expectation with an average of sampled outcomes. For a complete episode the return from step t is

$$\boxed{G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}}$$

and the action value of a policy is its expected return:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

If we observe N independent returns $G^{(1)}, \ldots, G^{(N)}$ that all started with (s, a) and then followed π, their average estimates $Q^\pi(s, a)$:

$$\boxed{\hat{Q}^\pi(s, a) = \frac{1}{N} \sum_{i=1}^{N} G^{(i)}}.$$

Compare targets:

Method	Target for $Q(s, a)$	Contains another estimate?
Q-learning	$r + \gamma \max_{a'} Q(s', a')$	Yes (bootstraps)
SARSA	$r + \gamma Q(s', a')$	Yes (bootstraps)
Monte Carlo	complete observed G_t	No

Because MC uses only observed rewards, it avoids bootstrap error. The price: returns can have **high variance** (one lucky or unlucky episode moves a single sample a lot), and an update must **wait until the episode ends**. Continuing tasks with no terminal state need a different treatment; cutting an episode at an arbitrary step is not a true terminal state.

4. Compute one return backward

Suppose an episode from state 2 goes `right, left, right, right`:

```
2 --R--> 3 --L--> 2 --R--> 3 --R--> 4
rewards:   0         0         0         1
```

It is easiest to compute returns **backward** with $G_t = r_t + \gamma G_{t+1}$, starting from $G_T = 0$:

t	(s_t, a_t)	r_t	$G_t = r_t + 0.95\, G_{t+1}$
3	(3, R)	1	$1 + 0.95(0) = 1$
2	(2, R)	0	$0 + 0.95(1) = 0.95$
1	(3, L)	0	$0 + 0.95(0.95) = 0.9025$
0	(2, R)	0	$0 + 0.95(0.9025) = 0.857375$

The code does exactly this in its reversed loop:

```
returns, total = [0.]*len(trajectory), 0.
for t in reversed(range(len(trajectory))):
    total = trajectory[t][2] + gamma*total
    returns[t] = total
```

5. Model-based Monte Carlo: plan by simulating

Here "model-based Monte Carlo" means **rollout planning with a generative transition model**: a function we may call with any state and action to get a sampled next state and reward. It is not a single universally named RL algorithm, and it does not automatically mean Monte Carlo Tree Search (MCTS).

In the script, that model is `model_step(state, action)`, which returns `(next_state, reward, done)`. For each candidate first action a, the planner simulates N complete episodes in which every *later* action is chosen by a continuation policy π, then picks the first action with the larger average:

$$\hat{Q}^{\pi}(s, a) = \frac{1}{N} \sum_{i=1}^{N} G_0^{(i)}, \qquad a_{\text{plan}} = \arg\max_a \hat{Q}^{\pi}(s, a).$$

The script uses $s = 2$, $N = 10000$ (`samples=10000`), and a **uniformly random continuation** (`rng.randrange(2)`). Its estimates are therefore Q^{random}, not Q^*: "what is my expected return if I take a now and then wander randomly?"

Trace one simulated rollout

`simulate(2, first_action=0, rng)` might go left, then randomly right, right, right:

Loop pass	Move	`reward`	`total += discount*reward`	`done ?`	`discount` after
1	2 → 1 (left, the fixed first action)	0	0	no	0.95
2	1 → 2 (random right)	0	0	no	0.9025
3	2 → 3 (random right)	0	0	no	0.857375
4	3 → 4 (random right)	1	0.857375	yes	—

This rollout returns 0.857375. Another rollout that drifts into state 0 returns exactly 0. The planner averages 10000 such numbers per first action.

What should the average be?

We can check the planner against an exact calculation. Let $V(s)$ be the value of the random policy (each action with probability 0.5). Then

$$V(1) = 0.5(0) + 0.5(0.95)V(2), \qquad V(3) = 0.5(1) + 0.5(0.95)V(2),$$

$$V(2) = 0.5(0.95)V(1) + 0.5(0.95)V(3).$$

Substituting the first two into the third:

$$V(2) = 0.475\,[0.475V(2) + 0.475V(2) + 0.5] = 0.45125\,V(2) + 0.2375,$$

$$V(2) = \frac{0.2375}{0.54875} \approx 0.43280.$$

Then

$$Q^{\text{random}}(2, L) = 0.95V(1) = 0.45125V(2) \approx 0.19530,$$

$$Q^{\text{random}}(2, R) = 0.95V(3) = 0.45125V(2) + 0.475 \approx 0.67030.$$

The script prints `[0.19299884270477186, 0.6678522142658854]`. Both are within about 0.0025 of the exact values. That gap is ordinary sampling noise: the standard deviation of a single rollout return is roughly 0.34 (left) and 0.40 (right), so with 10000 samples the standard error is about $0.34/\sqrt{10000} \approx 0.0034$ and $0.40/\sqrt{10000} = 0.004$. To halve the noise you need four times as many samples.

The planner chooses `argmax` = right (action 1), and the script asserts `action == 1`.

Caveats about the planner

- A better continuation policy, replanning at every step, or a search tree produces a different (usually better) planner. Random continuation is the simplest choice, not the best.
- The script's simulator is the true environment dynamics. A *learned* model can replace it, but then model error is added, and more simulated samples cannot remove that error, only the sampling noise.
- The planner only answers the question for state 2; it does not build a table.
- Simulating is "free" in the sense that it spends no real environment interactions; it is not free in compute.

6. Model-free Monte Carlo control: learn by acting

Now remove the planner's privilege. The learner `mc_control(step, ...)` is given a `step` function and only ever calls it on the **state it is actually in** with the action it actually chose. It never asks "what if I had gone left instead?" That is what "model-free" means here. (The script passes `model_step` as `step` for convenience; the learner simply does not use it counterfactually.)

Each of the `episodes=20000` iterations does three things:

1. **Play** one episode from state 2, choosing epsilon-greedy actions from the current table, and store `(state, action, reward)` triples.
2. **Compute** every return backward (Section 4).
3. **Update** the running average of each (s, a) pair, but only at its **first visit** in that episode.

The update is a running mean:

$$\boxed{N(s,a) \leftarrow N(s,a) + 1, \qquad \hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \frac{G_t - \hat{Q}(s,a)}{N(s,a)}}.$$

This is the same incremental-average trick as the bandit lesson: it gives exactly the ordinary mean of all returns seen so far for that pair. For example, with observations 0.5 then 1: first estimate $0 + (0.5 - 0)/1 = 0.5$, then $0.5 + (1 - 0.5)/2 = 0.75$, which equals $(0.5 + 1)/2$.

7. First visit versus every visit

Return to the episode in Section 4. The pair (2, R) occurs at $t = 0$ and $t = 2$.

Pair	Visits	First-visit MC uses	Every-visit MC uses
(2, R)	$t = 0, t = 2$	0.857375 only	0.857375 **and** 0.95
(3, L)	$t = 1$	0.9025	0.9025
(3, R)	$t = 3$	1	1

The script implements first visit with a `seen` set: the second (2, R) is skipped. Every-visit MC is a valid alternative that this file does not implement.

If these were the first updates ($N = 0$ everywhere), after this episode the table would hold $\hat{Q}(2, R) = 0.857375$, $\hat{Q}(3, L) = 0.9025$, $\hat{Q}(3, R) = 1$.

8. Epsilon-greedy exploration and why the first success takes a while

The script chooses actions with

```
greedy = max(range(2), key=lambda a: q[state][a])
action = rng.randrange(2) if rng.random() < epsilon else greedy
```

With probability $1 - \epsilon = 0.8$ it takes the greedy action; with probability $\epsilon = 0.2$ it picks uniformly between 2 actions. With m actions, the greedy action therefore has probability $1 - \epsilon + \epsilon/m = 0.8 + 0.1 = 0.9$ and the other action $\epsilon/m = 0.1$.

Tie-breaking matters. Python's `max` returns the *first* maximiser, so when `q[state] == [0.0, 0.0]` the "greedy" action is **left**. At the start every entry is zero, so the learner walks left 90% of the time. Episodes that go left from 2 then left from 1 end in state 0 with return 0, and averaging zeros keeps the table at zero, so left stays "greedy".

With the script's seed (`random.Random(11)`), the table remains all zeros for the first 103 episodes; every one of them ends in state 0. Episode 104 is the first to reach the goal: `2 --R--> 3 --R--> 4`, two exploratory right moves in a row. At that point the first-visit counts were:

Pair	Count after episode 104	Returns averaged	New $\hat{Q}$
(3, R)	1	just 1	1
(2, R)	10	nine 0s, then 0.95	$0.95/10 = 0.095$

The value of (2, R) is only 0.095 because nine earlier tries of "right from 2" ended up failing under the left-leaning behavior of that time. This is a concrete example of a broader caveat: **the running average mixes returns from every policy the learner has used**, not just the current one.

Once $\hat{Q}(3, R) > \hat{Q}(3, L)$ and $\hat{Q}(2, R) > \hat{Q}(2, L)$, the greedy actions flip to right, and successes become frequent.

9. What does the learner converge toward?

The learner keeps $\epsilon = 0.2$ forever, so it estimates the values of its own **exploratory** policy (right with probability 0.9 once right is greedy), not the optimal greedy values Q^*. Solving the Bellman equations for "right with probability 0.9 in every room" gives the values in the middle column:

Pair	Values of the 0.9-right policy	Script's learned $\hat{Q}$	Q^* (lesson 03)
(1, L)	0	0.0	0
(1, R)	0.8728	0.8707	0.9025
(2, L)	0.7463	0.7082	0.857375
(2, R)	0.9379	0.9379	0.95
(3, L)	0.8728	0.8694	0.9025
(3, R)	1	1.0	1

Most learned numbers are close to the exploratory-policy values. (2, L) is noticeably lower, consistent with the mixing effect from Section 8: many of its returns were collected early, when the policy still leaned left. (1, L) and (3, R) are exact because those moves always end immediately with reward 0 and 1. The learned numbers are a finite-sample result for one seed, not a proof of convergence.

Classical convergence of MC control to Q^* needs additional conditions, for example **GLIE**: *greedy in the limit with infinite exploration* (every pair keeps being tried, while ϵ shrinks toward zero). The script does not decay ϵ, so it does not satisfy GLIE. Its greedy policy is nonetheless correct: right is preferred in states 1–3.

10. Compare all the targets

Method	Target	Model access	Bootstraps?
Value iteration	$\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$	Full model	Yes
MC rollout planner	Mean of simulated complete returns	Generative model	No (here)
Model-free MC	Complete observed G_t	None	No
SARSA	$r + \gamma Q(s', a')$	None	Yes
Q-learning	$r + \gamma \max_{a'} Q(s', a')$	None	Yes

Two different axes appear here: *does the method need a model?* and *does its target use another estimate?* Monte Carlo can appear on either side of the first axis, but it never bootstraps.

11. Map the explanation to the script

Code	Meaning
`model_step(state, action)`	Generative model: `nxt = state ± 1`, reward `float(nxt == 4)`, done if `nxt in (0, 4)`
`simulate(state, first_action, rng, gamma=0.95)`	One rollout: fixed first action, then `rng.randrange(2)` random continuation; returns G_0
`total += discount*reward`; `discount *= gamma`	Accumulate $\sum_k \gamma^k r_k$ forward
`rollout_plan(state=2, samples=10000)`	Average 10000 rollouts per first action, using `random.Random(7)`
`max(range(2), key=lambda a: q[a])`	$\arg\max_a \hat{Q}^{\text{random}}(2, a)$; first index wins ties
`mc_control(step, episodes=20000, gamma=0.95, epsilon=0.2)`	Model-free first-visit MC control, `random.Random(11)`
`q, counts = [[0., 0.] ...], [[0, 0] ...]`	$\hat{Q}$ table and $N(s, a)$ counts, 5 rows × 2 actions
`rng.randrange(2) if rng.random() < epsilon else greedy`	Epsilon-greedy behavior
`trajectory.append((state, action, reward))`	Store the episode; no update until it ends
reversed loop `total = trajectory[t][2] + gamma*total`	Backward return computation $G_t = r_t + \gamma G_{t+1}$
`seen` set	First-visit rule
`q[state][action] += (total-q[state][action])/counts[state][action]`	Running mean of returns

12. Run it

From the course folder:

```
python3 03_reinforcement_learning/07_monte_carlo.py
```

Output on our machine:

```
model-based random-continuation Q: [0.19299884270477186, 0.6678522142658854] chosen action: 1
model-free MC Q: [[0.0, 0.8707445185535772], [0.708197832642094, 0.9379273909513608],
[0.8694426217271125, 1.0]]
```

The first line is $[\hat{Q}^{\text{random}}(2, L), \hat{Q}^{\text{random}}(2, R)]$ and the planned action. The second line shows `[left, right]` rows for states 1–3 of the learned table. Both random generators are seeded, so the numbers are reproducible.

The assertions check only that the planner chooses right at state 2 and that right has the larger learned value in each of states 1–3. They do not check that any estimate equals Q^{π} or Q^{*}.

13. Practice calculations

1. An episode has rewards `[0, 0, 1]` with $\gamma = 0.95$. What are G_0, G_1, G_2? $G_2 = 1$**,** $G_1 = 0.95$**,** $G_0 = 0.9025$**.**
2. $\hat{Q}(s, a) = 0.6$ after $N = 4$ returns. A fifth return is 1.0. What is the new estimate? $0.6 + (1.0 - 0.6)/5 = 0.68$**.**
3. In episode `2 -R-> 3 -L-> 2 -L-> 1 -L-> 0`, what return does first-visit MC assign to (2, R), and what does every-visit MC do with (2, L)? **All rewards are zero, so (2, R) gets 0; (2, L) occurs once, so both variants use its single return, also 0.**
4. With $\epsilon = 0.2$ and two actions, what is the probability of the non-greedy action? $\epsilon/2 = 0.1$**.**
5. Why does the planner's $\hat{Q}(2, R) \approx 0.668$ differ from the learner's ≈ 0.938? **Different continuation policies: uniformly random versus epsilon-greedy that mostly moves right.**
6. If you want the planner's standard error to fall from about 0.004 to 0.002, how many samples do you need? **About four times as many: 40000.**

Exercise: evaluate a fixed random policy with model-free MC (set every action to `rng.randrange(2)`) and compare its estimates for state 2 against the rollout planner over five seeds. Then change `samples` and observe how the spread of the planner's estimates changes.

Reference: Sutton and Barto, chapters 4–6. Next: A2C and A3C: advantage actor-critic.

Companion code: `07_monte_carlo.py`

03_reinforcement_learning/07_monte_carlo.py

```python
"""Model-based rollout planning and model-free first-visit Monte Carlo control.
Only standard library required. Both use complete episodes, no bootstrapping.
"""
import random


def model_step(state, action):
    nxt = state + (1 if action else -1)
    return nxt, float(nxt == 4), nxt in (0, 4)


def simulate(state, first_action, rng, gamma=0.95):
    # Simulator/model access: fixed random continuation, not optimal control.
    total, discount, action = 0., 1., first_action
    while True:
        state, reward, done = model_step(state, action)
        total += discount*reward
        if done:
            return total
        discount *= gamma
        action = rng.randrange(2)


def rollout_plan(state=2, samples=10000):
    rng = random.Random(7)
    q = [sum(simulate(state, a, rng) for _ in range(samples))/samples
         for a in (0, 1)]
    return q, max(range(2), key=lambda a: q[a])


def mc_control(step, episodes=20000, gamma=0.95, epsilon=0.2):
    # Learner only calls step: it never reads transitions or simulator internals.
    rng = random.Random(11)
    q, counts = [[0., 0.] for _ in range(5)], [[0, 0] for _ in range(5)]
    for _ in range(episodes):
        state, trajectory = 2, []
        while state not in (0, 4):
            greedy = max(range(2), key=lambda a: q[state][a])
            action = rng.randrange(2) if rng.random() < epsilon else greedy
            nxt, reward, done = step(state, action)
            trajectory.append((state, action, reward))
            state = nxt
            if done:
                break
        returns, total = [0.]*len(trajectory), 0.
        for t in reversed(range(len(trajectory))):
            total = trajectory[t][2] + gamma*total
            returns[t] = total
        seen = set()
        for (state, action, _), total in zip(trajectory, returns):
            if (state, action) in seen:
                continue
            seen.add((state, action))
            counts[state][action] += 1
            q[state][action] += (total-q[state][action])/counts[state][action]
    return q

```

```
58
59 if __name__ == '__main__':
60     q, action = rollout_plan()
61     print('model-based random-continuation Q:', q, 'chosen action:', action)
62     learned = mc_control(model_step)
63     print('model-free MC Q:', learned[1:4])
64     assert action == 1
65     assert all(row[1] > row[0] for row in learned[1:4])
```

3.8

A2C and A3C: advantage actor-critic

Companion code: 08_a2c_a3c.py. Requires PyTorch. Prerequisites: MDPs and returns, REINFORCE and actor-critic, Monte Carlo returns, and tensors and autograd. You should know softmax, log-probabilities, entropy, Bellman targets, and why we `detach` targets (see the math primer, sections 2–6).

1. What problem are we solving now?

Use the same corridor again:

```
   0          1          2          3          4
[END] <--> [room] <--> [START] <--> [room] <--> [GOAL]
 reward 0                                     reward +1
```

Left is action `0`, right is action `1`; entering 4 gives reward 1 and ends the episode, entering 0 gives 0 and ends it. The discount in the script is $\gamma = 0.95$.

REINFORCE (lesson 05) waits for a whole episode and uses the complete Monte Carlo return, just like lesson 07. That is unbiased but noisy, and nothing can be learned until the episode ends.

Advantage actor-critic keeps two things in one model:

- an **actor**, a policy $\pi_\theta(a \mid s)$ that chooses actions;
- a **critic**, a value estimate $V_\phi(s)$ of expected future return.

After a short **segment** of at most n steps, the critic fills in the rest of the return, so the agent can update without waiting for the episode to finish.

A2C and A3C are two ways to run several copies of the environment ("workers") at once. **A2C** waits for all workers and makes one synchronized update. **A3C** lets each worker update the shared parameters whenever it is ready. The loss is the same; only the coordination differs.

2. Notation

Symbol	Meaning	Code
s_t, a_t, r_t	State, sampled action, reward at step t of a segment	`state`, `action`, `rewards[t]`
$\pi_\theta(a \mid s)$	Actor: softmax of the first two outputs	`out[:2].softmax(-1)`
$V_\phi(s)$	Critic: the third output	`out[2]`
n	Maximum segment length	`steps=5`
γ	Discount factor	`gamma=0.95`

Symbol	Meaning	Code
R_t	Bootstrapped n-step return target	`targets[t]`
$\hat{A}_t$	Advantage estimate $R_t - V_\phi(s_t)$	`advantage[t]`
$H(\pi(\cdot \mid s))$	Entropy of the action distribution	`entropies[t]`
c_v, c_H	Critic and entropy coefficients	`0.5`, `0.01`
$\mathrm{sg}(\cdot)$	Stop-gradient: treat as a constant	`.detach()` / `torch.no_grad()`

The model is a single `nn.Linear(5, 3)` applied to a one-hot state `torch.eye(5)[state]`: two actor logits and one value per state. Because the input is one-hot and there is no hidden layer, each state effectively has its own three numbers, and the actor rows and the value row are separate weights inside one layer. Larger implementations often share a hidden network between actor and critic; this script does not have shared hidden features.

3. The n-step return: real rewards, then the critic

For a segment starting at t that runs n steps without a terminal,

$$\boxed{R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n})}.$$

Three cases decide what goes at the end:

What ended the segment	Bootstrap value used	Why
True terminal (entered 0 or 4)	0	There is no future after termination
Segment hit `steps=5` in a nonterminal state	$V_\phi(s_{t+n})$	The episode continues; the critic estimates the rest
(Not in this script) time-limit truncation	V_ϕ of the real next state	A cutoff is not a terminal

If a true terminal occurs before n steps, the segment is simply shorter. The script implements this with `break` when `terminal` is true, and then starts the next segment from state 2 (a reset). After a nonterminal segment the worker continues from the state where it stopped.

Small arithmetic check. Rewards $[0, 1]$, $\gamma = 0.9$, endpoint value 0.5:

$$R_0^{(2)} = 0 + 0.9(1) + 0.9^2(0.5) = 0.9 + 0.405 = 1.305.$$

If that endpoint were a true terminal, the bootstrap is 0 and $R_0 = 0.9$. With current value $V(s_0) = 0.4$, the advantages are $1.305 - 0.4 = 0.905$ and $0.9 - 0.4 = 0.5$.

Every step in a segment gets its own target

The code computes targets with a backward recursion, starting from the bootstrap:

```
total = torch.tensor(0.) if terminal else model(torch.eye(5)[state])[2]
for reward in reversed(rewards):
    total = reward + gamma*total
    targets.insert(0, total)
```

So the first step uses an n-step return, the second an $(n-1)$-step return, and the last step a 1-step return. Example with `steps=5`, $\gamma = 0.95$, all five rewards zero, and a nonterminal end with $V(s_5) = 0.8$:

t	Steps of real reward	$R_t = 0.95^{5-t}(0.8)$
4	1	0.76
3	2	0.722
2	3	0.6859
1	4	0.651605
0	5	0.61902475

The segment length is a dial: `steps=1` is a pure one-step TD target (maximal reliance on the critic, low variance, more bias); a very long segment approaches the Monte Carlo return of REINFORCE (no critic in the target, higher variance).

4. The loss: actor, critic, entropy

$$\hat{A}_t = R_t - V_\phi(s_t)$$

$$L_{\text{actor}} = -\frac{1}{B}\sum_t \log \pi_\theta(a_t \mid s_t)\ \text{sg}(\hat{A}_t), \qquad L_{\text{critic}} = \frac{1}{B}\sum_t \big(V_\phi(s_t) - \text{sg}(R_t)\big)^2,$$

$$\boxed{L = L_{\text{actor}} + c_v L_{\text{critic}} - c_H\,\overline{H}(\pi_\theta)}$$

with B the number of steps in the segment, $c_v = 0.5$, $c_H = 0.01$, and $\overline{H}$ the mean entropy over the segment.

Term	What it does
L_{actor}	Raises $\log \pi$ of actions with positive advantage, lowers it for negative
L_{critic}	Regresses $V_\phi(s_t)$ toward the target R_t
$-c_H\overline{H}$	Rewards uncertainty in the policy, encouraging exploration

Why the stop-gradients? The targets are computed under `torch.no_grad()`, so the critic loss only moves $V_\phi(s_t)$, not the bootstrap value. In the actor term, `advantage.detach()` makes the advantage a fixed weight. Without it, minimising the actor loss would also push gradients into the critic, letting the "actor objective" improve by distorting value predictions rather than by changing the policy.

In the script the critic term is written `0.5*advantage.square().mean()`; since `targets` has no gradient, that equals $0.5\,L_{\text{critic}}$.

5. A fully worked first segment from the script

The trainers call `torch.manual_seed(7)` before `make_model()`. The initial model gives, for state 2, logits $(0.0165, 0.3132)$ and value -0.5725; for state 3, logits $(0.0145, 0.0766)$ and value -0.6445. The initial right probabilities are 0.5736 in state 2 and 0.5155 in state 3.

A2C worker 0 uses `torch.Generator().manual_seed(20)`. Its first segment samples right, right: `2 --R--> 3 --R--> 4`, which ends at a true terminal.

Targets (bootstrap 0 because of termination):

$$R_1 = 1 + 0.95(0) = 1, \qquad R_0 = 0 + 0.95(1) = 0.95.$$

Advantages:

t	s_t	$V(s_t)$	R_t	$\hat{A}_t = R_t - V(s_t)$	$\pi(\text{right} \mid s_t)$	$\log \pi$	Entropy
0	2	−0.5725	0.95	1.5225	0.5736	−0.5558	0.6823
1	3	−0.6445	1	1.6445	0.5155	−0.6625	0.6927

The entropy of $(0.4264, 0.5736)$ is $-(0.4264 \ln 0.4264 + 0.5736 \ln 0.5736) \approx 0.6823$, slightly below the two-action maximum $\ln 2 \approx 0.6931$.

Loss pieces ($B = 2$):

$$L_{\text{actor}} = -\tfrac{1}{2}[(-0.5558)(1.5225) + (-0.6625)(1.6445)] = \tfrac{1}{2}(0.8462 + 1.0895) \approx 0.9679,$$

$$0.5\, L_{\text{critic}} = 0.5 \cdot \tfrac{1}{2}(1.5225^2 + 1.6445^2) = 0.5 \cdot \tfrac{1}{2}(2.3180 + 2.7044) \approx 1.2557,$$

$$c_H \overline{H} = 0.01 \cdot \tfrac{1}{2}(0.6823 + 0.6927) \approx 0.0069.$$

$$L \approx 0.9679 + 1.2557 - 0.0069 = 2.2167.$$

Calling `rollout_loss` on this model with generator seed 20 returns 2.21669 and next state 2 (reset after terminal). Both advantages are positive because the critic started pessimistic (negative values), so this segment's contribution to the gradient pushes the probability of right up in states 2 and 3 and pushes both values up. (In A2C it is averaged with three other workers' segments before the optimizer step.)

6. A2C: synchronous updates

`train_a2c(updates=250, workers=4)`:

1. Seed the model with `torch.manual_seed(7)`; create Adam with `lr=0.025`.
2. Give each of the 4 workers its own generator (seeds 20, 21, 22, 23) and its own environment state, all starting at 2.
3. For each of the 250 updates, every worker runs `rollout_loss` with the **same current parameters**, the 4 losses are averaged, and one optimizer step is taken.

```
params v0 ─┬─ worker 0 segment ─┐
           ├─ worker 1 segment ─┤
```

```
            ├─ worker 2 segment ─┼─ mean loss → one step → params v1 ─┬─ ...
            └─ worker 3 segment ─┘
```

Because all gradients come from one parameter version, nothing is stale. Collection here is a plain sequential loop for simplicity; vectorised environments can run the same logic in parallel. Note one detail: the script averages the four per-segment *means*, so a 2-step segment and a 5-step segment get equal weight in the update.

7. A3C: asynchronous updates

`train_a3c(updates_per_worker=250, workers=4)` runs 4 worker functions on a `ThreadPoolExecutor`. Each worker repeats 250 times:

1. **Copy** the global parameters into its local model (under the lock).
2. **Collect** a segment and **backpropagate** locally (no lock held).
3. **Apply** its local gradient to the global model with one SGD step (`lr=0.03`, under the lock).

While worker 0 is in step 2, other workers can finish step 3, so by the time worker 0 applies its gradient, the global parameters may already be newer than the ones it used. Its gradient is **stale**. This is the central distinction from A2C. See the original A3C paper.

```
worker 0:  copy v3 ── collect + backward ──────────── apply → v6
worker 1:     copy v4 ── collect + backward ── apply → v5
worker 2:  ... (each step applies whenever ready; no barrier)
```

What the teaching implementation does and does not do:

Feature	In this script?
Concurrent CPU threads, local model copies, independent generators (seeds 40–43)	Yes
Lock around parameter reads and writes	Yes, prevents conflicting writes
Barrier that waits for all workers	No
Lock-free Hogwild updates, shared RMSProp from the paper	No (plain SGD)
Deterministic results	No: thread scheduling changes the update order
Meant as a speed benchmark	No; at scale, processes are often preferable to Python threads

Both trainers process the same number of segments (4 × 250 = 1000), but A2C makes 250 Adam steps on averaged gradients while A3C makes 1000 SGD steps on single segments. The optimizers and learning rates differ, so the two printed results are not a controlled comparison of the algorithms.

8. Map the explanation to the script

Code	Meaning
`nn.Linear(5, 3)`	Two actor logits and one value per one-hot state
`probs = out[:2].softmax(-1)`	$\pi_\theta(\cdot \mid s)$
`torch.multinomial(probs, 1, generator=generator)`	Sample a_t with a per-worker generator
`logs.append(probs[action].log())`	$\log \pi_\theta(a_t \mid s_t)$
`-(probs*probs.log()).sum()`	Entropy H
`values.append(out[2])`	$V_\phi(s_t)$, with gradient
`rewards.append(float(state == 4))`	Reward 1 only on entering the goal
`if terminal: break`	Shorter segment at a true terminal
`torch.tensor(0.) if terminal else model(...)[2]`	Bootstrap: 0 at terminal, $V_\phi(s_{t+n})$ otherwise
`total = reward + gamma*total` (reversed)	n-step targets R_t
`advantage = targets-torch.stack(values)`	$\hat{A}_t$
`-(torch.stack(logs)*advantage.detach()).mean()`	Actor loss
`0.5*advantage.square().mean()`	Critic loss, $c_v = 0.5$
`-0.01*torch.stack(entropies).mean()`	Entropy bonus, $c_H = 0.01$
`return loss, 2 if terminal else state`	Reset to 2 after a terminal, else continue
`torch.stack(losses).mean().backward()` + `optimizer.step()`	A2C: one synchronized update
`local.load_state_dict(global_model.state_dict())`	A3C: refresh local copy
`shared.grad = private.grad.detach().clone()` + `optimizer.step()`	A3C: apply a possibly stale gradient

9. Run it

From the course folder:

```
python3 03_reinforcement_learning/08_a2c_a3c.py
```

Two runs on our machine printed:

```
A2C right probabilities: [0.9461103677749634, 0.9666398763656616, 0.9518125057220459]
A3C right probabilities: [0.9217504262924194, 0.9439137578010559, 0.9112730026245117]
```

```
A2C right probabilities: [0.9461103677749634, 0.9666398763656616, 0.9518125057220459]
A3C right probabilities: [0.9246646165847778, 0.9479199647903442, 0.9141947627067566]
```

Each list is $\pi(\text{right} \mid s)$ for states 1, 2, 3 after training, up from the initial $0.5819, 0.5736, 0.5155$. A2C is reproducible across runs (sequential, seeded). A3C's digits change from run to run because thread timing changes the order of updates, exactly as Section 7 predicts. Exact digits may also differ across PyTorch versions.

The only assertion is `torch.isfinite(p).all()`: it checks that training did not produce NaN or infinity. It does **not** check that the policy learned to move right; read the printed numbers for that.

10. Practice calculations

1. Rewards $[0, 0, 1]$ end in a true terminal, $\gamma = 0.95$. What are the targets? $[0.9025, 0.95, 1]$**.**
2. Rewards $[0, 0]$, nonterminal end, $V(s_2) = 0.6$, $\gamma = 0.95$. Targets? $R_1 = 0.57$**,** $R_0 = 0.5415$**.**
3. With $V(s_0) = 0.7$ in question 2, what is $\hat{A}_0$, and does the actor push the sampled action's probability up or down? $0.5415 - 0.7 = -0.1585$**; down.**
4. What is the entropy of $\pi = (0.5, 0.5)$, and what does the entropy term add to the loss? $\ln 2 \approx 0.6931$**;** $-0.01(0.6931) \approx -0.0069$**.**
5. If you remove `.detach()` from the actor's advantage, which parameters get extra gradient? **The value output (critic row), through** $-V_\phi(s_t)$ **inside the advantage.**
6. Why can A3C produce different numbers on two runs with the same seeds? **Threads interleave differently, so gradients are applied to different parameter versions in a different order.**

Exercises: set `steps` to 1, then 10, and explain the change in bootstrap reliance and return variance. Remove the entropy bonus and compare five seeds. Do not infer algorithm superiority from one short corridor run.

Next: TRPO: constrain each policy update.

Companion code: `08_a2c_a3c.py`

03_reinforcement_learning/08_a2c_a3c.py

```python
"""Small CPU A2C/A3C trainers. A3C uses concurrent threads and locked SGD writes.
Local rollout computation is asynchronous; no batch barrier between workers.
"""
import copy
import threading
from concurrent.futures import ThreadPoolExecutor
import torch
from torch import nn


def make_model():
    return nn.Linear(5, 3)  # two actor logits and one value


def rollout_loss(model, state, generator, steps=5, gamma=0.95):
    logs, values, rewards, entropies = [], [], [], []
    terminal = False
    for _ in range(steps):
        out = model(torch.eye(5)[state])
        probs = out[:2].softmax(-1)
        action = torch.multinomial(probs, 1, generator=generator).item()
        logs.append(probs[action].log())
        entropies.append(-(probs*probs.log()).sum())
        values.append(out[2])
        state += 1 if action else -1
        rewards.append(float(state == 4))
        terminal = state in (0, 4)
        if terminal:
            break
    with torch.no_grad():
        total = torch.tensor(0.) if terminal else model(torch.eye(5)[state])[2]
        targets = []
        for reward in reversed(rewards):
            total = reward + gamma*total
            targets.insert(0, total)
        targets = torch.stack(targets)
    advantage = targets-torch.stack(values)
    loss = -(torch.stack(logs)*advantage.detach()).mean()
    loss += 0.5*advantage.square().mean()-0.01*torch.stack(entropies).mean()
    return loss, 2 if terminal else state


def train_a2c(updates=250, workers=4):
    torch.manual_seed(7)
    model = make_model()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.025)
    generators = [torch.Generator().manual_seed(20+i) for i in range(workers)]
    states = [2]*workers
    for _ in range(updates):
        losses = []
        for i in range(workers):
            loss, states[i] = rollout_loss(model, states[i], generators[i])
            losses.append(loss)
        optimizer.zero_grad()
        torch.stack(losses).mean().backward()
        optimizer.step()  # one synchronized update for all workers
    return model
```

```


def train_a3c(updates_per_worker=250, workers=4):
    torch.manual_seed(7)
    global_model = make_model()
    optimizer = torch.optim.SGD(global_model.parameters(), lr=0.03)
    lock = threading.Lock()

    def worker(index):
        with lock:
            local = copy.deepcopy(global_model)
        generator = torch.Generator().manual_seed(40+index)
        state = 2
        for _ in range(updates_per_worker):
            with lock:
                local.load_state_dict(global_model.state_dict())
            local.zero_grad()
            loss, state = rollout_loss(local, state, generator)
            loss.backward()  # other workers can update global parameters here
            with lock:
                optimizer.zero_grad()
                for shared, private in zip(global_model.parameters(), local.parameters()):
                    shared.grad = private.grad.detach().clone()
                optimizer.step()  # stale local gradient, no synchronization barrier

    with ThreadPoolExecutor(max_workers=workers) as pool:
        list(pool.map(worker, range(workers)))
    return global_model


if __name__ == '__main__':
    torch.set_num_threads(1)
    for name, train in [('A2C', train_a2c), ('A3C', train_a3c)]:
        model = train()
        with torch.no_grad():
            p = model(torch.eye(5)[1:4])[:, :2].softmax(-1)[:, 1]
        print(name, 'right probabilities:', p.tolist())
        assert torch.isfinite(p).all()
```

3.9

TRPO: constrain each policy update

Companion code: 09_trpo.py, a PyTorch equation lab on a three-action bandit. Prerequisites: policy gradients, PPO and importance ratios, and the math primer sections 2–6 (derivatives, expectation, logarithms, KL divergence) and section 8 (linear systems and curvature). You should know what a gradient, a Hessian, and a positive definite matrix are. The lab computes exact action expectations, so it needs no sampling and no critic.

1. What problem are we solving now?

A policy-gradient step says *which direction* improves the policy, but not *how far* it is safe to go. A step that looks small in parameter space can change action probabilities a lot, and a policy that suddenly collapses onto a bad action collects bad data for the next update.

Trust Region Policy Optimization (TRPO) answers "how far?" by measuring the step in terms of **how much the policy's action distribution changes**, using KL divergence, and refusing steps that change it by more than a budget δ.

The script uses the smallest possible setting: a **bandit** with one state and three actions:

Action	0	1	2
Reward	0	0.3	1

The policy is a softmax over three logits $\theta = (\theta_0, \theta_1, \theta_2)$, starting at $\theta = 0$, i.e. probabilities $(1/3, 1/3, 1/3)$. Because there is one state and the rewards are known, every expectation can be computed exactly. The goal is to watch TRPO move probability toward action 2 in 15 constrained steps.

2. Notation

Symbol	Meaning	Example / Code
θ	Policy parameters (logits)	`theta`, shape `(3,)`
$\pi_\theta(a)$	Action probability, $\mathrm{softmax}(\theta)_a$	`x.log_softmax(-1)`
π_{old}	Policy before the step, held fixed	`old_probs`, `old_logs`
$\hat{A}(a)$	Advantage: reward minus expected reward under π_{old}	`advantages`
$\rho_\theta(a)$	Importance ratio $\pi_\theta(a)/\pi_{old}(a)$	`ratios`
$L(\theta)$	Surrogate objective	`surrogate(x)`
D_{KL}	KL divergence $\pi_{old} \mid \pi_\theta$	`kl(x)`
g	Gradient of L at θ_{old}	`gradient`
F	Hessian of KL at θ_{old} (Fisher matrix)	`fisher`
η	Damping added to F	`damping=1e-3`

Symbol	Meaning	Example / Code
δ	KL budget (trust-region size)	`delta=0.01`
d	Proposed parameter step $\theta - \theta_{old}$	`step`

3. The constrained problem

TRPO maximises an old-policy surrogate while constraining average KL:

$$\boxed{\max_{\theta} \; \mathbb{E}_{s,a\sim\pi_{old}}[\rho_\theta(s,a)\hat{A}(s,a)] \quad \text{s.t.} \quad \mathbb{E}_{s\sim d_{old}} D_{KL}(\pi_{old}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s)) \le \delta}$$

with $\rho_\theta = \pi_\theta(a \mid s)/\pi_{old}(a \mid s)$. Read it piece by piece:

- The **surrogate** reweights old-policy experience by how much more (or less) likely the new policy makes each action. At $\theta = \theta_{old}$ every ratio is 1, so its gradient is the ordinary policy gradient.
- The **constraint** limits how different the new action distribution may be, averaged over states visited by the old policy.

In the bandit there is one state, so the "average KL" is just one KL, and the expectations are exact sums over the three actions:

$$L(\theta) = \sum_a \pi_{old}(a)\,\frac{\pi_\theta(a)}{\pi_{old}(a)}\hat{A}(a), \qquad D_{KL} = \sum_a \pi_{old}(a)\big[\log \pi_{old}(a) - \log \pi_\theta(a)\big].$$

These are exactly `surrogate` and `kl` in the code. Since the π_{old} factors cancel, $L(\theta) = \sum_a \pi_\theta(a)\hat{A}(a)$, which in this exact bandit equals the expected reward of π_θ minus that of π_{old}.

4. Local approximation and the step formula

Let $d = \theta - \theta_{old}$. Near θ_{old}:

- the surrogate is approximately linear: $L \approx g^T d$, with $g = \nabla L$;
- the KL is approximately quadratic: $D_{KL} \approx \frac{1}{2} d^T F d$, with $F = \nabla^2 D_{KL}$.

Why no linear KL term? KL is minimised (equal to zero) when the policies match, so its first derivative there is zero; the Hessian describes the local curvature. This Hessian is the **Fisher information matrix** of the policy, which is why the code names it `fisher`.

The local problem is

$$\max_d \; g^T d \quad \text{subject to} \quad \tfrac{1}{2} d^T F d \le \delta.$$

The best direction is $d = \beta F^{-1} g$ (the "natural gradient"). Plugging it into the constraint at equality, $\frac{1}{2}\beta^2\, g^T F^{-1} g = \delta$, gives

$$\boxed{d = \sqrt{\frac{2\delta}{g^T F^{-1} g}}\; F^{-1} g}.$$

Scalar check. $g = 0.2$, $F = 0.5$, $\delta = 0.01$: $F^{-1}g = 0.4$, $g^T F^{-1} g = 0.2(0.4) = 0.08$, scale $\sqrt{0.02/0.08} = 0.5$, so $d = 0.5(0.4) = 0.2$. Its quadratic KL is $\frac{1}{2}(0.2)^2(0.5) = 0.01 = \delta$, exactly on the boundary. The actual (nonlinear) KL can differ, which motivates the line search below.

5. Why damping? The softmax redundancy

Adding the same constant to every logit does not change a softmax. So moving θ along $(1, 1, 1)$ changes nothing, the KL is flat in that direction, and $F(1,1,1)^T = 0$: F **is singular**. The code therefore solves

$$(F + \eta I)\, x = g, \qquad \eta = 10^{-3},$$

with a dense `torch.linalg.solve`. Damping makes the matrix invertible. Large networks cannot form F at all; they use conjugate gradient with Hessian-vector products instead. The dense solve is appropriate only at toy scale.

The code then measures `curvature = direction@metric@direction`, which is $x^T(F + \eta I)x = g^T(F + \eta I)^{-1} g$, the damped version of $g^T F^{-1} g$. If it is essentially zero (gradient vanishes) the step is rejected.

6. Worked example: the first TRPO step

Old policy and advantages. At $\theta = 0$, $\pi_{old} = (1/3, 1/3, 1/3)$. Expected reward is $(0 + 0.3 + 1)/3 = 0.43333$, so

$$\hat{A} = (0 - 0.43333,\ 0.3 - 0.43333,\ 1 - 0.43333) = (-0.43333,\ -0.13333,\ 0.56667).$$

Gradient. For a softmax, $\partial L / \partial \theta_j = \pi_j(\hat{A}_j - \sum_a \pi_a \hat{A}_a)$. The advantages already average to zero under π_{old}, so $g_j = \pi_j \hat{A}_j$:

$$g = (-0.14444,\ -0.04444,\ 0.18889).$$

Fisher matrix. For softmax, $F = \mathrm{diag}(\pi) - \pi\pi^T$:

$$F = \begin{pmatrix} 0.22222 & -0.11111 & -0.11111 \\ -0.11111 & 0.22222 & -0.11111 \\ -0.11111 & -0.11111 & 0.22222 \end{pmatrix}.$$

Each row sums to zero, confirming the singular direction $(1, 1, 1)$.

Solve. Because g sums to zero, F acts on it like $\frac{1}{3}I$, so $(F + 0.001I)x = g$ gives $x = g / 0.334333$:

$$x = (-0.43204,\ -0.13293,\ 0.56497).$$

Scale to the boundary. $g^T x = 0.058519 / 0.334333 = 0.17503$, so the scale is $\sqrt{2(0.01)/0.17503} = \sqrt{0.11427} = 0.33803$ and

$$d = 0.33803\, x = (-0.14604,\ -0.04494,\ 0.19098).$$

Its quadratic KL is $\frac{1}{2} d^T F d \approx 0.00997$ (slightly under 0.01 because of damping).

Backtracking line search. The code tries $\theta_{old} + 0.5^j d$ for $j = 0, 1, \ldots, 14$ and accepts the first candidate whose **actual** KL is at most δ **and** whose surrogate improves on the old value (which is 0):

j	Step fraction	New probabilities	Actual KL	Surrogate	Accepted?
0	1	(0.2851, 0.3155, 0.3994)	0.010153	0.060709	No: KL > 0.01
1	0.5	(0.3091, 0.3251, 0.3658)	0.002517	0.030010	**Yes**

The quadratic model predicted 0.00997, but the real KL of the full step is 0.010153, just over the budget. Halving the step gives roughly a quarter of the KL (0.0025), as a quadratic would suggest. Expected reward rises from 0.43333 to 0.46334, and $0.46334 - 0.43333 = 0.03001$ matches the surrogate, as Section 3 predicted for this exact bandit.

If all 15 candidates fail, `trpo_step` returns the unchanged parameters with `accepted=False`. Rejection is a valid, safe outcome.

7. Watch the whole run

The script runs `trpo_step` 15 times with the same rewards. We instrumented a copy to log each accepted step:

Step	Actual KL	$\pi(0)$	$\pi(1)$	$\pi(2)$	Expected reward	$\|d\|$
1	0.002517	0.3091	0.3251	0.3658	0.4633	0.122
2	0.002510	0.2855	0.3156	0.3989	0.4936	0.121
3	0.002502	0.2625	0.3049	0.4326	0.5241	0.120
4	0.009975	0.2189	0.2805	0.5006	0.5848	0.238
8	0.009482	0.0827	0.1604	0.7569	0.8050	0.269
12	0.008692	0.0147	0.0503	0.9350	0.9501	0.430
15	0.007056	0.0009	0.0062	0.9930	0.9948	0.918

Things to notice:

- Every step was accepted. Steps 1–3 needed one halving; from step 4 on the full step already satisfied the KL budget.
- The KL per step stays at or below 0.01 throughout, as the assertion requires.
- The **Euclidean size** $\|d\|$ of the accepted step grows from about 0.12 to 0.92. As the policy becomes peaked, the same change in logits moves the probabilities less, so TRPO allows larger parameter steps for the same KL. This is the point of measuring steps in distribution space rather than parameter space.
- The logits stay centred (their sum remains 0), since every step is a multiple of a zero-sum direction.

8. Caveats: what the lab does and does not show

- **Exact expectations.** Real TRPO estimates the surrogate and KL from sampled trajectories; this lab sums over all three actions exactly.
- **No trainer.** A complete TRPO implementation also needs environment collection, advantage estimation (often GAE), and critic fitting. None of that is here.
- **Dense Hessian.** `torch.autograd.functional.hessian` and `torch.linalg.solve` are fine for 3 parameters, not for a neural network.
- **No unconditional guarantee.** TRPO's theory motivates monotonic improvement for a related, more conservative objective. Practical sampled average-KL TRPO does not inherit an unconditional monotonic-return guarantee. See the primary paper.
- **Relation to PPO.** PPO replaces this constrained second-order step with a clipped first-order surrogate. Its clipping does not enforce a hard KL bound; TRPO's line search does check one (for the estimated KL).

9. Map the explanation to the script

Code	Meaning
`trpo_step(theta, rewards, delta=0.01, damping=1e-3)`	One constrained update; returns `(theta, kl, accepted)`
`old_logs = theta.detach().log_softmax(-1)`	$\log \pi_{old}$, frozen
`advantages = (rewards-(old_probs*rewards).sum()).detach()`	$\hat{A}(a) = r(a) - \mathbb{E}_{\pi_{old}}[r]$
`ratios = (x.log_softmax(-1)-old_logs).exp()`	$\rho_\theta(a)$
`(old_probs*ratios*advantages).sum()`	Exact surrogate $L(\theta)$
`(old_probs*(old_logs-x.log_softmax(-1))).sum()`	Exact $D_{KL}(\pi_{old}\|\pi_\theta)$
`torch.autograd.functional.jacobian(surrogate, theta)`	g
`torch.autograd.functional.hessian(kl, theta)`	F
`fisher+damping*torch.eye(...)`	$F + \eta I$
`torch.linalg.solve(metric, gradient)`	$x = (F + \eta I)^{-1} g$
`curvature = direction@metric@direction`	$g^T (F + \eta I)^{-1} g$
`(2*delta/curvature).sqrt()*direction`	Step scaled to the quadratic KL boundary
`for j in range(15)`, `(0.5**j)*step`	Backtracking line search
`candidate_kl <= delta and surrogate(candidate) > before`	Acceptance test: actual KL and improvement
`return theta.detach().clone(), 0., False`	Reject: parameters unchanged

10. Run it

From the course folder:

```
python3 03_reinforcement_learning/09_trpo.py
```

Output on our machine:

```
TRPO action probabilities: [0.0008788435235311761, 0.006167071159787495, 0.9929540853166813]
```

The computation is deterministic (no random numbers), so the digits should repeat, up to tiny floating-point differences across PyTorch versions. The assertions check that each step's reported KL is at most $0.01 + 10^{-10}$ and that the final probability of the best action exceeds 0.8. They do not check that every step was accepted or that the policy is optimal.

11. Practice calculations

1. Scalar case: $g = 0.3$, $F = 2$, $\delta = 0.01$. What is the step d? $F^{-1}g = 0.15$**,** $g^T F^{-1} g = 0.045$**,** **scale** $\sqrt{0.02/0.045} \approx 0.667$**,** $d = 0.1$**.**
2. What is its quadratic KL? $\frac{1}{2}(0.1)^2(2) = 0.01$**.**
3. A two-action bandit with rewards $(0, 1)$ starts at $\theta = 0$. What are the advantages and the gradient g? $\pi = (0.5, 0.5)$**,** $\hat{A} = (-0.5, 0.5)$**,** $g = (-0.25, 0.25)$**.**
4. If a full step has actual KL 0.05 and KL scales roughly quadratically with step size, how many halvings until it is under $\delta = 0.01$? **Two: one halving gives about** $0.05/4 = 0.0125$ **, two give about** $0.05/16 \approx 0.0031$**.**
5. Why is F singular for a softmax policy? **Adding a constant to all logits leaves the probabilities unchanged, so** $F(1, \ldots, 1)^T = 0$**.**
6. Why does $\|d\|$ grow in later steps? **A peaked softmax is less sensitive to logit changes, so a larger parameter step fits in the same KL.**

Exercise: increase δ and log accepted step sizes, actual KL, and reward. Explain why the same Euclidean parameter step can change different policies by different amounts.

This completes the reinforcement learning section. Next: Reward models: learning scores from pairwise preferences.

Companion code: `09_trpo.py`

03_reinforcement_learning/09_trpo.py

```python
"""TRPO equation lab: exact expectations for a three-action bandit.
Dense Hessian solve replaces conjugate gradient, appropriate only at toy scale.
Includes a KL constraint and backtracking; no environment/critic trainer.
"""
import torch


def trpo_step(theta, rewards, delta=0.01, damping=1e-3):
    old_logs = theta.detach().log_softmax(-1)
    old_probs = old_logs.exp()
    advantages = (rewards-(old_probs*rewards).sum()).detach()

    def surrogate(x):
        ratios = (x.log_softmax(-1)-old_logs).exp()
        return (old_probs*ratios*advantages).sum()

    def kl(x):
        return (old_probs*(old_logs-x.log_softmax(-1))).sum()

    gradient = torch.autograd.functional.jacobian(surrogate, theta)
    fisher = torch.autograd.functional.hessian(kl, theta)
    metric = fisher+damping*torch.eye(theta.numel(), dtype=theta.dtype)
    direction = torch.linalg.solve(metric, gradient)
    curvature = direction@metric@direction
    if curvature <= 1e-16:
        return theta.detach().clone(), 0., False
    step = (2*delta/curvature).sqrt()*direction
    before = surrogate(theta).detach()
    for j in range(15):
        candidate = theta.detach()+(0.5**j)*step
        candidate_kl = kl(candidate).item()
        if candidate_kl <= delta and surrogate(candidate) > before:
            return candidate, candidate_kl, True
    return theta.detach().clone(), 0., False


if __name__ == '__main__':
    theta = torch.zeros(3, dtype=torch.float64)
    rewards = torch.tensor([0., 0.3, 1.], dtype=torch.float64)
    for _ in range(15):
        theta, kl, accepted = trpo_step(theta, rewards)
        assert kl <= 0.01+1e-10
    print('TRPO action probabilities:', theta.softmax(-1).tolist())
    assert theta.softmax(-1)[2] > 0.8
```

PART 4

Alignment

4.1

Reward models: learning scores from pairwise preferences

Companion code: 01_reward_model.py. Prerequisites: sigmoid, logs, and derivatives, cross-entropy and gradient descent, and supervised fine-tuning. Read this explanation before running the code. No transformer is needed here.

1. What problem are we solving now?

Supervised fine-tuning (SFT) copies demonstrations: "for this prompt, write this answer." Often we cannot easily write the single best answer, but a person can look at **two** answers and say which one is better. People can usually make that choice more consistently than they can assign a precise absolute quality score such as "7.3 out of 10."

A preference dataset therefore stores triples:

```
prompt x:          "What is 12 * 3?"
chosen   y_w:      "12 * 3 = 36."                          (preferred, the "winner")
rejected y_l:      "Multiplication is repeated addition, so the answer is 38."
```

The goal of a **reward model** $r_\phi(x, y)$ is to output one number per (prompt, response) so that preferred responses tend to get larger numbers. Later lessons use that number as the reward an RL algorithm tries to increase.

The scalar is a **learned proxy for preference**, not a direct measurement of truth or helpfulness. Everything in this lesson is about fitting that proxy.

2. What tiny dataset does the script use?

A real reward model reads prompt and response tokens. The script replaces each response with three hand-written numbers so the mathematics is visible:

Pair	Chosen `[correct, clear, verbose]`	Rejected `[correct, clear, verbose]`	Chosen minus rejected
1	`[1.0, 1.0, 0.3]`	`[0.0, 1.0, 0.8]`	`[1.0, 0.0, -0.5]`
2	`[1.0, 0.8, 0.2]`	`[0.2, 0.8, 0.7]`	`[0.8, 0.0, -0.5]`
3	`[0.8, 1.0, 0.5]`	`[0.1, 0.5, 0.5]`	`[0.7, 0.5, 0.0]`

These are synthetic labels, not a practical way to measure quality automatically. Look at the last column before reading further:

- Correctness is higher for the chosen answer in **every** pair.
- Clarity differs only in pair 3.
- Verbosity is **lower** for the chosen answer in pairs 1 and 2 and equal in pair 3.

So in this data, “shorter” is perfectly correlated with “preferred.” Nobody told the model that brevity matters; the data happens to contain that pattern. Keep this in mind when we look at the learned weights in section 9.

3. Notation

Symbol	Meaning	Example / Code
x	Prompt	not represented in the script
y_w, y_l	Preferred (winner) and rejected (loser) response	rows of `chosen`, `rejected`
$f(y)$	Feature vector describing a response	`[1., 1., 0.3]`
ϕ	Trainable reward-model parameters	`model.weight`, shape `[1,3]`
$r_\phi(x, y)$	Scalar reward score	`model(chosen)`, shape `[3,1]`
r_w, r_l	Scores of winner and loser	`chosen_scores`, `rejected_scores`
Δ	Score difference $r_w - r_l$	`chosen_scores-rejected_scores`
$\sigma(u)$	Sigmoid $1/(1+e^{-u})$	inside `logsigmoid`
N	Number of pairs	3
L_{RM}	Reward-model loss averaged over pairs	`preference_loss(...)`

In the script the reward is linear: $r_\phi(y) = \phi^\top f(y)$, a weighted sum of the three features. In a real reward model, those quality features are not provided by an oracle; a network must infer useful representations from prompt and response tokens.

4. How do we turn two scores into a probability?

The **Bradley–Terry model** assumes each response has a positive “strength” e^r. The chance the winner beats the loser is its share of the total strength:

$$P(y_w \succ y_l \mid x) = \frac{e^{r_w}}{e^{r_w} + e^{r_l}} = \frac{1}{1 + e^{-(r_w - r_l)}}.$$

(Divide numerator and denominator by e^{r_w} to get the second form.) Hence

$$\boxed{P(y_w \succ y_l \mid x) = \sigma(r_w - r_l) = \sigma(\Delta)}$$

The symbol $\succ$ reads “is preferred to.” The sigmoid converts any score difference into a number between zero and one:

$\Delta = r_w - r_l$	$\sigma(\Delta)$	Interpretation
-2	0.1192	Model thinks the labelled winner is probably worse
-1	0.2689	
0	0.5000	Coin flip: no preference
1	0.7311	
2	0.8808	Fairly confident the winner is better

$\Delta = r_w - r_l$	$\sigma(\Delta)$	Interpretation
3	0.9526	

Only the **difference** of scores appears. That fact matters in section 7.

5. The loss: negative log-likelihood of the observed choices

We want large probability for every human choice in the data. Taking the negative log and averaging over pairs gives

$$\boxed{L_{RM} = -\frac{1}{N}\sum_{i=1}^{N} \log \sigma(r_{w,i} - r_{l,i})}$$

This is ordinary binary cross-entropy where the label is always "the first response won" and the logit is Δ.

Δ	$\sigma(\Delta)$	Loss $-\log \sigma(\Delta)$
-1	0.2689	1.3133
0	0.5000	$\log 2 = 0.6931$
0.5	0.6225	0.4741
1	0.7311	0.3133
2	0.8808	0.1269
3	0.9526	0.0486

With reward difference 0, preference probability is 0.5 and loss is $\log 2$. With difference 2, probability is about 0.881 and loss about 0.127. The loss favors assigning the preferred response a larger score, and a wrongly ordered pair (negative Δ) is penalized heavily.

The code uses `logsigmoid`, which computes $\log \sigma(\Delta)$ stably without first producing a possibly tiny sigmoid output and then taking its log.

6. Worked example: score all three pairs by hand

Pick illustrative weights $\phi = [1, 1, -1]$ (reward correctness and clarity, penalize verbosity). These are **not** the script's weights; they are chosen to keep the arithmetic easy.

Step 1: scores. For chosen pair 1: $1(1) + 1(1) - 1(0.3) = 1.7$. For rejected pair 1: $1(0) + 1(1) - 1(0.8) = 0.2$.

Pair	r_w	r_l	Δ	$\sigma(\Delta)$	Loss
1	1.7	0.2	1.5	0.8176	0.2014
2	1.6	0.3	1.3	0.7858	0.2410
3	1.3	0.1	1.2	0.7685	0.2633

Step 2: average. $L_{RM} = (0.2014 + 0.2410 + 0.2633)/3 = 0.2352$.

You can also compute Δ directly from the difference column of section 2: pair 3 gives $1(0.7) + 1(0.5) - 1(0) = 1.2$. A linear reward only ever sees $f(y_w) - f(y_l)$.

7. Which way do the gradients push?

Let $\Delta = r_w - r_l$. Using $\frac{d}{du}\log\sigma(u) = 1 - \sigma(u)$:

$$\boxed{\frac{d[-\log\sigma(\Delta)]}{d\Delta} = \sigma(\Delta) - 1}$$

This is always negative, so gradient descent **increases** the reward difference. By the chain rule, the gradient with respect to the winner's score is $\sigma(\Delta) - 1$ and with respect to the loser's score is $1 - \sigma(\Delta)$. At $\Delta = 0$, the winner-score gradient is -0.5 and loser-score gradient is $+0.5$: push the winner up and the loser down equally.

Δ	$\sigma(\Delta) - 1$
-2	-0.8808
0	-0.5
1	-0.2689
2	-0.1192
3	-0.0474

If the model already predicts the preference confidently, the gradient magnitude shrinks. Badly ranked pairs get the strongest push.

For the linear model, $\partial\Delta/\partial\phi = f(y_w) - f(y_l)$, so

$$\nabla_\phi L_{RM} = \frac{1}{N}\sum_i [\sigma(\Delta_i) - 1]\,[f(y_{w,i}) - f(y_{l,i})].$$

With $\phi = [1, 1, -1]$, the factors $\sigma(\Delta_i) - 1$ are $-0.1824,\ -0.2142,\ -0.2315$. The correctness component is

$$\frac{(-0.1824)(1.0) + (-0.2142)(0.8) + (-0.2315)(0.7)}{3} = \frac{-0.5158}{3} = -0.1719.$$

The full gradient is about $[-0.1719,\ -0.0386,\ +0.0661]$. Descent subtracts it, so the correctness and clarity weights rise and the verbosity weight becomes more negative, exactly following the patterns in the difference column.

8. Absolute reward is not identified by these labels

Add 5 to every score in the worked example: pair 1 becomes 6.7 versus 5.2, still $\Delta = 1.5$. Adding the same prompt-dependent constant to both response scores leaves their difference unchanged, so the probability and loss are unchanged too:

$$\sigma\big((r_w + c) - (r_l + c)\big) = \sigma(r_w - r_l).$$

Therefore pairwise comparisons cannot determine a unique absolute reward zero. Only score **differences** are identified. This explains why the teaching linear layer is `nn.Linear(3, 1, bias=False)` : a shared bias would cancel from every pair anyway. A practical consequence: "this response scored 4.4" means nothing alone; only comparisons with other responses to the same prompt do.

On perfectly separable data, unregularized logistic preference fitting can continue increasing score margins without reaching finite optimal weights. The script's data is separable (every chosen response is more correct), so longer training keeps growing the weights. In a scratch run continuing the same setup to 1000 Adam steps (not something the script does), the weights grew to roughly `[5.19, 5.61, -4.69]` and the loss fell to about 0.0012. A finite number of steps, regularization, and held-out evaluation matter. Correct pair ranking alone does not establish well-calibrated reward magnitudes.

9. What does the script actually learn?

With `torch.manual_seed(7)` the initial weights are about `[0.040, -0.348, 0.184]` . Every initial Δ is slightly negative (`-0.052, -0.060, -0.146`), so the starting pair accuracy is 0 and the loss is 0.7371, a little above $\log 2$.

After 100 Adam steps with learning rate 0.05:

Quantity	Value
Learned weights `[correct, clear, verbose]`	`[2.6420, 2.4584, -2.3417]`
Pair margins Δ	`3.813, 3.284, 3.079`
Pair probabilities $\sigma(\Delta)$	`0.978, 0.964, 0.956`
Final loss	about 0.0345
Training pair accuracy	1.0

Two observations deserve care:

1. **Clarity got a large weight from a single pair.** Only pair 3 differs in clarity, yet the weight reached 2.46. Three pairs are far too few to estimate how much clarity really matters.
2. **Verbosity got a large negative weight.** The model learned "shorter is better" because the rejected answers in pairs 1 and 2 happen to be longer. That is a **shortcut**: a feature correlated with preference in this data, not necessarily a cause of it.

Test the shortcut with held-out pairs (computed with the learned weights above; they are not in the script):

Held-out chosen	Held-out rejected	Δ	$\sigma(\Delta)$	Verdict
`[1.0, 0.8, 0.9]` long, correct	`[0.0, 0.8, 0.2]` short, wrong	$2.642(1) - 2.342(0.7) = 1.003$	0.732	Correct ranking, reduced confidence

Held-out chosen	Held-out rejected	Δ	$\sigma(\Delta)$	Verdict
`[0.6, 0.8, 1.0]` long, mostly correct	`[0.1, 0.8, 0.2]` short, wrong	$2.642(0.5) - 2.342(0.8) = -0.552$	0.365	**Wrong** ranking

Perfect training accuracy did not guarantee the intended preference.

10. Why this matters for RLHF

The next lessons optimize a policy **against** the reward model. A policy is very good at finding inputs where a learned score is high for unintended reasons, such as length, formatting, or flattering phrasing. This is called **reward hacking** or reward over-optimization. If our toy reward model were used as a target, a policy could gain score just by becoming terse.

Held-out prompts, disagreement analysis, and adversarial evaluation are essential before using such a score as an optimization target. Evaluate true task outcomes separately from learned reward.

Caveat on scope: the script shows the Bradley–Terry loss on three fixed feature pairs. It does not show a transformer reward head, prompt handling, label noise, annotator disagreement, ties, or reward-hacking during policy training. Claims about those settings above are general background, not things the script tests.

11. Map the explanation to the script

Code	Meaning
`chosen = torch.tensor([[1., 1., 0.3], ...])`	Feature rows $f(y_w)$ for the three winners
`rejected = torch.tensor([[0., 1., 0.8], ...])`	Feature rows $f(y_l)$ for the three losers
`model = nn.Linear(3, 1, bias=False)`	Linear reward $r_\phi(y) = \phi^\top f(y)$; bias omitted because it cancels
`model(chosen)`, `model(rejected)`	Scores r_w, r_l, each of shape `[3,1]`
`chosen_scores-rejected_scores`	Score differences Δ_i
`-nn.functional.logsigmoid(...).mean()`	$L_{RM} = -\frac{1}{N}\sum \log \sigma(\Delta_i)$, computed stably
`torch.optim.Adam(model.parameters(), lr=0.05)`	Optimizer updating the shared scoring weights
`for _ in range(100):`	Fixed number of steps, no convergence test
`(model(chosen)>model(rejected)).float().mean()`	Fraction of training pairs ranked correctly
`model.weight.detach().tolist()`	Learned ϕ = `[correct, clear, verbose]` weights

Adam updates the same scoring function using both sides of every pair. The printed pair accuracy measures only these three training comparisons.

12. Run and investigate

Run from the course folder (PyTorch must be installed; see the course README):

```
python3 04_alignment/01_reward_model.py
```

Output from the provided seed:

```
training pair accuracy: 1.0
learned feature weights: [[2.6419734954833984, 2.4584295749664307, -2.3417065143585205]]
```

The script has no command-line flags and no assertion; it only prints. Accuracy 1.0 means all three training pairs are ordered correctly. It does not mean the reward is correct on new responses, and the exact weight values depend on the seed, learning rate, and step count.

Experiments:

1. Add examples where long **correct** answers are chosen over short incorrect ones. Does the verbosity weight move toward zero?
2. Create a held-out pair that breaks a training correlation, such as a short correct answer versus a long incorrect one (or the reverse, as in section 9). Observe whether the model learned the intended quality or a shortcut.
3. Increase the step count and watch the weights keep growing (section 8).

13. Practice calculations

1. Scores $r_w = 2.0$, $r_l = 0.5$. What is $P(y_w \succ y_l)$ and the loss? $\Delta = 1.5$**,** $\sigma(1.5) = 0.8176$**, loss** $= 0.2014$**.**
2. What is $\partial L / \partial \Delta$ for that pair? $0.8176 - 1 = -0.1824$**.**
3. Scores $r_w = 0.5$, $r_l = 1.0$. Probability and loss? $\sigma(-0.5) = 0.3775$**, loss** 0.9741**. The pair is ranked wrongly.**
4. Add 10 to both scores in question 1. What changes? **Nothing; Δ is still 1.5.**
5. With weights $[1, 1, -1]$, what is Δ for pair 2 of the script? $1(0.8) + 1(0) - 1(-0.5) = 1.3$**.**
6. Why does a bias term not help? **It adds the same constant to both scores and cancels in Δ.**

Next: KL-regularized reward optimization.

Companion code: `01_reward_model.py`

04_alignment/01_reward_model.py

```python
"""Learn a reward score from chosen/rejected response pairs.
Bradley-Terry: P(chosen wins) = sigmoid(r_chosen - r_rejected).
Only score differences are identified. The absolute reward zero is arbitrary.
Real reward models score prompt+response tokens; here features expose the math.
Exercise: add a spurious length feature and watch shortcut learning appear.
"""
import torch
from torch import nn


def preference_loss(chosen_scores, rejected_scores):
    return -nn.functional.logsigmoid(chosen_scores-rejected_scores).mean()


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    # Features mean [correctness, clarity, verbosity]; these are synthetic labels,
    # not a practical way to measure response quality automatically.
    chosen = torch.tensor([[1., 1., 0.3], [1., 0.8, 0.2], [0.8, 1., 0.5]])
    rejected = torch.tensor([[0., 1., 0.8], [0.2, 0.8, 0.7], [0.1, 0.5, 0.5]])
    model = nn.Linear(3, 1, bias=False)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.05)
    for _ in range(100):
        loss = preference_loss(model(chosen), model(rejected))
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
    print('training pair accuracy:', (model(chosen)>model(rejected)).float().mean().item())
    print('learned feature weights:', model.weight.detach().tolist())
    # Held-out prompts, disagreement analysis, and adversarial evaluation are
    # essential before using such a score as an optimization target.

if __name__ == '__main__':
    main()
```

4.2

RLHF and KL regularization: improve reward without unrestricted drift

Companion code: 02_rlhf_kl_penalty.py. Prerequisites: reward modeling, PPO and GAE, softmax and logs, and supervised fine-tuning. Read this explanation before running the code.

1. What problem are we solving now?

A common RLHF workflow has three stages:

```
SFT model  --->  reward model from preferences  --->  policy optimization (e.g. PPO)
 (imitate)          (previous lesson)                  (maximize reward, stay close)
```

The frozen **reference policy** is usually the SFT model. The policy being trained starts as a copy of it. If we simply told the policy "maximize the reward model's score," it could drift far from sensible language and exploit the reward model's mistakes (remember the verbosity shortcut in the previous lesson). So we add a penalty for moving away from the reference.

This script isolates **that objective** using three possible actions instead of text. It computes every expectation exactly. It is not a PPO rollout trainer or a complete autoregressive RLHF pipeline. PPO is the larger piece that would perform the optimization from sampled responses.

2. The tiny example: a three-action bandit

Think of three canned responses to one prompt:

Action i	Imagined response	Reference prob q_i	Reward r_i
0	bland, safe answer	0.6	0.0
1	decent answer	0.3	0.5
2	best-scoring answer	0.1	1.0

The reference (SFT) model mostly gives the bland answer. The reward model prefers action 2. The penalty strength is $\beta = 0.2$.

Under the reference, expected reward is $0.6(0) + 0.3(0.5) + 0.1(1) = 0.25$.

3. Notation

Symbol	Meaning	Example / Code
p_i or $\pi(i)$	Trainable policy probability of action i	`probs`
q_i or $\pi_{ref}(i)$	Frozen reference probability	`reference = [0.6, 0.3, 0.1]`

Symbol	Meaning	Example / Code
r_i	Fixed reward of action i	`reward = [0.0, 0.5, 1.0]`
β	KL penalty strength	`beta = 0.2`
z_i	Unconstrained logit; $p = \mathrm{softmax}(z)$	`logits`
$D_{KL}(p\|q)$	Forward KL from policy to reference	`kl`
$J(p)$	Penalized objective to **maximize**	`objective`
p^*	Exact maximizer of J	`exact_optimum`

4. Define the objective and the KL direction

Maximize expected reward minus a KL penalty:

$$\boxed{J(p) = \sum_i p_i r_i - \beta\, D_{KL}(p\|q)}, \qquad D_{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i}.$$

- $\sum_i p_i r_i$ is the expected reward when actions are sampled from p.
- $\log(p_i/q_i)$ is the **log-ratio**: positive when the policy likes action i more than the reference did, negative when less.
- KL is the policy-weighted average of those log-ratios.

Assume $q_i > 0$ for the allowed actions. KL is nonnegative and is zero when the distributions agree. It is directional: $D_{KL}(p\|q)$ generally differs from $D_{KL}(q\|p)$.

Worked example. Take a candidate policy $p = [0.2, 0.3, 0.5]$.

i	p_i	q_i	$\log(p_i/q_i)$	$p_i \log(p_i/q_i)$	$p_i r_i$
0	0.2	0.6	$\log(1/3) = -1.0986$	-0.2197	0
1	0.3	0.3	0	0	0.15
2	0.5	0.1	$\log 5 = 1.6094$	0.8047	0.5
Sum				KL $= 0.5850$	$E[r] = 0.65$

$$J = 0.65 - 0.2(0.5850) = 0.65 - 0.1170 = 0.5330.$$

Notice that action 0's log-ratio is negative. A particular sampled log-ratio may be negative even though its expectation under p is nonnegative. The other direction, $D_{KL}(q\|p) = 0.6 \log 3 + 0 + 0.1 \log 0.2 = 0.4982$, is a different number.

5. Derive the exact best distribution

We maximize J subject to $\sum_i p_i = 1$. Introduce a Lagrange multiplier λ for that constraint:

$$\mathcal{L} = \sum_i p_i r_i - \beta \sum_i p_i \log(p_i/q_i) + \lambda\Big(\sum_i p_i - 1\Big).$$

Setting the derivative with respect to p_i to zero gives:

$$r_i - \beta[\log(p_i/q_i) + 1] + \lambda = 0 \quad \Longrightarrow \quad \log \frac{p_i}{q_i} = \frac{r_i}{\beta} + \text{const.}$$

Exponentiate, then normalize so the probabilities sum to one:

$$\boxed{p_i^* = \frac{q_i e^{r_i/\beta}}{\sum_j q_j e^{r_j/\beta}} = \text{softmax}_i(\log q + r/\beta)}$$

In words: **start from the reference and tilt it exponentially toward reward.** The denominator $Z = \sum_j q_j e^{r_j/\beta}$ is the normalizer (the DPO lesson calls it $Z(x)$).

Step-by-step for $\beta = 0.2$: $r/\beta = [0, 2.5, 5]$.

i	q_i	$e^{r_i/\beta}$	$q_i e^{r_i/\beta}$	p_i^*
0	0.6	1	0.6	$0.6/19.0961 = 0.0314$
1	0.3	12.1825	3.6547	0.1914
2	0.1	148.4132	14.8413	0.7772
Sum			$Z = 19.0961$	1

The unnormalized weights are approximately `[0.6,3.6547,14.8413]`, and normalizing gives `[0.0314,0.1914,0.7772]`. The high-reward action becomes much more probable, but the reference affects how much probability each action receives: action 1 keeps 19% rather than collapsing to zero.

At this optimum, $E[r] = 0.8729$, $D_{KL} = 1.4149$, and $J^* = 0.8729 - 0.2(1.4149) = 0.5899$. A neat identity: $J^* = \beta \log Z = 0.2 \log 19.0961 = 0.5899$.

6. Compare three policies

Policy	p	$E[r]$	$D_{KL}(p\|q)$	J **with** $\beta = 0.2$
Stay at reference	`[0.6, 0.3, 0.1]`	0.25	0	0.25
Candidate from section 4	`[0.2, 0.3, 0.5]`	0.65	0.5850	0.5330
Always pick action 2	`[0, 0, 1]`	1.00	$\log 10 = 2.3026$	0.5395
Exact optimum	`[0.0314, 0.1914, 0.7772]`	0.8729	1.4149	**0.5899**

The greedy policy gets the most reward but pays a large KL price. The optimum gives up a little reward in exchange for staying closer to the reference.

7. How does beta change the answer?

Larger beta penalizes drift more strongly; smaller beta allows reward to dominate more strongly. Exact optima from the boxed formula:

β	p^*	$E[r]$	D_{KL}	J^*
0.05	`[0.0000, 0.0001, 0.9999]`	0.9999	2.3011	0.8849

β	p^*	$E[r]$	D_{KL}	J^*
0.1	`[0.0003, 0.0198, 0.9799]`	0.9898	2.1806	0.7718
0.2	`[0.0314, 0.1914, 0.7772]`	0.8729	1.4149	0.5899
0.5	`[0.2785, 0.3785, 0.3430]`	0.5322	0.2970	0.3838
1.0	`[0.4391, 0.3620, 0.1989]`	0.3799	0.0677	0.3122
5.0	`[0.5694, 0.3147, 0.1159]`	0.2732	0.0023	0.2615

As $\beta \to 0$ the optimum becomes greedy; as $\beta \to \infty$ it returns to the reference. Always report reward **and** KL together: a higher reward with a huge KL may simply mean the policy found a reward-model loophole.

Reward scale and shift. Only r/β appears in p^*. If you multiply all rewards by ten without changing beta, reward dominates more strongly; it is exactly the same as using $\beta = 0.02$, and the optimum becomes essentially `[0, 0, 1]`. Reward scaling therefore changes the effective tradeoff. Adding the same constant to every reward does not change the best policy: it multiplies every weight $q_i e^{r_i/\beta}$ by the same factor, which cancels in the normalization (equivalently, it adds only a constant to expected reward).

This regularization can preserve behavior statistically, but cannot guarantee that all original capabilities remain intact.

8. Why optimize logits, and what is the first gradient?

The code stores unconstrained logits and converts them with log-softmax. This automatically keeps $p_i > 0$ and $\sum_i p_i = 1$ without a separate constrained optimizer. `probs * reward` computes the exact expected reward; `probs * (log_probs-reference.log())` computes exact forward KL. Adam minimizes `-objective`, which is the same as maximizing J.

The logits start at `reference.log()`, so the initial policy **is** the reference: $p = q$, $\mathrm{KL} = 0$, $J = 0.25$.

By the softmax chain rule, with $g_i = r_i - \beta \log(p_i/q_i)$,

$$\frac{\partial J}{\partial z_i} = p_i\Big(g_i - \sum_j p_j g_j\Big).$$

At the start, $p = q$ so $g = r$ and $\sum_j q_j r_j = 0.25$:

$$\nabla_z J = [0.6(0 - 0.25),\ 0.3(0.5 - 0.25),\ 0.1(1 - 0.25)] = [-0.15,\ 0.075,\ 0.075].$$

Below-average-reward action 0 is pushed down; the other two are pushed up. Note the KL term contributes zero gradient at $p = q$; it only starts resisting once the policy has moved. After one Adam step the policy is `[0.5758, 0.3182, 0.1061]` with $J = 0.2649$.

9. What does training actually reach?

Training trajectory from the script's settings (Adam, lr 0.05, $\beta = 0.2$):

Step	Policy	J
0	`[0.6000, 0.3000, 0.1000]`	0.2500
50	`[0.0749, 0.1790, 0.7461]`	0.5854
100	`[0.0423, 0.1888, 0.7689]`	0.5895
150 (printed)	`[0.0356, 0.1893, 0.7751]`	0.58984
exact optimum	`[0.0314, 0.1914, 0.7772]`	0.58990

After 150 steps the policy is close to, but not exactly at, the optimum: action 0 still has 0.0356 instead of 0.0314. The objective is already within about 0.00006 of its maximum, because J is flat near its peak. (Continuing the same loop for 1000 steps in a scratch run matches the optimum to four decimals; the script itself stops at 150.) The final `exact_optimum` comparison is an independent mathematical check on the learned distribution.

10. From actions to text

For a complete response, autoregressive probability is a product over generated tokens, so the log-ratio is a sum:

$$\log \frac{\pi(y \mid x)}{\pi_{ref}(y \mid x)} = \sum_t [\log \pi(y_t \mid x, y_{<t}) - \log \pi_{ref}(y_t \mid x, y_{<t})].$$

Example. A two-token response where the policy assigns 0.5 and 0.6 and the reference assigns 0.4 and 0.8:

Token	$\log \pi$	$\log \pi_{ref}$	Log-ratio
1	$\log 0.5$	$\log 0.4$	$+0.2231$
2	$\log 0.6$	$\log 0.8$	-0.2877
Sum			-0.0645

This single-sample estimate is negative, even though expected KL is never negative. That is why sampled KL penalties are noisy and only average out to the true KL.

Mask out prompt and padding positions (as in instruction masks). In sampled LLM training, this quantity can supply a reference penalty alongside a sequence reward. PPO additionally needs an **old rollout policy** for importance ratios. The reference policy and the old policy serve different roles and are not interchangeable:

Policy	Role	Updated?
Reference π_{ref}	Anchor for the KL penalty (usually SFT model)	Never
Old policy π_{old}	Generated the current rollout batch; PPO ratio denominator	Replaced after each collection
Current policy π_θ	Being trained	Every step

What the script does not show: sampling, token sequences, a value critic, PPO clipping, or a learned reward model that can be hacked. The comment about reward hacking is a general warning: higher learned reward need not imply higher human quality.

11. Map the explanation to the script

Code	Meaning
`reference = torch.tensor([0.6, 0.3, 0.1])`	Frozen reference q
`reward = torch.tensor([0.0, 0.5, 1.0])`	Fixed rewards r
`logits = torch.nn.Parameter(reference.log().clone())`	Trainable z, initialised so $p = q$
`beta = 0.2`	KL penalty strength
`log_probs = logits.log_softmax(-1)`	$\log p_i$, always a valid distribution
`kl = (probs*(log_probs-reference.log())).sum()`	Exact $D_{KL}(p\|q)$
`objective = (probs*reward).sum()-beta*kl`	$J(p) = E_p[r] - \beta D_{KL}$
`(-objective).backward()`	Gradient ascent on J via descent on $-J$
`for _ in range(150):`	Fixed 150 Adam steps, lr 0.05
`exact_optimum = (reference.log()+reward/beta).softmax(-1)`	Closed-form $p^* = \mathrm{softmax}(\log q + r/\beta)$

12. Run and experiment

Run from the course folder:

```
python3 04_alignment/02_rlhf_kl_penalty.py
```

Output:

```
learned policy: [0.03564076125621796, 0.1892627477645874, 0.7750964760780334] analytic optimum:
[0.03142009302973747, 0.19138753414154053, 0.7771924734115601]
```

The script has no command-line flags and no assertion. You compare the two lists by eye: the learned distribution should approach the analytic optimum, and here it agrees to about 0.004 in every entry.

Experiments (edit a copy of the script):

1. Sweep beta and compare reward and KL together; check against section 7.
2. Multiply the rewards by ten and confirm the result matches $\beta = 0.02$.
3. Add 3 to every reward and confirm the optimum does not change.
4. Increase the step count and watch the gap to the optimum shrink.

13. Practice calculations

1. For $p = [0.3, 0.3, 0.4]$, what are $E[r]$, $D_{KL}(p\|q)$, and J with $\beta = 0.2$? $E[r] = 0.55$**; KL** $= 0.3 \log 0.5 + 0 + 0.4 \log 4 = 0.3466$**;** $J = 0.55 - 0.0693 = 0.4807$**.**
2. What is the optimum for $\beta = 1$? **Weights** $[0.6,\ 0.3e^{0.5},\ 0.1e] = [0.6, 0.4946, 0.2718]$**, sum 1.3664;** $p^* = [0.4391, 0.3620, 0.1989]$**.**
3. Why is KL zero at the start of training? **The logits are initialised to** $\log q$**, so** $p = q$**.**
4. A token has $\pi = 0.2$ and $\pi_{ref} = 0.5$. Its log-ratio? $\log 0.4 = -0.9163$**.**

5. Rewards become $r + 7$. Does p^* change? **No; the common factor $e^{7/\beta}$ cancels.**
6. Which policy is the PPO ratio denominator: reference or old? **The old rollout policy.**

Next: DPO derives a preference loss from this optimum.

Companion code: `02_rlhf_kl_penalty.py`

04_alignment/02_rlhf_kl_penalty.py

```python
"""The bridge from supervised tuning to reward optimization.
Typical RLHF: SFT -> preference reward model -> policy optimization (e.g. PPO).
The frozen reference is usually the SFT policy; it anchors behavior.
Maximize E_pi[reward] - beta KL(pi || reference). This exact three-action
bandit illustrates the objective, NOT a complete autoregressive RLHF trainer.
Exercise: sweep beta and report both expected reward AND KL divergence.
"""
import torch


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    reference = torch.tensor([0.6, 0.3, 0.1])
    reward = torch.tensor([0.0, 0.5, 1.0])
    logits = torch.nn.Parameter(reference.log().clone())
    optimizer = torch.optim.Adam([logits], lr=0.05)
    beta = 0.2
    for _ in range(150):
        log_probs = logits.log_softmax(-1)
        probs = log_probs.exp()
        kl = (probs*(log_probs-reference.log())).sum()
        objective = (probs*reward).sum()-beta*kl
        optimizer.zero_grad(set_to_none=True)
        (-objective).backward()
        optimizer.step()
    with torch.no_grad():
        p = logits.softmax(-1)
        exact_optimum = (reference.log()+reward/beta).softmax(-1)
        print('learned policy:', p.tolist(), 'analytic optimum:', exact_optimum.tolist())
    # For sampled text, sum log_pi(token)-log_ref(token) over response tokens.
    # A single sampled log ratio can be negative even though expected KL >= 0.
    # Reward hacking: higher learned reward need not imply higher human quality.

if __name__ == '__main__':
    main()
```

4.3

DPO: derive preference optimization from a regularized reward objective

Companion code: 03_dpo.py. Prerequisites: pairwise reward likelihood, the KL-regularized optimum, and instruction data and loss masks. Read this explanation before running the code.

1. What problem does DPO simplify?

The previous two lessons describe a two-stage pipeline:

```
preferences --> fit reward model r_phi --> sample responses, optimize policy with RL + KL
penalty
```

That means training and storing an extra model, generating fresh responses during training, and tuning an RL algorithm such as PPO. **Direct Preference Optimization (DPO)** trains a policy directly from fixed prompt/chosen/rejected triples and a frozen reference. No online reward model or rollout collection is required for this objective.

DPO is offline preference optimization; it is not identical to on-policy reinforcement learning. It reuses the same preference data a reward model would use, but turns the policy's own log-probabilities into an *implicit* reward. The formulation comes from the DPO paper.

2. The tiny example in the script

The script's “responses” are three categorical actions for one implicit prompt:

Index	Role in the preference pair	Reference prob	Starting policy prob
0	rejected y_l	1/3	1/3
1	neutral (never compared)	1/3	1/3
2	chosen y_w	1/3	1/3

There is exactly one preference pair, “2 is better than 0,” repeated for 100 training steps. The reference is uniform and frozen; the policy starts uniform (all logits zero). Watch what happens to the neutral action, which no label mentions.

3. Notation

Symbol	Meaning	Example / Code
x	Prompt	implicit in the script
y_w, y_l	Chosen and rejected responses	indices 2 and 0

Symbol	Meaning	Example / Code
$\pi_\theta(y \mid x)$	Trainable policy probability	`logits.softmax(-1)`
$\pi_{ref}(y \mid x)$	Frozen reference probability	`reference_logs.exp()` = 1/3 each
$\log \frac{\pi_\theta}{\pi_{ref}}$	Policy log-ratio for one response	`policy_chosen-ref_chosen`
m	Unscaled margin: chosen log-ratio minus rejected log-ratio	`margin`
β	Strength of the implicit KL constraint	`beta=0.1`
z	Scaled margin βm	`beta*margin`
$\hat{r}(x, y)$	Implicit reward $\beta \log \frac{\pi_\theta}{\pi_{ref}}$	not printed by the script
$Z(x)$	Normaliser of the KL-regularized optimum	cancels; never computed

4. Derivation: replace rewards with policy log-ratios

Step 1: recall the optimum. In the previous lesson we found that maximizing $E[r] - \beta D_{KL}(\pi \| \pi_{ref})$ gives, for each prompt x:

$$\pi^*(y \mid x) = \frac{\pi_{ref}(y \mid x) e^{r(x,y)/\beta}}{Z(x)}.$$

Step 2: solve for the reward. Take logs and rearrange:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{ref}(y \mid x)} + \beta \log Z(x).$$

So a reward and its optimal policy carry the same information, up to $\beta \log Z(x)$.

Step 3: plug into Bradley–Terry. The reward-model lesson used $P(y_w \succ y_l) = \sigma(r_w - r_l)$. For two responses to the **same** prompt, the unknown $\beta \log Z(x)$ terms cancel in $r_w - r_l$. This is exactly the "only differences matter" property from section 8 of that lesson.

Step 4: replace π^* by the trainable model π_θ and maximize the likelihood of the observed preferences:

$$\boxed{z = \beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)} \right], \qquad L_{DPO} = -\mathbb{E}\big[\log \sigma(z) \big]}$$

This is the objective implemented in `dpo_loss`. In code, `margin` is the bracket (which we call m), and the loss is `-logsigmoid(beta*margin).mean()`.

The preference model and regularized-optimum assumptions motivate the transformation; they do not imply that any finite noisy dataset recovers human utility perfectly.

5. A numerical example and the loss value

Suppose the reference assigns chosen and rejected probabilities 0.2 and 0.4, while the current model assigns 0.4 and 0.2.

Response	π_θ	π_{ref}	Log-ratio
chosen	0.4	0.2	$\log 2 = 0.6931$
rejected	0.2	0.4	$\log 0.5 = -0.6931$

Then the unscaled margin is:

$$m = \log(0.4/0.2) - \log(0.2/0.4) = \log 4 \approx 1.3863.$$

With beta 0.1, $z = 0.1386$, preference probability is $\sigma(0.1386) \approx 0.5346$, and loss is $-\log 0.5346 \approx 0.6262$. If the policy equals the reference, both log-ratios are zero, $z = 0$, and the loss is $\log 2 = 0.6931$.

What is being compared? Not the raw probabilities, but how much the policy has moved each response **relative to the reference**. If the reference already preferred the chosen answer, the policy gets no credit for merely keeping that preference.

Sequence-scale example. Real responses have summed log-probabilities like -12. Suppose $\log \pi_\theta(y_w) = -12$, $\log \pi_\theta(y_l) = -15$, $\log \pi_{ref}(y_w) = -13$, $\log \pi_{ref}(y_l) = -14$. The chosen log-ratio is $+1$, the rejected is -1, so $m = 2$.

β	$z = \beta m$	$\sigma(z)$	Loss	$\partial L/\partial m$
0.1	0.2	0.5498	0.5981	-0.0450
0.5	1.0	0.7311	0.3133	-0.1345
1.0	2.0	0.8808	0.1269	-0.1192

6. Which way does the gradient push?

Let m be the unscaled margin. Its derivative is:

$$\boxed{\frac{\partial L}{\partial m} = \beta[\sigma(\beta m) - 1]}$$

This is negative, so descent encourages increasing the chosen log-probability relative to the rejected one and relative to the reference preference odds. For the first example ($m = 1.3863$, $\beta = 0.1$): $0.1(0.5346 - 1) = -0.0465$.

Through the chain rule, the parameter gradient is

$$\nabla_\theta L = -\beta\, \sigma(-z)\big[\nabla_\theta \log \pi_\theta(y_w) - \nabla_\theta \log \pi_\theta(y_l)\big],$$

using $1 - \sigma(z) = \sigma(-z)$. The weight $\sigma(-z)$ is large when the implicit reward ranks the pair wrongly and small when it already ranks it confidently, just like the reward-model gradient in the previous lessons.

Because probabilities are normalized and model parameters are shared, this does not guarantee the absolute probability of every chosen response always rises. DPO only promises to widen the *gap*; in real LLM training both log-probabilities can fall while the gap still grows.

Beta has two roles to distinguish. In the ideal regularized optimum it controls the reward/KL tradeoff: small beta means a weak tether to the reference. Inside the fixed DPO loss it also changes margin scale and gradient saturation. The table in section 5 shows the gradient is not monotonic in beta: going from 0.5 to 1.0 *shrinks* it because the sigmoid saturates. A larger beta is not simply "a larger learning rate."

7. Watch the script's first step by hand

With all logits zero, $\log \pi_\theta = \log \pi_{ref} = \log(1/3) = -1.0986$ for every action, so $m = 0$, $z = 0$, and the loss is $\log 2 = 0.6931$.

Because log-softmax subtracts the same log-normaliser from every entry, $\log \pi_\theta(2) - \log \pi_\theta(0) = z_2 - z_0$ (here z_i denotes logits). The reference is uniform, so its difference is zero. Therefore

$$L = -\log \sigma\big(0.1(z_2 - z_0)\big), \qquad \frac{\partial L}{\partial z_2} = 0.1(\sigma(0) - 1) = -0.05, \quad \frac{\partial L}{\partial z_0} = +0.05, \quad \frac{\partial L}{\partial z_1} = 0.$$

The logit gradient is `[0.05, 0.0, -0.05]`. **The neutral logit receives exactly zero gradient** for the whole run. Adam's first step moves each nonzero-gradient coordinate by about the learning rate 0.05, giving logits `[-0.05, 0, 0.05]` and probabilities `[0.3168, 0.3331, 0.3501]`.

8. What the script ends with

After 100 Adam steps (lr 0.05):

Quantity	Rejected (0)	Neutral (1)	Chosen (2)
Final logit	-4.5324	0	4.5324
Probability	0.000114	0.0106	0.9892
Implicit reward $0.1 \log(\pi_\theta / \frac{1}{3})$	-0.7977	-0.3445	$+0.1088$

The final margin is $m = 1.0878 - (-7.9770) = 9.0648$, so $z = 0.9065$, $\sigma(z) = 0.7123$, and the loss is still about 0.3393. The loss is nowhere near zero, yet the policy is already extreme. Two lessons:

1. **The neutral response lost almost all of its probability** (0.333 to 0.0106) although no label mentioned it and its logit never moved. Its probability fell only because softmax renormalised around the growing chosen logit.
2. **Adam moves at roughly constant speed.** The logits travelled 4.53 of a possible $100 \times 0.05 = 5$, because Adam normalises gradient size. This clean repeated preference can drive the distribution to an extreme; real training needs held-out evaluation and attention to label noise and coverage.

About the exercise in the script's docstring ("make beta smaller and compare gradient size and final preference"). The rows below come from edited copies of the script run separately; they are not outputs of the provided file:

Variant	Final probabilities `[rejected, neutral, chosen]`	Final loss
Adam, $\beta = 0.01$	`[0.000049, 0.0070, 0.9930]`	0.6448
Adam, $\beta = 0.1$ (script)	`[0.000114, 0.0106, 0.9892]`	0.3393
Adam, $\beta = 1.0$	`[0.0140, 0.1106, 0.8754]`	0.0158
Plain SGD lr 0.05, $\beta = 0.1$	`[0.2552, 0.3267, 0.4182]`	0.6688

The raw gradient is ten times smaller at $\beta = 0.01$, but Adam rescales it, so the smaller-beta run actually ends **more** extreme: its loss never saturates, so the push never fades. That matches the theory (small beta = weak tether to the reference). With plain SGD, the raw gradient size directly sets the step size, and the same 100 steps barely move the policy. Optimizer choice matters when you interpret beta experiments.

9. Sequence likelihood and masking

For response tokens y_t, sum conditional log-probabilities:

$$\log \pi_\theta(y \mid x) = \sum_t \log \pi_\theta(y_t \mid x, y_{<t}).$$

`sequence_logprob` applies log-softmax over vocabulary, shifts to next-token alignment, uses `gather` to select the actual target token, multiplies by the response mask, and sums over positions. This implements a sequence log-probability, not a token average. Replacing the sum by a mean changes the objective and its length dependence.

Worked example (batch 1, length 4, vocabulary 3). Sequence `token_ids=[[2, 1, 0, 0]]` = one prompt token, two response tokens, one pad; `response_mask=[[0, 1, 1, 0]]` marks the response tokens. The logit row at position t predicts token $t + 1$:

Logit row	Predicts position	Target token	Prob of target	Mask `response_mask[:,1:]`	Contribution
0, probs `[0.1, 0.8, 0.1]`	1	1	0.8	1	$\log 0.8 = -0.2231$
1, probs `[0.6, 0.2, 0.2]`	2	0	0.6	1	$\log 0.6 = -0.5108$
2, probs `[0.3, 0.3, 0.4]`	3	0 (pad)	0.3	0	0
3	dropped by `logits[:, :-1]`				

The function returns -0.7340 (checked by calling it). A token mean would give -0.3670, a different objective that changes how response length affects the margin.

`token_ids` must contain valid vocabulary indices even at masked positions; use the separate mask rather than placing -100 in IDs passed to `gather`. Chosen and rejected responses must share the same prompt and consistent template. Reference computations must be frozen (compute them under `torch.no_grad()` or precompute them once).

10. What the runnable example simplifies

`main` trains a three-action categorical distribution with action 2 preferred to action 0 and a uniform reference. The helper for sequences is available for understanding the LLM connection, but the main loop does not run a transformer and never calls `sequence_logprob`. There is one prompt, one pair, no batching, no label noise, and no held-out evaluation.

General claims in this lesson about LLM training (chosen probabilities falling, length effects of mean vs. sum, the need for frozen references) are background from the method's design, not behaviors the script demonstrates.

11. Map the explanation to the script

Code	Meaning
`logits[:, :-1].log_softmax(-1)`	Next-token log-probabilities; last position has no target
`.gather(` then `-1, token_ids[:, 1:, None]).squeeze(-1)`	Log-prob of each actual next token (one call split over two lines)
`(token_logs * response_mask[:, 1:]).sum(-1)`	Sum over response tokens only: $\log \pi(y \mid x)$
`margin = (policy_chosen-policy_rejected)-(ref_chosen-ref_rejected)`	Unscaled margin m (algebraically equal to chosen log-ratio minus rejected log-ratio)
`-nn.functional.logsigmoid(beta*margin).mean()`	$L_{DPO} = -\mathbb{E}[\log \sigma(\beta m)]$
`logits = nn.Parameter(torch.zeros(3))`	Trainable policy, starts uniform
`reference_logs = torch.zeros(3).log_softmax(-1)`	Frozen uniform reference, $\log(1/3)$ each
`dpo_loss(logs[2], logs[0], reference_logs[2], reference_logs[0])`	Preference “2 beats 0”, default `beta=0.1`
`torch.optim.Adam([logits], lr=0.05)`	Optimizer; 100 steps
`logits.softmax(-1)`	Printed final probabilities

12. Run and investigate

Run from the course folder:

```
python3 04_alignment/03_dpo.py
```

Output:

```
probabilities [rejected, neutral, chosen]: [0.00011442094546509907, 0.010639103129506111, 0.9892464280128479]
```

The script has no command-line flags and no assertion. The chosen action's probability increases markedly; the rejected action nearly vanishes; the neutral action shrinks only through normalisation. The course test suite (`tests/test_learning_invariants.py`) separately checks that `dpo_loss` gives the winner a negative gradient and the loser a positive one, and that `sequence_logprob` excludes prompt and padding positions.

Experiments:

1. Reverse the pair (`logs[0], logs[2]`) and observe the reversed learning direction.
2. Add conflicting preferences (also "0 beats 2") and examine the resulting compromise.
3. Change beta, then swap Adam for SGD, and compare with section 8.

13. Practice calculations

1. Policy log-probs: chosen -20, rejected -22; reference both -21; $\beta = 0.1$. What are m, z, and the loss? $m = (-20 + 22) - (-21 + 21) = 2$**,** $z = 0.2$**, loss** 0.5981**.**
2. What is $\partial L / \partial m$ there? $0.1(0.5498 - 1) = -0.0450$**.**
3. The policy equals the reference. What is the loss for any β? $\log 2 = 0.6931$**.**
4. Both policy log-probs drop by 3 from their reference values. What is m? **Zero; DPO sees no change in preference.**
5. In the script, what gradient does the neutral logit get? **Exactly 0.**
6. With token log-probs -0.2231 and -0.5108 on response tokens, what does `sequence_logprob` return? **Their sum,** -0.7340**.**

Next: GRPO: compare groups of answers instead of learning a value critic.

Companion code: `03_dpo.py`

04_alignment/03_dpo.py

```python
"""Direct Preference Optimization without an online reward model.
DPO uses fixed prompt/chosen/rejected triples and a frozen reference policy.
loss = -log sigmoid(beta * [(log pi_w-log ref_w)-(log pi_l-log ref_l)]).
DPO is offline preference optimization; it does not collect RL rollouts.
This categorical example trains end-to-end; sequence_logprob shows the LLM bridge.
Exercise: make beta smaller and compare gradient size and final preference.
"""
import torch
from torch import nn


def sequence_logprob(logits, token_ids, response_mask):
    # [B,T,V] logits; [B,T] IDs/mask. Mask marks REAL response tokens, not pads.
    token_logs = logits[:, :-1].log_softmax(-1).gather(
        -1, token_ids[:, 1:, None]).squeeze(-1)
    # Original DPO uses SUMMED completion log-probabilities, not token means.
    return (token_logs * response_mask[:, 1:]).sum(-1)


def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):
    margin = (policy_chosen-policy_rejected)-(ref_chosen-ref_rejected)
    return -nn.functional.logsigmoid(beta*margin).mean()


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    logits = nn.Parameter(torch.zeros(3))
    reference_logs = torch.zeros(3).log_softmax(-1)  # Frozen uniform reference.
    optimizer = torch.optim.Adam([logits], lr=0.05)
    for _ in range(100):
        logs = logits.log_softmax(-1)
        loss = dpo_loss(logs[2], logs[0], reference_logs[2], reference_logs[0])
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
    print('probabilities [rejected, neutral, chosen]:',
logits.softmax(-1).detach().tolist())
    # For LLMs, chosen and rejected must use the SAME prompt/template and the
    # reference must not be updated. Padding/prompt labels must be excluded.

if __name__ == '__main__':
    main()
```

4.4

GRPO: compare groups of answers instead of learning a value critic

Companion code: 04_grpo.py. Prerequisites: PPO and GAE for clipped probability ratios, REINFORCE and actor-critic for baselines, and RLHF with a KL penalty for the frozen reference policy. The math primer reviews softmax, standard deviation, and KL divergence. Read this explanation before running the code.

1. What problem are we solving now?

A language model can answer the same prompt in several ways. Some answers are correct; others are not. With a **verifiable reward** (for example, "does the final answer match the known solution?") we can score each answer with 1 or 0.

PPO needs an **advantage**: "how much better was this action than usual?" PPO usually obtains it from a separately trained value network, the **critic**. GRPO (Group Relative Policy Optimization) avoids the critic with a simple idea:

> Sample a **group** of answers to the **same** prompt, score them all, and judge each answer against the other members of its own group.

An answer that beats its siblings gets a positive advantage; one that does worse gets a negative advantage. The group itself supplies the baseline.

The companion script uses the smallest possible version of this idea:

```
prompt 0: possible one-token answers {0, 1, 2, 3}, correct answer = 1
prompt 1: possible one-token answers {0, 1, 2, 3}, correct answer = 3
```

Each "answer" is a single token, so a whole response is one action. Each iteration samples a group of $G = 8$ answers per prompt and awards reward 1 to a correct answer and 0 otherwise. The policy starts uniform: every answer has probability 0.25. Training should push probability toward answer 1 for prompt 0 and answer 3 for prompt 1.

2. Notation: read the symbols before the equations

Symbol	Meaning	Example / Code
x	A prompt (one row of the table)	row index 0 or 1 of `logits`
a_i	The i-th sampled answer for that prompt	`actions[prompt, i]`
G	Group size: answers sampled per prompt	8, from `.sample((8,))`
r_i	Reward of answer i: 1 if correct, else 0	`rewards`

Symbol	Meaning	Example / Code
$\bar{r}$	Mean reward in the group	`rewards.mean(-1, keepdim=True)`
s	Population standard deviation of the group's rewards	`rewards.std(-1, ..., unbiased=False)`
ε	Small constant that prevents division by zero	`1e-6`
A_i	Group-relative advantage of answer i	`advantages`
$\pi_\theta(a \mid x)$	Current policy: softmax of trainable logits	`logits.log_softmax(-1)`
π_{old}	Frozen copy of the policy that **generated** the group	`old_logs`
π_{ref}	Frozen **reference** policy: the uniform starting point	`reference_logs`
ρ_i	Ratio $\pi_\theta(a_i \mid x)/\pi_{old}(a_i \mid x)$	`ratio`
ϵ	PPO clip width, here 0.2	`ratio.clamp(0.8, 1.2)`
β	Weight of the reference KL penalty, here 0.02	`0.02*kl`

Two different "old" distributions appear, and confusing them is a common mistake: π_{old} changes every iteration (it is whatever generated the latest group), while π_{ref} never changes.

3. Step one: turn a group of rewards into advantages

For rewards $r_1, \ldots, r_G$ in one group, the script computes

$$\bar{r} = \frac{1}{G}\sum_i r_i, \qquad s = \sqrt{\frac{1}{G}\sum_i (r_i - \bar{r})^2}, \qquad \boxed{A_i = \frac{r_i - \bar{r}}{s + \varepsilon}}.$$

In words: subtract the group average (centering), then divide by the group's spread (scaling). The script uses the **population** standard deviation (`unbiased=False` , divide by G, not $G-1$) and $\varepsilon = 10^{-6}$. Other implementations may use different normalization conventions.

Worked example: rewards `[1, 0, 0, 1]`

1. Mean: $\bar{r} = (1+0+0+1)/4 = 0.5$.
2. Deviations: $[0.5, -0.5, -0.5, 0.5]$; squared: $[0.25, 0.25, 0.25, 0.25]$.
3. Population variance $= 0.25$, so $s = 0.5$.
4. Advantages: $0.5/0.500001 \approx 1$ and $-0.5/0.500001 \approx -1$.

Result: approximately `[1, -1, -1, 1]`. The two correct answers are pushed up and the two wrong answers pushed down by the same amount.

Worked example: rewards `[0, 0, 0, 1]`

1. $\bar{r} = 0.25$.
2. Deviations: $[-0.25, -0.25, -0.25, 0.75]$; squared: $[0.0625, 0.0625, 0.0625, 0.5625]$.
3. Variance $= 0.75/4 = 0.1875$, so $s = \sqrt{0.1875} \approx 0.4330$.
4. Advantages: $-0.25/0.4330 \approx -0.5774$ (three times) and $0.75/0.4330 \approx 1.7321$.

The rare successful answer receives a stronger positive relative signal.

The script's group size: eight answers

For binary rewards with k correct answers out of G, the mean is $p = k/G$ and the population standard deviation is $\sqrt{p(1-p)}$. Some group-of-8 cases:

Correct out of 8	$\bar{r}$	s	A for a correct answer	A for a wrong answer
0	0	0	— (none)	0
1	0.125	0.3307	2.6457	-0.3780
3	0.375	0.4841	1.2910	-0.7746
7	0.875	0.3307	0.3780	-2.6457
8	1	0	0	— (none)

Check the 3-of-8 row: $s = \sqrt{0.375 \times 0.625} = \sqrt{0.234375} \approx 0.4841$, so $0.625/0.4841 \approx 1.2910$ and $-0.375/0.4841 \approx -0.7746$. Notice also that the advantages in any group **sum to zero**: $3(1.2910) + 5(-0.7746) \approx 0$.

When every reward is equal

If every reward is equal (all 0 or all 1), every deviation is zero, so every advantage is zero. Epsilon avoids division by zero ($0/(0 + 10^{-6}) = 0$), but it cannot invent a preference signal. Such a group teaches the policy-gradient term nothing. More samples, better exploration, or more informative rewards may be needed when all attempts fail.

At the start of this toy problem the chance that all 8 answers to a prompt are wrong is $0.75^8 \approx 0.100$, so roughly one group in ten is uninformative even on the first iteration. Late in training the opposite happens: once the policy is almost always right, most groups are all-correct and also give zero advantage. In the script's actual run (seed 7), 122 of the 200 prompt-groups collected over 100 iterations had all-equal rewards.

A caveat about the baseline

Because the baseline and scale are computed from the sampled group itself, this is not simply the action-independent baseline proof from REINFORCE. Group standardization changes weighting and can introduce finite-group bias; it is a practical objective choice rather than a universal unbiased estimator.

4. Step two: a PPO-style clipped update, without a critic

Store the log-probabilities of the policy that generated the group. For answer a_i, the ratio of new to old probability is

$$\rho_i = \exp(\log \pi_\theta(a_i \mid x) - \log \pi_{old}(a_i \mid x)).$$

The minimized teaching loss is

$$\boxed{L = -\,\mathrm{mean}_i \min\big(\rho_i A_i,\ \mathrm{clip}(\rho_i, 0.8, 1.2) A_i\big) + 0.02\, D_{KL}(\pi_\theta \| \pi_{ref})}.$$

Part	Interpretation
$\rho_i A_i$	Raise probability of positive-advantage answers, lower negative ones
$\text{clip}(\rho_i, 0.8, 1.2) A_i$	Same, but the ratio's influence is capped at ±20%
$\min(\cdot, \cdot)$	Take the more pessimistic of the two, as in PPO
mean_i	Average over all 16 entries (2 prompts × 8 answers)
Leading minus sign	Optimizers minimize; we want to maximize the surrogate
$0.02\, D_{KL}$	Penalty for drifting away from the reference policy

This is exactly PPO's clipped surrogate with **group advantages replacing critic-based advantages**. If you need a refresher on why the minimum is used, see sections 9–10 of the PPO lesson.

Worked clipping example

Take the 3-of-8 advantages $A = 1.2910$ (correct) and $A = -0.7746$ (wrong):

A	ρ	ρA	$\text{clip}(\rho) A$	Selected minimum	Gradient through ρ?
1.2910	1.0	1.2910	1.2910	1.2910	Yes
1.2910	1.3	1.6783	$1.2(1.2910) = 1.5492$	1.5492	No: clipped, flat
1.2910	0.7	0.9037	$0.8(1.2910) = 1.0328$	0.9037	Yes
-0.7746	0.7	-0.5422	$0.8(-0.7746) = -0.6197$	-0.6197	No: clipped, flat
-0.7746	1.3	-1.0070	$1.2(-0.7746) = -0.9295$	-1.0070	Yes

The pattern matches PPO: once a good answer's probability has already risen by more than 20%, or a bad answer's probability has already fallen by more than 20%, that sample stops pushing. Moves in the *harmful* direction are never capped, so the objective stays pessimistic.

Two frozen distributions with two different jobs

`old_logs` stays fixed for two update epochs, then fresh answers are sampled. The reference remains the original uniform distribution throughout training. Old-policy ratios limit changes relative to collection; reference KL penalizes long-term drift from the starting policy.

Because there are only four actions, the code computes **exact** categorical KL by summing over all of them:

$$D_{KL}(\pi_\theta \| \pi_{ref}) = \sum_{a=0}^{3} \pi_\theta(a \mid x) \log \frac{\pi_\theta(a \mid x)}{\pi_{ref}(a \mid x)},$$

averaged over the two prompts. It does not use a sampled token-level KL estimator. The actor receives gradients; sampled actions, rewards, and advantages do not (they are computed under `torch.no_grad()`).

KL example: for a prompt whose policy is $[0.1, 0.7, 0.1, 0.1]$ against the uniform reference 0.25,

$$3(0.1) \ln \frac{0.1}{0.25} + 0.7 \ln \frac{0.7}{0.25} = 0.3(-0.9163) + 0.7(1.0296) = -0.2749 + 0.7207 \approx 0.4458.$$

The largest possible value here is $\ln 4 \approx 1.386$, reached when all probability sits on one answer. With $\beta = 0.02$ the penalty is at most about 0.028, which is small compared with advantages near 1. In this toy run the reference term therefore slows drift slightly but does not prevent the policy from becoming nearly deterministic: the final probabilities give KL values of about 1.346 and 1.309.

5. Watch the script's first group, step by step

With `torch.manual_seed(7)`, the first iteration samples:

Prompt	Sampled answers	Rewards	Advantages
0 (correct = 1)	`[1, 1, 3, 2, 3, 3, 1, 2]`	`[1,1,0,0,0,0,1,0]`	1.2910 for the three 1s, -0.7746 otherwise
1 (correct = 3)	`[3, 3, 0, 0, 0, 0, 0, 3]`	`[1,1,0,0,0,0,0,1]`	1.2910 for the three 3s, -0.7746 otherwise

These are the exact values the script produces with this seed, matching the 3-of-8 row of the earlier table.

Update epoch 1. The policy has not moved yet, so $\pi_\theta = \pi_{old}$ and every $\rho_i = 1$. The surrogate mean is the mean advantage, which is 0, and the KL to the uniform reference is 0. The printed loss would be about 10^{-8}, essentially zero. **A zero loss value does not mean a zero gradient.**

For a softmax policy, $\partial \log \pi(a_i)/\partial u_v = 1[a_i = v] - \pi(v)$, where u_v is the logit of answer v. Because the group's advantages sum to zero, the $-\pi(v)$ terms cancel, and the gradient of the loss for logit v of one prompt is

$$\frac{\partial L}{\partial u_v} = -\frac{1}{16} \sum_{i:\, a_i = v} A_i.$$

For prompt 0:

Logit	Samples choosing it	Gradient
u_0	none	0
u_1	three, each $A = 1.2910$	$-3(1.2910)/16 = -0.2421$
u_2	two, each $A = -0.7746$	$+2(0.7746)/16 = +0.0968$
u_3	three, each $A = -0.7746$	$+3(0.7746)/16 = +0.1452$

The script's autograd values are -0.24206, 0.09682, 0.14524, matching. Adam's first step moves each logit by about the learning rate in the direction opposite its gradient sign, so u_1 becomes 0.04 and u_2, u_3 become -0.04. (u_0 moves by about 0.005 rather than 0 because its gradient is a tiny floating-point residue, about -1.4×10^{-9}, which Adam's per-coordinate normalization amplifies. This is a numerical side effect, not part of GRPO.)

Update epoch 2. The probabilities for prompt 0 are now about $[0.2533, 0.2623, 0.2422, 0.2422]$, so the ratios are about $0.2623/0.25 = 1.049$ for correct answers and $0.2422/0.25 = 0.969$ for the wrong ones. Both lie inside $[0.8, 1.2]$, so no clipping happens. The loss is now about -0.0388.

In fact, in this run clipping **never** activates: the largest second-epoch ratio deviation over all 100 iterations is about 0.066. The clip is present to show the objective's form; this toy run does not exercise it. Larger learning rates or more update epochs per group would.

Over training. The probabilities of the correct answers (prompt 0, prompt 1) after selected iterations of the seeded run are:

Iterations	$\pi(1 \mid x_0)$	$\pi(3 \mid x_1)$
10	0.583	0.545
25	0.914	0.892
50	0.976	0.976
100	0.994	0.988

6. Shapes and implementation mapping

`logits` has shape `[2,4]`: prompts by possible answers. Sampling with shape `(8,)` initially gives `[8,2]`; transposing produces `[2,8]`, prompts by group members. `gather(1,actions)` retrieves the selected answers' log-probabilities. Normalization along the last dimension keeps each prompt's baseline separate. Mixing rewards across prompts would compare different tasks rather than answers to the same task.

7. Map the explanation to the script

Code	Meaning
`logits = nn.Parameter(torch.zeros(2, 4))`	Trainable policy; zeros mean a uniform start
`reference_logs = torch.zeros(2, 4).log_softmax(-1)`	Frozen uniform reference π_{ref}
`correct_answers = torch.tensor([1, 3])`	The verifier's answer key
`Categorical(logits=logits).sample((8,)).T`	Group of 8 answers per prompt, shape `[2,8]`
`old_logs = old_logs_all.gather(1, actions)`	$\log \pi_{old}(a_i \mid x)$, fixed for this group
`rewards = (actions == correct_answers[:, None]).float()`	Binary verifiable reward
`group_advantages(rewards)`	$(r_i - \bar{r})/(s + 10^{-6})$ within each prompt
`for _ in range(2):`	Two update epochs reuse the same group
`ratio = (new_logs-old_logs).exp()`	ρ_i
`torch.minimum(ratio*advantages, ratio.clamp(0.8, 1.2)*advantages)`	Clipped surrogate
`kl = (logs_all.exp()*(logs_all-reference_logs)).sum(-1).mean()`	Exact KL to the reference, averaged over prompts
`loss = -surrogate.mean()+0.02*kl`	Minimized objective
`torch.optim.Adam([logits], lr=0.04)`	Optimizer and step size

8. Extending to language requires additional decisions

Real completions have many tokens and different lengths. A sequence reward must be assigned to token-level learning terms with explicit masks and reduction rules. Length normalization, reward scaling, clipping variants, and KL estimators all affect behavior. This small objective is not a claim to reproduce every production GRPO variant or current trainer default. See the TRL GRPO documentation for those distinctions, and the DeepSeekMath paper listed in SOURCES for the method's original setting.

What the script demonstrates: group-normalized advantages, a clipped ratio, and an exact reference KL can steer a one-step categorical policy to the answers a verifier rewards. What it does not demonstrate: anything about multi-token generation, reasoning quality, sample efficiency, or comparisons with PPO.

9. Run it and read the output

Run from the course folder:

```
python3 04_alignment/04_grpo.py
```

The script prints the final answer probabilities, one row per prompt. With the fixed seed, the output is approximately

```
answer probabilities: [[0.0012, 0.9945, 0.0020, 0.0023], [0.0059, 0.0027, 0.0034, 0.9880]]
```

(the real print shows full floating-point digits). Expect high probability on answer 1 for prompt 0 and answer 3 for prompt 1.

The assertion checks only that `group_advantages` returns all zeros when every reward in a `[2,8]` group equals one. It does **not** check that training converged or that the correct answers won; you read that from the printed table.

Experiments to try:

- Set all rewards to one: the policy-gradient term disappears; only reference regularization can act. From the uniform start nothing moves at all (the KL and its gradient are zero there); a policy that had already moved would be pulled back toward uniform.
- Test a flawed verifier that rewards a wrong action to see why optimizing reward does not guarantee optimizing true correctness.
- Raise the learning rate or the number of update epochs and count how often `ratio` leaves `[0.8, 1.2]`.

10. Practice calculations

1. Rewards `[1,1,0,0,0,0,0,0]`. What are $\bar{r}$ and s? $\bar{r} = 0.25$**,** $s = \sqrt{0.25 \times 0.75} \approx 0.4330$**.**

2. What advantages do the correct and wrong answers receive? $0.75/0.4330 \approx 1.732$ **and** $-0.25/0.4330 \approx -0.577$**.**
3. Rewards `[0,0,0,0]`. What are the advantages, and why? **All zero: every reward equals the mean, so every deviation is zero.**
4. $A = 1.732$ and $\rho = 1.25$. Which surrogate term is selected? $\min(2.165,\ 1.2 \times 1.732 = 2.078) = 2.078$**, the clipped one; its gradient through** ρ **is zero.**
5. $A = -0.577$ and $\rho = 1.25$. Which term is selected? $\min(-0.721,\ -0.693) = -0.721$**, the unclipped one; the harmful increase is still penalized.**
6. Policy $[0.25, 0.25, 0.25, 0.25]$ against the uniform reference. What is the KL? **Zero: the distributions are identical.**
7. Does GRPO in this script train a value network? **No. The group mean replaces the critic's baseline; there is no** `critic` **module anywhere in the file.**

Next: SDPO, RLSD, and OPSD: learning from a privileged self-teacher.

Companion code: `04_grpo.py`

04_alignment/04_grpo.py

```python
"""Group Relative Policy Optimization, with one-token responses.
Sample several responses for the SAME prompt; normalize their rewards within
that group; use those relative advantages in a PPO-like objective. No critic.
This teaching variant uses population standard deviation, clipped ratios and
an exact categorical KL term. Production GRPO variants differ in normalization,
length weighting, KL estimator and clipping; this is not a TRL default replica.
Exercise: give every response the same reward and explain the zero advantage.
"""
import torch
from torch import nn
from torch.distributions import Categorical


def group_advantages(rewards):
    return (rewards-rewards.mean(-1, keepdim=True)) / (rewards.std(-1, keepdim=True, unbiased=False)+1e-6)


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    # Each row is a prompt. Each action is a one-token possible answer.
    logits = nn.Parameter(torch.zeros(2, 4))
    reference_logs = torch.zeros(2, 4).log_softmax(-1)
    correct_answers = torch.tensor([1, 3])
    optimizer = torch.optim.Adam([logits], lr=0.04)
    for iteration in range(100):
        with torch.no_grad():
            old_logs_all = logits.log_softmax(-1)
            actions = Categorical(logits=logits).sample((8,)).T  # [prompts, group].
            old_logs = old_logs_all.gather(1, actions)
            rewards = (actions == correct_answers[:, None]).float()
            advantages = group_advantages(rewards)
        for _ in range(2):
            logs_all = logits.log_softmax(-1)
            new_logs = logs_all.gather(1, actions)
            ratio = (new_logs-old_logs).exp()
            surrogate = torch.minimum(ratio*advantages, ratio.clamp(0.8, 1.2)*advantages)
            kl = (logs_all.exp()*(logs_all-reference_logs)).sum(-1).mean()
            loss = -surrogate.mean()+0.02*kl
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
    print('answer probabilities:', logits.softmax(-1).detach().tolist())
    assert torch.equal(group_advantages(torch.ones(2, 8)), torch.zeros(2, 8))
    # Verifiable rewards still require tests for shortcuts and incorrect verifiers.

if __name__ == '__main__':
    main()
```

4.5

SDPO, RLSD, and OPSD: learning from a privileged self-teacher

Companion code: 05_sdpo_rlsd_opsd.py, PyTorch objective labs (loss functions on supplied tensors, not complete LLM trainers). Prerequisites: causal language modeling, response masks, policy gradients, PPO, GRPO, KL direction, and stop-gradient. Read tensors and autograd, neural network training, tokens and attention, a tiny transformer, instruction data and masks, REINFORCE and actor-critic, PPO and GAE, RLHF and KL, and GRPO first. The math primer reviews KL divergence.

This chapter fixes the meanings of these acronyms to the linked paper versions (also listed in SOURCES); other papers can reuse the same acronyms. Statements about what the papers propose are summaries of the cited versions, not something the script verifies. Read the papers before relying on any detail for your own experiments.

1. What problem are we solving now?

In GRPO the only learning signal was a single number per answer: correct (1) or wrong (0). Every token of a long answer gets the same credit, even though perhaps only one step was the actual mistake.

Suppose we could show the model something it will **not** see at test time:

- the error message from a failing test (feedback), or
- a reference solution to the problem.

With that **privileged information** z in its context, the same model can judge, token by token, which choices look sensible. That privileged copy acts as a **self-teacher**. The **student** is the ordinary model, which sees only the prompt. All three methods in this chapter use the teacher's per-token probabilities to give the student a richer signal than one scalar reward:

Method	Privileged z	How the teacher is used
SDPO	Environment feedback, or successful same-question attempts	Distill: minimize reverse KL, student ‖ teacher
OPSD	Reference answer or reasoning trace y^*	Distill: a divergence such as forward KL, teacher ‖ student
RLSD	Teacher evidence alongside a verifier reward	Reweight the verifier's group advantage per token

A tiny example used throughout: a vocabulary of 4 tokens at one position. The student is uniform (zero logits, exactly as the script initializes it), and the teacher prefers token 0:

```
student p = [0.25, 0.25, 0.25, 0.25]
teacher q = [0.40, 0.30, 0.20, 0.10]
```

2. Notation and the implementation contract

For prompt x, student-generated response y, prefix $h_t = (x, y_{<t})$, and privileged information z, define $p_t = \pi_\theta(\cdot \mid h_t)$ and $q_t = \mathrm{sg}(\pi_{\bar{\theta}}(\cdot \mid x, z, y_{<t}))$.

Symbol	Meaning	Example / Code
x	Prompt	(outside the lab)
y, y_t	Response sampled from the student; its t-th token	`tokens`
h_t	Student prefix: prompt plus earlier response tokens	(outside the lab)
z	Privileged information: feedback f or reference y^*	(outside the lab)
p_t	Student next-token distribution	`student_logits.log_softmax(-1)`
q_t	Teacher next-token distribution on the **same** prefix	`teacher_logits.detach().log_softmax(-1)`
sg	Stop-gradient: treat as a constant	`.detach()` / `torch.no_grad()`
π_{old}	Student policy that generated the rollout	`old_token_logs`
A_i	Verifier-derived group advantage of sequence i	`sequence_advantages`
λ	RLSD mixing weight	`mixing=0.5`
ϵ_w	RLSD evidence-weight clip	`weight_clip=0.2`
ϵ	PPO ratio clip	`policy_clip=0.2`
$	y	$

A teacher pass scores the **same student prefix**. Inference uses the student context. In the lab, logits are supplied tensors; teacher prompting, autoregressive collection, verification, and teacher synchronization are outside the example.

All losses average over valid response tokens per sequence, then sequences. Prompt/padding positions are excluded; EOS remains a response token (a convention for real data; the lab has no EOS token, only a mask). Computed rollouts and prefixes are fixed during each update: no differentiation through sampling or through teacher targets. `[B,T,V]` logits become `[B,T]` token losses.

3. Three ways to compare two distributions

KL divergence is **not symmetric**, so the order matters:

$$D_{KL}(p\|q) = \sum_v p_v \log \frac{p_v}{q_v} \quad \text{(reverse: student first)}, \qquad D_{KL}(q\|p) = \sum_v q_v \log \frac{q_v}{p_v} \quad \text{(forward: teacher fi}$$

The Jensen-Shannon divergence (JSD) averages both against the mixture $m = (p+q)/2$:

$$\boxed{\mathrm{JSD}(p,q) = \tfrac{1}{2} D_{KL}(q\|m) + \tfrac{1}{2} D_{KL}(p\|m)}.$$

Worked example with the tiny p and q:

Token v	p_v	q_v	$p_v \ln(p_v/q_v)$	$q_v \ln(q_v/p_v)$	m_v
0	0.25	0.40	$0.25 \ln 0.625 = -0.1175$	$0.40 \ln 1.6 = 0.1880$	0.325
1	0.25	0.30	$0.25 \ln 0.833 = -0.0456$	$0.30 \ln 1.2 = 0.0547$	0.275
2	0.25	0.20	$0.25 \ln 1.25 = 0.0558$	$0.20 \ln 0.8 = -0.0446$	0.225
3	0.25	0.10	$0.25 \ln 2.5 = 0.2291$	$0.10 \ln 0.4 = -0.0916$	0.175
Sum			**reverse KL ≈ 0.1218**	**forward KL ≈ 0.1064**	

For JSD: $D_{KL}(q\|m) \approx 0.0296$ and $D_{KL}(p\|m) \approx 0.0261$, so JSD $\approx \frac{1}{2}(0.0296 + 0.0261) \approx 0.0279$. JSD is always between 0 and $\ln 2 \approx 0.693$; both KLs are 0 exactly when $p = q$. These three values match what `distillation_loss` returns for this single token with kinds `'sdpo'`, `'opsd'`, and `'jsd'`.

4. SDPO: Self-Distillation Policy Optimization

Use feedback $z = f$ from the environment, such as a failing test, or successful same-question attempts. The foundational reverse-KL objective is

$$\boxed{L_{SDPO} = \mathbb{E}_{y \sim \pi_{old}} \frac{1}{|y|} \sum_t D_{KL}(p_t \| q_t)}.$$

Read it right to left: at every response position, measure how far the student's distribution is from the feedback-informed teacher's, average over the response's tokens, then average over sampled responses.

For a fixed teacher and prefix, the gradient with respect to student logit u_v is

$$\frac{\partial L}{\partial u_v} = p_v \Big[\log \frac{p_v}{q_v} - D_{KL}(p\|q) \Big].$$

Worked example (with $D_{KL}(p\|q) = 0.1218$):

v	$\log(p_v/q_v)$	$p_v[\log(p_v/q_v) - 0.1218]$	**Effect of a descent step**
0	-0.4700	$0.25(-0.5918) = -0.1479$	raise logit (teacher likes it more)
1	-0.1823	$0.25(-0.3041) = -0.0760$	raise logit
2	0.2231	$0.25(0.1014) = 0.0253$	lower logit
3	0.9163	$0.25(0.7945) = 0.1986$	lower logit (teacher likes it least)

These match the autograd gradient of `distillation_loss(..., kind='sdpo')`.

Sampled-token view. The sampled-token policy-gradient view uses detached advantage $\log q_t(y_t) - \log p_t(y_t)$; a constant baseline cancels in expectation. In the example, sampling token 0 gives "advantage" $\ln(0.40/0.25) = +0.470$ (the teacher approves more than the student), while token 3 gives $\ln(0.10/0.25) = -0.916$. This connects SDPO to the policy gradients of earlier lessons: the teacher's log-ratio plays the role of a dense, per-token reward.

The lab evaluates **full-vocabulary** reverse KL, not this sampled estimate. Practical variants can use JSD, teacher regularization, and importance correction for reused rollouts. Paper v1, equation 1.

5. OPSD: On-Policy Self-Distillation

Here $z = y^*$ is a reference answer or reasoning trace. The student samples; the privileged self-teacher evaluates those prefixes. Minimize

$$\boxed{L_{OPSD} = \mathbb{E}_{y \sim \pi_{old}} \frac{1}{|y|} \sum_t D(q_t \| p_t)}.$$

"On-policy" means the prefixes come from the student's own samples, so the student is corrected on the situations it actually produces, not on prefixes of the reference answer.

The paper allows divergence choices. The lab uses forward KL, whose student-logit gradient is simply

$$\frac{\partial D_{KL}(q\|p)}{\partial u_v} = p_v - q_v,$$

and also offers symmetric JSD with $m = (p+q)/2$: $\frac{1}{2}KL(q\|m) + \frac{1}{2}KL(p\|m)$. OPSD does not require a scalar verifier reward in its distillation loss. The teacher is detached even if its parameters are shared with the student. Paper v1, section 3.2.

Gradient comparison for the tiny example (computed by autograd on the script's `distillation_loss`):

Logit	Reverse KL (`sdpo`)	Forward KL (`opsd`), $p_v - q_v$	JSD (`jsd`)
u_0	-0.1479	$0.25 - 0.40 = -0.15$	-0.0361
u_1	-0.0760	$0.25 - 0.30 = -0.05$	-0.0152
u_2	0.0253	$0.25 - 0.20 = 0.05$	0.0099
u_3	0.1986	$0.25 - 0.10 = 0.15$	0.0413

All three push in the same directions here; they differ in magnitude, and they can differ more when one distribution puts near-zero probability where the other does not. This example does not show that any one divergence is better.

Why must the teacher be detached? If gradients reached the teacher, the loss could be lowered by moving the *teacher* toward the student instead of the student toward the teacher. Since the teacher may share parameters with the student, that would also corrupt the student. `teacher_logits.detach()` makes q a fixed target.

6. Averaging correctly over masked tokens

`masked_mean` first averages each sequence's valid tokens, then averages sequences. Using the script's mask `[[1,1,1],[1,1,0]]` and example token losses:

```
sequence 0 token losses: [0.3, 0.1, 0.2]      mask [1, 1, 1]
sequence 1 token losses: [0.4, 0.6, 9.0]      mask [1, 1, 0]   (9.0 is padding)
```

1. Sequence 0: $(0.3 + 0.1 + 0.2)/3 = 0.2$.
2. Sequence 1: $(0.4 + 0.6)/2 = 0.5$. The padded 9.0 is multiplied by 0.
3. Mean over sequences: $(0.2 + 0.5)/2 = 0.35$.

A flat average over all five valid tokens would give $1.6/5 = 0.32$ instead: that convention weights long responses more. Neither is universally correct; the script chooses per-sequence averaging. A sequence with no valid token raises `ValueError` rather than dividing by zero.

7. RLSD: RLVR with Self-Distillation

RLSD keeps the verifier (as in GRPO) and uses the teacher only to decide **how much** each token shares in the sequence's credit. Use verifier-derived group advantage A_i, retaining its sign while the teacher modulates token credit:

$$\Delta_{it} = \mathrm{sg}(\log q_{it}(y_{it}) - \log \pi_{old}(y_{it} \mid h_{it})),$$

$$w_{it} = \exp(\mathrm{sign}(A_i)\Delta_{it}), \qquad \boxed{\tilde{A}_{it} = \mathrm{sg}\{A_i[(1-\lambda) + \lambda\ \mathrm{clip}(w_{it}, 1-\epsilon_w, 1+\epsilon_w)]\}}.$$

Part	Interpretation
Δ_{it}	Teacher-vs-student log evidence for the sampled token
$\exp(\Delta)$	Evidence ratio $q(y_t)/\pi_{old}(y_t)$
$\mathrm{sign}(A_i)$	For a failed sequence, invert the ratio
$\mathrm{clip}(w, 0.8, 1.2)$	Teacher can change a token's credit by at most ±20%
λ	Blend between uniform credit ($\lambda = 0$) and teacher-weighted credit

Insert $\tilde{A}$ into the PPO/GRPO clipped policy surrogate using a separate current/old importance ratio. This follows Algorithm 1's credit construction; the lab supplies the differentiable policy surrogate explicitly. According to the cited paper version, teacher weights remain fixed across reuse epochs and no auxiliary KL distillation loss is needed (the lab has no reuse epochs, so it does not exercise the first point). $\lambda = 0$ recovers uniform sequence credit. Zero group advantage remains zero. Paper v1, equations 13–15 and Algorithm 1.

Hand calculation

Suppose $A = -1$, teacher probability 0.4, old student probability 0.2, $\epsilon_w = 0.2$, and $\lambda = 0.5$.

1. $\Delta = \ln 0.4 - \ln 0.2 = \ln 2$, so the evidence ratio is 2.
2. Negative advantage inverts it: $w = \exp(-\ln 2) = 0.5$.
3. Clip to $[0.8, 1.2]$: 0.8.
4. $\tilde{A} = -1[0.5 + 0.5(0.8)] = -0.9$.

The token is still penalized, with a smaller magnitude: the teacher thinks this token was reasonable, so it receives less blame for the failure. For $A = 1$, $w = 2$ clips to 1.2 and gives $\tilde{A} = 1[0.5 + 0.5(1.2)] = 1.1$: a teacher-endorsed token in a successful answer gets extra credit.

A	$q(y_t)$	$\pi_{old}(y_t)$	Ratio	w	Clipped	$\tilde{A}$ ($\lambda = 0.5$)
+1	0.40	0.20	2.0	2.0	1.2	1.1
-1	0.40	0.20	2.0	0.5	0.8	-0.9
+1	0.10	0.25	0.4	0.4	0.8	0.9
-1	0.10	0.25	0.4	2.5	1.2	-1.1
+1	0.26	0.25	1.04	1.04	1.04	1.02
0	0.40	0.20	2.0	1	1	0

Every row has been checked against `rlsd_advantages`. Note that the sign of $\tilde{A}$ always equals the sign of A: because the clipped weight is at least $0.8 > 0$, the teacher can only rescale credit, never flip it. With the first row's numbers, $\lambda = 0$ gives $\tilde{A} = -1$ (uniform) and $\lambda = 1$ gives -0.8 (fully teacher-weighted). In code, `sign(0)=0`, so $w = e^0 = 1$ and $\tilde{A} = 0 \times 1 = 0$.

The script also clamps the signed gap to $[-30, 30]$ before `exp`; this only prevents overflow and does not change any result after the $[0.8, 1.2]$ clip.

8. Two ratios with two different jobs

The teacher/student ratio measures evidence; the current/old ratio handles policy reuse. They have different purposes.

Ratio	Formula	Role	Gradient?
Evidence ratio	$q(y_t)/\pi_{old}(y_t)$	Scales token credit inside $\tilde{A}$	No, detached
Importance ratio	$\pi_\theta(y_t)/\pi_{old}(y_t)$	PPO-style clipped update	Yes, through π_θ

`rlsd_loss` computes `ratio = (new_token_logs-old_token_logs.detach()).exp()` and the clipped surrogate $\min(\rho\tilde{A}, \text{clip}(\rho, 0.8, 1.2)\tilde{A})$, then returns its negative masked mean.

9. Reproduce the script's RLSD number

The script samples tokens from a uniform old student (all zero logits, so $\pi_{old} = 0.25$ for every token) and scores them with random teacher logits. Sequence 0 gets $A = +1$, sequence 1 gets $A = -1$, and the mask is `[[1,1,1],[1,1,0]]`. With seed 7:

Sequence, position	$q(y_t)$	Evidence ratio $q/0.25$	w after sign	Clipped	$\tilde{A}$
0, 0	0.0539	0.215	0.215	0.8	0.9
0, 1	0.4274	1.709	1.709	1.2	1.1
0, 2	0.7033	2.813	2.813	1.2	1.1
1, 0	0.5063	2.025	0.494	0.8	-0.9

Sequence, position	$q(y_t)$	Evidence ratio $q/0.25$	w after sign	Clipped	$\tilde{A}$
1, 1	0.3131	1.252	0.798	0.8	-0.9
1, 2 (padding)	0.3179	—	—	—	masked out

Before any update the current policy equals the old one, so every $\rho = 1$ and the surrogate is just $\tilde{A}$:

1. Sequence 0: $(0.9 + 1.1 + 1.1)/3 = 1.0333$.
2. Sequence 1: $(-0.9 - 0.9)/2 = -0.9$.
3. Mean: $(1.0333 - 0.9)/2 = 0.0667$; the loss is its negative, **-0.0667**.

This is exactly the printed `RLSD sampled-token loss: -0.06666666269302368`.

Gradient check for sequence 0, position 0 (sampled token 3, $\tilde{A} = 0.9$): each valid token of sequence 0 carries weight $\frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}$, and $\partial \log \pi(3)/\partial u_v = 1[v = 3] - 0.25$. So the loss gradient is $-\frac{0.9}{6}(0.75) = -0.1125$ for the sampled token and $+\frac{0.9}{6}(0.25) = 0.0375$ for the others, which matches autograd. The padded position (sequence 1, position 2) receives a gradient of exactly zero.

10. Map the explanation to the script

Code	Meaning
`masked_mean(values, mask)`	Per-sequence mean over valid tokens, then mean over sequences
`teacher = teacher_logits.detach().log_softmax(-1)`	$\log q_t$, a stop-gradient target
`(student.exp()*(student-teacher)).sum(-1)`	SDPO reverse KL $D_{KL}(p\|q)$ per token
`(teacher.exp()*(teacher-student)).sum(-1)`	OPSD forward KL $D_{KL}(q\|p)$ per token
`torch.logaddexp(student, teacher)-torch.log(torch.tensor(2.))`	$\log m$, mixture for JSD
`gap = teacher_token_logs-old_token_logs`	Δ_{it} (inside `torch.no_grad()`)
`sequence_advantages[:, None].sign()*gap`	$\mathrm{sign}(A_i)\Delta_{it}$
`.clamp(-30, 30).exp().clamp(1-weight_clip, 1+weight_clip)`	$\mathrm{clip}(w_{it}, 0.8, 1.2)$ with overflow guard
`sequence_advantages[:, None]*((1-mixing)+mixing*weights)`	$\tilde{A}_{it}$
`ratio = (new_token_logs-old_token_logs.detach()).exp()`	Current/old importance ratio
`ratio.clamp(1-policy_clip, 1+policy_clip)*advantages`	PPO-clipped term
`teacher = torch.randn(2, 3, 4, requires_grad=True)`	Random teacher; `requires_grad` makes the "no teacher gradient" assertion meaningful
`optimizer = torch.optim.SGD([student], lr=0.5)`	50 steps of plain gradient descent on student logits

`distillation_loss` computes reverse KL, forward KL, or JSD. `rlsd_advantages` constructs detached credit. `rlsd_loss` applies the current/old policy ratio. `masked_mean` enforces valid tokens. Note that `rlsd_loss` does not expose `weight_clip`, so through that function ϵ_w is always the default 0.2.

11. Run it and read the output

Run from the course folder:

```
python3 04_alignment/05_sdpo_rlsd_opsd.py
```

With the fixed seed the output is:

```
sdpo loss: 0.29050520062446594 -> 0.02199612557888031
opsd loss: 0.24910327792167664 -> 0.02752235345542431
jsd loss: 0.06422306597232819 -> 0.03564245253801346
RLSD sampled-token loss: -0.06666666269302368
```

Each distillation line shows the loss before and after 50 SGD steps that move fresh zero-initialized student logits toward the same random teacher. The "before" values differ because they are different divergences of the same pair of distributions; compare each line only with itself. The RLSD line is the single loss value reproduced in section 9; no RLSD optimization steps are taken.

The assertions check that (1) each distillation loss decreased, (2) the teacher tensor never received a gradient, and (3) the RLSD gradient on the student logits is finite, with the teacher still gradient-free. They do not check convergence to zero, the correctness of any particular number, or any downstream task benefit. Random teacher logits illustrate the equations only; they do not demonstrate improved reasoning.

12. What a real experiment would need

When designing an experiment, evaluate held-out prompts without privileged information, correctness, answer leakage, response lengths, and performance on prior tasks. A falling distillation loss alone does not establish better task performance or justify general claims about the relative merits of these methods.

Answer leakage deserves special attention: the teacher sees z, and if the student is pushed to copy tokens that are only predictable from z (for example, the reference answer's exact numbers), it may learn to guess rather than reason. This lab cannot measure that risk.

13. Exercises

Exercise: verify a padded position has zero gradient. Set teacher equal to student and check both KLs vanish. Compare positive and negative A for the same evidence ratio. Explain why detached teacher targets are necessary.

14. Practice calculations

1. Student $p = [0.5, 0.5]$, teacher $q = [0.8, 0.2]$. Reverse KL $D_{KL}(p\|q)$? $0.5\ln(0.5/0.8) + 0.5\ln(0.5/0.2) = 0.5(-0.470) + 0.5(0.916) \approx 0.223$.
2. Forward KL $D_{KL}(q\|p)$ for the same pair? $0.8\ln 1.6 + 0.2\ln 0.4 = 0.376 - 0.183 \approx 0.193$.
3. Forward-KL gradient for the two student logits? $p - q = [-0.3,\ 0.3]$.
4. Token losses `[[0.2,0.4],[0.9,0.0]]` with mask `[[1,1],[1,0]]`. Masked mean? $(0.3 + 0.9)/2 = 0.6$.
5. RLSD with $A = +1$, $q(y_t) = 0.15$, $\pi_{old}(y_t) = 0.25$, $\lambda = 0.5$, $\epsilon_w = 0.2$. **Ratio 0.6, clipped to 0.8,** $\tilde{A} = 0.5 + 0.4 = 0.9$.
6. Same token but $A = -1$? $w = 1/0.6 \approx 1.667$**, clipped to 1.2,** $\tilde{A} = -(0.5 + 0.6) = -1.1$**: a token the teacher dislikes gets extra blame.**
7. $A = -2$, ratio 2, $\lambda = 0.5$? $w = 0.5 \to 0.8$**,** $\tilde{A} = -2(0.9) = -1.8$.
8. Can RLSD's teacher turn a negative advantage into a positive one? **No. The clipped weight lies in** $[0.8, 1.2]$**, so** $\tilde{A}$ **keeps the sign of** A.

Next: efficient training.

Companion code: `05_sdpo_rlsd_opsd.py`

04_alignment/05_sdpo_rlsd_opsd.py

```python
"""SDPO, OPSD and RLSD objective labs, not complete LLM trainers.
Shapes: logits [batch, time, vocabulary], token IDs and masks [batch, time].
Read the companion chapter for the chosen paper versions and simplifications.
"""
import torch


def masked_mean(values, mask):
    # Average tokens within each response, then average valid responses.
    counts = mask.sum(-1)
    if not (counts > 0).all():
        raise ValueError('Each response must contain at least one valid token')
    return ((values*mask).sum(-1)/counts).mean()


def distillation_loss(student_logits, teacher_logits, mask, kind='sdpo'):
    student = student_logits.log_softmax(-1)
    teacher = teacher_logits.detach().log_softmax(-1)
    if kind == 'sdpo':  # reverse KL: student || feedback-conditioned teacher
        token_loss = (student.exp()*(student-teacher)).sum(-1)
    elif kind == 'opsd':  # forward-KL teaching choice: teacher || student
        token_loss = (teacher.exp()*(teacher-student)).sum(-1)
    elif kind == 'jsd':
        mixture = torch.logaddexp(student, teacher)-torch.log(torch.tensor(2.))
        token_loss = 0.5*((teacher.exp()*(teacher-mixture)).sum(-1)
                          +(student.exp()*(student-mixture)).sum(-1))
    else:
        raise ValueError(kind)
    return masked_mean(token_loss, mask)


def rlsd_advantages(sequence_advantages, teacher_token_logs, old_token_logs,
                    mixing=0.5, weight_clip=0.2):
    if not 0 <= mixing <= 1 or not 0 <= weight_clip < 1:
        raise ValueError('mixing in [0,1], weight_clip in [0,1) required')
    with torch.no_grad():
        gap = teacher_token_logs-old_token_logs
        signed_gap = sequence_advantages[:, None].sign()*gap
        weights = signed_gap.clamp(-30, 30).exp().clamp(1-weight_clip, 1+weight_clip)
        return sequence_advantages[:, None]*((1-mixing)+mixing*weights)


def rlsd_loss(new_token_logs, old_token_logs, teacher_token_logs,
              sequence_advantages, mask, mixing=0.5, policy_clip=0.2):
    advantages = rlsd_advantages(sequence_advantages, teacher_token_logs,
                                 old_token_logs, mixing)
    ratio = (new_token_logs-old_token_logs.detach()).exp()
    objective = torch.minimum(ratio*advantages,
                              ratio.clamp(1-policy_clip, 1+policy_clip)*advantages)
    return -masked_mean(objective, mask)


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    mask = torch.tensor([[1., 1., 1.], [1., 1., 0.]])
    teacher = torch.randn(2, 3, 4, requires_grad=True)
```

```
    for kind in ('sdpo', 'opsd', 'jsd'):
        student = torch.zeros(2, 3, 4, requires_grad=True)
        optimizer = torch.optim.SGD([student], lr=0.5)
        before = distillation_loss(student, teacher, mask, kind).item()
        for _ in range(50):
            optimizer.zero_grad()
            loss = distillation_loss(student, teacher, mask, kind)
            loss.backward()
            optimizer.step()
        after = distillation_loss(student, teacher, mask, kind).item()
        print(kind, 'loss:', before, '->', after)
        assert after < before and teacher.grad is None
    # Actions sampled from the unprivileged old student, not the teacher.
    logits = torch.zeros(2, 3, 4, requires_grad=True)
    with torch.no_grad():
        old_all = logits.log_softmax(-1)
        tokens = torch.distributions.Categorical(logits=logits).sample()
        old = old_all.gather(-1, tokens[..., None]).squeeze(-1)
        teacher_logs = teacher.log_softmax(-1).gather(-1, tokens[..., None]).squeeze(-1)
    new = logits.log_softmax(-1).gather(-1, tokens[..., None]).squeeze(-1)
    loss = rlsd_loss(new, old, teacher_logs, torch.tensor([1., -1.]), mask)
    loss.backward()
    print('RLSD sampled-token loss:', loss.item())
    assert torch.isfinite(logits.grad).all() and teacher.grad is None


if __name__ == '__main__':
    main()
```

PART 5

Advanced topics

5.1

Efficient training: accumulation, precision, clipping, schedules, and checkpoints

Companion code: 01_training_efficiency.py. Prerequisites: neural-network training loops and tensors and autograd. Read this explanation before running the code. No GPU is required; on a CPU the script quietly turns the mixed-precision parts off (section 11 explains how).

1. What problem are we solving now?

A real fine-tuning run is usually limited by memory and time, not by ideas. You may want 16 examples per update, but only 4 fit on the device at once. You may want faster arithmetic, but lower-precision numbers can round small gradients to zero. You may need to stop a run and resume it tomorrow.

The script packs five standard tools into one tiny regression loop:

Tool	Question it answers
Gradient accumulation	How can 4-example forward passes produce a 16-example update?
Mixed precision + loss scaling	How can FP16 arithmetic avoid losing small gradients?
Gradient clipping	How do we stop one unusually large gradient from causing a huge step?
Warmup + cosine schedule	How should the learning rate change across updates?
Disk checkpoint	What must be saved so training can be resumed later?

The toy setup: 36 random inputs with 8 features, 36 random targets, a two-layer network `Linear(8, 32) -> ReLU -> Linear(32, 1)`, mean-squared-error loss, and AdamW with base learning rate 0.003. The targets are random noise, so nothing meaningful is learned. The point is the **mechanics** of each update.

2. First identify which memory you need to reduce

Training stores four kinds of numbers:

Stored item	Grows with	Main technique that reduces it
Parameters	Model size	Smaller model, quantization (lesson 04)
Gradients	Number of trainable parameters	Freezing, LoRA (LoRA lesson)
Optimizer state (AdamW keeps two moments per parameter)	Number of trainable parameters	LoRA, sharded optimizers
Intermediate activations	Batch size x sequence length x depth	Gradient accumulation, activation checkpointing

Gradient accumulation reduces the per-forward batch of activations; mixed precision changes numeric storage and computation; LoRA reduces trainable parameter state; activation checkpointing recomputes intermediate values. None is a universal memory fix. In particular, accumulation does **not** shrink parameters, gradients, or optimizer state at all.

3. Notation

Symbol	Meaning	Example / Code
N	Examples in one effective (optimizer-step) batch	`samples` (16, 16, then 4)
n_k	Examples in microbatch k	`len(inputs)` = 4
L_k	Mean loss over microbatch k	`(model(inputs)-labels).square().mean()`
L	Mean loss over the whole effective batch	sum of weighted microbatch losses
K	Accumulation count (microbatches per update)	`accumulation = 4`
W	Number of data-parallel workers	1 here (distributed lesson)
S	Loss scale for FP16	`scaler` (65536 by default on CUDA; disabled on CPU)
c	Gradient-norm clipping threshold	`1.0` in `clip_grad_norm_`
u	Index of optimizer update, starting at 0	`updates`
$m(u)$	Learning-rate multiplier	`multiplier`
η	Actual learning rate	`0.003*multiplier`

4. How many updates does the script make?

Count step by step:

1. 36 examples split into microbatches of 4 gives $36/4 = 9$ microbatches.
2. Groups of $K = 4$ microbatches: microbatches 1-4, 5-8, and 9 alone.
3. So `total_updates = math.ceil(9/4) = 3` optimizer updates.

Update u	Microbatches in group	Examples N	Weight per microbatch n_k/N
0	4	16	$4/16 = 0.25$
1	4	16	$4/16 = 0.25$
2	1	4	$4/4 = 1$

The script docstring states the general rule:

$$\boxed{\text{effective batch} = \text{microbatch} \times K \times W}$$

Here $4 \times 4 \times 1 = 16$ for full groups. With 8 data-parallel workers each doing the same, one update would see $4 \times 4 \times 8 = 128$ examples. The last group is short (4 examples), which is why the weighting below uses the **actual** count rather than the nominal 16.

5. Why gradient accumulation can equal a larger batch

Suppose an effective batch has N examples split into microbatches of sizes n_k, with microbatch mean loss L_k. Then:

$$\boxed{L = \sum_k \frac{n_k}{N} L_k, \qquad \nabla L = \sum_k \frac{n_k}{N} \nabla L_k.}$$

The second equality follows from linearity of differentiation. Backward calls **add** gradients into parameter `.grad` buffers, so: weight each microbatch loss, call backward for each, and step the optimizer only after the group is complete. Clear gradients before the group, not between its microbatches.

Worked example: six examples, one weight

Take a one-parameter model $\hat{y} = wx$ with $w = 0.5$ and six examples, split into a microbatch of 4 and a microbatch of 2:

x	1	2	3	4	5	6
y	2	1	4	3	6	9
$wx - y$	-1.5	0	-2.5	-1	-3.5	-6
$(wx - y)^2$	2.25	0	6.25	1	12.25	36

Microbatch losses: $L_1 = (2.25 + 0 + 6.25 + 1)/4 = 2.375$ and $L_2 = (12.25 + 36)/2 = 24.125$. Full-batch loss: $57.75/6 = 9.625$, which equals $\frac{4}{6}(2.375) + \frac{2}{6}(24.125)$.

The gradient of a mean squared error is $\frac{1}{n}\sum 2x(wx - y)$:

- Microbatch 1: $\frac{2}{4}[1(-1.5) + 2(0) + 3(-2.5) + 4(-1)] = \frac{2}{4}(-13) = -6.5$.
- Microbatch 2: $\frac{2}{2}[5(-3.5) + 6(-6)] = -53.5$.

How microbatch gradients are combined	Accumulated gradient
Weight by size: $\frac{4}{6}(-6.5) + \frac{2}{6}(-53.5) = -4.333 - 17.833$	**-22.167** (equals full batch)
Naive average: $(-6.5 - 53.5)/2$	-30.0 (wrong: over-weights the 2 examples)

The correct accumulated value matches the gradient computed on all six at once (both -22.1667 in PyTorch). The script's line `loss = loss*len(inputs)/samples` is exactly the n_k/N weight.

The short last group

In the script's third update there is only one microbatch. Its weight is $4/4 = 1$. Blindly dividing every loss by $K = 4$ would give it weight 0.25, shrinking that update's gradient by a factor of 4. (AdamW partly normalizes gradient scale, but its moment estimates average over earlier steps, so the shrink is not cleanly cancelled.) For token losses, weight by supervised-token count rather than example count if that is the intended objective.

When the equivalence breaks

Exact large-batch equivalence also requires compatible randomness, reductions, and batch-dependent operations. BatchNorm, for example, sees different statistics on a microbatch than on a combined batch. Dropout masks differ unless random streams match. Optimizer steps between microbatches also break the equivalence because parameters and optimizer state change. Floating-point addition order differs, so expect agreement up to small rounding differences, not bit-identity.

6. Mixed precision: why small gradients disappear

FP16 uses less memory and can run faster on supported hardware, but has limited range. Its key limits (from `numpy.finfo(numpy.float16)`):

FP16 property	Value
Largest finite value	65504
Smallest normal value	$\approx 6.10 \times 10^{-5}$
Smallest positive (subnormal) value	$\approx 5.96 \times 10^{-8}$

A gradient smaller than about 3×10^{-8} rounds to zero: it **underflows**. Values just above that keep very few significant bits.

Fix: scale the loss by S before backward. By linearity,

$$\boxed{\nabla_\theta(SL) = S\nabla_\theta L.}$$

Every backward value is S times larger, so small values move into FP16's representable range. Before the optimizer uses them, the gradients are divided by S again in FP32.

Worked example with $S = 65536 = 2^{16}$ (PyTorch's default initial scale)

True gradient	Stored directly in FP16	Scaled $\times 65536$ in FP16	Unscaled back (FP32)
1×10^{-8}	0 (lost)	0.000655	$\approx 1.0 \times 10^{-8}$
3×10^{-8}	6×10^{-8} (100% error)	0.001966	$\approx 3.0 \times 10^{-8}$
1×10^{-4}	0.0001	6.555	$\approx 1.0 \times 10^{-4}$
2.0	2.0	**inf** (131072 > 65504)	step must be skipped

The last row shows the opposite danger: **overflow**. Dynamic loss scaling handles it. With PyTorch's `GradScaler` defaults, if any gradient is inf or NaN the optimizer step is skipped and S is multiplied by 0.5 (65536 -> 32768); after 2000 consecutive successful steps S is multiplied by 2. These are library defaults (`backoff_factor=0.5`, `growth_factor=2.0`, `growth_interval=2000`), not values chosen in this script.

In PyTorch autocast, parameters and their `.grad` buffers usually stay FP32; the FP16 values are activations and intermediate results of eligible operations. Underflow therefore happens to gradients flowing backward through FP16 operations, which is exactly what scaling

protects.

BF16 has a wider exponent range (about the same as FP32) and often does not need loss scaling, but supported kernels and hardware still matter. The script uses FP16 on CUDA and ordinary FP32 on CPU; it does not demonstrate BF16 execution.

7. The order of operations matters

For each update the script does:

1. Clear gradients once: `optimizer.zero_grad(set_to_none=True)`.
2. For every microbatch: autocast the forward pass (CUDA only), weight the loss, then `scaler.scale(loss).backward()` so gradients accumulate.
3. Unscale gradients **once** after accumulation: `scaler.unscale_(optimizer)`.
4. Clip the actual gradient norm: `clip_grad_norm_(..., 1.0)`.
5. Attempt the optimizer step (`scaler.step`) and update the scale (`scaler.update`).

Why must unscaling come before clipping? Gradient clipping applies

$$\boxed{g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|}\right)}$$

with $c = 1$ here. Take a true gradient $g = (3, 4)$, so $\|g\| = 5$:

- **Correct order** (unscale, then clip): $g \cdot \min(1, 1/5) = (0.6, 0.8)$, norm 1.
- **Wrong order** with $S = 1024$: the scaled gradient is $(3072, 4096)$, norm 5120. Clipping gives $(0.6, 0.8)$; unscaling then divides by 1024, leaving $(0.000586, 0.000781)$, norm $1/1024 \approx 0.000977$.

Clipping before unscaling enforced the threshold on artificially enlarged gradients, making the real step about 1000 times too small.

`clip_grad_norm_` uses one **global** norm over all parameters together, not a separate norm per tensor. Clipping controls gradient magnitude, not the exact AdamW parameter step magnitude, because AdamW divides by its own running second-moment estimate.

8. Learning-rate schedule: warmup, then cosine decay

Warmup increases the multiplier over initial updates. Afterwards a cosine multiplier takes the form:

$$\boxed{m(u) = \tfrac{1}{2}\left[1 + \cos(\pi p(u))\right], \qquad p(u) = \frac{u - \text{warmup}}{\max(1,\ \text{total} - \text{warmup})}}$$

where p measures progress after warmup. During warmup the code uses $m(u) = (u + 1)/\text{warmup}$. Schedules should count effective **optimizer updates**, not microbatches, which is why the script computes the multiplier once per group.

With `warmup = 1` and `total_updates = 3`:

u	Branch	Computation	$m(u)$	Learning rate
0	warmup	$(0+1)/1$	1.0	0.003
1	cosine, $p = 0/2 = 0$	$\frac{1}{2}(1+\cos 0)$	1.0	0.003
2	cosine, $p = 1/2$	$\frac{1}{2}(1+\cos(\pi/2))$	0.5	0.0015

Two honest observations about this tiny setting:

- With `warmup = 1` there is no visible ramp: the single warmup step already has multiplier 1. A real run would use many warmup updates (e.g. hundreds), starting from a small fraction of the base rate.
- The last multiplier is 0.5 rather than reaching zero. That follows from the code's indexing (p never reaches 1 in three updates), not a convergence promise. The saved optimizer state indeed records `lr = 0.0015`.

CUDA overflow-skipped steps are not specially accounted for in this teaching counter (`updates` increases even if `scaler.step` skipped the step); a production scheduler should define its skipped-step behavior.

9. What a resumable checkpoint needs

Model weights define predictions, but future training also depends on optimizer moments, scaler state, RNG state, scheduler progress, and data order. The file saves:

Key in the saved dictionary	Why it is needed
`'model'`	Network weights
`'optimizer'`	AdamW moments, step counts, and current learning rate
`'scaler'`	Current loss scale and growth tracker (an empty dict `{}` on CPU, where the scaler is disabled)
`'updates'`	Update counter (3 at the end), which drives the schedule
`'torch_rng'`	CPU PyTorch random-number-generator state

It then reloads them with `weights_only=True` as a demonstration.

It does not capture Python/NumPy/CUDA RNG, sampler cursor, all configuration, or a general scheduler state. Thus it is not an exact full-run-resume system. Also note the script saves **after** training finishes, so the reload only proves the round trip works; it does not continue training afterwards. Activation checkpointing in lesson 06 is a separate memory technique, not this disk-saving operation, despite the shared word.

10. Map the explanation to the script

Code	Meaning
`device = 'cuda' if torch.cuda.is_available() else 'cpu'`	Choose GPU if present; everything else adapts

Code	Meaning
`scaler = torch.amp.GradScaler('cuda', enabled=device == 'cuda')`	Dynamic loss scaling, only active on CUDA
`microbatch, accumulation = 4, 4`	$n_k = 4$, $K = 4$
`total_updates = math.ceil(len(microbatches)/accumulation)`	$\lceil 9/4 \rceil = 3$ optimizer updates
`samples = sum(len(a) for a, _ in group)`	N for this group: 16, 16, then 4
`optimizer.zero_grad(set_to_none=True)`	Clear gradients once per group, before accumulation
`multiplier = (updates+1)/warmup if updates < warmup else 0.5*(1+math.cos(...))`	Warmup then cosine $m(u)$
`param_group['lr'] = 0.003*multiplier`	Set actual learning rate η
`torch.autocast('cuda', dtype=torch.float16) if device == 'cuda' else nullcontext()`	FP16 forward on CUDA; plain FP32 on CPU
`loss = loss*len(inputs)/samples`	Weight n_k/N, correct for the short last group
`scaler.scale(loss).backward()`	Backward on $S\cdot$ loss; gradients add up
`scaler.unscale_(optimizer)`	Divide accumulated gradients by S once
`nn.utils.clip_grad_norm_(model.parameters(), 1.0)`	Global-norm clipping with $c = 1$
`scaler.step(optimizer)` / `scaler.update()`	Step (skipped on inf/NaN) and adjust S
`torch.save({...}, output)`	Write checkpoint to `outputs/training_checkpoint.pt`
`torch.load(output, map_location=device, weights_only=True)`	Reload safely (tensors and plain containers only)

11. Run it

Run from the course folder:

```
python3 05_advanced/01_training_efficiency.py
```

The only output is one line, for example:

```
saved and reloaded checkpoint: <course folder>/outputs/training_checkpoint.pt
```

There is no assertion and no printed loss. Success means: three updates ran, the checkpoint was written to `outputs/`, and all pieces loaded back without error.

On a CPU-only machine (the setting used to verify this lesson) the script still runs, but `GradScaler(..., enabled=False)` makes `scale` return the loss unchanged, `unscale_` and `update` do nothing, and `step` simply calls `optimizer.step()`; autocast is replaced by `nullcontext()`. So on CPU you are exercising accumulation, clipping, the schedule, and checkpointing in FP32, not FP16 loss scaling. The CUDA mixed-precision path was not tested here.

12. Practice calculations

1. 100 examples, microbatch 4, accumulation 8, one worker. How many optimizer updates, and how big is the last group? **25 microbatches;** $\lceil 25/8 \rceil = 4$ **updates; the last group has 1 microbatch = 4 examples.**
2. In that last group, what weight should its single microbatch loss get? $4/4 = 1$**, not** $1/8$**.**
3. Microbatch 8, accumulation 4, 4 GPUs. Effective batch? $8 \times 4 \times 4 = 128$**.**
4. Gradient $(6, 8)$ with $c = 1$. Clipped gradient? **Norm 10, so** $(0.6, 0.8)$**.** With $c = 20$? **Unchanged,** $(6, 8)$**, because** $\min(1, 20/10) = 1$**.**
5. A true gradient of 5×10^{-9} with $S = 65536$: what is the scaled value, and does it survive FP16? **About** 3.3×10^{-4}**; yes, it is representable.**
6. Warmup 1, total 5 updates. Multipliers for $u = 0, \ldots, 4$? $p = 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}$ **for** $u = 1..4$**, giving 1, 1, 0.854, 0.5, 0.146.**
7. Exercise from the script: compare one FP32 accumulated gradient with the same full-batch gradient before stepping, using a model without dropout or BatchNorm. **They should agree up to floating-point summation differences.**

Next: Soft Actor-Critic: continuous actions and entropy.

Companion code: `01_training_efficiency.py`

05_advanced/01_training_efficiency.py

```python
"""Accumulation, mixed precision, clipping, scheduling, checkpointing.
Effective batch = microbatch * accumulation_steps * number_of_data_parallel_workers.
Accumulation saves activation memory, not model/optimizer memory. It matches a
large batch only with compatible reduction, randomness and batch-dependent layers.
Exercise: compare this FP32 update with a single full-batch update.
"""
from contextlib import nullcontext
from pathlib import Path
import math
import torch
from torch import nn


def main():
    torch.manual_seed(7)
    torch.set_num_threads(1)
    device = 'cuda' if torch.cuda.is_available() else 'cpu'
    model = nn.Sequential(nn.Linear(8, 32), nn.ReLU(), nn.Linear(32, 1)).to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=0.003)
    # FP16 needs loss scaling to reduce underflow. BF16 often does not.
    scaler = torch.amp.GradScaler('cuda', enabled=device == 'cuda')
    x, y = torch.randn(36, 8, device=device), torch.randn(36, 1, device=device)
    microbatch, accumulation = 4, 4
    microbatches = [(x[i:i+microbatch], y[i:i+microbatch]) for i in range(0, len(x),
microbatch)]
    total_updates = math.ceil(len(microbatches)/accumulation)
    updates = 0
    for start in range(0, len(microbatches), accumulation):
        group = microbatches[start:start+accumulation]
        samples = sum(len(a) for a, _ in group)
        optimizer.zero_grad(set_to_none=True)
        # Warmup then cosine decay, stepped per optimizer update, not microbatch.
        warmup = 1
        multiplier = (updates+1)/warmup if updates < warmup else 0.5*(1+math.cos(
            math.pi*(updates-warmup)/max(1, total_updates-warmup)))
        for param_group in optimizer.param_groups:
            param_group['lr'] = 0.003*multiplier
        for inputs, labels in group:
            context = torch.autocast('cuda', dtype=torch.float16) if device == 'cuda' else
nullcontext()
            with context:
                loss = (model(inputs)-labels).square().mean()
                # Weight by actual sample count, including the final short group.
                # Token losses instead need weighting by supervised token count.
                loss = loss*len(inputs)/samples
            scaler.scale(loss).backward()
        scaler.unscale_(optimizer)  # Clip real gradients, not scaled gradients.
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
        updates += 1
    output = Path(__file__).resolve().parents[1]/'outputs/training_checkpoint.pt'
    output.parent.mkdir(exist_ok=True)
    torch.save({'model': model.state_dict(), 'optimizer': optimizer.state_dict(),
                'scaler': scaler.state_dict(), 'updates': updates,
                'torch_rng': torch.get_rng_state()}, output)
    checkpoint = torch.load(output, map_location=device, weights_only=True)
```

```
    model.load_state_dict(checkpoint['model'])
    optimizer.load_state_dict(checkpoint['optimizer'])
    scaler.load_state_dict(checkpoint['scaler'])
    torch.set_rng_state(checkpoint['torch_rng'].cpu())
    print('saved and reloaded checkpoint:', output)
    # Exact full-run resume also needs sampler position, Python/NumPy/CUDA RNG,
    # scheduler state and data preprocessing/config versions.

if __name__ == '__main__':
    main()
```

5.2

Soft Actor-Critic: continuous actions, entropy, and twin value estimates

Companion code: 02_sac_update.py. Prerequisites: Q-learning targets, DQN targets and target networks, policy gradients and actor-critic, and autograd. Read this explanation before running the code.

1. What problem are we solving now?

The corridor in earlier lessons had two actions: left or right. Many control problems instead have **continuous** actions: a steering angle, a torque, or a joint velocity. Here each action is a vector of two numbers, each in $(-1, 1)$.

Two consequences follow:

- DQN's `max` over actions is no longer a lookup over a short list. We cannot enumerate infinitely many actions, so a separate **actor** network proposes them.
- Exploring by occasionally choosing a uniformly random action (epsilon-greedy) is clumsy. SAC instead uses a **stochastic** policy and adds a reward for keeping it random, which encourages exploration and avoids choosing one narrow action too early.

SAC learns a stochastic continuous-action policy using **off-policy replay**: it reuses stored transitions collected by older versions of the policy.

What this file actually does. It executes one full set of SAC update equations on 32 **synthetic** replay transitions: random states with 3 numbers, random actions with 2 numbers, random rewards. It does not collect real transitions, run an environment, or demonstrate a learned control policy. It is an equation lab: you can check every tensor shape and gradient path.

2. Notation

Symbol	Meaning	Example / Code
s, s'	State and next state, 3 numbers each	`states`, `next_states`, shape `[32,3]`
a	Stored replay action, 2 numbers in $[-1, 1)$	`actions`, shape `[32,2]`
r	Reward	`rewards`, shape `[32,1]`
d	1 if the transition terminated the task	`terminated` (all zeros here)
γ	Discount	`0.99`
$\pi_\theta(a \mid s)$	Actor's action density	`actor.sample(...)`
μ, σ	Gaussian mean and standard deviation before squashing	`mean`, `log_std.clamp(-5, 2).exp()`
u	Unsquashed Gaussian sample	`raw`

Symbol	Meaning	Example / Code
Q_{ϕ_1}, Q_{ϕ_2}	Two online critics	`q1`, `q2`
$Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$	Slow target critics	`target1`, `target2`
α	Entropy temperature, $\alpha = e^{\ell_\alpha}$	`log_alpha.exp()`, starts at $e^{-1} \approx 0.3679$
H_{target}	Target entropy	`target_entropy = -2.0`
τ	Polyak averaging rate	`0.005`

Alpha here controls the reward/exploration balance. It is unrelated to LoRA's adapter scale and to the learning rate α of the Q-learning lesson.

3. The maximum-entropy objective

Conceptually SAC optimizes discounted reward **plus** entropy:

$$\boxed{J(\pi) = \mathbb{E}\left[\sum_t \gamma^t \{r_t + \alpha H(\pi(\cdot \mid s_t))\}\right]}$$

Entropy H measures how spread out the action distribution is. Because entropy is $-\mathbb{E}[\log \pi(a \mid s)]$, sampled objectives use $-\alpha \log \pi(a \mid s)$ as a per-sample bonus: an unlikely (low $\log \pi$) action gets a larger bonus.

In continuous spaces this is **differential** entropy, which can be negative and changes under action rescaling. A density can exceed 1, so $\log \pi$ can be positive and the "entropy bonus" can be a penalty. You will see this in the worked example of section 5.

4. A differentiable stochastic actor

The actor's single linear layer maps the 3 state numbers to 4 outputs: two action means and two log standard deviations. It then samples:

$$\boxed{u = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \qquad a = \tanh(u).}$$

This is **reparameterization**: randomness enters through independent noise ϵ, while the sampled value remains differentiable with respect to the actor's parameters. `Normal.rsample()` supplies that path. Ordinary `.sample()` would not preserve this pathwise gradient, and the actor update in section 7 would receive no signal from the critic.

The code clamps the log standard deviation to $[-5, 2]$, so σ stays between $e^{-5} \approx 0.0067$ and $e^2 \approx 7.39$.

5. Why the tanh squash needs a log-probability correction

Tanh bounds each action to $(-1, 1)$. But the Gaussian density is a density over u, not over the transformed a. Squashing piles probability up near ± 1, so the density of a is different. Change of variables gives, per dimension j:

$$\boxed{\log \pi(a \mid s) = \sum_j \left[\log \mathcal{N}(u_j; \mu_j, \sigma_j) - \log(1 - \tanh^2 u_j)\right]}$$

because $\frac{d}{du} \tanh u = 1 - \tanh^2 u$. The script evaluates the correction with the stable identity

$$\boxed{\log(1 - \tanh^2 u) = 2[\log 2 - u - \operatorname{softplus}(-2u)]}$$

which follows from $1 - \tanh^2 u = \frac{4e^{-2u}}{(1+e^{-2u})^2}$ and $\operatorname{softplus}(x) = \log(1 + e^x)$.

Worked example: one two-dimensional sample

Suppose for some state the actor outputs $\mu = (0.2, -0.4)$ and $\log \sigma = (-0.5, 0)$, so $\sigma = (0.6065, 1)$, and the noise is $\epsilon = (0.5, -1.0)$.

Gaussian log-density per dimension is $\log \mathcal{N} = -\frac{1}{2}\epsilon^2 - \log \sigma - \frac{1}{2}\log(2\pi)$, with $\frac{1}{2}\log(2\pi) = 0.9189$.

Step	Dimension 1	Dimension 2
$u = \mu + \sigma\epsilon$	$0.2 + 0.6065(0.5) = 0.5033$	$-0.4 + 1(-1) = -1.4$
$a = \tanh u$	0.4647	-0.8854
$\log \mathcal{N}(u)$	$-0.125 + 0.5 - 0.9189 = -0.5439$	$-0.5 - 0 - 0.9189 = -1.4189$
$\log(1 - \tanh^2 u)$	-0.2433	-1.5318
corrected $\log \mathcal{N} - \log(1 - \tanh^2 u)$	-0.3007	+0.1128

Sum over dimensions: $\log \pi(a \mid s) = -0.3007 + 0.1128 = -0.1879$. Without the correction you would have used -1.9629, which is wrong by 1.775. Note dimension 2's corrected log-density is **positive**: near $a = -0.885$ the squashed density is above 1. That is the negative-differential-entropy effect from section 3.

Why the stable identity matters

For $u = 10$ in float32, $\tanh(10)$ rounds to exactly 1, so the naive $\log(1 - \tanh^2 u) = \log 0 = -\infty$. The identity gives $2[0.6931 - 10 - \operatorname{softplus}(-20)] = -18.6137$, a finite correct value. It avoids subtracting nearly equal floating-point numbers when tanh saturates.

If an environment requires actions outside $[-1, 1]$, affine rescaling and the corresponding density correction need to be handled explicitly.

6. Critic target: bootstrap value, entropy, and the twin minimum

SAC uses two critics and slow target copies of each. For a replay transition, sample a **fresh** next action a' from the current actor at s' (not the action stored in replay), then compute:

$$\boxed{y = r + \gamma(1 - d)\left[\min_{i=1,2} Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a' \mid s')\right]}$$

Each critic minimizes $\mathbb{E}[(Q_{\phi_i}(s,a)-y)^2]$ with a **detached** target (the script builds y under `torch.no_grad()`). The minimum of two estimates can reduce the optimistic value errors that a single critic's maximization tends to accumulate, the same overestimation issue seen with DQN.

Worked example (earlier toy numbers)

Target critics predict 3 and 5, $\log\pi(a'\mid s')=-1$, $\alpha=0.2$, $r=1$, $\gamma=0.9$, nonterminal:

$$y=1+0.9[\min(3,5)-0.2(-1)]=1+0.9(3.2)=3.88.$$

For a terminal transition ($d=1$) the bracket is multiplied by zero and $y=1$.

Same example with the script's settings

Now use $\gamma=0.99$ and the initial $\alpha=e^{-1}=0.36788$:

1. Twin minimum: $\min(3,5)=3$.
2. Entropy term: $-\alpha\log\pi=-0.36788(-1)=+0.36788$.
3. Soft next value: $3+0.36788=3.36788$.
4. Target: $y=1+0.99(3.36788)=4.3342$.

Variant of the next-value estimate	Target y
Twin minimum (what SAC uses)	4.3342
Average of the two critics, 4	5.3242
Larger critic, 5	6.3142
Twin minimum but $\alpha=0$ (no entropy term)	3.9700

The min choice gives the most cautious target; the entropy term adds value to next states where the policy can remain random.

An implementation limitation you must notice

In the script `q2` is a deep copy of `q1`, and both critics receive the same data, target, and symmetric optimizer updates. They remain **identical** in this toy update, so `torch.minimum(q1(...), q2(...))` always equals `q1(...)` and the printed critic loss is exactly twice one critic's MSE ($1.18791/2\approx0.594$ each). The code shows the twin-critic formula, but does not actually demonstrate its error-diversification benefit. A practical extension should initialize critics independently and evaluate whether their errors differ.

7. Actor update: increase soft value through the sampled action

The actor minimizes:

$$\boxed{L_\pi=\mathbb{E}_s\left[\alpha\log\pi_\theta(a\mid s)-\min_i Q_{\phi_i}(s,a)\right],\quad a=f_\theta(s,\epsilon).}$$

Read it as: prefer actions the critics value highly, but pay α for every unit of $\log \pi$ (i.e. reward staying random). For one sample with $\alpha = 0.36788$, $\log \pi = -1$, and critic values 3 and 5, the contribution is $0.36788(-1) - 3 = -3.3679$. The script's printed actor loss is the average of such terms over 32 states.

Gradients reach θ along two paths: through $\log \pi_\theta$, and through the critic's output with respect to the sampled action $a = f_\theta(s, \epsilon)$.

Freeze critic **parameters**, but retain the derivative of the critic output with respect to the sampled action. Otherwise the actor cannot learn which actions increase value. The script's `requires_grad_(False)` does this; wrapping the entire actor-loss critic computation in `no_grad()` would remove the useful path too. The actor loss also uses `log_alpha.exp().detach()`, so this step does not change alpha.

8. Temperature update: aim for a target entropy

Alpha must stay positive, so the script learns $\ell_\alpha = \log \alpha$ and uses $\alpha = e^{\ell_\alpha}$. Its common log-temperature surrogate is:

$$\boxed{L_\alpha = -\mathbb{E}\left[\ell_\alpha \ \text{stopgrad}(\log \pi(a \mid s) + H_{target})\right]}$$

Here $H_{target} = -2$ for two action dimensions (the common "negative action dimensionality" heuristic). The derivative is

$$\frac{\partial L_\alpha}{\partial \ell_\alpha} = -\mathbb{E}[\log \pi + H_{target}] = \hat{H} - H_{target},$$

where $\hat{H} = -\mathbb{E}[\log \pi]$ is the sampled entropy estimate. Gradient descent therefore lowers alpha when entropy is above target and raises it when entropy is below target.

Worked example: two cases, starting from $\ell_\alpha = -1$

Mean $\log \pi$	Entropy estimate $\hat{H}$	Gradient $\hat{H} - (-2)$	Direction	ℓ_α after one Adam step (lr 0.001)	New α
-1	1	+3	entropy above target: lower α	-1.001	0.36751
3	-3	-1	entropy below target: raise α	-0.999	0.36825

Why exactly ± 0.001? On Adam's **first** step, bias correction makes the update approximately $\text{lr} \times \text{sign}(g)$, regardless of how large g is (a gradient of 300 also moves ℓ_α by 0.001). This is an Adam property, not a SAC property.

The log-alpha surrogate has a different gradient scaling from optimizing the exact alpha-parameterized dual objective, but the same desired balance point. In the script the alpha update uses the `log_prob` sampled **before** the actor step, detached.

9. Target networks: Polyak averaging

Targets move slowly by Polyak averaging:

$$\boxed{\bar{\phi} \leftarrow (1-\tau)\bar{\phi} + \tau\phi, \qquad \tau = 0.005.}$$

If a target weight is 2.0 and the online weight is 3.0, the new target weight is $0.995(2) + 0.005(3) = 2.005$. `target_p.lerp_(p, 0.005)` computes exactly this. Repeated updates close the gap geometrically: half of the gap to an unchanging online weight disappears after $\ln 0.5 / \ln 0.995 \approx 138$ updates. This slow movement keeps the critic's regression target from chasing itself, as with DQN's target network.

10. Code sequence and shapes

The code updates critics first, actor second, temperature third, then target networks. States have shape `[32,3]`, actions `[32,2]`, and concatenated critic inputs `[32,5]`. Rewards, log-probabilities, and values are `[32,1]` (note `keepdim=True` in the log-prob sum) to avoid accidental broadcasting: a `[32]` tensor minus a `[32,1]` tensor would silently become `[32,32]`.

11. Map the explanation to the script

Code	Meaning
`self.net = nn.Linear(3, 4)`	State to two means and two log standard deviations
`mean, log_std = self.net(states).chunk(2, dim=-1)`	Split the 4 outputs into μ and $\log\sigma$
`Normal(mean, log_std.clamp(-5, 2).exp())`	Gaussian with $\sigma \in [e^{-5}, e^{2}]$
`raw = distribution.rsample()`	Reparameterized sample $u = \mu + \sigma\epsilon$
`actions = raw.tanh()`	Squash to $(-1, 1)$
`correction = 2*(torch.log(torch.tensor(2.0))-raw-nn.functional.softplus(-2*raw))`	Stable $\log(1-\tanh^2 u)$
`log_prob = (distribution.log_prob(raw)-correction).sum(-1, keepdim=True)`	Corrected $\log\pi(a \mid s)$, shape `[32,1]`
`q2 = copy.deepcopy(q1)`	Second critic, **identical** initialization
`target1, target2 = copy.deepcopy(q1).requires_grad_(False), ...`	Frozen target critics
`log_alpha = nn.Parameter(torch.tensor(-1.0))`	ℓ_α, so α starts at 0.3679
`next_actions, next_logs = actor.sample(next_states)`	Fresh a' and $\log\pi(a' \mid s')$ under `no_grad`
`next_q = torch.minimum(target1(inputs), target2(inputs))-log_alpha.exp()*next_logs`	Soft next value with twin minimum
`target = rewards + 0.99*(1-terminated)*next_q`	Bellman target y
`critic_loss = (q1(inputs)-target).square().mean()+(q2(inputs)-target).square().mean()`	Sum of both critics' MSE

Code	Meaning
`p.requires_grad_(False)`	Freeze critic parameters, keep action gradients
`actor_loss = (log_alpha.exp().detach()*log_prob-torch.minimum(q1(inputs), q2(inputs))).mean()`	L_π
`alpha_loss = -(log_alpha*(log_prob.detach()+target_entropy)).mean()`	L_α
`target_p.lerp_(p, 0.005)`	Polyak update with $\tau = 0.005$

12. Run it

Run from the course folder:

```
python3 05_advanced/02_sac_update.py
```

On the CPU used to verify this lesson (PyTorch 2.14, seed 7) the output was:

```
critic loss: 1.1879132986068726 actor loss: -0.4474215507507324
```

Exact digits can differ across PyTorch versions or hardware. There is no assertion. Printed losses only confirm execution of a synthetic update; they are not learning curves, and one number cannot show whether SAC "works". The script needs no GPU.

Extend the example with independently initialized critics, a real environment, replay collection, a warmup policy, and held-out episode evaluation before claiming control learning.

13. Practice calculations

1. Target critics give 2 and 2.5, $\log \pi(a' \mid s') = -0.5$, $\alpha = 0.2$, $r = 0$, $\gamma = 0.99$, nonterminal. Target? $0.99[2 - 0.2(-0.5)] = 0.99(2.1) = 2.079$**.**
2. Same transition but terminal with $r = 0$? $y = 0$**.**
3. One dimension with $u = 0$: what is $\log(1 - \tanh^2 0)$, and from the identity? $\log 1 = 0$**;** $2[\log 2 - 0 - \log 2] = 0$**.**
4. Batch mean $\log \pi = -2.5$ with $H_{target} = -2$. Sign of $\partial L_\alpha / \partial \ell_\alpha$, and does alpha go up or down? $\hat{H} - H_{target} = 2.5 + 2 = +4.5 > 0$**, so descent lowers alpha.**
5. Target weight 1.0, online weight 0.0, $\tau = 0.005$. After one update? **0.995.** After two (online still 0)? $0.995^2 = 0.990025$**.**
6. Why can't the actor loss be computed inside `torch.no_grad()` ? **The gradient through $Q(s, a)$ with respect to the sampled action would vanish, leaving only the entropy term.**

Reference: SAC equations and algorithm. Next: offline RL and the conservative Q-learning penalty.

Companion code: `02_sac_update.py`

05_advanced/02_sac_update.py

```python
"""Soft Actor-Critic: continuous actions and entropy regularization.
ADVANCED EQUATION LAB: one full actor/critic/temperature/target update on fake
replay data. This is NOT a complete environment-training implementation.
SAC is off-policy; it uses replay and twin critics to reduce overestimation.
Reparameterization a=tanh(mu+sigma*noise) lets gradients flow through sampling.
Exercise: replace fake replay with real transitions from a continuous environment.
"""
import copy
import torch
from torch import nn
from torch.distributions import Normal


class Actor(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Linear(3, 4)  # Two action means and two log standard deviations.

    def sample(self, states):
        mean, log_std = self.net(states).chunk(2, dim=-1)
        distribution = Normal(mean, log_std.clamp(-5, 2).exp())
        raw = distribution.rsample()  # sample() would break pathwise gradients.
        actions = raw.tanh()
        # Change-of-variables correction for tanh; actions lie in [-1,1].
        # The softplus identity is stable even when tanh saturates near +/-1.
        correction = 2*(torch.log(torch.tensor(2.0))-raw-nn.functional.softplus(-2*raw))
        log_prob = (distribution.log_prob(raw)-correction).sum(-1, keepdim=True)
        return actions, log_prob


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    actor = Actor()
    q1 = nn.Sequential(nn.Linear(5, 32), nn.ReLU(), nn.Linear(32, 1))
    q2 = copy.deepcopy(q1)
    target1, target2 = copy.deepcopy(q1).requires_grad_(False),
copy.deepcopy(q2).requires_grad_(False)
    critics = list(q1.parameters())+list(q2.parameters())
    q_optimizer = torch.optim.Adam(critics, lr=0.001)
    policy_optimizer = torch.optim.Adam(actor.parameters(), lr=0.001)
    log_alpha = nn.Parameter(torch.tensor(-1.0))
    alpha_optimizer = torch.optim.Adam([log_alpha], lr=0.001)
    states, next_states = torch.randn(32, 3), torch.randn(32, 3)
    actions, rewards = torch.rand(32, 2)*2-1, torch.randn(32, 1)
    terminated = torch.zeros(32, 1)
    with torch.no_grad():
        next_actions, next_logs = actor.sample(next_states)
        inputs = torch.cat([next_states, next_actions], -1)
        next_q = torch.minimum(target1(inputs), target2(inputs))-log_alpha.exp()*next_logs
        target = rewards + 0.99*(1-terminated)*next_q
    inputs = torch.cat([states, actions], -1)
    critic_loss = (q1(inputs)-target).square().mean()+(q2(inputs)-target).square().mean()
    q_optimizer.zero_grad(set_to_none=True)
    critic_loss.backward()
    q_optimizer.step()
    for p in critics:
```

```
57          p.requires_grad_(False)  # Still differentiate Q with respect to ACTIONS.
58      sampled, log_prob = actor.sample(states)
59      inputs = torch.cat([states, sampled], -1)
60      actor_loss = (log_alpha.exp().detach()*log_prob-torch.minimum(q1(inputs),
    q2(inputs))).mean()
61      policy_optimizer.zero_grad(set_to_none=True)
62      actor_loss.backward()
63      policy_optimizer.step()
64      for p in critics:
65          p.requires_grad_(True)
66      target_entropy = -2.0  # Common heuristic: negative action dimensionality.
67      alpha_loss = -(log_alpha*(log_prob.detach()+target_entropy)).mean()
68      alpha_optimizer.zero_grad(set_to_none=True)
69      alpha_loss.backward()
70      alpha_optimizer.step()
71      with torch.no_grad():
72          for online, target_net in ((q1, target1), (q2, target2)):
73              for p, target_p in zip(online.parameters(), target_net.parameters()):
74                  target_p.lerp_(p, 0.005)  # Polyak averaging: slow target movement.
75      print('critic loss:', critic_loss.item(), 'actor loss:', actor_loss.item())
76
77  if __name__ == '__main__':
78      main()
```

5.3

Offline RL and the conservative Q-learning penalty

Companion code: 03_offline_rl_cql.py. Prerequisites: Q-learning and SARSA, DQN and target networks, and softmax and cross-entropy. Read this explanation before running the code.

1. What problem are we solving now?

Every learner so far could **act**: try an action, see the result, and correct a wrong estimate. Offline RL removes that ability. It learns from a fixed dataset collected earlier by some other **behavior policy** (a person, an old controller, a logged system), and it may not collect anything new.

Why is that hard? Actions absent from the dataset may receive arbitrary neural-network values. A Bellman `max` or an actor can exploit those unsupported high values, causing errors to reinforce themselves without any new environment evidence to correct them. Online, the agent would try the overrated action, get a poor reward, and fix the value. Offline, that correction never arrives.

The script's toy setup: two dataset states, three discrete actions each, and a Q-table given directly as a trainable tensor:

```
                action 0   action 1   action 2
state A   Q =     1          5          2        dataset action: 0, Bellman target 1.2
state B   Q =     2          4          1        dataset action: 0, Bellman target 1.8
```

In both states the dataset only shows action 0, yet action 1 has the largest Q-value. Nothing in the data supports that 5 or that 4. This file is a discrete **CQL regularizer lab**, not a complete offline RL trainer.

2. Notation

Symbol	Meaning	Example / Code
D	Fixed offline dataset	two rows here
s	Dataset state	row index of `q`
a_D	Action recorded in the dataset at s	`actions = torch.tensor([0, 0])`
$Q(s, a)$	Estimated action value	`q[s][a]`
K	Number of discrete actions	3
y	Detached Bellman target	`bellman_targets = [1.2, 1.8]`
α	Weight of the conservative penalty	`0.5`
$C(Q)$	Conservative penalty	`conservative_penalty(q, actions)`

This α is unrelated to SAC's temperature or Q-learning's step size.

3. Start with Bellman regression

For dataset action a_D and a detached target y:

$$\boxed{L_{Bellman} = \mathbb{E}_{(s,a_D)\sim D}[(Q_\theta(s,a_D) - y)^2]}$$

In a real loop $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$ (or a policy-based variant); the script simply supplies constants.

This directly constrains observed actions. It need not constrain unobserved ones well. For a state with Q values `[0,5]` but only action 0 observed, ordinary regression on action 0 says little about whether the value 5 is justified.

Worked numbers for the script. Predictions for the dataset actions are $Q(A,0) = 1$ and $Q(B,0) = 2$:

$$L_{Bellman} = \tfrac{1}{2}[(1-1.2)^2 + (2-1.8)^2] = \tfrac{1}{2}(0.04 + 0.04) = 0.04.$$

Its gradient touches **only** column 0: $\partial L/\partial Q(s,0) = (Q - y)$ (the 2 from squaring cancels the $\frac{1}{2}$ mean), giving -0.2 for state A and $+0.2$ for state B. Columns 1 and 2 get exactly zero. That is the problem in one line: plain Bellman fitting never touches the suspicious 5 and 4. And in a real loop that unchecked 5 would feed into other states' targets through $\max_{a'}$.

4. Add a conservative contrast

The simplified discrete penalty is:

$$\boxed{C(Q) = \mathbb{E}_{s\sim D}\left[\log \sum_a e^{Q(s,a)} - Q(s,a_D)\right], \qquad L = L_{Bellman} + \alpha\, C(Q).}$$

Log-sum-exp is a smooth approximation to the maximum:

$$\max_a Q_a \le \log \sum_a e^{Q_a} \le \max_a Q_a + \log K.$$

So C is roughly "best value anywhere minus value of what the data did". It is large when some action is rated far above the dataset action.

Subtracting the observed-action value makes the penalty favor dataset-supported actions relative to other actions. It is not simply subtracting a constant from every Q value: adding a constant to all action values cancels in C. (Checked numerically: adding 10 to every entry leaves both per-state gaps unchanged.)

5. Worked example: compute the penalty row by row

State A, Q row `[1, 5, 2]`:

Quantity	Action 0	Action 1	Action 2	Total
e^Q	2.7183	148.4132	7.3891	158.5205
softmax $= e^Q/\text{total}$	0.01715	0.93624	0.04661	1

$\log 158.5205 = 5.0659$. The gap is $5.0659 - Q(A,0) = 5.0659 - 1 = 4.0659$. Bound check: $5 \le 5.0659 \le 5 + \log 3 = 6.0986$.

State B, Q row `[2, 4, 1]`:

Quantity	Action 0	Action 1	Action 2	Total
e^Q	7.3891	54.5982	2.7183	64.7055
softmax	0.11420	0.84379	0.04201	1

$\log 64.7055 = 4.1698$. The gap is $4.1698 - 2 = 2.1698$.

Average over the two states:

$$C = \tfrac{1}{2}(4.0659 + 2.1698) = 3.1179.$$

This is the `conservative penalty` the script prints (3.11787). The total loss is $0.04 + 0.5(3.1179) = 1.5989$ (not printed).

6. The derivative reveals the mechanism

For one state:

$$\boxed{\frac{\partial C_s}{\partial Q_a} = \text{softmax}(Q)_a - \mathbf{1}[a = a_D]}$$

The log-sum-exp part pushes every action down in proportion to its softmax weight; the $-Q(s, a_D)$ part pushes the dataset action up by 1.

For `[0,5]` and observed action 0, softmax is approximately `[0.0067,0.9933]`. Gradients are `[-0.9933,+0.9933]`. Gradient descent pushes the observed value up and the unsupported high value down. Bellman regression simultaneously anchors observed values toward reward-consistent targets.

Full gradient of the script's loss

Because C is averaged over 2 states and multiplied by $\alpha = 0.5$, each penalty gradient entry is $0.5 \times \frac{1}{2} \times (\text{softmax} - \text{one-hot}) = 0.25(\text{softmax} - \text{one-hot})$.

Entry	Bellman part	Penalty part $0.25(p - \mathbf{1})$	Total = printed `q.grad`
$Q(A,0)$	-0.2	$0.25(0.01715 - 1) = -0.24571$	**-0.44571**
$Q(A,1)$	0	$0.25(0.93624) = +0.23406$	**+0.23406**
$Q(A,2)$	0	$0.25(0.04661) = +0.01165$	**+0.01165**
$Q(B,0)$	$+0.2$	$0.25(0.11420 - 1) = -0.22145$	**-0.02145**

Entry	Bellman part	Penalty part $0.25(p-1)$	Total = printed `q.grad`
$Q(B,1)$	0	$0.25(0.84379) = +0.21095$	**+0.21095**
$Q(B,2)$	0	$0.25(0.04201) = +0.01050$	**+0.01050**

How to read it under gradient descent ($Q \leftarrow Q - \eta\,\text{grad}$):

- **Positive** entries (all unsupported actions) are pushed **down**. The biggest push hits the most overrated action, because it has the largest softmax weight.
- State A's dataset action is pushed **up** by both terms: its value 1 is below its target 1.2, and the penalty also favors it.
- State B's dataset action shows a tug-of-war: Bellman wants $2 \to 1.8$ (down), the penalty wants it up. The net gradient is tiny (-0.021). CQL does not blindly inflate observed actions; Bellman fitting still matters.

Hypothetical illustration (not in the script). One plain gradient step with step size 1 would give rows `[1.446, 4.766, 1.988]` and `[2.021, 3.789, 0.989]`, lowering the penalty from 3.118 to 2.695 and the total loss from 1.599 to 1.402. The script itself calls only `backward()`; it never updates `q`.

Mathematically this discrete penalty equals negative log-softmax of the observed action when Q values are treated as logits (check: $-\log 0.01715 = 4.0659$, the state-A gap). Its interpretation here is conservative value regularization, combined with Bellman fitting, rather than an isolated supervised action classifier.

7. What changes when alpha changes?

Rerunning the same loss with different α (computed separately; the script fixes 0.5):

α	grad $Q(A,0)$	grad $Q(A,1)$	grad $Q(B,0)$	grad $Q(B,1)$
0	-0.2	0	+0.2	0
0.5	-0.4457	+0.2341	-0.0215	+0.2109
2	-1.1829	+0.9362	-0.6858	+0.8438
5	-2.6571	+2.3406	-2.0145	+2.1095

With $\alpha = 0$ there is no pressure on unsupported actions at all. As α grows the conservative contribution dominates, and by $\alpha = 2$ state B's dataset action is pushed up even though Bellman wanted it lower. Too much pessimism can suppress useful actions and prevent improvement beyond the behavior dataset; too little lets extrapolated values leak into targets. Choosing α is a real tuning decision, not a solved detail.

8. When the dataset contains more than one action

The penalty is an average over dataset rows. If the same state appears twice, once with action 0 and once with action 1, the averaged gradient is

$$\text{softmax}(Q) - (\text{empirical action frequencies}).$$

For Q `[0,5]` and data frequencies $(0.5, 0.5)$: $[0.0067 - 0.5,\ 0.9933 - 0.5] = [-0.4933, +0.4933]$. Action 1's push-down shrank from 0.9933 to 0.4933 compared with the single-action case, because the data now partly supports it. The penalty is at its stationary point when the softmax of Q matches how often the dataset chose each action, which is why it is said to keep the policy near the data's support.

9. Map the explanation to the script

Code	Meaning
`chosen = q_values.gather(1, dataset_actions[:, None]).squeeze(1)`	Select $Q(s, a_D)$ for each row
`torch.logsumexp(q_values, dim=-1)`	Stable $\log \sum_a e^{Q(s,a)}$ per row
`(torch.logsumexp(q_values, dim=-1)-chosen).mean()`	$C(Q)$, averaged over states
`q = torch.tensor([[1., 5., 2.], [2., 4., 1.]], requires_grad=True)`	Q-table as a trainable tensor (no network)
`actions = torch.tensor([0, 0])`	Dataset actions a_D
`bellman_targets = torch.tensor([1.2, 1.8])`	Hand-supplied y (would be detached target-net outputs)
`bellman = (predicted-bellman_targets).square().mean()`	$L_{Bellman} = 0.04$
`loss = bellman+0.5*penalty`	$L = L_{Bellman} + \alpha C$ with $\alpha = 0.5$
`loss.backward()`	Fill `q.grad` ; no optimizer step follows

`torch.logsumexp` subtracts the row maximum internally, so it does not overflow even for very large Q values, unlike a naive `q.exp().sum().log()` .

In `main` , `q` is directly a trainable tensor, targets are hand-supplied constants, and alpha is 0.5. Backward exposes derivatives; there is no optimizer loop, replay construction, or policy evaluation.

10. Run it

Run from the course folder:

```
python3 05_advanced/03_offline_rl_cql.py
```

On the CPU used to verify this lesson the output was (one line, wrapped here):

```
conservative penalty: 3.1178650856018066 gradient: [[-0.4457131028175354,
0.23405984044075012, 0.011653153225779533], [-0.02145114541053772,
0.21094867587089539, 0.01050251629203558]]
```

These match the hand calculation in sections 5 and 6. The script has no assertion; check by eye that the unsupported high-Q actions (column 1) have positive gradients, so descent would lower them. The course's test suite (`tests/`) separately checks that the penalty pushes down an unsupported action. No GPU is needed.

11. What the script does not show

The CQL paper develops broader objectives (including continuous-action versions that estimate the log-sum-exp by sampling actions) and theoretical results under stated assumptions. This tiny regularizer demonstration does not establish a universal lower bound or guarantee safe deployment. See the original CQL paper.

Compare against **behavior cloning**, which simply predicts dataset actions and avoids Bellman extrapolation but may not improve beyond the behavior policy. It is a useful baseline; never trust offline Q values alone, and evaluate a learned policy carefully before deploying it.

12. Practice calculations

1. Q row `[0, 0, 0]`, dataset action 0. Penalty for this state? $\log 3 - 0 = 1.0986$**; it is never below 0 and equals** $\log K$ **when all values tie.**
2. Q row `[3, 0]`, dataset action 0. Penalty and gradient? $\log(e^3 + 1) - 3 = 0.0486$**; softmax** $\approx [0.9526, 0.0474]$**, gradient** $[-0.0474, +0.0474]$**: almost nothing to fix.**
3. Add 100 to every entry of state A. Does its gap change? **No, still 4.0659.**
4. With $\alpha = 1$ instead of 0.5, what is the total gradient on $Q(A, 1)$? $0.5 \times 0.93624 = 0.46812$ **(penalty averaged over 2 states, Bellman part 0).**
5. With $\alpha = 0$, what is the gradient on $Q(A, 1)$? **Zero: Bellman regression alone never touches unsupported actions.**
6. Exercise: add more observed examples of the previously unsupported action. **Its average penalty gradient becomes softmax minus its new data frequency, so it is pushed down less (section 8).**

Next: distillation and quantization.

Companion code: `03_offline_rl_cql.py`

05_advanced/03_offline_rl_cql.py

```python
"""Offline RL and a simplified discrete Conservative Q-Learning loss.
Offline RL learns from a FIXED dataset; it cannot explore to correct mistakes.
Bootstrapping on unsupported actions can produce overestimated Q values.
CQL adds alpha*(logsumexp_a Q(s,a)-Q(s,a_dataset)) to a Bellman loss.
This equation lab shows the regularizer, not a full CQL benchmark trainer.
Exercise: increase alpha and inspect gradients on actions absent from the data.
"""
import torch


def conservative_penalty(q_values, dataset_actions):
    chosen = q_values.gather(1, dataset_actions[:, None]).squeeze(1)
    return (torch.logsumexp(q_values, dim=-1)-chosen).mean()


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    q = torch.tensor([[1., 5., 2.], [2., 4., 1.]], requires_grad=True)
    actions = torch.tensor([0, 0])
    bellman_targets = torch.tensor([1.2, 1.8])  # In a real loop: detached target-net outputs.
    predicted = q.gather(1, actions[:, None]).squeeze(1)
    bellman = (predicted-bellman_targets).square().mean()
    penalty = conservative_penalty(q, actions)
    loss = bellman+0.5*penalty
    loss.backward()
    print('conservative penalty:', penalty.item(), 'gradient:', q.grad.tolist())
    # Positive gradients on unsupported high-Q actions push their values DOWN
    # under gradient descent. Too much pessimism can also prevent improvement.
    # Behavior cloning is a useful baseline; never trust offline Q values alone.

if __name__ == '__main__':
    main()
```

5.4

Distillation and quantization: two different ways to make models smaller

Companion code: 04_distillation_and_quantization.py. Prerequisites: softmax, cross-entropy training, KL divergence in RLHF, and the QLoRA section of Hugging Face SFT. Read this explanation before running the code.

1. What problem are we solving now?

A large trained model can be too slow, too memory-hungry, or too expensive to serve. There are two very different ways to shrink it:

Idea	What changes	Requires training?
Distillation	A *student* model learns to imitate a *teacher's* output distribution	Yes, the student is trained
Quantization	The same numbers are stored or computed with fewer representable values	Not in the simple post-training form used here

Distillation is about **learning**; quantization is about **numeric compression**. They solve different problems and can be combined: distill a smaller model, then quantize its weights. The script demonstrates the two ideas separately, on purpose, so you can see each one's arithmetic.

Two honest warnings before we start:

- The "student" in the script is a trainable **2×3 matrix of logits**, not a deployable small network. There is no input, no hidden layer, and no generalization to new examples. It only shows what the loss pushes toward.
- The quantization part is symmetric per-tensor INT8 **fake quantization** of a random 8×8 matrix. It is not NF4, not full QLoRA, and does not run integer matrix multiplications.

2. The tiny distillation setup in the script

The teacher is fixed. It has two examples ("rows"), each with three class logits:

```
teacher = torch.tensor([[4., 2., -1.], [-1., 2., 4.]])
```

Row 0 prefers class 0; row 1 is the mirror image and prefers class 2. The hard labels are `torch.tensor([0, 2])`, which agree with the teacher's top choice. The student starts at all-zero logits, so initially it says "1/3, 1/3, 1/3" for every class.

Symbol	Meaning	Example / Code
z^t	Teacher logits for one example	`[4, 2, -1]`, `teacher_logits`

Symbol	Meaning	Example / Code
z^s	Student logits for that example	starts at `[0, 0, 0]`, `student_logits`
T	Temperature, $T > 0$	`temperature=2.0`
q_T	Teacher probabilities at temperature T	`(teacher_logits.detach()/temperature).softmax(-1)`
p_T	Student probabilities at temperature T	`exp` of `(student_logits/temperature).log_softmax(-1)`
y	Hard label (correct class index)	`labels`, here `[0, 2]`
α	Weight on the soft (teacher) loss	`alpha=0.7`
B	Batch size (number of rows)	2

3. Why can soft labels teach more than a single class?

A hard label says only which class is correct. A teacher distribution might say class A has probability 0.7, B has 0.25, and C has 0.05. The relative probability of incorrect classes can communicate similarity or uncertainty unavailable in a one-hot label: "B is a plausible confusion, C is not." Hinton et al. called this information "dark knowledge". A mistaken teacher can also transfer its errors, so soft labels are only as good as the teacher.

4. What does temperature do?

At temperature $T > 0$, teacher and student probabilities are

$$\boxed{q_i = \mathrm{softmax}(z^t/T)_i, \qquad p_i = \mathrm{softmax}(z^s/T)_i.}$$

Dividing logits by $T > 1$ shrinks the gaps between them, so the distribution becomes flatter ("softer"). Worked example for the teacher row `[4, 2, -1]`:

T	Scaled logits	e^{scaled}	Sum	Probabilities q_T
1	`[4, 2, -1]`	`[54.598, 7.389, 0.368]`	62.355	`[0.8756, 0.1185, 0.0059]`
2	`[2, 1, -0.5]`	`[7.389, 2.718, 0.607]`	10.714	`[0.6897, 0.2537, 0.0566]`
4	`[1, 0.5, -0.25]`	`[2.718, 1.649, 0.779]`	5.146	`[0.5283, 0.3204, 0.1513]`

For example, at $T = 2$: $7.389/10.714 = 0.6897$. At $T = 1$ class 2 gets only 0.6% of the mass; at $T = 2$ it gets 5.7%. Softer targets expose differences among lower-ranked classes instead of almost all mass going to the winner. Relative **order** never changes, because dividing by a positive T preserves order.

5. The soft-target loss: KL divergence from teacher to student

The soft loss is the Kullback–Leibler divergence

$$\boxed{D_{KL}(q\|p) = \sum_i q_i(\log q_i - \log p_i).}$$

It is zero exactly when $p = q$ and positive otherwise. Because teacher probabilities are fixed, the $\sum_i q_i \log q_i$ part (minus the teacher's entropy) is a constant with respect to the student. Minimizing this KL is therefore equivalent to minimizing the cross-entropy $-\sum_i q_i \log p_i$: it encourages the student to match the teacher distribution.

`teacher_logits.detach()` enforces that the teacher is not optimized by the student loss. In this script the teacher is a plain tensor that never required gradients, so `detach()` is a safety habit here; it matters when the teacher is itself a network that is also being trained or shares parameters.

Worked example at initialization ($T = 2$). The student is uniform, $p = [1/3, 1/3, 1/3]$, and $q = [0.6897, 0.2537, 0.0566]$:

Class i	q_i	$q_i/p_i = 3q_i$	$\log(3q_i)$	$q_i \log(3q_i)$
0	0.6897	2.0690	0.7271	0.5014
1	0.2537	0.7611	−0.2729	−0.0692
2	0.0566	0.1698	−1.7731	−0.1004
Sum	1			**0.3318**

Individual terms can be negative; the total KL cannot.

6. The combined objective and the T^2 factor

The script mixes soft and hard targets:

$$\boxed{L = \alpha\, T^2\, D_{KL}(q_T \| p_T) + (1 - \alpha)\, \mathrm{CE}(z^s, y), \qquad \alpha = 0.7,\ T = 2.}$$

The hard cross-entropy uses the student's **untempered** logits ($T = 1$), as in `nn.functional.cross_entropy(student_logits, labels)`.

Worked initial loss. Both rows give the same numbers by symmetry, so the batch average equals one row's value:

$$L = 0.7 \cdot 4 \cdot 0.3318 + 0.3 \cdot \ln 3 = 0.9291 + 0.3296 = 1.2587.$$

Running the script's `distillation_loss` on the zero student indeed returns `1.2587`.

Why T^2? The derivative of the unscaled soft loss with respect to a student logit is

$$\frac{\partial D_{KL}(q_T \| p_T)}{\partial z_i^s} = \frac{p_i - q_i}{T}.$$

One factor $1/T$ comes from the chain rule through z/T. At high temperatures, probability differences $p_i - q_i$ also tend to shrink roughly with $1/T$, so the soft gradient shrinks roughly like $1/T^2$, motivating the usual T^2 compensation. It is a heuristic for keeping soft and hard terms on a comparable scale, not an exact statement that all gradient magnitudes remain invariant at every temperature.

Worked initial gradient (row 0). With T^2 scaling, the soft gradient is $\alpha T(p_i - q_i)$; the hard gradient is $(1-\alpha)(\text{softmax}(z^s)_i - [i = y])$; both are divided by $B = 2$ because of the batch average:

Class	Soft: $0.7 \cdot 2\,(p-q)/2$	Hard: $0.3\,(p - \text{onehot})/2$	Total gradient
0	$0.7(0.3333 - 0.6897) = -0.2494$	$0.15(0.3333 - 1) = -0.1000$	**−0.3494**
1	$0.7(0.3333 - 0.2537) = +0.0557$	$0.15(0.3333) = +0.0500$	**+0.1057**
2	$0.7(0.3333 - 0.0566) = +0.1937$	$0.15(0.3333) = +0.0500$	**+0.2437**

These match `student.grad` from PyTorch for the first row. Gradient descent subtracts the gradient, so class 0's logit goes up and classes 1 and 2 go down, class 2 most strongly. (Adam rescales each coordinate, so its actual step sizes differ, but the signs are the same on the first step.)

7. PyTorch details that are easy to get wrong

`nn.functional.kl_div(input, target)` expects **log** student probabilities as `input` and ordinary teacher probabilities as `target`. It computes $\sum \text{target} \cdot (\log \text{target} - \text{input}) = D_{KL}(q \| p)$. Passing the arguments backwards would compute a different KL direction (or nonsense, if probabilities are passed where log-probabilities are expected).

`reduction='batchmean'` sums class contributions and averages over examples, giving the mathematical KL per example. The default `'mean'` would also divide by the number of classes.

8. What does training produce?

The script runs 100 Adam steps with learning rate 0.05 and prints `student.softmax(-1)`, the student's probabilities at $T = 1$, not at $T = 2$. Observed output for row 0, compared with the teacher:

Quantity	Class 0	Class 1	Class 2
Teacher at $T = 1$	0.8756	0.1185	0.0059
Student at $T = 1$ (printed)	0.8960	0.0957	0.0083
Teacher at $T = 2$ (soft target q)	0.6897	0.2537	0.0566
Student at $T = 2$	0.7027	0.2297	0.0676

Row 1 prints the mirror image `[0.0083, 0.0957, 0.8960]`. What to notice:

- The student's ranking matches the teacher and the labels.
- The student does **not** copy the teacher exactly. The hard-label term pulls extra probability onto the correct class (0.896 vs. 0.876), so the optimum is a compromise between the two terms; the final loss is about 0.0396, not 0.
- 100 steps are a short run, not a convergence test. In a separate check (not part of the script), 2000 steps gave approximately `[0.898, 0.096, 0.0055]`.

9. Symmetric INT8 quantization: the formula

Now the second, unrelated idea. For a floating weight tensor W, choose one scale for the whole tensor:

$$\boxed{s = \frac{\max |W|}{127}, \qquad q = \mathrm{clip}(\mathrm{round}(W/s), -127, 127), \qquad \hat{W} = s\,q.}$$

Symbol	Meaning	Code
s	Scale: float value of one integer step	`scale`
q	Stored integer code in $[-127, 127]$	`integers` (dtype `torch.int8`)
$\hat{W}$	Dequantized (reconstructed) float weights	`reconstructed`
$W - \hat{W}$	Quantization error	`weights-reconstructed`

"Symmetric" means zero maps exactly to integer 0 and there is no separate zero point (offset). INT8 can represent -128, but this scheme uses only $[-127, 127]$ so positive and negative ranges are equal. The script also guards against an all-zero tensor with `clamp_min(1e-8)` so it never divides by zero.

10. Worked example: quantize five weights by hand

Take $W = [0.30,\ -1.00,\ 0.05,\ 0.62,\ -0.004]$. Then $\max|W| = 1$ and $s = 1/127 \approx 0.0078740$.

W	W/s	$q = \mathrm{round}$	$\hat{W} = q/127$	Error $W - \hat{W}$
0.30	38.10	38	0.299213	+0.000787
−1.00	−127.00	−127	−1.000000	0
0.05	6.35	6	0.047244	+0.002756
0.62	78.74	79	0.622047	−0.002047
−0.004	−0.508	−1	−0.007874	+0.003874

The largest-magnitude weight is reconstructed exactly. Mean squared error:

$$\mathrm{MSE} = \tfrac{1}{5}(0.000787^2 + 0 + 0.002756^2 + 0.002047^2 + 0.003874^2) \approx 5.48 \times 10^{-6}.$$

Without clipping, nearest rounding gives absolute error at most $s/2 \approx 0.003937$ per weight, ignoring floating-point roundoff; the worst entry above (0.003874) is just under that bound. Notice that -0.004 became -0.0079: tiny weights have large **relative** error even when their absolute error is small.

(Detail: `torch.round` rounds exact halves to the nearest even integer, e.g. $2.5 \to 2$. Exact halves are rare with real weights.)

11. Why can one outlier ruin a per-tensor scale?

One global scale can be dominated by an outlier. Consider a 2×3 matrix whose first row has small weights and whose second row contains 100:

$$W = \begin{bmatrix} 0.012 & -0.02 & 0.015 \\ 100 & -40 & 3 \end{bmatrix}.$$

Per-tensor: $s = 100/127 \approx 0.7874$. Row 0 becomes $[0.0152, -0.0254, 0.0190]/1 \to$ codes `[0, 0, 0]`. Every small weight is erased. Row 1 becomes codes `[127, -51, 4]`, reconstructed as `[100, -40.157, 3.150]`.

Per-row (per-channel): row 0 gets its own scale $0.02/127 \approx 0.00015748$, codes `[76, -127, 95]`, reconstructed as `[0.011969, -0.02, 0.014961]`; row 1 keeps scale 0.7874.

Scheme	Row-0 reconstruction	Row-0 relative error	Whole-matrix MSE
Per-tensor	`[0, 0, 0]`	100%	0.007992
Per-row	`[0.011969, -0.02, 0.014961]`	below 0.3%	0.007864

The overall MSE barely changes because row 1's errors dominate it, yet row 0 went from destroyed to nearly exact. A single average error number can hide which channels lost their information. Per-channel scales are standard practice for weights for exactly this reason. (These per-row numbers are a hand calculation for the exercise; the script itself only does per-tensor.)

12. What the script's quantization demonstrates, and what it does not

The script draws `weights = torch.randn(8, 8)` (deterministic because of `torch.manual_seed(7)` and because Adam consumes no random numbers). In the verification environment (PyTorch 2.14.1, CPU), $\max|W| \approx 2.9662$, so $s \approx 0.023356$. The printed MSE was about 4.98×10^{-5}. A rough sanity check: if rounding errors were uniform on $[-s/2, s/2]$, the expected MSE would be $s^2/12 \approx 4.55 \times 10^{-5}$, the same order of magnitude. The largest error was 0.01160, below $s/2 \approx 0.01168$.

Storage arithmetic (not measured by the script): 64 float32 values need 256 bytes; 64 int8 codes need 64 bytes plus one 4-byte scale, about a 3.8× reduction for this tiny tensor.

What the script does **not** do: it stores an INT8 tensor and then reconstructs float values before any use. It does not use accelerated integer matrix multiplication kernels, pack a complete model, quantize activations, or implement NF4/double quantization as used in QLoRA. Storage savings and runtime speed are separate claims that require separate measurement on real hardware.

13. Map the explanation to the script

Code	Meaning
`(student_logits/temperature).log_softmax(-1)`	$\log p_T$, the student's tempered log-probabilities

Code	Meaning
`(teacher_logits.detach()/temperature).softmax(-1)`	q_T, fixed teacher soft targets, no gradient to the teacher
`nn.functional.kl_div(..., reduction='batchmean')`	$D_{KL}(q_T \mid p_T)$ averaged over examples
`nn.functional.cross_entropy(student_logits, labels)`	Hard-label loss at $T = 1$
`alpha*temperature**2*soft+(1-alpha)*hard`	Combined objective L with $\alpha = 0.7$, $T = 2$
`teacher = torch.tensor([[4., 2., -1.], [-1., 2., 4.]])`	Two fixed teacher rows
`student = nn.Parameter(torch.zeros_like(teacher))`	Student logits, starting uniform
`torch.optim.Adam([student], lr=0.05)`	Optimizer, run `for _ in range(100):`
`distillation_loss(student, teacher, torch.tensor([0, 2]))`	Labels agree with the teacher's top classes
`scale = weights.abs().max().clamp_min(1e-8)/127`	Per-tensor scale s
`integers = (weights/scale).round().clamp(-127, 127).to(torch.int8)`	Integer codes q
`reconstructed = integers.float()*scale`	Dequantized $\hat{W} = sq$
`student.softmax(-1)`	Printed student probabilities at $T = 1$
`(weights-reconstructed).square().mean()`	Printed quantization MSE

14. Run and investigate

This script needs PyTorch (see the course README for setup). Run from the course folder:

```
python3 05_advanced/04_distillation_and_quantization.py
```

The script has no command-line flags. Observed output:

```
student probabilities: [[0.8960..., 0.0956..., 0.0082...], [0.0082..., 0.0956..., 0.8960...]]
quantization MSE: 4.98...e-05
```

Expect the student's preferred classes to match teacher and labels, and a small reconstruction error on random weights. The script contains no assertion; you check these properties by reading the output. Exact trailing digits may differ across PyTorch versions or hardware.

Experiments:

1. Increase one weight to create an outlier (e.g. `weights[0, 0] = 100.`) and compare per-tensor versus per-row quantization, using `weights.abs().amax(dim=1, keepdim=True)/127` as per-row scales.
2. Change `temperature` to 1 and 4 and compare the student's final probabilities.
3. Set `alpha=1.0` (pure distillation) and check that the student moves closer to the teacher's $T = 1$ probabilities.

Distillation reference: Distilling the Knowledge in a Neural Network.

15. Practice calculations

1. Teacher logits `[3, 1]` at $T = 2$: what is q? **Scaled logits `[1.5, 0.5]`, $q_0 = 1/(1 + e^{-1}) = 0.7311$, $q_1 = 0.2689$.**
2. With a uniform student $p = [0.5, 0.5]$, what is $D_{KL}(q\|p)$? $0.7311 \ln(1.4621) + 0.2689 \ln(0.5379) = 0.2777 - 0.1668 = 0.1109$.
3. With $\alpha = 0.7$ and $T = 2$, what is the scaled soft term $\alpha T^2 D_{KL}$? $0.7 \cdot 4 \cdot 0.1109 = 0.3106$.
4. Weights `[0.5, -0.25, 0.1]` with symmetric INT8: scale, codes, and reconstruction? $s = 0.5/127 = 0.003937$**; codes `[127, -64, 25]` (from 127, −63.5, 25.4; the exact half −63.5 rounds to the even −64);** $\hat{W} = [0.5, -0.25197, 0.098425]$.
5. What is the maximum per-weight rounding error in question 4? $s/2 \approx 0.00197$**; the actual largest is** $|0.1 - 0.098425| = 0.001575$.
6. Does quantizing to INT8 by itself make inference faster? **Not necessarily. It reduces storage; speed requires kernels and hardware that compute with the quantized format. This script dequantizes before use.**

Next: Evaluation and uncertainty.

Companion code: `04_distillation_and_quantization.py`

05_advanced/04_distillation_and_quantization.py

```python
"""Distillation and the arithmetic behind simple quantization.
Distillation teaches a student using teacher probabilities (soft targets).
This is symmetric per-tensor INT8 fake quantization, NOT NF4 or full QLoRA.
Exercise: compare per-tensor with per-row scales on weights with outliers.
"""
import torch
from torch import nn


def distillation_loss(student_logits, teacher_logits, labels, temperature=2.0, alpha=0.7):
    # PyTorch kl_div takes LOG student probabilities and ordinary teacher probs.
    soft = nn.functional.kl_div(
        (student_logits/temperature).log_softmax(-1),
        (teacher_logits.detach()/temperature).softmax(-1), reduction='batchmean')
    # T^2 compensates for the temperature's effect on gradient scale.
    hard = nn.functional.cross_entropy(student_logits, labels)
    return alpha*temperature**2*soft+(1-alpha)*hard


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    teacher = torch.tensor([[4., 2., -1.], [-1., 2., 4.]])
    student = nn.Parameter(torch.zeros_like(teacher))
    optimizer = torch.optim.Adam([student], lr=0.05)
    for _ in range(100):
        loss = distillation_loss(student, teacher, torch.tensor([0, 2]))
        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        optimizer.step()
    weights = torch.randn(8, 8)
    scale = weights.abs().max().clamp_min(1e-8)/127
    integers = (weights/scale).round().clamp(-127, 127).to(torch.int8)
    reconstructed = integers.float()*scale
    print('student probabilities:', student.softmax(-1).detach().tolist())
    print('quantization MSE:', (weights-reconstructed).square().mean().item())
    # This stores INT8 weights but dequantizes before arithmetic; accelerated
    # inference needs appropriate quantized kernels, packing, and hardware.

if __name__ == '__main__':
    main()
```

5.5

Evaluation and uncertainty: how much should you trust an improvement?

Companion code: 05_evaluation_and_uncertainty.py. Prerequisites: averages from bandits, sampling and variance from Monte Carlo, and the idea of comparing several seeds from Q-learning and SARSA. The script needs only the Python standard library. Read this explanation before running the code.

1. What problem are we solving now?

You fine-tuned a model and it scores higher than the baseline. Is it actually better, or did it get lucky on this particular set of test tasks?

A higher score is an observation, not yet a robust conclusion. Two models evaluated on a small sample can differ by chance. We need three things:

1. A clear **metric** measuring the outcome we care about (task success, not just training loss).
2. **Held-out data** that was not used for training or model selection.
3. An **uncertainty estimate** that matches how the examples were sampled.

This script estimates uncertainty in a **paired mean improvement**: both methods are tested on the same tasks, and we ask how much the average per-task improvement could vary if we had drawn a different set of tasks.

2. The tiny example in the script

Ten evaluation tasks, each scored 1 (solved) or 0 (failed):

Task	0	1	2	3	4	5	6	7	8	9
Baseline a_i (`before`)	0	1	0	1	0	1	0	0	1	0
Candidate b_i (`after`)	1	1	0	1	1	1	0	1	1	0
Difference $d_i = b_i - a_i$	1	0	0	0	1	0	0	1	0	0

The baseline solves 4/10, the candidate 7/10. Three tasks improve by one; seven do not change; none get worse. These are synthetic numbers. Ten tasks are far too few for a serious scientific claim, as the script's own comment says.

Symbol	Meaning	Example / Code
N	Number of paired evaluation tasks	10, `len(differences)`
a_i, b_i	Baseline and candidate score on task i	`baseline`, `candidate`
d_i	Paired difference $b_i - a_i$	`differences`

Symbol	Meaning	Example / Code
$\hat{\Delta}$	Estimated mean improvement	`statistics.mean(differences)`
s_d	Sample standard deviation of the d_i	not computed by the script
SE	Standard error of $\hat{\Delta}$	not computed by the script
R	Number of bootstrap replicates	`samples=2000`
$\hat{\Delta}^*_r$	Mean of bootstrap replicate r	entries of `means`

3. The point estimate

$$\boxed{\hat{\Delta} = \frac{1}{N}\sum_{i=1}^{N} d_i} \qquad \Rightarrow \qquad \hat{\Delta} = \frac{1+0+0+0+1+0+0+1+0+0}{10} = 0.3.$$

This equals $0.7 - 0.4$, the difference of the two accuracies. On its own it says nothing about how stable it is.

4. A classical standard error and t-interval, computed by hand

The script does not compute this, but it gives a useful reference. For independent task differences, the standard error of the mean is

$$\boxed{\mathrm{SE} = \frac{s_d}{\sqrt{N}}, \qquad s_d^2 = \frac{1}{N-1}\sum_{i=1}^{N}(d_i - \hat{\Delta})^2.}$$

Step by step:

Quantity	Arithmetic	Value
Deviations	three of $1 - 0.3 = 0.7$, seven of $0 - 0.3 = -0.3$	
Sum of squares	$3(0.49) + 7(0.09) = 1.47 + 0.63$	2.10
s_d^2	$2.10/9$	0.2333
s_d	$\sqrt{0.2333}$	0.4830
SE	$0.4830/\sqrt{10} = 0.4830/3.1623$	**0.1528**

A 95% t-interval uses $t_{0.975,\,N-1} = t_{0.975,9} \approx 2.262$:

$$0.3 \pm 2.262(0.1528) = 0.3 \pm 0.3456 = [-0.046,\ 0.646].$$

The t-interval assumes the mean is roughly normally distributed, which is shaky for ten 0/1 differences, and it extends below 0 even though no task got worse. The bootstrap below provides another way to approximate the estimator's sampling variation without assuming a particular distribution for the task scores.

5. Why does pairing matter mathematically?

For two scores on the same task:

$$\boxed{\mathrm{Var}(b-a) = \mathrm{Var}(b) + \mathrm{Var}(a) - 2\,\mathrm{Cov}(a,b).}$$

Task difficulty often causes positive covariance: both methods do well on easy tasks and poorly on hard ones. Pairing preserves this relationship and can make comparison less noisy. With the example data (sample variances, divisor $N-1$):

Quantity	Value
$\mathrm{Var}(a)$	0.2667
$\mathrm{Var}(b)$	0.2333
$\mathrm{Cov}(a,b)$	0.1333
$\mathrm{Var}(d) = 0.2333 + 0.2667 - 2(0.1333)$	0.2333
Paired SE $= \sqrt{0.2333/10}$	**0.153**
Unpaired SE $= \sqrt{0.2667/10 + 0.2333/10}$	**0.224**

Ignoring the pairing would inflate the standard error by almost half here. Independently resampling baseline and candidate scores would lose the correspondence and estimate a different uncertainty structure. As a separate check (not in the script), resampling the two lists independently with the same 2000 replicates gave an interval of about `[-0.1, 0.7]`, wider than the paired result below.

The script resamples the differences directly, which is equivalent to drawing the same task indices for both methods.

6. Bootstrap step by step

Treat the empirical task distribution as an approximation to the population of tasks. For each of $R = 2000$ repetitions:

1. Draw N observed differences **with replacement**.
2. Compute their mean $\hat{\Delta}^*_r$.
3. Store that mean.

Sort these means and take approximately the 2.5th and 97.5th percentiles as a **percentile-bootstrap interval**:

$$\boxed{\mathrm{CI}_{95\%} \approx \left[\hat{\Delta}^*_{(\lfloor 0.025R \rfloor)},\ \hat{\Delta}^*_{(\lfloor 0.975R \rfloor)}\right]}$$

where the subscript is a 0-based position in the sorted list. With $R = 2000$, that is `means[50]` and `means[1950]`.

"With replacement" means one task can be drawn several times while another is omitted in a replicate. This simulates sampling variability around the observed dataset. For example:

Replicate draws (differences)	Ones drawn	Mean
1,1,1,1,1,0,0,0,0,0	5	0.5
1,0,0,0,0,0,0,0,0,0	1	0.1
0,0,0,0,0,0,0,0,0,0	0	0.0

Because each draw is a 1 with probability $3/10$, the number of ones in a replicate follows a Binomial(10, 0.3) distribution, so the bootstrap means can only be 0.0, 0.1, …, 1.0. We can compare the script's actual 2000 replicates (seed 7) with the exact probabilities:

Replicate mean	Exact probability	Exact cumulative	Count in script's run
0.0	0.0282	0.0282	67
0.1	0.1211	0.1493	252
0.2	0.2335	0.3828	467
0.3	0.2668	0.6496	544
0.4	0.2001	0.8497	382
0.5	0.1029	0.9527	194
0.6	0.0368	0.9894	78
0.7	0.0090	0.9984	13
0.8	0.0014	0.9999	3

Reading the endpoints: the first 67 sorted values (positions 0–66) are 0, so position 50 is **0**. The values up to 0.5 fill positions 0–1905, so position 1950 is **0.6**. The interval is `[0, 0.6]`. The exact cumulative column agrees: 2.5% falls inside the 0.0 bucket (cumulative 0.0282) and 97.5% falls inside the 0.6 bucket (from 0.9527 to 0.9894).

The interval is discrete and approximate because the dataset is tiny. The standard deviation of the 2000 bootstrap means was 0.145, close to the plug-in value $\sqrt{0.3 \cdot 0.7/10} = 0.145$ and a bit smaller than the t-based SE of 0.153 (the bootstrap uses divisor N rather than $N - 1$).

7. Interpret the interval carefully

A frequentist 95% confidence procedure aims to cover the population quantity in about 95% of repeated studies under its assumptions. It is not a statement that this fixed computed interval contains the fixed truth with 95% posterior probability. Bootstrap coverage can be poor with small samples, severe skew, or a mismatched sampling model.

Here the interval's lower end touches 0. That does not prove "no effect", and it does not prove a reliable effect either. A reasonable summary is: "On 10 tasks, the candidate improved 3 and worsened none; the paired bootstrap interval for the mean improvement is roughly 0 to 0.6, so more tasks are needed for a precise estimate."

8. What is the unit of resampling?

If examples share users/documents, individual rows are not independent units. Resample whole users/documents (a **cluster bootstrap**), or use an appropriate hierarchical procedure. Otherwise the interval will look narrower than it should.

For RL, multiple evaluation episodes of one trained agent do not replace multiple independent training seeds: training randomness is another source of variation. Report the return distribution across held-out evaluation seeds; a single high-return episode does not demonstrate reliable learning. Match the resampling design to the claim you want to make.

Worked seed example (hypothetical numbers, not from the script). Five independent training seeds give final mean returns $[0.82, 0.75, 0.91, 0.68, 0.79]$:

Step	Arithmetic	Value
Mean	$3.95/5$	0.79
Deviations	$0.03, -0.04, 0.12, -0.11, 0.00$	
Sum of squares	$0.0009 + 0.0016 + 0.0144 + 0.0121 + 0$	0.0290
s	$\sqrt{0.0290/4} = \sqrt{0.00725}$	0.0851
SE	$0.0851/\sqrt{5}$	0.0381
95% t-interval, $t_{0.975,4} \approx 2.776$	$0.79 \pm 2.776(0.0381) = 0.79 \pm 0.106$	$[0.684,\ 0.896]$

With only five seeds the t multiplier is large (2.776 rather than about 1.96), which is the honest price of few independent runs.

9. Code mapping and practical limitations

Code	Meaning
`if not baseline or len(baseline) != len(candidate):`	Refuse empty or unpaired inputs with `ValueError`
`rng = random.Random(seed)`	Private random generator, `seed=7` by default
`differences = [b-a for a, b in zip(baseline, candidate)]`	Paired differences d_i
`rng.choices(differences, k=len(differences))`	One bootstrap replicate: N draws with replacement
`statistics.mean(...)`	Replicate mean $\hat{\Delta}^*_r$
`means = sorted(... for _ in range(samples))`	All $R = 2000$ replicate means, sorted
`means[int(0.025*samples)], means[int(0.975*samples)]`	Percentile endpoints, positions 50 and 1950
`return statistics.mean(differences), (...)`	Point estimate $\hat{\Delta}$ and interval
`before = [0, 1, 0, 1, 0, 1, 0, 0, 1, 0]`	Baseline scores a_i
`after = [1, 1, 0, 1, 1, 1, 0, 1, 1, 0]`	Candidate scores b_i

The seed fixes the bootstrap simulation, not uncertainty in the original experiment. Increasing bootstrap repetitions makes the numerical percentiles more stable; it does not create more independent evaluation tasks. Increasing genuine task coverage is different

from resampling the same ten tasks. (Checked separately: `samples=20000` and `seed=1` both still give `(0, 0.6)`.)

10. Run and check

Run from the course folder:

```
python3 05_advanced/05_evaluation_and_uncertainty.py
```

The script has no command-line flags. Observed output:

```
mean improvement: 0.3 approximate 95% bootstrap interval: (0, 0.6)
```

Expect mean improvement 0.3 and a wide interval. The lower endpoint prints as `0` rather than `0.0` because `statistics.mean` of a list of integer zeros returns the integer 0. There is no assertion in the script; the only built-in check is the `ValueError` for empty or mismatched score lists.

Exercises:

1. Use identical candidate and baseline scores. Every difference and every bootstrap mean must be zero, so the result is `(0, (0, 0))`.
2. Compare a model selected using the same test set: explain why resampling cannot undo the resulting selection bias. (The bootstrap only resamples the tasks you have; it cannot tell that they were also used to pick the winner.)
3. Following the docstring, compare independent vs. paired bootstrapping on strongly correlated tasks and confirm that the paired interval is narrower.

11. Practice calculations

1. Differences `[1, 1, 0, -1, 1]`: what is $\hat{\Delta}$? $2/5 = 0.4$**.**
2. What are s_d and SE? **Deviations** $0.6, 0.6, -0.4, -1.4, 0.6$**; sum of squares** $0.36 \cdot 3 + 0.16 + 1.96 = 3.2$**;** $s_d = \sqrt{3.2/4} = 0.894$**; SE** $= 0.894/\sqrt{5} = 0.4$**.**
3. With $R = 1000$ replicates, which sorted positions does the script's rule use? **`int(0.025*1000)=25` and `int(0.975*1000)=975`.**
4. $\mathrm{Var}(a) = 0.25$, $\mathrm{Var}(b) = 0.25$, $\mathrm{Cov}(a, b) = 0.2$: what is $\mathrm{Var}(b - a)$? $0.25 + 0.25 - 0.4 = 0.1$**; without the covariance term it would be 0.5.**
5. Three seeds give returns `[0.6, 0.8, 0.7]`. Mean and SE? **Mean 0.7;** $s = \sqrt{(0.01 + 0.01 + 0)/2} = 0.1$**; SE** $= 0.1/\sqrt{3} = 0.0577$**.**
6. Does running 100,000 bootstrap replicates instead of 2000 narrow the interval substantially? **No. It only reduces Monte Carlo noise in the percentiles; the width is set by the ten real tasks.**

Next: Activation checkpointing.

Companion code: `05_evaluation_and_uncertainty.py`

05_advanced/05_evaluation_and_uncertainty.py

```python
"""Evaluate outcomes, not just loss; quantify uncertainty.
Use held-out tasks and multiple seeds. Compare methods under equal data/compute.
This paired bootstrap uses the same sampled task IDs for both methods.
Exercise: compare independent vs paired bootstrapping on strongly correlated tasks.
"""
import random
import statistics


def paired_bootstrap(baseline, candidate, samples=2000, seed=7):
    if not baseline or len(baseline) != len(candidate):
        raise ValueError('Need nonempty paired scores on the same evaluation tasks.')
    rng = random.Random(seed)
    differences = [b-a for a, b in zip(baseline, candidate)]
    means = sorted(statistics.mean(rng.choices(differences, k=len(differences))) for _ in range(samples))
    return statistics.mean(differences), (means[int(0.025*samples)], means[int(0.975*samples)])


if __name__ == '__main__':
    before = [0, 1, 0, 1, 0, 1, 0, 0, 1, 0]
    after =  [1, 1, 0, 1, 1, 1, 0, 1, 1, 0]
    delta, interval = paired_bootstrap(before, after)
    print('mean improvement:', delta, 'approximate 95% bootstrap interval:', interval)
    # Ten synthetic tasks are far too few for a serious scientific claim.
    # If examples share documents/users, resample clusters, not individual rows.
    # RL: report return distribution across held-out evaluation seeds; a single
    # high-return episode does not demonstrate reliable learning.
```

5.6

Activation checkpointing: recompute values to save training memory

Companion code: 06_activation_checkpointing.py. Prerequisites: tensors and autograd, neural network training, and training efficiency. Read this explanation before running the code.

1. What problem are we solving now?

Training memory has several parts:

Memory item	Depends on batch size?	Changed by activation checkpointing?
Weights	No	No
Gradients	No	No
Optimizer state (e.g. Adam moments)	No	No
Saved activations for backward	Yes, and on sequence length	**Yes**

A forward pass computes intermediate tensors called **activations**. Backward needs many of them to apply the chain rule, so autograd keeps them alive until backward reaches the layer that needs them. For deep networks with long sequences and large batches, these saved activations can use more memory than the weights.

Activation checkpointing (also called gradient checkpointing) keeps only selected tensors, usually the inputs at block boundaries. During backward it re-runs the forward pass of each block to rebuild the missing intermediates. It trades **extra computation** for **less saved activation memory**.

This is different from saving a model checkpoint to disk. Nothing here writes files; the word "checkpoint" means "a saved point in the computation".

2. Why does backward need forward values?

Start with the smallest possible chain. Let

$$u = wx, \qquad y = u^2.$$

The chain rule gives

$$\frac{\partial y}{\partial w} = \frac{\partial y}{\partial u}\,\frac{\partial u}{\partial w} = 2u\,x.$$

The derivative contains u, which was produced during the forward pass. There are two ways to have u available when backward arrives:

Strategy	What is kept after forward	What backward does
Ordinary	x and u	Uses the stored u
Checkpointed	Only x (and w, which is a parameter anyway)	Recomputes $u = wx$, then uses it

With $w = 3$ and $x = 2$:

1. Forward: $u = 3 \cdot 2 = 6$, $y = 6^2 = 36$.
2. Ordinary backward: read stored $u = 6$; derivative $= 2(6)(2) = 24$.
3. Checkpointed backward: recompute $u = 3 \cdot 2 = 6$; derivative $= 2(6)(2) = 24$.

Same answer, one extra multiplication, one fewer stored value. A large block is the same idea at scale: its internal tensors can be reconstructed from its input and parameters.
Checkpointing does not stop those tensors from existing **temporarily** during computation; it stops them from being **retained** from the forward pass until backward reaches the block.

3. Notation

Symbol	Meaning	Example / Code
B	Batch size (examples)	8 in the script: `torch.randn(8, 16)`
S	Sequence length (tokens per example)	1024 in the worked example
H	Hidden size (features per token)	16 input, 64 hidden in the script
L	Number of layers / blocks	32 in the worked example
b	Bytes per number	2 for fp16/bf16, 4 for fp32
c	Retained internal tensors per layer, in units of one $B \times S \times H$ tensor	Assumed 16 in the worked example
k	Layers per checkpointed segment	8 in the $\sqrt{L}$ example
M_{act}	Activation memory retained for backward	—

One "hidden-sized" activation tensor costs

$$\boxed{\text{bytes} = B \times S \times H \times b}.$$

4. The script's tiny model: what is actually saved?

The script builds a linear–GELU–linear network and an identical deep copy:

```
ordinary = nn.Sequential(nn.Linear(16, 64), nn.GELU(), nn.Linear(64, 16))
recomputed = copy.deepcopy(ordinary)
x = torch.randn(8, 16)
```

Name the intermediate tensors:

```
x [8,16] --Linear--> h [8,64] --GELU--> a [8,64] --Linear--> out [8,16] --square,mean--> loss
```

- The second `Linear` needs `a` to compute its weight gradient.

- `GELU` needs `h` to compute its local derivative.
- The first `Linear` needs `x` for its weight gradient.

`h` and `a` each have $8 \times 64 = 512$ numbers. In the script's default fp32 ($b = 4$), each costs $512 \times 4 = 2048$ bytes, so the two internal activations together cost **4096 bytes**.

As an extra check (not part of the script), wrapping each forward in `torch.autograd.graph.saved_tensors_hooks` and recording what autograd saves gave, for the same seed and shapes:

Run	Tensors saved for backward (shapes)	Hidden activations `h`, `a` retained?
Ordinary	`[8,16]`, `[8,64]`, `[8,64]`, `[64,16]`, `[8,16]`	Yes, 1024 numbers
Checkpointed	`[8,16]`, `[8,16]`	No

The `[64,16]` entry is a view of a weight that exists anyway, `x` exists anyway, and the final `[8,16]` is the block output saved by `.square()`. The two `[8,64]` tensors are the activations that checkpointing avoided keeping. A forward hook on the checkpointed copy also showed its first `Linear` running **once during forward and once more during backward**: that second call is the recomputation.

These bytes are trivial; the script is about **correctness**, not savings.

5. Why should the gradients agree?

If recomputation uses the same function, parameters, inputs, and relevant random state, the rebuilt activations equal the originals, so the chain rule produces the same gradients up to floating-point effects. Checkpointing changes the **execution schedule**, not the mathematical objective.

The script runs one model normally and the copy through:

```
checkpoint(recomputed, x, use_reentrant=False)
```

Each computes the mean square of its output and backpropagates. The file then compares every pair of corresponding parameter gradients with `torch.allclose(..., atol=1e-6)`. In the run used to write this lesson, both losses were `0.04553...` and the largest absolute gradient difference was 0.

`use_reentrant=False` selects the non-reentrant implementation explicitly. It also handles this case, where the input `x` does not itself require gradients but the module parameters do.

6. Worked example: how much memory could a real model save?

The script does not measure this. The following is a **back-of-envelope model** with an assumed constant, intended only to show the arithmetic.

Take a transformer-like stack with $B = 8$, $S = 1024$, $H = 4096$, $L = 32$, in fp16 ($b = 2$). One hidden-sized tensor:

$$8 \times 1024 \times 4096 \times 2 = 67{,}108{,}864 \text{ bytes} = 64 \text{ MiB}.$$

Assume each layer retains internal tensors worth $c = 16$ such tensors (attention and MLP intermediates; the true value depends on the architecture, the attention kernel, and what each operator saves).

Ordinary training keeps everything for all layers:

$$M_{act} = L \times c \times 64 \text{ MiB} = 32 \times 16 \times 64 \text{ MiB} = 32{,}768 \text{ MiB} = 32 \text{ GiB}.$$

Checkpoint every layer keeps only each layer's input, then rebuilds one layer's internals at a time during backward:

Part	Arithmetic	Memory
Stored boundaries	32×64 MiB	2 GiB
One layer rebuilt during backward	16×64 MiB	1 GiB
Peak activation total	$2 + 1$	**about 3 GiB**

Under these assumptions activation memory drops by about $32/3 \approx 10.7\times$. Weights, gradients and optimizer state are unchanged, so the **total** memory reduction is smaller than that ratio.

7. What does it cost in compute?

A common rule of thumb: backward costs about twice a forward pass. Then one training step costs about $1 + 2 = 3$ forward-equivalents. Checkpointing every block re-runs each block's forward once more:

$$\frac{1 + 1 + 2}{1 + 2} = \frac{4}{3} \quad \Rightarrow \quad \text{about } 33\% \text{ more compute per step.}$$

This is an estimate, not a measurement. Real overhead varies with which layers are checkpointed, kernel efficiency, and whether the run was limited by compute or memory. Sometimes the freed memory allows a larger batch, which can recover some throughput.

8. The $\sqrt{L}$ idea: checkpoint segments, not every layer

Use a simpler unit model: a chain of L layers where each layer's output is one unit of memory. Ordinary backprop stores all L units.

Split the chain into segments of k layers. Keep only the L/k segment boundaries. During backward, recompute one segment at a time, which temporarily needs its k layer outputs:

$$\boxed{M(k) \approx \frac{L}{k} + k}.$$

This is smallest near $k = \sqrt{L}$, giving $M \approx 2\sqrt{L}$. For $L = 64$:

Segment size k	Boundaries L/k	Rebuilt k	Total units
1	64	1	65

Segment size k	Boundaries L/k	Rebuilt k	Total units
2	32	2	34
4	16	4	20
8	**8**	**8**	**16**
16	4	16	20
64	1	64	65

So 16 units instead of 64, while still recomputing each layer's forward about once (roughly the same one-extra-forward cost as in Section 7). In practice, libraries often checkpoint each transformer block because a block contains many internal tensors relative to its single input, as in Section 6.

9. When can recomputation surprise you?

- **Randomness.** Dropout consumes random numbers; the recomputed mask must match the original. PyTorch's checkpoint utilities handle supported RNG preservation, but arbitrary state, external randomness, or device movement inside a block can require care.
- **Side effects.** Incrementing counters, logging, or updating running statistics (such as batch-norm buffers) may execute again during recomputation. Do not put side effects or state-changing operations inside recomputed blocks.
- **Library switches.** Many transformer libraries expose `gradient_checkpointing_enable()`. Training usually disables generation KV caching (`use_cache`); caching and activation recomputation serve different purposes. Inspect the model's implementation rather than assuming one switch covers every block.

10. What the script shows and what it does not

The script shows	The script does not show
Checkpointed and ordinary gradients match for one small network	Any peak-memory reduction
How to call `checkpoint(..., use_reentrant=False)` on a module	Runtime overhead
Gradient equality as a test of correct recomputation	Behavior with dropout, batch norm, or GPUs

Do not infer a particular memory-reduction ratio from this script. To measure savings, use a larger workload, reset peak-memory counters, synchronize GPU operations when timing, and compare identical model/data configurations.

11. Map the explanation to the script

Code	Meaning
`torch.manual_seed(7)`	Fix initial weights and the input for reproducibility
`nn.Sequential(nn.Linear(16, 64), nn.GELU(), nn.Linear(64, 16))`	The block whose activations are saved or recomputed

Code	Meaning
`recomputed = copy.deepcopy(ordinary)`	Identical weights, separate gradient storage
`x = torch.randn(8, 16)`	Batch of 8 inputs with 16 features; no `requires_grad`
`ordinary(x).square().mean()`	Ordinary forward: autograd retains `h` and `a`
`checkpoint(recomputed, x, use_reentrant=False)`	Retain only the input; rerun the block in backward
`.square().mean()`	Same mean-square loss on both outputs
`regular_loss.backward()` / `checkpoint_loss.backward()`	Compute gradients; the second triggers recomputation
`torch.allclose(p.grad, q.grad, atol=1e-6)`	Check each parameter's gradients agree

12. Run and check

Run from the course folder:

```
python3 05_advanced/06_activation_checkpointing.py
```

Expected output:

```
recomputed and ordinary gradients match
```

The assertion checks that all four gradient tensors (two weights, two biases) agree within `atol=1e-6`. It does not check memory or speed.

Experiment: change only the copied model's weights, for example add 0.01 to the first layer's weight inside `torch.no_grad()`. The assertion should then fail, because the two compared functions differ. Restore equality afterward. Then profile a larger model on a GPU, as the script's docstring suggests, to study the actual memory-versus-runtime tradeoff.

13. Practice calculations

1. An fp16 activation has $B = 4$, $S = 2048$, $H = 4096$. How many MiB is one hidden-sized tensor? $4 \times 2048 \times 4096 \times 2 = 67{,}108{,}864$ **bytes** $= 64$ **MiB.**
2. In the script, how many bytes do `h` and `a` take together in fp32? $2 \times 8 \times 64 \times 4 = 4096$ **bytes.**
3. Using the Section 6 assumptions but $L = 48$, what is ordinary activation memory, and the per-layer-checkpoint peak? $48 \times 16 \times 64$ **MiB** $= 48$ **GiB;** 48×64 **MiB** $+16 \times 64$ **MiB** $= 3 + 1 = 4$ **GiB.**
4. For a unit chain of $L = 100$, what segment size minimizes $L/k + k$, and what is the total? $k = 10$**, total** $10 + 10 = 20$ **units instead of 100.**
5. If backward costs twice a forward, what is the extra compute of recomputing every forward once? $4/3 - 1 \approx 33\%$**.**
6. Does checkpointing reduce optimizer-state memory? **No. It only changes how many activations are retained for backward.**

Next: distributed data parallel training.

Companion code: `06_activation_checkpointing.py`

05_advanced/06_activation_checkpointing.py

```python
"""Trade extra computation for lower activation memory.
Activation checkpointing discards intermediate activations and recomputes them
in backward. It differs from saving model checkpoints to disk.
Exercise: profile peak CUDA memory on a larger network with/without checkpointing.
"""
import copy
import torch
from torch import nn
from torch.utils.checkpoint import checkpoint


def main():
    torch.set_num_threads(1)  # Small examples run faster without large thread pools.
    torch.manual_seed(7)
    ordinary = nn.Sequential(nn.Linear(16, 64), nn.GELU(), nn.Linear(64, 16))
    recomputed = copy.deepcopy(ordinary)
    x = torch.randn(8, 16)
    regular_loss = ordinary(x).square().mean()
    checkpoint_loss = checkpoint(recomputed, x, use_reentrant=False).square().mean()
    regular_loss.backward()
    checkpoint_loss.backward()
    for p, q in zip(ordinary.parameters(), recomputed.parameters()):
        assert torch.allclose(p.grad, q.grad, atol=1e-6)
    print('recomputed and ordinary gradients match')
    # Real transformers: model.gradient_checkpointing_enable(); usually disable
    # use_cache during training. Savings depend on checkpoint boundaries.
    # Do not put side effects or state-changing operations inside recomputed blocks.

if __name__ == '__main__':
    main()
```

5.7

Distributed data parallel training: split data and average gradients

Companion code: 07_distributed_training.py. Prerequisites: training loops, gradient accumulation, and activation checkpointing. Read this explanation before running the code.

1. What problem are we solving now?

One process can only work through so many examples per second. **Data parallel training** runs several workers at once:

1. Every worker holds a **full copy** (replica) of the model.
2. Each worker receives a **different slice** of the data.
3. Each worker computes gradients on its own slice.
4. Workers **average** their gradients, so every replica sees the same gradient.
5. Every replica applies the same optimizer step and stays identical.

This increases how much data is processed per distributed step. It does **not** divide one model's weight memory among workers: each one still stores all weights, gradients and optimizer state.

The example uses two CPU processes with the Gloo communication backend. No GPU is required.

2. The toy task in the script

The synthetic target is

$$y = x_1 + x_2 + x_3 + x_4.$$

The script draws 128 random inputs with 4 features each and trains `nn.Linear(4, 1)`. A linear model can represent this target exactly with weight vector `[1,1,1,1]` and bias 0, so we know what a correct run should learn.

Setting	Value in the script
Dataset size N	128
Workers P (`--nproc_per_node`)	2
Local batch size B_{local}	16
Optimizer	SGD, learning rate 0.1
Epochs	10
Loss	Mean squared error on each local batch

3. Notation

Symbol	Meaning	Example / Code
P	World size: number of worker processes	2; `--nproc_per_node=2`
p or rank	This worker's index, $0, \ldots, P-1$	`dist.get_rank()`
B_{local}	Examples per worker per step	`batch_size=16`
K	Gradient-accumulation steps	1 (none in the script)
B_{eff}	Examples contributing to one optimizer update	32
L_p	Mean loss on rank p's local batch	`loss` on each rank
g_p	Gradient of L_p	`param.grad` before syncing
$\bar{g}$	Averaged gradient after all-reduce	`param.grad` after `backward()`
η	Learning rate	`lr=0.1`

All-reduce is a collective operation: every rank contributes a tensor and every rank receives the combined result. DDP all-reduces gradients (sum, then divide by P) during `loss.backward()`.

4. Why is averaging gradients mathematically sensible?

Suppose P workers each receive n examples, with local mean loss L_p. The mean loss over all Pn examples is

$$L = \frac{1}{P}\sum_{p=1}^{P} L_p, \qquad \boxed{\nabla L = \frac{1}{P}\sum_{p=1}^{P} \nabla L_p = \bar{g}}.$$

So the averaged gradient equals the gradient of one big batch containing every worker's examples, **when local batch sizes are equal**.

Worked 2-rank example

Use a one-weight model $\hat{y} = wx$, squared error, $w = 0$, $\eta = 0.1$. For mean squared error, $\partial L/\partial w = \frac{2}{n}\sum_i x_i(wx_i - y_i)$.

Rank	Local data (x, y)	Per-example $x(wx - y)$ at $w = 0$	Local gradient g_p
0	$(1,1), (2,2)$	$-1,\ -4$	$\frac{2}{2}(-1-4) = -5$
1	$(1,3), (1,3)$	$-3,\ -3$	$\frac{2}{2}(-3-3) = -6$

Step by step:

1. All-reduce sum: $-5 + (-6) = -11$.
2. Divide by $P = 2$: $\bar{g} = -5.5$ on **both** ranks.
3. Check against one big batch of all four examples: $\frac{2}{4}(-1-4-3-3) = \frac{2}{4}(-11) = -5.5$. Same.
4. SGD on each rank: $w \leftarrow 0 - 0.1(-5.5) = 0.55$. Both replicas now hold $w = 0.55$.

If both replicas start from the same weights and optimizer state, identical steps keep them synchronized. (DDP also broadcasts rank 0's parameters to other ranks when the wrapper is constructed; in this script both ranks additionally use the same seed.) Without synchronization they would drift into separate models fitting separate data streams.

The original two-number example works the same way: local gradients 2 and 6 average to 4 on each worker.

Caveat: unequal batch sizes

Equal weighting of ranks equals equal weighting of examples only when local batch sizes match. If rank 0 has 8 examples with mean gradient 2 and rank 1 has 24 examples with mean gradient 6:

Method	Arithmetic	Result
Default DDP average	$\frac{1}{2}(2) + \frac{1}{2}(6)$	4
True mean over 32 examples	$\frac{8}{32}(2) + \frac{24}{32}(6) = 0.5 + 4.5$	5

The correct weights are $1/4$ and $3/4$, not $1/2$ and $1/2$. Unequal token counts create the analogous issue for masked language-model objectives, where each rank's "mean over tokens" covers a different number of tokens.

5. How is the dataset split across ranks?

Every rank in the script builds the **same** 128-example dataset (same seed). `DistributedSampler` then decides which indices each rank reads. Roughly: shuffle all indices with a seed that depends on the epoch, pad if needed, and give rank p every P-th index starting at p.

Script case: 128 examples, 2 ranks

- Per rank: $128/2 = 64$ examples.
- Batches per rank per epoch: $64/16 = 4$.
- Synchronized optimizer updates: 4 per epoch, $4 \times 10 = 40$ in total.
- The two ranks' index sets are disjoint and together cover all 128 indices (checked by building both samplers in one process; 0 overlap, 128 union).

For the shuffle used in epoch 0 (seed 0 is the sampler default), the first eight indices were rank 0: `[44, 121, 31, 71, 105, 15, 48, 30]` and rank 1: `[94, 13, 116, 84, 14, 22, 11, 77]`. After `sampler.set_epoch(1)` they change, e.g. rank 0 starts `[37, 20, 103, 33, ...]`. Without `set_epoch`, every epoch would reuse the same order.

Small uneven case: 10 examples, 3 ranks, no shuffle

10 is not divisible by 3. Verified with `DistributedSampler(..., num_replicas=3, rank=r, shuffle=False)`:

Setting	Samples per rank	Rank 0	Rank 1	Rank 2
`drop_last=False` (default)	$\lceil 10/3 \rceil = 4$	`[0,3,6,9]`	`[1,4,7,0]`	`[2,5,8,1]`
`drop_last=True`	$\lfloor 10/3 \rfloor = 3$	`[0,3,6]`	`[1,4,7]`	`[2,5,8]`

With padding, indices 0 and 1 are seen twice (12 slots for 10 examples). With `drop_last=True`, example 9 is skipped. Account for duplicates when computing evaluation metrics across ranks.

6. Effective batch size and learning rate

The number of examples behind one optimizer update is

$$\boxed{B_{eff} = B_{local} \times P \times K},$$

assuming equal-sized batches and correctly weighted accumulation. In the script: $B_{eff} = 16 \times 2 \times 1 = 32$.

Run	B_{local}	P	K	B_{eff}	Updates per epoch ($128/B_{eff}$)
One worker	16	1	1	16	8
Script	16	2	1	32	4
One worker, matched	32	1	1	32	4
One worker, accumulation	16	1	2	32	4

Two consequences:

- Increasing worker count with fixed local batch **changes the global batch** and **reduces updates per epoch**. Ten epochs on one worker with batch 16 is 80 updates; ten epochs in the script is 40. Matched epoch count does not imply matched optimizer-update count.
- A larger batch gives a less noisy gradient but fewer steps. A common heuristic, the "linear scaling rule", multiplies the learning rate by the batch-size factor (here $0.1 \times 2 = 0.2$), often with warmup. This is a heuristic, not a guarantee: it can fail at large batch sizes and interacts with the optimizer. The script keeps `lr=0.1` and does not test any scaling rule.

As an extra check (not part of the script), a single-process replay that used the same per-rank index order and averaged the two ranks' batch losses for 40 updates produced the same weights as the two-process run, to all printed digits. That is the equivalence in Section 4 holding in practice.

7. Read the process lifecycle

Code	Role
`torchrun`	Launch workers and set rank/world-size environment variables
`'RANK' not in os.environ` check	Refuse to run as a plain `python` script

Code	Role
`init_process_group('gloo')`	Establish the distributed communication group
`DistributedSampler`	Partition data consistently across ranks
`DistributedDataParallel(model)`	Synchronize replicated model training
`loss.backward()`	Trigger gradient synchronization through DDP hooks
Rank-zero condition	Print once instead of duplicating output from all workers
`destroy_process_group()` in `finally`	Clean up communication resources

Every participating worker must execute compatible collective operations. A worker taking a different branch and skipping backward can leave the others waiting forever. This equal-sized teaching dataset avoids uneven-input complications. Save checkpoints only on rank zero, and aggregate evaluation metrics across ranks rather than reporting one rank's numbers.

8. DDP versus sharding

DDP normally keeps full weights, gradients, and optimizer state on each worker. FSDP and ZeRO variants **shard** some of these states to reduce per-worker memory. Those methods add gathering/scattering and have different memory/communication tradeoffs. They are not implemented in this file.

For multiple NVIDIA GPUs, a typical adaptation uses one process per GPU, selects the local device, moves inputs and model there, and uses NCCL. Do not assume this CPU script can use GPUs merely by changing the backend string.

9. Map the explanation to the script

Code	Meaning
`if 'RANK' not in os.environ:`	Detect a missing `torchrun` launch and exit with instructions
`dist.init_process_group('gloo')`	Join the CPU communication group
`torch.manual_seed(7)`	Same dataset and initial weights on every rank
`x = torch.randn(128, 4)`	128 examples, 4 features
`TensorDataset(x, x.sum(-1, keepdim=True))`	Target $y = x_1 + x_2 + x_3 + x_4$
`DistributedSampler(dataset, shuffle=True)`	Give each rank its own 64 indices
`DataLoader(dataset, batch_size=16, sampler=sampler)`	Local batch $B_{local} = 16$
`DistributedDataParallel(nn.Linear(4, 1))`	Wrap the replica so gradients are averaged
`torch.optim.SGD(model.parameters(), lr=0.1)`	Identical optimizer on every rank
`sampler.set_epoch(epoch)`	Different deterministic shuffling each epoch
`(model(inputs)-targets).square().mean()`	Local mean squared error L_p
`loss.backward()`	Compute g_p and all-reduce to $\bar{g}$
`optimizer.step()`	Same update on every rank

Code	Meaning
`if dist.get_rank() == 0:`	Print from one rank only
`model.module.weight`	The wrapped `nn.Linear` inside DDP
`dist.destroy_process_group()`	Clean up in `finally`

10. Run and investigate

Run from the course directory:

```
torchrun --standalone --nproc_per_node=2 05_advanced/07_distributed_training.py
```

`--standalone` runs a single-node rendezvous on this machine; `--nproc_per_node=2` starts two worker processes (ranks 0 and 1). Local socket access is needed for the rendezvous even though both workers are on the same machine. Running the file with plain `python` exits with a message telling you to use `torchrun`.

`torchrun` may first print a warning that it set `OMP_NUM_THREADS=1`. Rank zero then prints the learned weight matrix as a nested list, e.g.

```
DDP learned weights: [[0.99894118309021, 0.9991607069969177, 1.0007399320602417,
0.9972922205924988]]
```

These are approximately `[0.999, 0.999, 1.001, 0.997]`, close to the target of all ones, matching the record in `VERIFICATION.md`. The bias is not printed. The script has no assertion; you judge success by these numbers. Exact digits can vary with PyTorch version and platform.

Investigations:

1. Print sample indices per rank (the script's exercise) and verify that the two ranks' sets are disjoint and cover all 128 examples.
2. Compare one and two workers at matched **global** batch size and update count. Explain why matched epoch count alone may not imply matched update count.
3. Try `--nproc_per_node=4` with the same local batch size and predict the new B_{eff} and updates per epoch before running.

11. Practice calculations

1. Two ranks have local gradients -2 and 8. What gradient does each rank use after DDP? $(-2+8)/2 = 3$**.**
2. With $\eta = 0.1$ and starting weight 1.0, what is the new weight on each rank? $1.0 - 0.1(3) = 0.7$ **on both.**
3. Four ranks, local batch 16, accumulation 2. What is B_{eff}? $16 \times 4 \times 2 = 128$**.**
4. With 4 ranks and local batch 16 on the 128-example dataset, how many examples per rank and updates per epoch? $128/4 = 32$ **examples;** $32/16 = 2$ **updates.**

5. Ranks hold 10 and 30 examples with mean gradients 1 and 5. What does default DDP average give, and what is the true per-example mean? **DDP: 3. True:** $\frac{10}{40}(1) + \frac{30}{40}(5) = 0.25 + 3.75 = 4$**.**
6. 7 examples on 2 ranks with `drop_last=False` and no shuffle: what does each rank get? $\lceil 7/2 \rceil = 4$ **each: rank 0** `[0,2,4,6]`**, rank 1** `[1,3,5,0]` **(index 0 is repeated as padding).**
7. Does DDP let a model too large for one worker's memory fit across two workers? **No. Each rank holds a full replica; that needs sharding (FSDP/ZeRO) or model parallelism.**

Next: projects.

Companion code: `07_distributed_training.py`

05_advanced/07_distributed_training.py

```python
"""Minimal CPU DistributedDataParallel training, two processes.
Run from the course directory:
  torchrun --standalone --nproc_per_node=2 05_advanced/07_distributed_training.py
DDP replicates the model, splits data across workers, and averages gradients.
It does NOT shard model weights/optimizer state; FSDP/ZeRO address that memory.
This CPU/gloo lesson avoids GPU assumptions; use NCCL and one GPU per rank for CUDA.
Exercise: print sample indices per rank and verify that training data is partitioned.
"""
import os
import torch
import torch.distributed as dist
from torch import nn
from torch.nn.parallel import DistributedDataParallel
from torch.utils.data import DataLoader, TensorDataset, DistributedSampler


def main():
    if 'RANK' not in os.environ:
        raise SystemExit('Launch with torchrun --standalone --nproc_per_node=2 '+__file__)
    torch.set_num_threads(1)
    dist.init_process_group('gloo')
    try:
        torch.manual_seed(7)
        x = torch.randn(128, 4)
        dataset = TensorDataset(x, x.sum(-1, keepdim=True))
        sampler = DistributedSampler(dataset, shuffle=True)
        loader = DataLoader(dataset, batch_size=16, sampler=sampler)
        model = DistributedDataParallel(nn.Linear(4, 1))
        optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
        for epoch in range(10):
            sampler.set_epoch(epoch)  # Different deterministic shuffling each epoch.
            for inputs, targets in loader:
                optimizer.zero_grad(set_to_none=True)
                loss = (model(inputs)-targets).square().mean()
                loss.backward()  # DDP synchronizes gradient buckets here.
                optimizer.step()
        if dist.get_rank() == 0:
            print('DDP learned weights:', model.module.weight.detach().tolist())
        # Save only on rank zero. Aggregate evaluation metrics across ranks.
        # Uneven local batch sizes require care: rank means != global sample mean.
    finally:
        dist.destroy_process_group()

if __name__ == '__main__':
    main()
```

PART 6

Projects and appendices

Capstones and thesis experiments

Each project should produce runnable code, a configuration file, a data/split manifest, an evaluation table, and a short report of failures and limitations. Start with the smallest version that answers your question.

1. Full fine-tuning versus LoRA

Question: when does a higher adapter rank improve held-out task performance? Build a small domain-specific instruction dataset with non-overlapping source splits. Compare the unchanged base, full tuning, and LoRA ranks 2/8/16. Keep training data, token budget, and evaluation prompts fixed. Report task accuracy, validation loss, trainable parameters, time, peak memory, and source capability retention. Run multiple seeds. Do not infer quality from loss alone.

Stretch: repeat with QLoRA and separate numerical compression effects from changes in hardware, batch size, and optimizer.

2. DQN versus PPO

Question: how do sample efficiency and stability differ on a small discrete task? First use the chain environment. Then implement a Gymnasium adapter and move to a task such as CartPole. Preserve termination/truncation semantics. Compare at equal environment steps, log learning curves, and evaluate multiple seeds.

Ablations: DQN without replay, DQN without a target network, PPO without entropy, and PPO with different rollout/update ratios. Include a random-action baseline.

3. DPO under preference noise

Question: how robust is preference optimization when pair labels are corrupted? Begin with the categorical DPO lesson. Generate train/validation/test preference pairs from a known utility. Flip 0/10/30 percent of TRAINING labels only. Sweep beta using validation data. Measure held-out preference accuracy, KL to the reference, and whether chosen and rejected probabilities move as expected.

Stretch: use a small language model and examine response-length confounding.

4. GRPO reward design

Question: what happens when a reward is easy to game? Expand the one-token task into generated arithmetic answers. Compare exact correctness rewards with a flawed format/length-based reward. Keep a separate correctness evaluator. Measure true accuracy, reward, diversity, response length, and the fraction of groups with zero reward variance.

Stretch: compare group sizes at equal TOTAL generated-token budgets.

5. Offline RL with missing action coverage

Question: how does dataset coverage change the usefulness of pessimism? Collect fixed chain transitions with different behavior policies. Compare behavior cloning, ordinary offline Q-learning, and the discrete CQL penalty. Use held-out environment evaluation without adding those transitions to training. Plot return against coverage and conservative penalty coefficient.

A report template

- Research question and testable hypothesis.
- Baselines and why they are appropriate.
- Data provenance, split policy, and leakage checks.
- Exact configuration, versions, hardware, seeds and compute budget.
- Evaluation protocol with uncertainty and paired comparisons where applicable.
- Results, representative failures, and alternative explanations.
- Limitations and the next experiment that could disprove your interpretation.

These are starting experiments, not claims of novel thesis contributions. Check the literature before describing any result as new.

Primary sources for deeper study

These links are references, not runtime dependencies. Online documentation can change; match it to the package version you installed. Implementations here are original teaching examples and deliberately smaller than production trainers.

Foundations and fine-tuning

- PyTorch autograd tutorial: gradients and computational graphs.
- Attention Is All You Need: transformer architecture.
- LoRA paper and PEFT LoRA guide: low-rank adaptation.
- PEFT quicktour: adapters and pretrained model integration.
- QLoRA paper and bitsandbytes integration: quantized base-model tuning.

Reinforcement learning

- Sutton and Barto book, author's site: MDPs, dynamic programming, Monte Carlo, TD, policy gradients.
- Gymnasium basic usage: reset/step API and episode boundaries.
- DQN paper: deep Q-learning.
- Double DQN paper: reducing overestimation.
- GAE paper: advantage estimation.
- PPO paper: clipped policy optimization.
- SAC documentation: entropy-regularized continuous control.
- CQL paper: conservative offline Q-learning.

Alignment

- InstructGPT paper: instruction tuning and human feedback.
- DPO paper and TRL DPO documentation: direct preference optimization.
- DeepSeekMath paper and TRL GRPO documentation: group-relative policy optimization and implementation variants.

Suggested paper-reading order: problem -> objective equation -> assumptions -> algorithm -> experiments -> limitations. Reproduce one equation before reading all implementation details. The advanced roadmap names further topics without claiming that this folder implements them all.

Additional requested methods

- A3C: asynchronous advantage actor-critic.
- TRPO: constrained policy updates.
- SDPO, v1: feedback-conditioned self-teacher, reverse-KL objective.
- OPSD, v1: privileged reference context and on-policy distillation.

- Self-Distilled RLVR / RLSD, v1: verifier advantages with teacher-weighted token credit.

These versioned research references were checked on 2026-10-22. The companion objective labs state their divergence choices and do not reproduce full trainers.

Verification record

Verified on 2026-09-23 using Python 3.12.3 on CPU. Packages: PyTorch 2.14.0+cpu, NumPy 2.5.3, Transformers 5.17.0, PEFT 0.21.0, Accelerate 1.15.0, and Safetensors 0.8.0. See `requirements-tested.txt` for top-level version pins. The broader requirements ranges are convenience ranges, not a guarantee that every possible combination has been tested.

Executed successfully

- All 27 lesson scripts passed Python syntax compilation.
- `python run_lessons.py`: 24/24 offline single-process lessons completed.
- `python -m unittest discover -s tests -v`: 13/13 numerical checks passed.
- Two CPU processes ran the DDP lesson successfully. Learned regression weights were approximately [0.999, 0.999, 1.001, 0.997], against a target of all ones.
- The real-model SFT script loaded `sshleifer/tiny-gpt2`, ran five LoRA updates, evaluated the validation split, and saved the adapter and tokenizer.
- The saved LoRA adapter was reloaded with its original base and generated text.
- The full fine-tuning branch ran two updates, saved a full model checkpoint, and successfully reloaded it for generation.

The numerical tests cover future-token leakage, prompt/padding supervision, LoRA initialization and merging, Bellman values, discounted returns, terminal versus truncated GAE behavior, DPO gradient direction, masked sequence log-probabilities, constant-reward GRPO, conservative Q gradients, and preserving real EOS targets when EOS is also the padding token.

Limits of this verification

These are execution and mathematical-invariant checks, not benchmark claims. The tiny GPT-2 model produced repetitive text and its validation loss did not improve in the short test. That is expected for a plumbing check and is not a useful assistant. The larger default `distilgpt2` model was not downloaded/tested.

CUDA mixed precision, NVIDIA QLoRA, large models, multi-GPU training, and other package versions were not tested. SAC/CQL and the categorical alignment labs teach objectives; they are not complete large-scale training systems.

DDP initially could not open local sockets in the restricted execution sandbox; it passed when run with local socket access. Package/model downloads likewise needed network access. Core lessons run offline after installing dependencies.

Theory companion update

Added a matching `.md` chapter for each of the 27 lesson scripts, plus a math primer and a paired theory/code index. This update changes documentation only.

Checked all 27 companion files exist, all local Markdown links resolve, and fenced code/display-equation delimiters are balanced. Independently recomputed the worked GAE recurrence and return targets, PPO clipping table, finite-difference gradient, broadcasting-error loss, categorical RLHF optimum, and DPO example. These checks passed. They are arithmetic/document-structure checks, not a claim that every Markdown viewer renders LaTeX identically.

The SAC companion explicitly explains the existing example's identical critic initialization, and the policy-gradient companion distinguishes its simplified finite-horizon and discount-weighting conventions from a general exact objective.

Requested RL syllabus update — 2026-10-22

Verified using Python 3.12.3, PyTorch 2.14.1+cpu, and NumPy 2.5.2 in `/tmp/thesis-rl-venv` (temporary verification environment).

- All 31 lesson scripts passed Python syntax parsing; all local Markdown links resolved.
- Offline lesson runner: **28/28 passed**; standard-library subset: **6/6 passed**.
- Numerical invariant suite: **17/17 passed**. Added checks for policy-iteration agreement, actual TRPO KL and improvement, masked distillation/teacher stop-gradient, and RLSD sign preservation and zero-mixing behavior.
- Original DQN (`--vanilla-dqn`) also ran successfully, selecting right in all three nonterminal states. The default Double DQN passed in the full runner.
- A2C and concurrent A3C trained successfully. A3C scheduling is nondeterministic; observed right probabilities were approximately [0.922, 0.945, 0.912].
- TRPO's bandit lab reached approximately 0.993 probability on the best action.
- SDPO, OPSD, and JSD objective losses decreased, with no teacher gradients.

These checks exercise teaching examples, not published LLM results. The TRPO and self-distillation additions are objective/update labs; they do not implement complete trajectory-based TRPO or real language-model post-training pipelines. The existing external-download and DDP scripts were not rerun for this update.